

Matt Weisfeld

Fifth Edition



Tradução PT-BR - página original 1

O PROCESSO DE PENSAMENTO ORIENTADO A OBJETOS

Quinta Edição

Matt Weisfeld

The Object-Oriented Thought Process



The Object-Oriented Thought Process

Fifth Edition

Tradução PT-BR - página original 2

O Processo de Pensamento Orientado a Objetos

Quinta Edição

The Object-Oriented Thought Process

Fifth Edition

Matt Weisfeld

Addison-Wesley

Boston • Columbus • New York • San Francisco • Amsterdam • Cape Town
Dubai • London • Madrid • Milan • Munich • Paris • Montreal • Toronto • Delhi
Mexico City • São Paulo • Sydney • Hong Kong • Seoul • Singapore • Taipei • Tokyo

Tradução PT-BR - página original 3

O Processo de Pensamento Orientado a Objetos

Quinta Edição

Matt Weisfeld

Boston - Columbus - Nova York - São Francisco - Amsterdã - Cidade do Cabo
Dubai - Londres - Madri - Milão - Munique - Paris - Montreal - Toronto - Délhi
Cidade do México - São Paulo - Sydney - Hong Kong - Seul - Singapura - Taipei - Tóquio

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in this book, and the publisher was aware of a trademark claim, the designations have been printed with initial capital letters or in all capitals.

The author and publisher have taken care in the preparation of this book, but make no expressed or implied warranty of any kind and assume no responsibility for errors or omissions. No liability is assumed for incidental or consequential damages in connection with or arising out of the use of the information or programs contained herein.

For information about buying this title in bulk quantities, or for special sales opportunities (which may include electronic versions; custom cover designs; and content particular to your business, training goals, marketing focus, or branding interests), please contact our corporate sales department at corpsales@pearsoned.com or (800) 382-3419.

For government sales inquiries, please contact governmentsales@pearsoned.com.

For questions about sales outside the U.S., please contact intlcs@pearson.com.

Visit us on the Web: informit.com/aw

Library of Congress Control Number: 2019930825

Copyright © 2019 Pearson Education, Inc.

All rights reserved. Printed in the United States of America. This publication is protected by copyright, and permission must be obtained from the publisher prior to any prohibited reproduction, storage in a retrieval system, or transmission in any form or by any means, electronic, mechanical, photocopying, recording, or likewise. For information regarding permissions, request forms and the appropriate contacts within the Pearson Education Global Rights & Permissions Department, please visit www.pearsoned.com/permissions/.

ISBN-13: 978-0-13-518196-6

ISBN-10: 0-13-518196-8

1 19

Microsoft and/or its respective suppliers make no representations about the suitability of the information contained in the documents and related graphics published as part of the services for any purpose. All such documents and related graphics are provided “as is” without warranty of any kind. Microsoft and/ or its respective suppliers hereby disclaim all warranties and conditions with regard to this information, including all warranties and conditions of merchantability, whether express, implied or statutory, fitness for a particular purpose, title and non-infringement. In no event shall Microsoft and/or its respective sup-pliers be liable for any special, indirect or consequential damages or any damages whatsoever resulting from loss of use, data or profits, whether in an action of contract, negligence or other tortious action, arising out of or in connection with the use or performance of information available from the services.

The documents and related graphics contained herein could include technical inaccuracies or typographical errors. Changes are periodically added to the information herein. Microsoft and/or its respective sup-pliers may make improvements and/or changes in the product(s)

Muitas das designações usadas por fabricantes e vendedores para distinguir seus produtos são reivindicadas como marcas registradas. Quando essas designações aparecem neste livro e a editora tinha conhecimento de uma reivindicação de marca, elas foram impressas com iniciais maiúsculas ou inteiramente em maiúsculas.

O autor e a editora tiveram cuidado na preparação deste livro, mas não oferecem garantia expressa ou implícita de qualquer tipo e não assumem responsabilidade por erros ou omissões. Não se assume responsabilidade por danos incidentais ou consequenciais relacionados ao uso das informações ou programas aqui contidos.

Para informações sobre compras em grandes quantidades ou oportunidades de vendas especiais, entre em contato com o departamento de vendas corporativas da Pearson pelos dados apresentados na página original. Consultas sobre vendas governamentais e internacionais também podem ser feitas pelos endereços indicados no original.

Visite: informit.com/aw

Número de Controle da Library of Congress: 2019930825

Copyright © 2019 Pearson Education, Inc.

Todos os direitos reservados. Impresso nos Estados Unidos da América. Esta publicação é protegida por direitos autorais e é necessária autorização da editora antes de qualquer reprodução, armazenamento ou transmissão proibida, por meios eletrônicos, mecânicos, fotocópia, gravação ou similares.

ISBN-13: 978-0-13-518196-6

ISBN-10: 0-13-518196-8

A Microsoft e/ou seus fornecedores não fazem declarações sobre a adequação, para qualquer finalidade, das informações contidas nos documentos e gráficos relacionados publicados como parte dos serviços. Esses materiais são fornecidos “como estão”, sem garantias. A Microsoft e/ou seus fornecedores rejeitam garantias e condições relativas a essas informações, inclusive comercialização, adequação a finalidade específica, titularidade e não violação de direitos. Em nenhuma hipótese serão responsáveis por danos especiais, indiretos ou consequenciais, nem por perdas de uso, dados ou lucros decorrentes do uso ou desempenho das informações disponíveis nos serviços.

Os documentos e gráficos relacionados podem conter imprecisões técnicas ou erros tipográficos. Alterações são adicionadas periodicamente. A Microsoft e/ou seus fornecedores podem realizar melhorias e/ou alterações nos produtos e programas descritos a qualquer momento.

and/or the program(s) described herein at any time. Partial screenshots may be viewed in full within the software version specified.

Microsoft® and Windows® are registered trademarks of the Microsoft Corporation in the U.S.A. and other countries. Screenshots and icons reprinted with permission from the Microsoft Corporation. This book is not sponsored or endorsed by or affiliated with the Microsoft Corporation.

Editor-in-Chief

Mark Taub

Development Editor

Mark Taber

Managing Editor

Sandra Schroeder

Senior Project Editor

Tonya Simpson

Indexer

Erika Millen

Proofreader

Abigail Manheim

Technical Reviewer

John Upchurch

Editorial Assistant

Cindy Teeters

Cover Designer

Chuti Prasertsith

Compositor

codeMantra

Tradução PT-BR - página original 5

...e/ou nos programas descritos, a qualquer momento. Capturas de tela parciais podem ser vistas integralmente na versão de software especificada.

Microsoft® e Windows® são marcas registradas da Microsoft Corporation nos Estados Unidos e em outros países. Capturas de tela e ícones foram reproduzidos com permissão da Microsoft Corporation. Este livro não é patrocinado, endossado nem afiliado à Microsoft Corporation.

Editor-chefe
Mark Taub

Editor de desenvolvimento
Mark Taber

Editora-gerente
Sandra Schroeder

Editora sênior de projeto
Tonya Simpson

Responsável pelo índice
Erika Millen

Revisora
Abigail Manheim

Revisor técnico
John Upchurch

Assistente editorial
Cindy Teeters

Designer da capa
Chuti Prasertsith

Composição
codeMantra

Acknowledgments

As with the first four editions, this book required the combined efforts of many people. I would like to take the time to acknowledge as many of these people as possible, for without them, this book would never have happened.

First and foremost, I would like to thank my wife Sharon for all her help. Not only did she provide support and encouragement throughout this lengthy process, she is also the first line editor for all my writing.

I would also like to thank my mom and the rest of my family for their continued support.

It is hard to believe that the work on the first edition of this book began in 1998. For all these years, I have thoroughly enjoyed working with everyone at Pearson—on all five editions. Working with editors Mark Taber and Tonya Simpson on this edition has been a pleasure.

A special thanks goes to Jon Upchurch for his expertise with much of the code as well as the technical editing of the manuscript. Jon’s insights into an amazing range of technical topics have been of great help to me.

Finally, thanks to my daughters, Stacy and Stephanie, and my cat, Paulo, for always keeping me on my toes.

About the Author

Matt Weisfeld is a college professor, software developer, and author based in Cleveland, Ohio. Prior to teaching college full time, he spent 20 years in the information technology industry as a software developer, entrepreneur, and adjunct professor. Weisfeld holds an MS in computer science and an MBA. Besides several editions of *The Object-Oriented Thought Process*, Matt has authored two other software development books and published many articles in magazines and journals, such as *informit.com*, *developer.com*, *Dr. Dobb’s Journal*, *The C/C++ Users Journal*, *Software Development Magazine*, *Java Report*, and the international journal *Project Management*.

AGRADECIMENTOS

Assim como nas quatro primeiras edições, este livro exigiu o esforço conjunto de muitas pessoas. Gostaria de reconhecer o maior número possível delas, pois sem sua colaboração este livro nunca teria acontecido.

Antes de tudo, agradeço à minha esposa, Sharon, por toda a ajuda. Além de oferecer apoio e incentivo durante este longo processo, ela também é a primeira editora de tudo o que escrevo.

Também agradeço à minha mãe e ao restante da minha família pelo apoio contínuo.

É difícil acreditar que o trabalho na primeira edição deste livro começou em 1998. Durante todos esses anos, tive enorme prazer em trabalhar com todos na Pearson, nas cinco edições. Trabalhar com os editores Mark Taber e Tonya Simpson nesta edição foi um prazer.

Um agradecimento especial a Jon Upchurch por sua experiência com grande parte do código e pela revisão técnica do manuscrito. As percepções de Jon sobre uma gama impressionante de temas técnicos foram de grande ajuda.

Por fim, obrigado às minhas filhas, Stacy e Stephanie, e ao meu gato, Paulo, por sempre me manterem atento.

SOBRE O AUTOR

Matt Weisfeld é professor universitário, desenvolvedor de software e autor, residente em Cleveland, Ohio. Antes de lecionar em tempo integral, trabalhou por 20 anos na indústria de tecnologia da informação como desenvolvedor de software, empreendedor e professor adjunto. Weisfeld possui mestrado em Ciência da Computação e MBA. Além de várias edições de *The Object-Oriented Thought Process*, escreveu outros dois livros sobre desenvolvimento de software e publicou numerosos artigos em revistas e periódicos, incluindo *informit.com*, *developer.com*, *Dr. Dobb’s Journal*, *The C/C++ Users Journal*, *Software Development Magazine*, *Java Report* e o periódico internacional *Project Management*.

We Want to Hear from You!

As the reader of this book, *you* are our most important critic and commentator. We value your opinion and want to know what we’re doing right, what we could do better, what areas you’d like to see us publish in, and any other words of wisdom you’re willing to pass our way.

We welcome your comments. You can email or write to let us know what you did or didn’t like about this book—as well as what we can do to make our books better.

Please note that we cannot help you with technical problems related to the topic of this book.

When you write, please be sure to include this book’s title and author as well as your name and email address. We will carefully review your comments and share them with the author and editors who worked on the book.

Email: community@informit.com

Reader Services

Visit our website and register this book at www.informit.com/register for convenient access to any updates, downloads, or errata that might be available for this book.

QUEREMOS OUVIR VOCÊ!

Como leitor deste livro, você é nosso crítico e comentarista mais importante. Valorizamos sua opinião e queremos saber o que estamos fazendo bem, o que poderíamos fazer melhor, em quais áreas você gostaria que publicássemos e quaisquer outras sugestões que queira compartilhar.

Seus comentários são bem-vindos. Você pode enviar um e-mail ou escrever para dizer do que gostou ou não gostou neste livro e o que podemos fazer para melhorar nossos livros.

Observe que não podemos ajudar com problemas técnicos relacionados ao tema deste livro.

Ao escrever, inclua o título e o autor do livro, além de seu nome e endereço de e-mail. Seus comentários serão analisados cuidadosamente e compartilhados com o autor e com os editores que trabalharam no livro.

E-mail: community@informit.com

SERVIÇOS AO LEITOR

Visite nosso site e registre este livro em www.informit.com/register para ter acesso conveniente a atualizações, downloads ou erratas que possam estar disponíveis.

Introduction

THIS BOOK’S SCOPE

As the title suggests, this book is about the object-oriented (OO) thought process. Although choosing the theme and title of a book are important decisions, these decisions are not at all straightforward when dealing with a highly conceptual topic. Many books deal with one level or another of programming and object orientation. Several popular books cover topics including OO analysis, OO design, OO programming, design patterns, OO data (XML), the Unified Modeling Language (UML), OO web development, OO mobile development, various OO programming languages, and many other topics related to OO programming (OOP).

However, while poring over all these books, many people forget that all these topics are built on a single foundation: how you think in OO ways. Often, many software professionals, as well as students, dive into these books without taking the appropriate time and effort to really understand the design concepts behind the code.

I contend that learning OO concepts is not accomplished by learning a specific development method, a programming language, or a set of design tools. Object-oriented development is, simply put, a way of thinking. This book is all about the OO thought process.

Separating the languages, development practices, and tools from the OO thought process is not an easy task. Often, people are introduced to OO concepts by diving headfirst into a programming language. For example, many years ago a large number of C programmers were first introduced to object orientation by migrating directly to C++ before they were even remotely exposed to OO concepts.

It is important to understand the significant difference between learning OO concepts and programming in an OO language. This came into sharp focus for me well before I worked on the first edition of this book, when I read articles like Craig Larman’s “What the UML Is—and Isn’t,” In this article he states,

Unfortunately, in the context of software engineering and the UML diagramming language, acquiring the skills to read and write UML notation seems to sometimes be equated with skill in object-oriented analysis and design. Of course, this is not so, and the latter is much more important than the former. Therefore, I recommend seeking education and educational materials in which intellectual skill in object-oriented analysis and design is paramount rather than UML notation or the use of a case tool.

Thus, although learning a modeling language is an important step, it is much more important to learn OO skills first. Learning UML before fully understanding OO concepts is similar to learning how to read an electrical diagram without first knowing anything about electricity.

The same problem occurs with programming languages. As stated earlier, many C programmers moved into the realm of object orientation by migrating to C++ before being directly exposed to OO concepts. This would always come out in an interview. Quite often developers who claim to be C++ programmers are simply C programmers using C++ compilers. Even now, with languages such as C# .NET, VB .NET, Objective-C, Swift, and Java well established, a few key questions in a job interview can quickly uncover a lack of OO understanding.

INTRODUÇÃO

O ESCOPO DESTE LIVRO

Como o título sugere, este livro trata do processo de pensamento orientado a objetos (OO). Embora a escolha do tema e do título de um livro seja importante, essas decisões não são simples quando se lida com um assunto altamente conceitual. Muitos livros abordam diferentes níveis de programação e orientação a objetos. Há obras populares sobre análise OO, projeto OO, programação OO, padrões de projeto, dados OO (XML), Unified Modeling Language (UML), desenvolvimento web OO, desenvolvimento móvel OO, linguagens de programação OO e muitos outros tópicos relacionados à programação orientada a objetos (POO).

Ao estudar todas essas obras, porém, muitas pessoas esquecem que todos esses assuntos se apoiam em uma única base: a forma de pensar de maneira orientada a objetos. Profissionais de software e estudantes frequentemente mergulham nesses livros sem dedicar tempo e esforço suficientes para compreender de fato os conceitos de projeto por trás do código.

Defendo que aprender os conceitos de OO não significa aprender um método específico de desenvolvimento, uma linguagem de programação ou um conjunto de ferramentas de projeto. Desenvolvimento orientado a objetos é, de forma simples, uma maneira de pensar. Este livro é sobre esse processo de pensamento.

Separar linguagens, práticas de desenvolvimento e ferramentas do processo de pensamento OO não é fácil. Muitas vezes, as pessoas são apresentadas aos conceitos OO mergulhando diretamente em uma linguagem. Anos atrás, por exemplo, muitos programadores C conheceram a orientação a objetos migrando diretamente para C++ antes de terem qualquer contato real com os conceitos OO.

É importante compreender a grande diferença entre aprender conceitos OO e programar em uma linguagem OO. Isso ficou muito claro para mim antes mesmo da primeira edição, quando li textos como “What the UML Is - and Isn’t”, de Craig Larman. Em essência, Larman observa que saber ler e escrever notação UML não equivale a dominar análise e projeto orientados a objetos; a habilidade intelectual de analisar e projetar sistemas OO é muito mais importante do que a notação ou o uso de uma ferramenta CASE.

Portanto, embora aprender uma linguagem de modelagem seja importante, é muito mais importante aprender primeiro as habilidades OO. Aprender UML antes de compreender plenamente os conceitos OO é como aprender a ler um diagrama elétrico sem saber nada de eletricidade.

O mesmo problema ocorre com linguagens de programação. Muitos programadores C migraram para C++ antes de serem expostos diretamente aos conceitos OO. Em entrevistas, isso fica evidente: alguns desenvolvedores que se apresentam como programadores C++ são, na prática, programadores C usando um compilador C++. Mesmo hoje, com C# .NET, VB .NET, Objective-C, Swift e Java bem estabelecidos, poucas perguntas bem escolhidas revelam rapidamente a falta de compreensão de OO.

Early versions of Visual Basic are not OO. C is not OO, and C++ was developed to be backward compatible with C. Because of this, it is quite possible to use a C++ compiler while using only C syntax while forsaking all of C++’s OO features. Objective-C was designed as an extension to the standard ANSI C language. Even worse, a programmer can use just enough OO features to make a program incomprehensible to OO and non-OO programmers alike.

Thus, it is of vital importance that while you’re learning to use OO development environments, you first learn the fundamental OO concepts. Resist the temptation to jump directly into a programming language, and instead take the time to learn the object-oriented thought process first.

WHAT’S NEW IN THE FIFTH EDITION

As stated often in this introduction, my vision for the first edition was to stick to the concepts rather than focus on a specific emerging technology. Although I still adhere to this goal for the fifth edition, I also introduce more of the “counter-arguments” than were present in the earlier editions. By that I mean that although object-oriented development is, by far, the biggest game in town, it is not the only one.

Since the first edition of this book was completed in 1999, many technologies have emerged and some have faded. At the time, Java was just establishing itself and was the primary OO development language. Web pages would soon become a part of daily life and business. We all know how ubiquitous mobile devices have become. In the past 20 years software developers have encountered XML, JSON, CSS, XSLT, SOAP, and RESTful Web Services. Android devices use Java and now Kotlin, while iOS devices use Objective-C and Swift.

The point I am trying to make is that we have embraced a lot of technologies in the past 20 years (and four editions of the book). My primary goal for this edition is to condense all of this down to the original intent of the first edition, fundamental object-oriented concepts. I like to think that whatever success the first edition of the book had was because it focused on fundamental object-oriented concepts. In some ways we have gone full circle because this edition encapsulates all the technologies mentioned above.

Finally, the concepts that ultimately encapsulate these technologies into a design methodology are represented by SOLID, which is woven into all the chapters of this edition as well as two new chapters at the end of the book.

The five SOLID principles are

- **SRP**—Single Responsibility Principle
- **OCP**—Open/Close Principle
- **LSP**—Liskov Substitution Principle
- **IPS**—Interface Segregation Principle
- **DIP**—Dependency Inversion Principle

I often think of the first nine chapters as representing what I consider classical object-oriented principles. The last three chapters on design patterns, avoiding dependencies, and SOLID build on the classical principles and present a strong methodology.

As primeiras versões do Visual Basic não eram orientadas a objetos. C não é OO, e C++ foi desenvolvido para manter compatibilidade retroativa com C. Por isso, é perfeitamente possível usar um compilador C++ escrevendo apenas sintaxe C e ignorando todos os recursos OO da linguagem. Objective-C foi projetada como extensão da linguagem ANSI C. Pior ainda, um programador pode usar apenas recursos OO suficientes para tornar o programa incompreensível tanto para programadores OO quanto para não OO.

Assim, enquanto você aprende a usar ambientes de desenvolvimento OO, é essencial aprender primeiro os conceitos fundamentais de orientação a objetos. Resista à tentação de ir diretamente para uma linguagem de programação; dedique tempo a aprender primeiro o processo de pensamento orientado a objetos.

O QUE HÁ DE NOVO NA QUINTA EDIÇÃO

Como afirmado várias vezes nesta introdução, minha visão para a primeira edição era manter o foco nos conceitos, e não em uma tecnologia emergente específica. Continuo seguindo esse objetivo na quinta edição, mas agora apresento mais “contra-argumentos” do que nas edições anteriores. Embora o desenvolvimento orientado a objetos seja, de longe, a abordagem predominante, ele não é a única.

Desde a conclusão da primeira edição, em 1999, muitas tecnologias surgiram e algumas desapareceram. Naquele momento, Java começava a se estabelecer e era a principal linguagem de desenvolvimento OO. As páginas web logo passariam a fazer parte da vida cotidiana e dos negócios. Dispositivos móveis tornaram-se onipresentes. Nas últimas duas décadas, desenvolvedores conviveram com XML, JSON, CSS, XSLT, SOAP e serviços web RESTful. Android usa Java e, atualmente, também Kotlin; iOS utiliza Objective-C e Swift.

O ponto é que incorporamos muitas tecnologias ao longo de 20 anos e quatro edições. Meu objetivo principal nesta edição é condensar tudo isso de volta à intenção original da primeira edição: os conceitos fundamentais de orientação a objetos. Acredito que o sucesso da primeira edição ocorreu justamente porque ela se concentrou nesses fundamentos. De certo modo, completamos um ciclo, pois esta edição encapsula todas as tecnologias mencionadas.

Por fim, os conceitos que organizam essas tecnologias em uma metodologia de projeto são representados por SOLID, presente ao longo dos capítulos desta edição e em dois novos capítulos no final do livro.

Os cinco princípios SOLID são:

- SRP - Princípio da Responsabilidade Única (Single Responsibility Principle)
- OCP - Princípio Aberto/Fechado (Open/Closed Principle)
- LSP - Princípio da Substituição de Liskov (Liskov Substitution Principle)
- ISP - Princípio da Segregação de Interfaces (Interface Segregation Principle)
- DIP - Princípio da Inversão de Dependência (Dependency Inversion Principle)

Costumo considerar os nove primeiros capítulos como uma apresentação dos princípios “clássicos” de orientação a objetos. Os três últimos, sobre padrões de projeto, prevenção de dependências e SOLID, constroem sobre esses princípios clássicos e apresentam uma metodologia consistente.

THE INTENDED AUDIENCE

This book is a general introduction to the concepts of object-oriented programming. The term *concepts* is important because, while code is certainly used to reinforce the topics covered, the primary focus of this book is to ground the reader in the object-oriented thought process. It is also important for programmers to understand that OOP does not represent a distinct paradigm (as many believe)—OOP is simply one part of a vast toolkit available to modern software developers.

When the material for the first edition of this book was initially created in 1995, OOP was in its infancy. I can say this because, other than pockets of OO languages such as Smalltalk, there really were no true object-oriented languages in play at the time. C++, which does not enforce object-oriented constructs, was the dominant C-based language. Java 1.0 was released in 1996 and C# 1.0 in 2002. In fact, when the first edition of this book was published in 1999, there was no certainty that OO would actually become the leading development paradigm. (Java 2 wasn’t even released until December 1998.) Despite its current dominance, there are some interesting chinks in the OOP armor to be addressed.

Thus, the audience for the first edition differs from the audience today.

From 1995 until as late as 2010, I was basically retraining many structured programmers in the art of OOP. The vast majority of these students had grown up with COBOL, FORTRAN, C, and VB, both in college and on the job. Today, students graduating college, writing video games, creating websites, or producing mobile apps have almost certainly learned programming using an object-oriented language. Thus, the approach of the fifth edition of this book is significantly different from the first edition, or second, etc. Rather than teaching structured programmers to become OO developers, we are now teaching programmers who have grown up with OO languages.

The intended audience for this book includes business managers, designers, developers, programmers, and project managers: in short, anyone who wants to gain a general understanding of what object orientation is all about. My hope is that reading this book will provide a strong foundation for moving to other books covering more advanced topics.

THE BOOK’S APPROACH

It should be obvious by now that I am a firm believer in becoming comfortable with the object-oriented thought process before jumping into a programming language or modeling language. This book is filled with examples of code and UML class diagrams; however, you do not need to know a specific programming language or UML to read it. After all I have said about learning the concepts first, why is there so much code and class diagrams?

First, code and class diagrams are great for illustrating OO concepts. Second, they are integral to the OO process and should be addressed at an introductory level. The key is not to focus on Java, C#, and so on but to use them as aids in the understanding of the underlying concepts.

Note that I really like using UML class diagrams as a visual aid to illustrate classes, and their attributes and methods. In fact, the class diagrams are the only component of UML used in this book. I believe that the UML class diagrams offer a great way to model the conceptual nature of object models. I continue to use object models as an educational tool to illustrate class design and how classes relate to one another.

O PÚBLICO-ALVO

Este livro é uma introdução geral aos conceitos de programação orientada a objetos. A palavra “conceitos” é importante: embora código seja usado para reforçar os assuntos, o foco principal é oferecer ao leitor uma base sólida no processo de pensamento orientado a objetos. Também é importante entender que POO não representa um paradigma isolado, como muitos acreditam; ela é apenas uma parte do amplo conjunto de ferramentas disponíveis aos desenvolvedores de software modernos.

Quando o material da primeira edição começou a ser criado, em 1995, a POO ainda estava em sua infância. Fora alguns nichos de linguagens OO, como Smalltalk, havia poucas linguagens verdadeiramente orientadas a objetos em uso. C++, que não obriga o uso de construções OO, era a linguagem dominante baseada em C. Java 1.0 foi lançado em 1996 e C# 1.0 em 2002. Quando a primeira edição saiu, em 1999, nem sequer havia certeza de que OO se tornaria o paradigma dominante. Apesar de sua predominância atual, existem pontos vulneráveis e limitações interessantes na POO que merecem discussão.

Assim, o público da primeira edição é diferente do público de hoje.

De 1995 até aproximadamente 2010, eu basicamente retreinava muitos programadores estruturados na arte da POO. A maioria desses alunos tinha crescido profissionalmente com COBOL, FORTRAN, C e VB. Hoje, estudantes que saem da universidade, desenvolvem jogos, criam sites ou produzem aplicativos móveis quase certamente aprenderam programação usando alguma linguagem orientada a objetos. Por isso, a abordagem da quinta edição é significativamente diferente das anteriores: em vez de ensinar programadores estruturados a se tornarem desenvolvedores OO, ensinamos programadores que cresceram usando linguagens OO.

O público pretendido inclui gestores de negócios, projetistas, desenvolvedores, programadores e gerentes de projeto; em resumo, qualquer pessoa que queira obter uma compreensão geral do que é orientação a objetos. Espero que a leitura forneça uma base sólida para avançar a livros sobre assuntos mais especializados.

A ABORDAGEM DO LIVRO

A esta altura deve estar claro que acredito firmemente em adquirir familiaridade com o processo de pensamento orientado a objetos antes de mergulhar em uma linguagem de programação ou de modelagem. O livro contém muitos exemplos de código e diagramas de classes UML; ainda assim, não é necessário conhecer uma linguagem específica nem UML para lê-lo. Então por que usar tanto código e tantos diagramas?

Primeiro, código e diagramas de classes são excelentes para ilustrar conceitos OO. Segundo, são partes integrantes do processo OO e devem ser apresentados em nível introdutório. A chave não é focar em Java, C# ou outra linguagem, mas usá-las como auxílio à compreensão dos conceitos subjacentes.

Gosto especialmente de diagramas de classes UML como recurso visual para mostrar classes, atributos e métodos. Na verdade, os diagramas de classes são o único componente de UML usado neste livro. Eles oferecem uma ótima maneira de modelar a natureza conceitual de modelos de objetos, ilustrando o projeto de classes e as relações entre elas.

The code examples in the book illustrate concepts such as loops and functions; however, understanding the code itself is not a prerequisite for understanding the concepts. It might be helpful to have a book at hand that covers specific languages’ syntax if you want to get more detailed.

I cannot state too strongly that this book does *not* teach Java, C# .NET, VB .NET, Objective-C, Swift, or UML, all of which can command volumes unto themselves. It is also important to understand that this is a book of concepts, and the intent of the examples in this book is not, necessarily, to describe the *optimal* way to design your classes; they are an educational exercise meant to get you thinking about OO concepts. For example, it is obvious that you won’t create many models using penguins and barkless dogs on the job—but using them is a fun way to demonstrate the concepts. With all of this in mind, it is my hope that this book will whet your appetite for other OO topics, such as OO analysis, object-oriented design, and OO programming.

SOURCE CODE USED IN THIS BOOK

The sample code described throughout this book is available on the publisher’s website. Go to informit.com/register and register your book for access to downloads.

Os exemplos de código do livro ilustram conceitos como laços e funções; porém, compreender o código em si não é pré-requisito para compreender os conceitos. Se você quiser aprofundar detalhes de sintaxe, pode ser útil ter à mão um livro específico sobre a linguagem utilizada.

É importante reforçar que este livro não ensina Java, C# .NET, VB .NET, Objective-C, Swift nem UML; cada um desses assuntos poderia ocupar vários volumes. Este é um livro de conceitos, e os exemplos não pretendem necessariamente mostrar a maneira ótima de projetar classes. Eles são exercícios educacionais destinados a fazer você pensar sobre conceitos OO. É evidente que no trabalho você provavelmente não criará muitos modelos envolvendo pinguins e cães que não latem, mas esses exemplos são maneiras divertidas de demonstrar os conceitos. Espero que o livro desperte seu interesse por outros temas de OO, como análise orientada a objetos, projeto orientado a objetos e programação OO.

CÓDIGO-FONTE USADO NESTE LIVRO

O código de exemplo descrito ao longo do livro está disponível no site da editora. Acesse informit.com/register e registre seu livro para obter acesso aos downloads.

1. Introduction to Object-Oriented Concepts

Although many programmers don’t realize it, object-oriented (OO) software development has been around since the early 1960s. It wasn’t until the mid to late 1990s that the object-oriented paradigm started to gain momentum, despite the fact that popular object-oriented programming languages such as Smalltalk and C++ were already widely used.

The rise of OO methodologies coincides with the emergence of the Internet as a business and entertainment platform. In short, objects work well over a network. And after it became obvious that the Internet was here to stay, object-oriented technologies were already well positioned to develop the new web-based technologies.

It is important to note that the title of this first chapter is “Introduction to Object-Oriented Concepts.” The operative word here is “concepts” and not “technologies.” Technologies change very quickly in the software industry, whereas concepts evolve. I use the term “evolve” because, although they remain relatively stable, they do change. And this is what is really cool about focusing on the concepts. Despite their consistency, they are always undergoing reinterpretations, and this allows for some very interesting discussions.

This evolution can be easily traced over the past 25 years or so as we follow the progression of the various industry technologies from the first primitive browsers of the mid to late 1990s to the mobile/phone/web applications that dominate today. As always, new developments are just around the corner as we explore hybrid apps and more. Throughout this journey, OO concepts have been there every step of the way. That is why the topics of this chapter are so important. These concepts are just as relevant today as they were 25 years ago.

THE FUNDAMENTAL CONCEPTS

The primary point of this book is to get you thinking about how the concepts are used in designing object-oriented systems. Historically, object-oriented languages are defined by the following: encapsulation, inheritance, and polymorphism (what I call “classical” OO). Thus, if a language does not implement all of these, it is generally not considered completely object-oriented. Along with these three terms, I always include composition in the mix; thus, my list of object-oriented concepts looks like this:

- Encapsulation
- Inheritance
- Polymorphism
- Composition

We will discuss all these in detail as we proceed through the rest of the book.

One of the issues that I have struggled with right from the first edition of this book is how these concepts relate directly to current design practices, which are always changing. For example, there has always been debate about using inheritance in an OO design. Does inheritance actually break encapsulation? (This topic will be covered in later chapters.) Even now, many developers try to avoid inheritance as much as possible. So this raises the question: Should inheritance be used at all?

1. INTRODUÇÃO AOS CONCEITOS ORIENTADOS A OBJETOS

Embora muitos programadores não percebam, o desenvolvimento de software orientado a objetos existe desde o início da década de 1960. Foi somente entre meados e o final dos anos 1990 que o paradigma OO começou a ganhar grande impulso, apesar de linguagens populares como Smalltalk e C++ já serem amplamente utilizadas.

A ascensão das metodologias OO coincide com o surgimento da Internet como plataforma de negócios e entretenimento. Em resumo, objetos funcionam bem em redes. Quando ficou evidente que a Internet havia chegado para ficar, as tecnologias orientadas a objetos já estavam bem posicionadas para desenvolver as novas tecnologias baseadas na web.

É importante observar que o título deste capítulo é “Introdução aos Conceitos Orientados a Objetos”. A palavra decisiva é “conceitos”, não “tecnologias”. Tecnologias mudam muito rapidamente na indústria de software, enquanto conceitos evoluem. Uso o termo “evoluem” porque, embora permaneçam relativamente estáveis, eles também mudam. Essa é uma das vantagens de concentrar a atenção nos conceitos: mesmo consistentes, eles passam por novas interpretações, abrindo espaço para discussões muito interessantes.

Essa evolução pode ser acompanhada nas últimas décadas, desde os primeiros navegadores rudimentares dos anos 1990 até os aplicativos móveis, de telefone e web que predominam atualmente. Novas tecnologias continuam surgindo, como aplicações híbridas, mas os conceitos OO estiveram presentes durante todo esse percurso. Por isso, os assuntos deste capítulo continuam tão relevantes hoje quanto eram há 25 anos.

OS CONCEITOS FUNDAMENTAIS

O objetivo central deste livro é fazer você pensar em como os conceitos são utilizados no projeto de sistemas orientados a objetos. Historicamente, linguagens OO são definidas por encapsulamento, herança e polimorfismo - o que chamo de OO “clássica”. Se uma linguagem não implementa esses conceitos, normalmente não é considerada totalmente orientada a objetos. Além desses três, sempre incluo composição. Assim, nossa lista é:

- Encapsulamento
- Herança
- Polimorfismo
- Composição

Todos serão discutidos em detalhe ao longo do livro.

Desde a primeira edição, uma das questões que mais me desafiam é a relação entre esses conceitos e as práticas atuais de projeto, que estão sempre mudando. A herança, por exemplo, sempre foi objeto de debate em projetos OO. Ela realmente rompe o encapsulamento? Muitos desenvolvedores procuram evitá-la sempre que possível. Isso leva à pergunta: devemos usar herança?

My approach is, as always, to stick to concepts. Whether or not you use inheritance, you at least need to understand what inheritance is, thus enabling you to make an educated design choice. It is important not to forget that inheritance will almost certainly be encountered in code maintenance, so you need to learn it regardless.

As mentioned in the introduction, the intended audience is those who want a general introduction to fundamental OO concepts. With this statement in mind, in this chapter I present the fundamental object-oriented concepts with the hope that you will then gain a solid foundation for making important design decisions. The concepts covered here touch on most, if not all, of the topics covered in subsequent chapters, which explore these issues in much greater detail.

OBJECTS AND LEGACY SYSTEMS

As OO moved into the mainstream, one of the issues facing developers was the integration of new OO technologies with existing systems. Lines were being drawn between OO and structured (or procedural) programming, which was the dominant development paradigm at the time. I always found this odd because, in my mind, object-oriented and structured programming do not compete with each other. They are complementary because objects integrate well with structured code. Even now, I often hear this question: are you a structured programmer or an object-oriented programmer? Without hesitation, I would answer: both.

In the same vein, object-oriented code is not meant to replace structured code. Many non-OO *legacy systems* (that is, older systems that are already in place) are doing the job quite well, so why risk potential disaster by changing or replacing them? In most cases you should not change them, at least not for the sake of change. There is nothing inherently wrong with systems written in non-OO code. However, brand-new development definitely warrants the consideration of using OO technologies (in some cases, there is no choice but to do so).

Although there has been a steady and significant growth in OO development in the past 25 years, the global community’s dependence on networks such as the Internet and mobile infrastructures has helped catapult it even further into the mainstream. The explosion of transactions performed on browsers and mobile apps has opened up brand-new markets, where much of the software development is new and mostly unencumbered by legacy concerns. Even when there are legacy concerns, there is a trend to wrap the legacy systems in object wrappers.

Object Wrappers

Object wrappers are object-oriented code that includes other code inside. For example, you can take structured code (such as loops and conditions) and *wrap* it inside an object to make it look like an object. You can also use object wrappers to *wrap* functionality such as security features, nonportable hardware features, and so on. Wrapping structured code is covered in detail in [Chapter 6](#), “[Designing with Objects](#).”

One of the most interesting areas of software development is the integration of legacy code with mobile- and web-based systems. In many cases, a mobile web front end ultimately connects to data that resides on a mainframe. Developers who can combine the skills of mainframe and mobile web development are in demand.

You probably experience objects in your daily life without even realizing it. These experiences can take place in your car, when you’re talking on your cell phone, using your home entertainment system, playing computer games, and many other situations. The electronic highway has, in essence, become an object-based highway. As businesses gravitate toward the

Minha abordagem continua sendo concentrar-me nos conceitos. Independentemente de você usar ou não herança, precisa entender o que ela é para poder tomar uma decisão de projeto consciente. Além disso, é quase certo que você encontrará herança ao fazer manutenção de código, portanto precisa aprendê-la de qualquer forma.

Como mencionado na introdução, o público deste livro inclui pessoas que desejam uma introdução geral aos conceitos fundamentais de OO. Este capítulo apresenta esses fundamentos para fornecer uma base sólida para decisões importantes de projeto. Os conceitos aqui abordados aparecem em praticamente todos os capítulos seguintes, onde serão examinados com maior profundidade.

OBJETOS E SISTEMAS LEGADOS

Quando OO passou a fazer parte do desenvolvimento convencional, um dos problemas foi integrar novas tecnologias OO aos sistemas já existentes. Criava-se uma divisão entre programação orientada a objetos e programação estruturada ou procedural, que era o paradigma dominante. Sempre considerei essa oposição estranha: programação OO e programação estruturada não precisam competir; elas se complementam, porque objetos se integram bem a código estruturado. Mesmo hoje, quando alguém pergunta “você é programador estruturado ou orientado a objetos?”, minha resposta é: ambos.

Da mesma forma, código OO não foi criado para substituir todo código estruturado. Muitos sistemas legados não OO fazem seu trabalho muito bem. Por que correr o risco de alterá-los ou substituí-los apenas por mudança? Na maioria dos casos, não se deve fazer isso. Não há nada inerentemente errado em um sistema escrito sem OO. Em desenvolvimento novo, porém, o uso de tecnologias OO merece ser considerado - e em certos contextos praticamente não há escolha.

O crescimento constante de OO nas últimas décadas foi acelerado pela dependência mundial de redes, da Internet e de infraestruturas móveis. A explosão de transações feitas em navegadores e aplicativos móveis abriu mercados em que grande parte do software é novo e pouco limitada por sistemas legados. Mesmo quando há legado, é comum envolvê-lo em wrappers de objetos.

WRAPPERS DE OBJETOS

Wrappers de objetos são código orientado a objetos que envolve outro código. É possível, por exemplo, pegar código estruturado - laços, condições etc. - e encapsulá-lo dentro de um objeto, fazendo-o parecer parte de um modelo OO. Também se pode envolver funcionalidades como mecanismos de segurança ou recursos de hardware não portáteis. Esse assunto é tratado em detalhe no Capítulo 6, “Projetando com Objetos”.

Uma das áreas mais interessantes do desenvolvimento de software é a integração de código legado com sistemas móveis e web. Muitas vezes, uma interface web móvel se conecta, no fim, a dados residentes em um mainframe. Desenvolvedores capazes de combinar conhecimento de mainframe com desenvolvimento web/móvel são bastante procurados.

Você provavelmente interage com objetos todos os dias sem perceber: no carro, ao usar o celular, sistemas de entretenimento doméstico, jogos e muitas outras situações. A “rodovia eletrônica” tornou-se, em essência, uma rodovia baseada em objetos. À medida que empresas migram para a web móvel, também se aproximam de objetos, pois as tecnologias usadas no comércio eletrônico são em grande parte orientadas a objetos.

mobile web, they are gravitating toward objects because the technologies used for electronic commerce are mostly OO in nature.

Mobile Web

No doubt, the emergence of the Internet provided a major impetus for the shift to object-oriented technologies. This is because objects are well suited for use on networks. Although the Internet was at the forefront of this paradigm shift, mobile networks have now joined the mix in a major way. In this book, the term *mobile web* will be used in the context of concepts that pertain to both mobile app development and web development. The term *hybrid* app is sometimes used to refer to applications that render in browsers on both web and mobile devices.

PROCEDURAL VERSUS OO PROGRAMMING

Before we delve deeper into the advantages of OO development, let’s consider a more fundamental question: What exactly is an object? This is both a complex and a simple question. It is complex because learning any method of software development is not trivial. It is simple because people already think in terms of objects.

TIP

In watching a YouTube video lecture presented by OO guru Robert Martin, his view is that the statement that “people think in terms of objects” was coined by marketing people. Just some food for thought.

For example, when you look at a person, you see the person as an object. And an object is defined by two components: attributes and behaviors. A person has attributes, such as eye color, age, height, and so on. A person also has behaviors, such as walking, talking, breathing, and so on. In its basic definition, an *object* is an entity that contains *both* data and behavior. The word *both* is the key difference between OO programming and other programming methodologies. In procedural programming, for example, code is placed into totally distinct functions or procedures. Ideally, as shown in [Figure 1.1](#), these procedures then become “black boxes,” where inputs go in and outputs come out. Data is placed into separate structures and is manipulated by these functions or procedures.

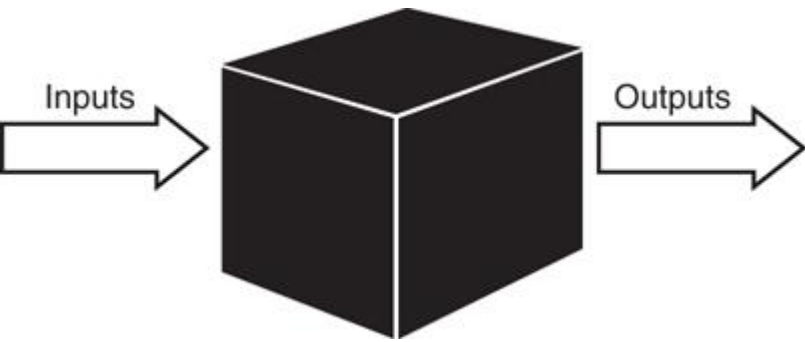


Figure 1.1 Black boxes.

Difference Between OO and Procedural

In OO design, the attributes and behaviors are contained within a single object, whereas in procedural, or structured, design the attributes and behaviors are normally separated.

As OO design grew in popularity, one of the realities that initially slowed its acceptance was that there were a lot of non-OO systems in place that worked perfectly fine. Thus, it did not make any business sense to change the systems for the sake of change. Anyone who is familiar with any computer system knows that any change can spell disaster—even if the change is perceived to be slight.

WEB MÓVEL

Sem dúvida, o surgimento da Internet deu grande impulso à adoção de tecnologias orientadas a objetos, pois objetos são adequados ao uso em redes. A Internet esteve na linha de frente dessa mudança de paradigma e, hoje, as redes móveis também têm enorme importância. Neste livro, o termo “web móvel” será usado para conceitos que se aplicam tanto ao desenvolvimento de aplicativos móveis quanto ao desenvolvimento web. O termo “aplicativo híbrido” é usado às vezes para aplicações executadas em navegadores tanto na web quanto em dispositivos móveis.

PROGRAMAÇÃO PROCEDURAL VERSUS PROGRAMAÇÃO OO

Antes de aprofundarmos as vantagens do desenvolvimento OO, considere uma pergunta mais fundamental: o que exatamente é um objeto? A pergunta é ao mesmo tempo complexa e simples. É complexa porque aprender qualquer método de desenvolvimento de software exige esforço. É simples porque as pessoas já costumam pensar em termos de objetos.

DICA

Robert Martin, conhecido especialista em OO, já comentou em uma palestra que a afirmação de que “as pessoas pensam em termos de objetos” teria sido criada pelo marketing. É um ponto interessante para reflexão.

Quando olhamos para uma pessoa, por exemplo, podemos vê-la como um objeto. Um objeto é definido por dois componentes: atributos e comportamentos. Uma pessoa possui atributos, como cor dos olhos, idade e altura, e comportamentos, como andar, falar e respirar. Em sua definição básica, um objeto é uma entidade que contém tanto dados quanto comportamento. A palavra “tanto” é a diferença essencial entre programação OO e outras metodologias.

Na programação procedural, o código é organizado em funções ou procedimentos separados. Idealmente, como na Figura 1.1, esses procedimentos funcionam como “caixas-pretas”: entradas chegam, saídas são produzidas. Os dados ficam em estruturas separadas e são manipulados por essas funções ou procedimentos.

Figura 1.1 - Caixas-pretas.

DIFERENÇA ENTRE OO E PROCEDURAL

No projeto OO, atributos e comportamentos ficam contidos em um único objeto. No projeto procedural ou estruturado, atributos e comportamentos normalmente ficam separados.

Quando o projeto OO ganhou popularidade, sua adoção foi inicialmente desacelerada pela existência de muitos sistemas não OO que funcionavam perfeitamente. Não havia sentido comercial em substituir sistemas apenas por substituí-los. Qualquer pessoa familiarizada com sistemas computacionais sabe que até uma pequena alteração pode provocar problemas sérios.

This situation came into play with the lack of acceptance of OO databases. At one point in the emergence of OO development, it seemed somewhat likely that OO databases would replace relational databases. However, this never happened. Businesses have a lot of money invested in relational databases, and one overriding factor discouraged conversion: they worked. When all the costs and risks of converting systems from relational to OO databases became apparent, there was no compelling reason to switch.

In fact, the business forces have now found a happy middle ground. Much of the software development practices today have flavors of several development methodologies, such as OO and structured.

As illustrated in [Figure 1.2](#), in structured programming the data is often separated from the procedures, and often the data is global, so it is easy to modify data that is outside the scope of your code. This means that access to data is uncontrolled and unpredictable (that is, multiple functions may have access to the global data). Second, because you have no control over who has access to the data, testing and debugging are much more difficult. Objects address these problems by combining data and behavior into a nice, complete package.

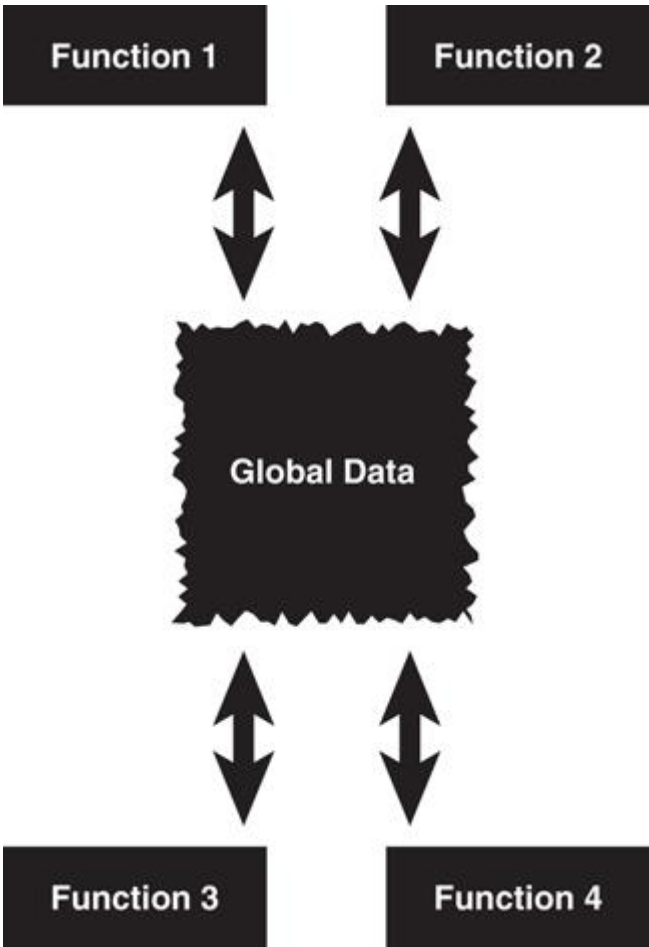


Figure 1.2 Using global data.

Proper Design

We can state that when properly designed, there is no such thing as global data in an OO model. This fact provides a high amount of data integrity in OO systems.

Rather than replacing other software development paradigms, objects are an evolutionary response. Structured programs have complex data structures, such as arrays, and so on. C++ has structures, which have many of the characteristics of objects (classes).

Essa situação apareceu claramente na pouca adoção de bancos de dados orientados a objetos. Em determinado momento, parecia possível que bancos OO substituíssem bancos relacionais, mas isso não ocorreu. As empresas já tinham muito dinheiro investido em bancos relacionais e havia um fator decisivo contra a conversão: eles funcionavam. Quando ficaram claros os custos e riscos da migração de sistemas relacionais para bancos OO, não havia motivo convincente para mudar.

Na prática, o mercado encontrou um meio-termo. Muitas práticas atuais de desenvolvimento combinam elementos de diferentes metodologias, como orientação a objetos e programação estruturada.

Como mostra a Figura 1.2, em programação estruturada os dados muitas vezes ficam separados dos procedimentos e podem ser globais. Assim, é fácil modificar dados fora do escopo de um trecho de código. O acesso se torna pouco controlado e imprevisível, pois várias funções podem acessar os mesmos dados globais. Além disso, como não há controle claro sobre quem pode alterá-los, testes e depuração se tornam mais difíceis. Objetos ajudam a resolver esses problemas combinando dados e comportamento em um pacote coeso.

Figura 1.2 - Uso de dados globais.

PROJETO ADEQUADO

Em um modelo OO bem projetado, podemos dizer que não existem dados globais no sentido tradicional. Isso proporciona alto nível de integridade dos dados.

Em vez de substituir outros paradigmas de desenvolvimento, os objetos são uma resposta evolutiva. Programas estruturados já usam estruturas de dados complexas, como arrays; C++ possui estruturas que compartilham várias características com objetos e classes.

However, objects are much more than data structures and primitive data types, such as integers and strings. Although objects do contain entities such as integers and strings, which are used to represent attributes, they also contain methods, which represent behaviors. In an object, methods are used to perform operations on the data as well as other actions. Perhaps more important, you can control access to members of an object (both attributes and methods). This means that some members, both attributes and methods, can be hidden from other objects. For instance, an object called `Math` might contain two integers, called `myInt1` and `myInt2`. Most likely, the `Math` object also contains the necessary methods to set and retrieve the values of `myInt1` and `myInt2`. It might also contain a method called `sum()` to add the two integers together.

Data Hiding

In OO terminology, data are referred to as *attributes*, and behaviors are referred to as methods. Restricting access to certain attributes and/or methods is called data *hiding*.

By combining the attributes and methods in the same entity, which in OO parlance is called *encapsulation*, we can control access to the data in the `Math` object. By defining these integers as off-limits, another logically unconnected function cannot manipulate the integers `myInt1` and `myInt2`—only the `Math` object can do that.

Sound Class Design Guidelines

Keep in mind that it is possible to create poorly designed OO classes that do not restrict access to class attributes. The bottom line is that you can design bad code just as efficiently with OO design as with any other programming methodology. Simply take care to adhere to sound class design guidelines (see [Chapter 5](#), “[Class Design Guidelines](#)”).

What happens when another object—for example, `myObject`—wants to gain access to the sum of `myInt1` and `myInt2`? It asks the `Math` object: `myObject` sends a message to the `Math` object. **Figure 1.3** shows how the two objects communicate with each other via their methods. The message is really a call to the `Math` object’s `sum` method. The `sum` method then returns the value to `myObject`. The beauty of this is that `myObject` does not need to know how the sum is calculated (although I’m sure it can guess). With this design methodology in place, you can change how the `Math` object calculates the sum without making a change to `myObject` (as long as the means to retrieve the sum do not change). All you want is the sum—you *don’t care* how it is calculated.

Entretanto, objetos são muito mais do que estruturas de dados e tipos primitivos, como inteiros e strings. Embora objetos contenham valores usados para representar atributos, eles também contêm métodos, que representam comportamentos. Em um objeto, os métodos realizam operações sobre os dados e executam outras ações. Mais importante ainda, é possível controlar o acesso aos membros de um objeto - tanto atributos quanto métodos. Assim, alguns membros podem ser ocultados de outros objetos.

Por exemplo, um objeto chamado `Math` pode conter dois inteiros, `myInt1` e `myInt2`. Provavelmente ele também possui métodos para definir e recuperar esses valores e talvez um método `sum()` para somá-los.

OCULTAÇÃO DE DADOS

Na terminologia OO, dados são chamados de atributos e comportamentos são chamados de métodos. Restringir o acesso a determinados atributos e/ou métodos é chamado de ocultação de dados.

Ao combinar atributos e métodos na mesma entidade - o que em OO chamamos de encapsulamento - podemos controlar o acesso aos dados do objeto `Math`. Se os inteiros forem definidos como inacessíveis externamente, uma função sem relação lógica com `Math` não poderá manipulá-los diretamente; apenas o próprio objeto `Math` poderá fazê-lo.

DIRETRIZES PARA UM BOM PROJETO DE CLASSES

É perfeitamente possível criar classes OO mal projetadas que não restringem o acesso aos atributos. Em outras palavras, OO não impede código ruim. É necessário seguir boas diretrizes de projeto de classes, discutidas no Capítulo 5.

O que acontece se outro objeto, como `myObject`, quiser obter a soma de `myInt1` e `myInt2`? Ele solicita essa informação ao objeto `Math`, enviando-lhe uma mensagem. A Figura 1.3 mostra a comunicação entre os dois objetos por meio de seus métodos. A mensagem corresponde, na prática, a uma chamada ao método `sum()` de `Math`, que devolve o valor a `myObject`.

A vantagem é que `myObject` não precisa saber como a soma é calculada. Podemos alterar a implementação do cálculo em `Math` sem mudar `myObject`, desde que a forma de solicitar o resultado continue a mesma. O que interessa ao cliente é o resultado, não o mecanismo interno usado para obtê-lo.

Figura 1.3 - Comunicação entre objetos.

Um exemplo simples com uma calculadora ajuda a ilustrar o conceito. Para calcular uma soma, você usa apenas a interface da calculadora - teclado e visor. A calculadora possui um comportamento de soma que é acionado pela sequência adequada de teclas. Você recebe o resultado, mas não precisa saber como ele foi obtido eletrônica ou algoritmicamente.

Calcular a soma não é responsabilidade de myObject; é responsabilidade do objeto Math. Enquanto myObject tiver acesso à interface de Math, poderá enviar as mensagens apropriadas e obter o resultado. De modo geral, objetos não devem manipular diretamente os dados internos de outros objetos. Assim, myObject não deveria alterar myInt1 e myInt2 por conta própria. Como veremos depois, também costuma ser melhor construir objetos pequenos, com responsabilidades específicas, em vez de objetos grandes que fazem muitas coisas.

DA PROGRAMAÇÃO PROCEDURAL AO DESENVOLVIMENTO ORIENTADO A OBJETOS

Agora que temos uma visão geral das diferenças entre tecnologias procedurais e orientadas a objetos, podemos aprofundar essas duas abordagens.

PROGRAMAÇÃO PROCEDURAL

A programação procedural normalmente separa os dados de um sistema das operações que os manipulam. Por exemplo, ao enviar informações através de uma rede, somente os dados relevantes podem ser transmitidos, como na Figura 1.4, esperando-se que o programa na outra extremidade saiba o que fazer com eles. Em outras palavras, cliente e servidor precisam compartilhar algum acordo ou protocolo de comunicação.

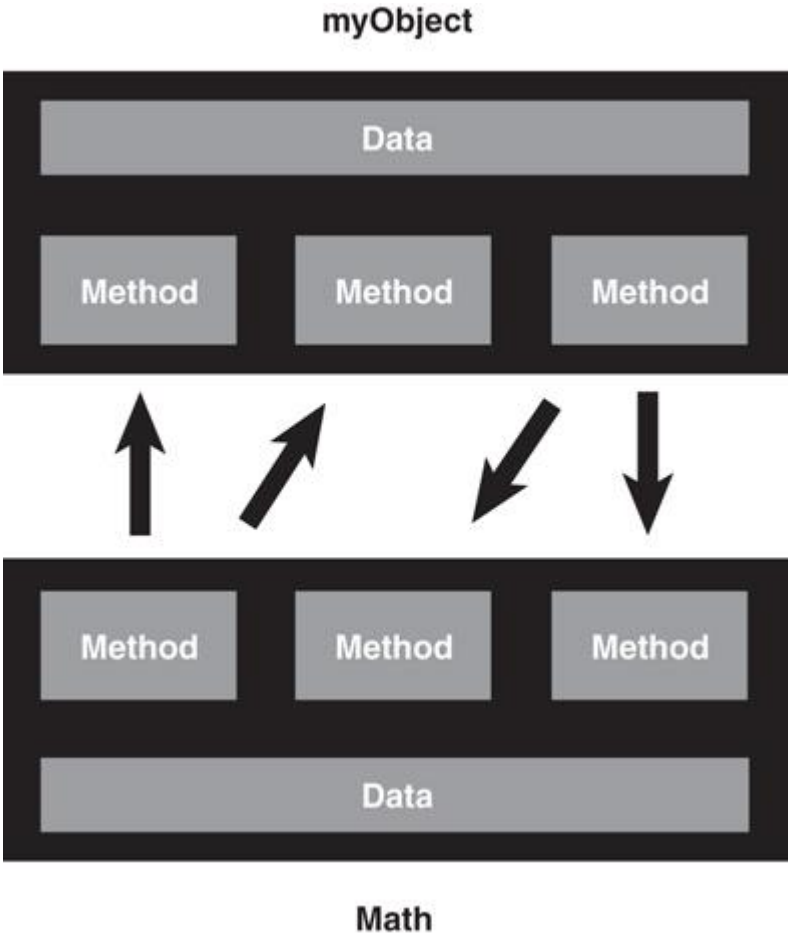


Figure 1.3 Object-to-object communication.

Using a simple calculator example illustrates this concept. When determining a sum with a calculator, all you use is the calculator’s interface—the keypad and LED display. The calculator has a sum method that is invoked when you press the correct key sequence. You may get the correct answer back; however, you have no idea how the result was obtained—either electronically or algorithmically.

Calculating the sum is not the responsibility of myObject—it’s the Math object’s responsibility. As long as myObject has access to the Math object, it can send the appropriate messages and obtain the requested result. In general, objects should not manipulate the internal data of other objects (that is, myObject should not directly change the value of myInt1 and myInt2). And, for reasons we will explore later, it is normally better to build small objects with specific tasks rather than build large objects that perform many.

MOVING FROM PROCEDURAL TO OBJECT-ORIENTED DEVELOPMENT

Now that we have a general understanding about some of the differences between procedural and object-oriented technologies, let’s delve a bit deeper into both.

Procedural Programming

Procedural programming normally separates the data of a system from the operations that manipulate the data. For example, if you want to send information across a network, only the relevant data is sent (see [Figure 1.4](#)), with the expectation that the program at the other end of the network pipe knows what to do with it. In other words, some sort of handshaking agreement

must be in place between the client and the server to transmit the data. In this model, it is quite possible that no code is actually sent over the wire.

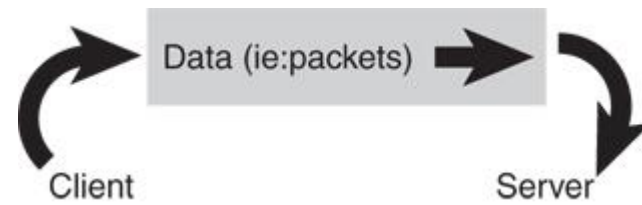


Figure 1.4 Data transmitted over a wire.

OO Programming

The fundamental advantage of OO programming is that the data and the operations that manipulate the data (the code) are both encapsulated in the object. For example, when an object is transported across a network, the entire object, including the data and behavior, goes with it.

A Single Entity

Although thinking in terms of a single entity is great in theory, in many cases, the behaviors themselves may not be sent because both sides have copies of the code. However, it is important to think in terms of the entire object being sent across the network as a single entity.

In [Figure 1.5](#), the `Employee` object is sent over the network.

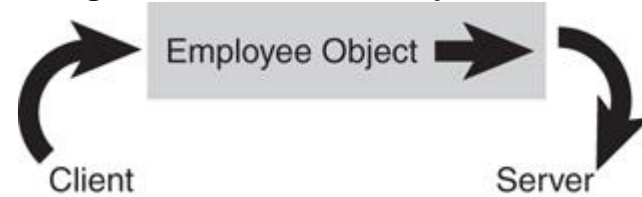


Figure 1.5 Objects transmitted over a wire.

Proper Design

A good example of this concept is an object that is loaded by a browser. Often, the browser has no idea of what the object will do ahead of time because the code is not there previously. When the object is loaded, the browser executes the code within the object and uses the data contained within the object.

WHAT EXACTLY IS AN OBJECT?

Objects are the building blocks of an OO program. A program that uses OO technology is basically a collection of objects. To illustrate, let's consider that a corporate system contains objects that represent employees of that company. Each of these objects is made up of the data and behavior described in the following sections.

Object Data

The data stored within an object represents the state of the object. In OO programming terminology, this data is called *attributes*. In our example, as shown in [Figure 1.6](#), employee attributes could be Social Security numbers, date of birth, gender, phone number, and so on. The attributes contain the information that differentiates between the various objects, in this case the employees. Attributes are covered in more detail later in this chapter in the discussion on classes.

Nesse modelo, é perfeitamente possível que nenhum código seja efetivamente enviado pela rede.

Figura 1.4 - Dados transmitidos por uma conexão.

PROGRAMAÇÃO OO

A vantagem fundamental da programação OO é que os dados e as operações que manipulam esses dados - isto é, o código - são considerados conjuntamente dentro do objeto. Conceitualmente, quando um objeto é transportado por uma rede, pensamos na entidade inteira, incluindo seus dados e seu comportamento.

UMA ÚNICA ENTIDADE

Embora pensar no objeto como uma única entidade seja importante, na prática o comportamento pode não ser transmitido, porque os dois lados já podem possuir cópias do código. Ainda assim, do ponto de vista conceitual, deve-se pensar no objeto como uma entidade única.

Na Figura 1.5, um objeto `Employee` é representado sendo enviado pela rede.

Figura 1.5 - Objetos transmitidos por uma conexão.

PROJETO ADEQUADO

Um exemplo é um objeto carregado por um navegador. O navegador pode não saber previamente o que esse objeto fará. Quando o objeto é carregado, o navegador executa seu código e utiliza os dados associados.

O QUE EXATAMENTE É UM OBJETO?

Objetos são os blocos fundamentais de um programa OO. Um programa que usa tecnologia orientada a objetos é, essencialmente, uma coleção de objetos. Imagine um sistema corporativo que contém objetos representando os funcionários da empresa. Cada objeto é composto por dados e comportamentos.

DADOS DO OBJETO

Os dados armazenados em um objeto representam seu estado. Na terminologia de POO, esses dados são chamados de atributos. No exemplo, como mostrado na Figura 1.6, os atributos de um funcionário poderiam incluir número de identificação, data de nascimento, gênero, telefone etc. Esses atributos contêm informações que distinguem um objeto de outro. O assunto será retomado adiante, na discussão sobre classes.

Figura 1.6 - Atributos de Employee.

COMPORTAMENTOS DO OBJETO

O comportamento de um objeto representa o que ele é capaz de fazer. Em linguagens procedurais, comportamentos são definidos por procedimentos, funções e sub-rotinas. Em POO, esses comportamentos ficam em métodos, e um método é acionado por uma mensagem.

No exemplo de Employee, um comportamento necessário é definir e retornar os valores dos atributos. Assim, um atributo poderia ter métodos correspondentes como `setGender()` e `getGender()`. Quando outro objeto precisa dessa informação, envia uma mensagem ao objeto Employee solicitando o valor.

O uso de getters e setters, como vários aspectos da tecnologia OO, evoluiu desde a primeira edição deste livro, principalmente no tratamento de dados. Uma das vantagens mais poderosas dos objetos é justamente que os dados fazem parte do pacote: não ficam completamente separados do código.

O surgimento de XML chamou atenção para a apresentação portátil de dados e também favoreceu maneiras alternativas de o código acessá-los. Em tecnologias .NET, getters e setters podem ser tratados como propriedades dos próprios dados.

Por exemplo, um atributo Java chamado Name pode ser declarado assim:

```
public String Name;
```

Seus métodos de leitura e escrita poderiam ser:

```
public void setName(String n) { name = n; }
public String getName() { return name; }
```

Em C# .NET, uma propriedade equivalente poderia ser declarada com `get` e `set`. O código original é mantido na página à esquerda.



Figure 1.6 Employee

attributes.

Object Behaviors

The *behavior* of an object represents what the object can do. In procedural languages the behavior is defined by procedures, functions, and subroutines. In OO programming terminology, these behaviors are contained in *methods*, and you invoke a method by sending a message to it. In our employee example, consider that one of the behaviors required of an employee object is to set and return the values of the various attributes. Thus, each attribute would have corresponding methods, such as `setGender()` and `getGender()`. In this case, when another object needs this information, it can send a message to an employee object and ask it what its gender is.

Not surprisingly, the application of getters and setters, as with much of object-oriented technology, has evolved since the first edition of this book was published. This is especially true when it comes to data. Remember that one of the most interesting, not to mention powerful, advantages of using objects is that the data is part of the package—it is not separated from the code.

The emergence of XML has not only focused attention on presenting data in a portable manner; it also has facilitated alternative ways for the code to access the data. In .NET techniques, the getters and setters are considered properties of the data itself.

For example, consider an attribute called `Name`, using Java, that looks like the following:

```
public String Name;
```

The corresponding getter and setter would look like this:

```
public void setName (String n) {name = n;};
public String getName() {return name;};
```

Now, when creating an XML attribute called `Name`, the definition in C# .NET may look something like this, although you can certainly use the same approach as the Java example:

```
private string strName;
```

```
public String Name
{
    get { return this.strName; }
    set {
        if (value == null) return;
```

```
    this.strName = value;
}
```

In this technique, the getters and setters are actually *properties* of the attributes—in this case, `Name`.

Regardless of the approach, the purpose is the same—controlled access to the attribute. For this chapter, I want to first concentrate on the conceptual nature of accessor methods; we will get more into properties in later chapters.

Getters and Setters

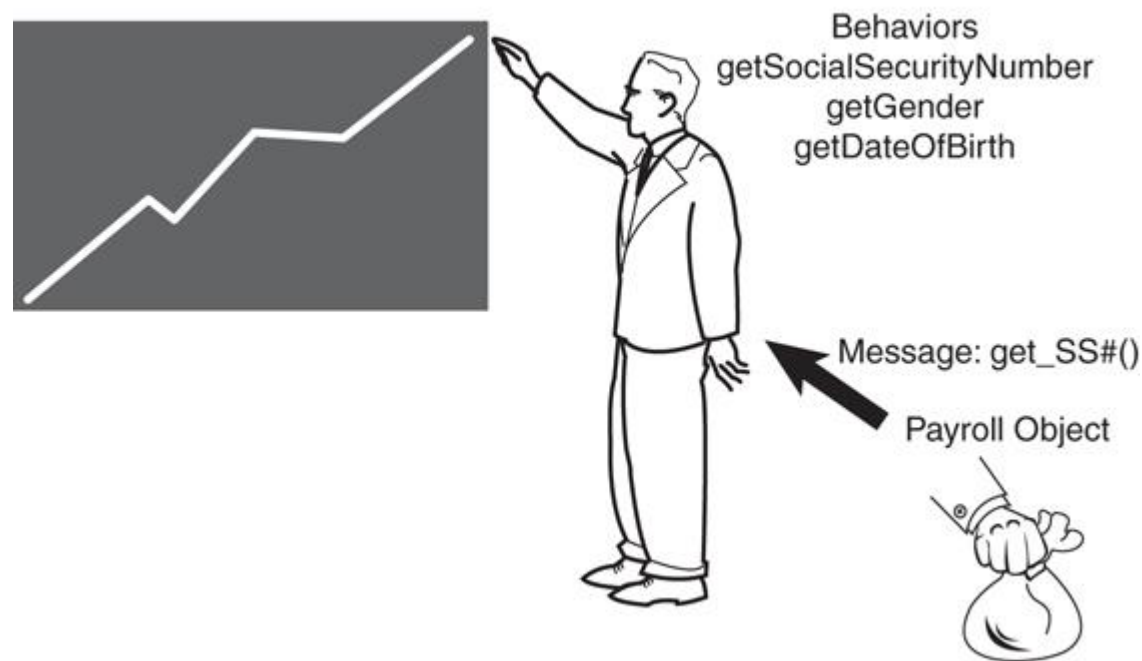
The concept of getters and setters supports the concept of data hiding. Because other objects should not directly manipulate data within another object, the getters and setters provide controlled access to an object's data. Getters and setters are sometimes called accessor methods and mutator methods, respectively.

Note that we are showing only the interface of the methods, and not the implementation. The following information is all the user needs to know to effectively use the methods:

- The name of the method
- The parameters passed to the method
- The return type of the method

To illustrate behaviors, consider [Figure 1.7](#).

Employee Object



Figure

1.7 Employee behaviors.

In [Figure 1.7](#), the `Payroll` object contains a method called `CalculatePay()` that calculates the pay for a specific employee. Among other information, the `Payroll` object must obtain the Social Security number of this employee. To get this information, the payroll object must send a message to the `Employee` object (in this case, the `getSocialSecurityNumber()` method). Basically, this means that the `Payroll` object calls the `getSocialSecurityNumber()` method of

Nessa técnica, getters e setters tornam-se propriedades associadas ao atributo `Name`.

Independentemente da abordagem usada, o objetivo é o mesmo: oferecer acesso controlado ao atributo. Neste capítulo, o foco inicial está na natureza conceitual dos métodos de acesso; propriedades serão discutidas posteriormente.

GETTERS E SETTERS

O conceito de getters e setters dá suporte à ocultação de dados. Como um objeto externo não deveria manipular diretamente os dados internos de outro objeto, esses métodos oferecem uma forma controlada de acesso. Getters também são chamados de métodos de acesso (accessor methods) e setters de métodos modificadores (mutator methods).

Observe que mostramos apenas a interface dos métodos, e não sua implementação. Para usar um método corretamente, o cliente precisa conhecer:

- o nome do método;
- os parâmetros passados ao método;
- o tipo de retorno do método.

Para ilustrar comportamentos, considere a [Figura 1.7](#).

Figura 1.7 - Comportamentos de Employee.

Na figura, o objeto `Payroll` contém um método chamado `CalculatePay()`, que calcula o pagamento de determinado funcionário. Entre outras informações, `Payroll` precisa obter o número de identificação desse funcionário. Para isso, envia uma mensagem ao objeto `Employee` - neste caso, chama `getSocialSecurityNumber()`. Em termos práticos, `Payroll` invoca esse método de `Employee` e aguarda a informação solicitada.

the `Employee` object. The employee object recognizes the message and returns the requested information. To illustrate further, [Figure 1.8](#) is a class diagram representing the `Employee/Payroll` system we have been talking about.

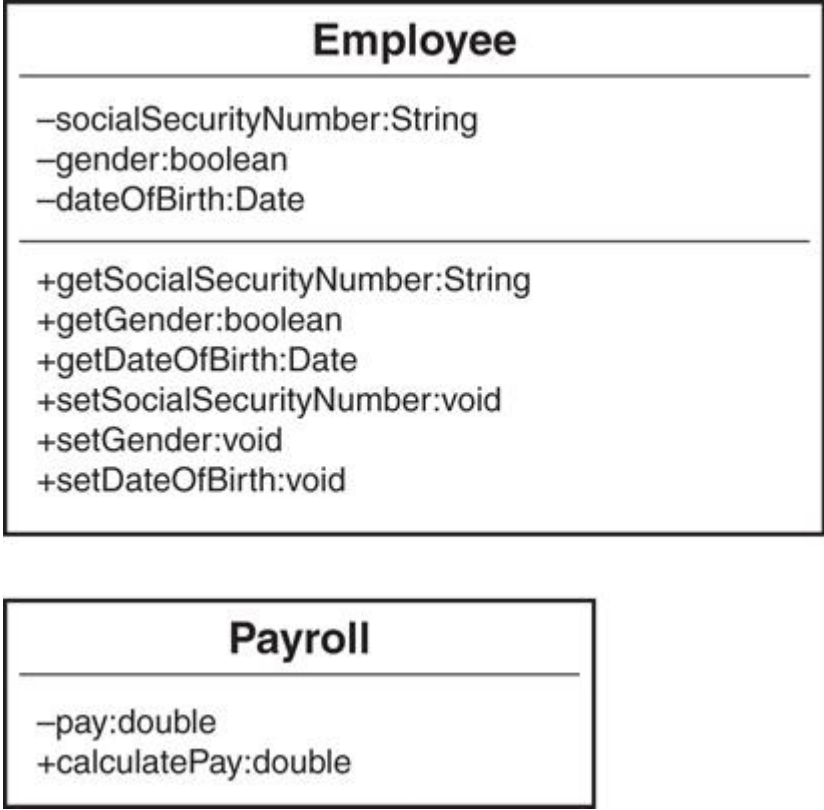


Figure 1.8 Employee and payroll class

diagrams.

UML Class Diagrams

Because this is the first class diagram we have seen, it is very basic and lacks some of the constructs (such as constructors) that a proper class should contain. Fear not—we will discuss class diagrams and constructors in more detail in [Chapter 3](#), “[More Object-Oriented Concepts](#).”

Each class diagram is defined by three separate sections: the name itself, the data (attributes), and the behaviors (methods). In [Figure 1.8](#), the `Employee` class diagram’s attribute section contains `SocialSecurityNumber`, `Gender`, and `DateOfBirth`, whereas the method section contains the methods that operate on these attributes. You can use UML modeling tools to create and maintain class diagrams that correspond to real code.

Modeling Tools

Visual modeling tools provide a mechanism to create and manipulate class diagrams using the Unified Modeling Language (UML). Class diagrams are used and discussed throughout this book. They are used as a tool to help visualize classes and their relationships to other classes. The use of UML in this book is limited to class diagrams.

We will get into the relationships between classes and objects later in this chapter, but for now you can think of a class as a template from which objects are made. When an object is created, we say that the objects are instantiated. Thus, if we create three employees, we are actually creating three totally distinct instances of an `Employee` class. Each object contains its own copy of the attributes and methods. For example, consider [Figure 1.9](#). An employee object called `John` (John is its identity) has its own copy of all the attributes and methods defined in

O objeto `Employee` reconhece a mensagem e retorna a informação solicitada.

A [Figura 1.8](#) apresenta um diagrama de classes representando o sistema `Employee/Payroll` utilizado no exemplo.

Figura 1.8 - Diagramas das classes `Employee` e `Payroll`.

DIAGRAMAS DE CLASSES UML

Como este é o primeiro diagrama de classes apresentado, ele é bastante básico e ainda não mostra todos os elementos que uma classe adequada deve conter, como construtores. Diagramas de classes e construtores serão detalhados no [Capítulo 3](#).

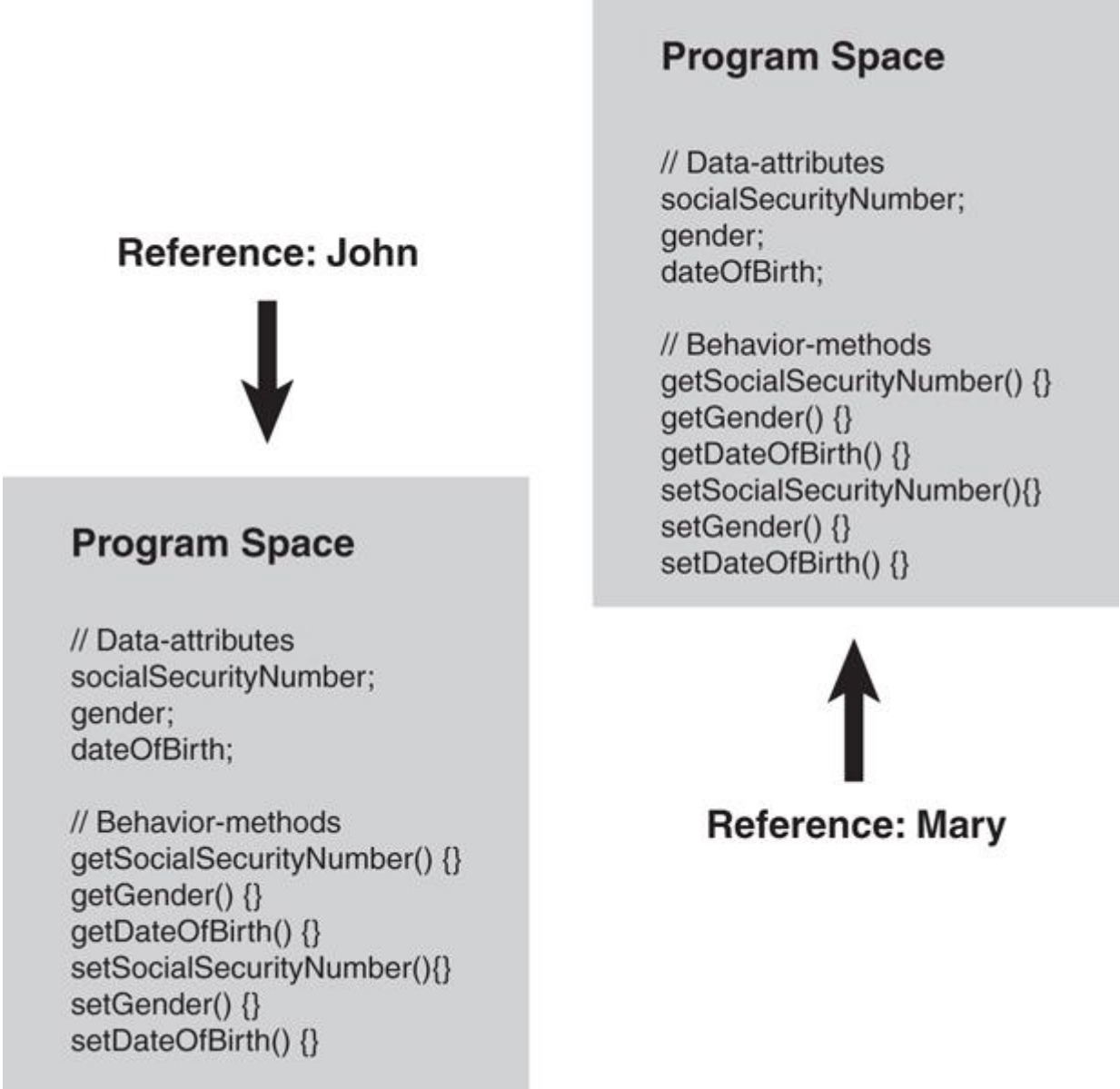
Cada diagrama de classe é dividido em três seções: nome, dados (atributos) e comportamentos (métodos). Na [Figura 1.8](#), a seção de atributos de `Employee` contém `SocialSecurityNumber`, `Gender` e `DateOfBirth`; a seção de métodos contém operações relacionadas a esses atributos. Ferramentas de modelagem UML podem ser usadas para criar e manter diagramas que correspondam ao código real.

FERRAMENTAS DE MODELAGEM

Ferramentas de modelagem visual fornecem mecanismos para criar e manipular diagramas de classes com a Unified Modeling Language (UML). Esses diagramas serão usados ao longo do livro para visualizar classes e seus relacionamentos. O uso de UML nesta obra fica limitado a diagramas de classes.

Mais adiante trataremos das relações entre classes e objetos. Por enquanto, considere uma classe como um molde a partir do qual objetos são produzidos. Quando um objeto é criado, dizemos que ele foi instanciado. Se criarmos três funcionários, estaremos criando três instâncias distintas da classe `Employee`. Cada objeto possui seus próprios valores de atributos. Conceitualmente, também podemos imaginar que cada um possui os comportamentos definidos pela classe. Um objeto chamado `John`, por exemplo, tem sua própria identidade e seus próprios dados conforme a definição de `Employee`.

the `Employee` class. An employee object called `Mary` has its own copy of attributes and methods. They both have a separate copy of the `DateOfBirth` attribute and the `getDateOfBirth` method.



1.9 Program spaces.

An Implementation Issue

Be aware that there is not necessarily a physical copy of each method for each object. Rather, each object points to the same implementation. However, this is an issue left up to the compiler/operating platform. From a conceptual level, you can think of objects as being wholly independent and having their own attributes and methods.

WHAT EXACTLY IS A CLASS?

In short, a class is a blueprint for an object. When you instantiate an object, you use a class as the basis for how the object is built. In fact, trying to explain classes and objects is really a chicken-and-egg dilemma. It is difficult to describe a class without using the term *object* and vice versa. For example, a specific individual bike is an object. However, someone had to have created the blueprints (that is, the class) to build the bike. In OO software, unlike the

Um objeto `Employee` chamado `Mary` também possui seus próprios atributos. John e Mary têm, por exemplo, valores independentes para `DateOfBirth`, embora utilizem o mesmo comportamento definido por `getDateOfBirth()`.

Figura 1.9 - Espaços de programa.

UMA QUESTÃO DE IMPLEMENTAÇÃO

Não existe necessariamente uma cópia física de cada método para cada objeto. Em geral, os objetos apontam para uma mesma implementação do método. Esse detalhe fica a cargo do compilador e da plataforma. No nível conceitual, porém, é útil pensar em cada objeto como uma entidade independente, com seus próprios atributos e comportamentos.

O QUE EXATAMENTE É UMA CLASSE?

Em resumo, uma classe é o projeto ou molde de um objeto. Quando um objeto é instanciado, a classe serve de base para sua construção. Explicar classes e objetos cria um problema parecido com o dilema do ovo e da galinha: é difícil definir classe sem falar em objeto e vice-versa.

Uma bicicleta específica, por exemplo, é um objeto; mas alguém precisou criar antes o projeto usado para construir bicicletas desse tipo - isto é, a classe. Em software OO, ao contrário do dilema do ovo e da galinha, sabemos o que vem primeiro: a classe.

Figure

chicken-and-egg dilemma, we do know what comes first—the class. An object cannot be instantiated without a class. Thus, many of the concepts in this section are similar to those presented earlier in the chapter, especially when we talk about attributes and methods.

Although this book focuses on the concepts of OO software and not on a specific implementation, it is often helpful to use code examples to explain some concepts, so Java code fragments are used throughout the book to help explain some concepts when appropriate. However, for certain key examples, the code is provided in several languages as downloads.

The following sections describe some of the fundamental concepts of classes and how they interact.

Creating Objects

Classes can be thought of as the templates, or cookie cutters, for objects as seen in [Figure 1.10](#). A class is used to create an object.

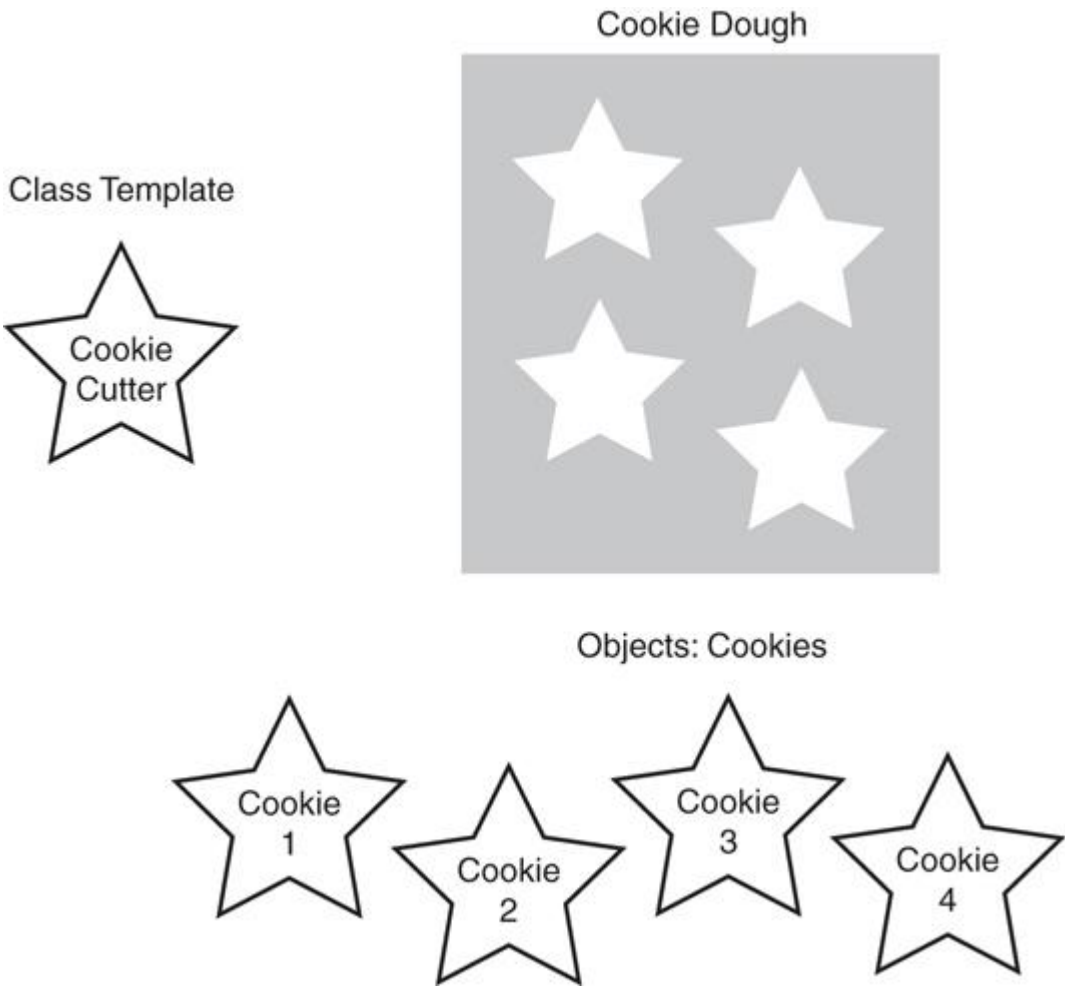


Figure 1.10 Class

template.

A class can be thought of as a sort of higher-level data type. For example, just as you create an integer or a float:

```
int x;
float y;
```

you can also create an object by using a predefined class:

Um objeto não pode ser instanciado sem uma classe. Por isso, vários conceitos desta seção são semelhantes aos apresentados anteriormente, especialmente atributos e métodos.

Embora o livro se concentre em conceitos de software OO e não em uma implementação específica, exemplos de código ajudam a explicar certos pontos. Fragmentos Java são usados ao longo do texto quando apropriado e, para alguns exemplos importantes, versões em outras linguagens são disponibilizadas como download.

As próximas seções apresentam alguns conceitos fundamentais das classes e de sua interação.

CRIANDO OBJETOS

Classes podem ser entendidas como moldes ou cortadores de biscoitos para objetos, como mostra a Figura 1.10. A classe é usada para criar o objeto.

Figura 1.10 - Molde de uma classe.

Uma classe também pode ser vista como um tipo de dado de nível mais alto. Assim como declaramos variáveis primitivas:

```
int x;
float y;
```

também podemos declarar um objeto com base em uma classe previamente definida. O código correspondente permanece preservado no original à esquerda.

myClassmyObject;

In this example, the names themselves make it obvious that `myClass` is the class and `myObject` is the object.

Remember that each object has its own attributes (data) and behaviors (functions or routines). A class defines the attributes and behaviors that all objects created with this class will possess. Classes are pieces of code. Objects instantiated from classes can be distributed individually or as part of a library. Because objects are created from classes, it follows that classes must define the basic building blocks of objects (attributes, behavior, and messages). In short, you must design a class before you can create an object.

For example, here is a definition of a `Person` class:

```
public class Person{

    //Attributes
    private String name;
    private String address;

    //Methods
    public String getName(){
        return name;
    }
    public void set        Name(String n){
        name = n;
    }

    public String getAddress(){
        return address;
    }
    public void setAddress(String adr){
        address = adr;
    }

}
```

Attributes

As you already saw, the data of a class is represented by attributes. Each class must define the attributes that will store the state of each object instantiated from that class. In the `Person` class example in the previous section, the `Person` class defines attributes for `name` and `address`.

Access Designations

When a data type or method is defined as `public`, other objects can directly access it. When a data type or method is defined as `private`, only that specific object can access it. Another access modifier, `protected`, allows access by related objects, which you'll learn about in [Chapter 3](#).

Methods

As you learned earlier in the chapter, methods implement the required behavior of a class. Every object instantiated from this class includes methods as defined by the class. Methods may implement behaviors that are called from other objects (messages) or provide the fundamental, internal behavior of the class. Internal behaviors are private methods that are not accessible by other objects. In the `Person` class, the behaviors are `getName()`, `setName()`, `getAddress()`, and `setAddress()`. These methods allow other objects to inspect and change the values of the object's attributes. This is a common technique in OO systems. In all cases, access to attributes within an object should be controlled by the object itself—no other object should directly change an attribute of another.

No exemplo, os próprios nomes tornam evidente que `myClass` representa a classe e `myObject` o objeto.

Lembre-se de que cada objeto possui seus próprios atributos (dados) e comportamentos (funções ou rotinas). A classe define os atributos e comportamentos que todos os objetos criados a partir dela possuirão. Classes são código; objetos instanciados dessas classes podem ser distribuídos individualmente ou como parte de uma biblioteca. Como objetos são criados a partir de classes, as classes devem definir seus blocos fundamentais: atributos, comportamento e mensagens. Portanto, é necessário projetar uma classe antes de criar seus objetos.

A página original mostra uma definição Java da classe `Person`, com atributos privados `name` e `address` e métodos `getName()`, `setName()`, `getAddress()` e `setAddress()`. O código é mantido sem tradução.

ATRIBUTOS

Como vimos, os dados de uma classe são representados por atributos. Cada classe define os atributos que armazenam o estado de cada objeto instanciado. No exemplo `Person`, a classe define atributos para nome e endereço.

MODIFICADORES DE ACESSO

Quando um tipo de dado ou método é definido como `public`, outros objetos podem acessá-lo diretamente. Quando é `private`, somente o próprio objeto pode acessá-lo. O modificador `protected` permite acesso a objetos relacionados e será discutido no Capítulo 3.

MÉTODOS

Métodos implementam o comportamento exigido de uma classe. Todo objeto instanciado da classe dispõe dos métodos definidos por ela. Alguns comportamentos são chamados por outros objetos, por meio de mensagens; outros oferecem comportamento interno da própria classe. Esses comportamentos internos podem ser métodos `private` e não ficam acessíveis a objetos externos.

Na classe `Person`, `getName()`, `setName()`, `getAddress()` e `setAddress()` permitem que outros objetos consultem ou alterem os valores dos atributos de maneira controlada. Em todos os casos, o acesso aos atributos internos de um objeto deve ser controlado pelo próprio objeto; outro objeto não deveria modificar diretamente um atributo que não lhe pertence.

Messages

Messages are the communication mechanism between objects. For example, when Object A invokes a method of Object B, Object A is sending a message to Object B. Object B’s response is defined by its return value. Only the public methods, not the private methods, of an object can be invoked by another object. The following code illustrates this concept:

```
public class Payroll{

    String name;
    Person p = new Person();
    p.setName("Joe");

    ... code

    name = p.getName();
}
```

In this example (assuming that a `Payroll` object is instantiated), the `Payroll` object is sending a message to a `Person` object, with the purpose of retrieving the name via the `getName()` method. Again, don’t worry too much about the actual code, because we are really interested in the concepts. We address the code in detail as we progress through the book.

USING CLASS DIAGRAMS AS A VISUAL TOOL

Over the years, many tools and modeling methodologies have been developed to assist in designing software systems. Right from the start, I have used UML class diagrams to assist in the educational process. Although it is beyond the scope of this book to describe UML in any detail, we will use UML class diagrams to illustrate the classes that we build. In fact, we have already used class diagrams in this chapter. [Figure 1.11](#) shows the `Person` class diagram we discussed earlier in the chapter.

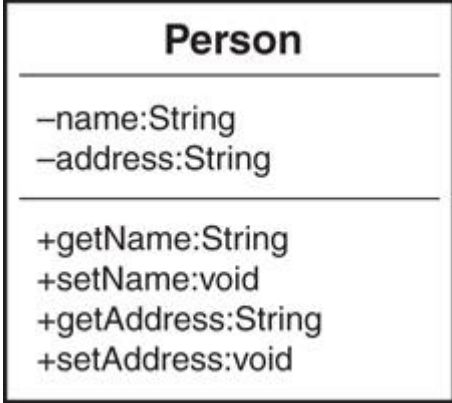


Figure 1.11 The `Person` class diagram.

As we saw previously, notice that the attributes and methods are separated (the attributes on the top and the methods on the bottom). As we delve more deeply into OO design, these class diagrams will get much more sophisticated and convey much more information on how the different classes interact with each other.

ENCAPSULATION AND DATA HIDING

One of the primary advantages of using objects is that the object need not reveal all its attributes and behaviors. In good OO design (at least what is generally accepted as good), an object should reveal only the interfaces that other objects must have to interact with it. Details not pertinent to the use of the object should be hidden from all other objects—basically a “need to know” basis.

MENSAGENS

Mensagens são o mecanismo de comunicação entre objetos. Quando o Objeto A invoca um método do Objeto B, A está enviando uma mensagem a B. A resposta de B é definida pelo valor retornado pelo método. Apenas métodos públicos - e não métodos privados - podem ser invocados diretamente por outro objeto.

A página original apresenta um exemplo em Java no qual um objeto `Payroll` cria ou utiliza um objeto `Person`, chama `p.setName("Joe")` e depois obtém o nome com `p.getName()`. Nesse caso, `Payroll` envia uma mensagem a `Person` com o objetivo de recuperar o nome por meio de `getName()`. O código em si não é o foco aqui; o conceito importante é a comunicação por mensagens.

USANDO DIAGRAMAS DE CLASSES COMO FERRAMENTA VISUAL

Ao longo dos anos, várias ferramentas e metodologias de modelagem foram criadas para auxiliar no projeto de sistemas de software. Desde a primeira edição, o autor utiliza diagramas de classes UML como recurso educacional. Embora uma descrição detalhada de UML esteja fora do escopo do livro, os diagramas serão usados para ilustrar as classes construídas.

A Figura 1.11 apresenta o diagrama da classe `Person` discutida anteriormente.

Figura 1.11 - Diagrama da classe `Person`.

Observe que atributos e métodos aparecem separados: os atributos na parte superior e os métodos na inferior. À medida que avançarmos em projeto OO, os diagramas ficarão mais sofisticados e fornecerão mais informações sobre a maneira como as classes interagem.

ENCAPSULAMENTO E OCULTAÇÃO DE DADOS

Uma das principais vantagens de objetos é que eles não precisam revelar todos os seus atributos e comportamentos. Em um bom projeto OO, o objeto deve expor apenas as interfaces de que outros objetos precisam para interagir com ele. Detalhes que não são relevantes para seu uso devem permanecer ocultos - em essência, seguindo o princípio de “necessidade de saber”.

Encapsulation is defined by the fact that objects contain both the attributes and behaviors. Data hiding is a major part of encapsulation.

For example, an object that calculates the square of a number must provide an interface to obtain the result. However, the internal attributes and algorithms used to calculate the square need not be made available to the requesting object. Robust classes are designed with encapsulation in mind. In the next sections, we cover the concepts of interface and implementation, which are the basis of encapsulation.

Interfaces

We have seen that the interface defines the fundamental means of communication between objects. Each class design specifies the interfaces for the proper instantiation and operation of objects. Any behavior that the object provides must be invoked by a message sent using one of the provided interfaces. The interface should completely describe how users of the class interact with the class. In most OO languages, the methods that are part of the interface are designated as `public`.

Private Data

For data hiding to work properly, all attributes should be declared as `private`. Thus, attributes are never part of the interface. Only the public methods are part of the class interface. Declaring an attribute as `public` breaks the concept of data hiding.

Let’s look at the example just mentioned: calculating the square of a number. In this example, the interface would consist of two pieces:

- How to instantiate a `Square` object
- How to send a value to the object and get the square of that value in return

As discussed earlier in the chapter, if a user needs access to an attribute, a method is created to return the value of the attribute (a `getter`). If a user then wants to obtain the value of an attribute, a method is called to return its value. In this way, the object that contains the attribute controls access to it. This is of vital importance, especially in security, testing, and maintenance. If you control the access to the attribute, when a problem arises, you do not have to worry about tracking down every piece of code that might have changed the attribute—it can be changed in only one place (the `setter`).

From a security perspective, you don’t want uncontrolled code to change or retrieve sensitive data. For example, when you use an ATM, access to data is controlled by asking for a PIN.

Signatures—Interfaces Versus Interfaces

Don't confuse the interfaces used to extend classes with the interface of a class. I like to equate the interfaces, represented by methods, as “signatures.”

Implementations

Only the public attributes and methods are considered the interface. The user should not see any part of the internal implementation, interacting with an object solely through class interfaces. Thus, anything defined as `private` is inaccessible to the user and considered part of the class’s internal implementation.

In the previous example, for instance the `Employee` class, only the attributes were hidden. In many cases, there will be methods that also should be hidden and thus not part of the interface. Continuing the example of the square root from the previous section, the user does not care how

O encapsulamento é definido pelo fato de que objetos reúnem atributos e comportamentos. A ocultação de dados é uma parte importante desse conceito.

Por exemplo, um objeto que calcula o quadrado de um número precisa fornecer uma interface para que o resultado seja obtido. Porém, os atributos internos e os algoritmos usados no cálculo não precisam ficar disponíveis para o objeto solicitante. Classes robustas são projetadas com encapsulamento em mente. As ideias de interface e implementação, que formam a base do encapsulamento, são discutidas a seguir.

INTERFACES

A interface define o meio fundamental de comunicação entre objetos. O projeto de cada classe especifica as interfaces necessárias para a criação e operação adequadas de seus objetos. Qualquer comportamento oferecido pelo objeto deve ser acionado por uma mensagem enviada através de uma dessas interfaces. A interface deve descrever completamente como o usuário da classe interage com ela. Na maioria das linguagens OO, os métodos pertencentes à interface são declarados `public`.

DADOS PRIVADOS

Para que a ocultação de dados funcione corretamente, os atributos devem ser declarados `private`. Portanto, atributos não fazem parte da interface pública; apenas os métodos públicos fazem. Declarar um atributo como `public` rompe o princípio de ocultação de dados.

Considere novamente o exemplo do quadrado de um número. A interface precisa definir:

- como instanciar um objeto `Square`;
- como enviar um valor ao objeto e receber seu quadrado.

Se um cliente precisar ler um atributo, cria-se normalmente um método que devolve seu valor, um `getter`. Assim, o próprio objeto que contém o atributo controla o acesso. Isso é essencial para segurança, testes e manutenção. Quando o acesso ao atributo é controlado, um problema pode ser investigado em poucos pontos, em vez de procurar em todo o código por trechos capazes de alterar o valor; a modificação é centralizada, por exemplo, no `setter`.

Do ponto de vista da segurança, não se deseja que código não controlado leia ou altere dados sensíveis. Em um caixa eletrônico, por exemplo, o acesso é controlado pela solicitação de um PIN.

ASSINATURAS - DOIS SENTIDOS DE “INTERFACE”

Não confunda a interface pública de uma classe, representada por seus métodos disponíveis ao cliente, com o recurso de linguagem interface usado para estender tipos. O autor costuma associar a interface pública aos “contratos” ou assinaturas que a classe oferece.

IMPLEMENTAÇÕES

Somente os elementos públicos fazem parte da interface. O usuário não deveria precisar conhecer a implementação interna; deve interagir com o objeto exclusivamente através de suas interfaces públicas. Tudo o que é `private` fica inacessível ao cliente e pertence à implementação interna. Métodos também podem ser ocultos, e não apenas atributos.

the square root is calculated—as long as it is the correct answer. Thus, the implementation can change, and it will not affect the user’s code. For example, the company that produces the calculator can change the algorithm (perhaps because it is more efficient) without affecting the result.

A Real-World Example of the Interface/Implementation Paradigm

Figure 1.12 illustrates the interface/implementation paradigm using real-world objects rather than code. The toaster requires electricity. To get this electricity, the cord from the toaster must be plugged into the electrical outlet, which is the interface. All the toaster needs to do to obtain the required electricity is to implement a cord that complies with the electrical outlet specifications; this is the interface between the toaster and the power company (actually the power industry). That the actual implementation is a coal-powered electric plant is not the concern of the toaster. In fact, for all the toaster cares, the implementation could be a nuclear power plant or a local power generator. With this model, any appliance can get electricity, as long as it conforms to the interface specification as shown in **Figure 1.12**.

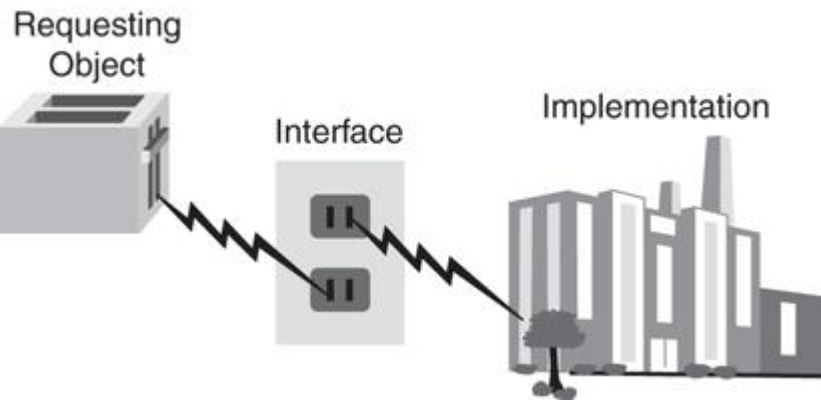


Figure 1.12 Power plant example.

A Model of the Interface/Implementation Paradigm

Let’s explore the `Square` class further. Assume that you are writing a class that calculates the squares of integers. You must provide a separate interface and implementation. That is, you must specify a way for the user to invoke and obtain the square value. You must also provide the implementation that calculates the square; however, the user should not know anything about the specific implementation. **Figure 1.13** shows one way to do this. Note that in the class diagram, the plus sign (+) designates public and the minus sign (-) designates private. Thus, you can identify the interface by the methods, prefaced with plus signs.

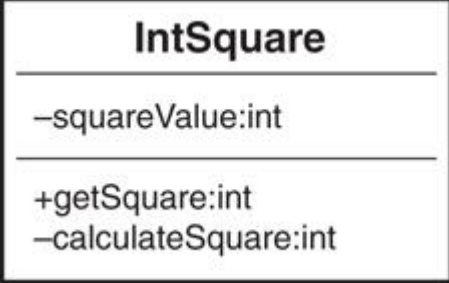


Figure 1.13 The Square class.

This class diagram corresponds to the following code:

[Click here to view code image](#)

```
public class IntSquare {

    // private attribute
    private int squareValue;
```

No exemplo do cálculo, o usuário não se importa com a maneira como o quadrado é calculado, desde que o resultado seja correto. A implementação pode mudar sem afetar o código do cliente. O fabricante de uma calculadora, por exemplo, pode substituir o algoritmo por outro mais eficiente sem alterar a forma como o usuário solicita o resultado.

UM EXEMPLO DO MUNDO REAL DO PARADIGMA INTERFACE/IMPLEMENTAÇÃO

A Figura 1.12 ilustra o paradigma usando objetos reais. Uma torradeira precisa de eletricidade. Para recebê-la, seu cabo deve ser conectado a uma tomada, que funciona como interface. A torradeira precisa apenas implementar um cabo compatível com a especificação da tomada. Essa é a interface entre a torradeira e a rede elétrica.

A implementação real da geração de energia - uma usina a carvão, nuclear ou um gerador local - não importa para a torradeira. Qualquer aparelho pode receber eletricidade desde que respeite a especificação da interface.

Figura 1.12 - Exemplo de uma usina e uma interface elétrica.

UM MODELO DO PARADIGMA INTERFACE/IMPLEMENTAÇÃO

Considere uma classe que calcula o quadrado de inteiros. Ela precisa fornecer uma interface para o cliente solicitar o valor e, separadamente, uma implementação que realize o cálculo. O usuário não precisa saber como a implementação funciona.

A Figura 1.13 mostra uma possibilidade. No diagrama de classes, o sinal + indica membros públicos e o sinal - indica membros privados. Assim, os métodos marcados com + compõem a interface disponível ao cliente.

Figura 1.13 - A classe Square/IntSquare.

O código Java correspondente é preservado na página original. A classe possui um atributo privado para armazenar o valor e um método público `getSquare()`, que constitui a interface.

O único elemento que o usuário precisa acessar é o método público `getSquare()`, isto é, a interface. O algoritmo de cálculo fica em `calculateSquare()`, um método privado. O atributo `squareValue` também é privado porque o cliente não precisa nem saber que ele existe.

Esse é o objetivo do encapsulamento: o objeto revela somente as interfaces necessárias para sua utilização e oculta detalhes de implementação que não dizem respeito ao cliente.

Se a implementação mudar - por exemplo, substituindo uma multiplicação pela função `Math.pow` da biblioteca Java - a interface não precisa mudar. O cliente continua recebendo a mesma funcionalidade pelo mesmo ponto de acesso. Esse princípio é especialmente importante em código que lida com dados: é possível, por exemplo, mover dados de um arquivo para um banco de dados sem obrigar o usuário da classe a alterar o código da aplicação, desde que a interface permaneça estável.

HERANÇA

Herança permite que uma classe receba atributos e métodos de outra classe. Com ela, novas classes podem ser construídas extraíndo-se atributos e comportamentos comuns para uma classe mais geral.

Um dos grandes problemas de projeto em POO é identificar o que há de comum entre diferentes classes. Suponha que existam as classes `Dog` e `Cat` e que ambas tenham um atributo de cor dos olhos. Em um modelo procedural, esse atributo poderia ser repetido nos dois trechos de código. Em um projeto OO, ele pode ser movido para uma classe mais geral, `Mammal`, juntamente com outros atributos e comportamentos comuns. `Dog` e `Cat` passam então a herdar de `Mammal`, como mostra a Figura 1.14.

```
// public interface
public intgetSquare (int value) {

    squareValue = calculateSquare(value);

    return squareValue;

}

// private implementation
private intcalculateSquare (int value) {

    return value*value;

}
}
```

Note that the only part of the class that the user has access to is the public method `getSquare`, which is the interface. The implementation of the square algorithm is in the method `calculateSquare`, which is private. Also notice that the attribute `squareValue` is private because users do not need to know that this attribute exists. Therefore, we have hidden the part of the implementation: The object reveals only the interfaces the user needs to interact with it, and details that are not pertinent to the use of the object are hidden from other objects. If the implementation were to change—suppose you wanted to use the language’s built-in square function—you would not need to change the interface. Here the code uses the Java library method `Math.pow`, which performs the same function, but note that the interface is still `calculateSquare`.

Click here to view code image

```
// private implementation
private intcalculateSquare (int value) {

    return = Math.pow(value,2);

}
}
```

The user would get the same functionality using the same interface, but the implementation would have changed. This is very important when you’re writing code that deals with data; for example, you can move data from a file to a database without forcing the user to change any application code.

INHERITANCE

Inheritance enables a class to inherit the attributes and methods of another class. This provides the ability to create new classes by abstracting out common attributes and behaviors from another class.

One of the major design issues in OO programming is to factor out commonality of the various classes. For example, suppose you have a `Dog` class and a `Cat` class, and each will have an attribute for eye color. In a procedural model, the code for `Dog` and `Cat` would each contain this attribute. In an OO design, the color attribute could be moved up to a class called `Mammal`—along with any other common attributes and methods. In this case, both `Dog` and `Cat` inherit from the `Mammal` class, as shown in [Figure 1.14](#).

Figura 1.14 - Hierarquia Mammal.

As classes Dog e Cat herdam de Mammal. Isso significa que Dog possui, por exemplo, eyeColor herdado de Mammal e barkFrequency definido especificamente para cães. Da mesma forma, um objeto Dog pode possuir getEyeColor() herdado de Mammal e bark() definido apenas em Dog.

Quando Dog ou Cat são instanciadas, os objetos possuem tudo o que foi definido em sua própria classe e também o que foi herdado da classe pai. Assim, Dog reúne suas propriedades específicas com as propriedades de Mammal.

COMPORTAMENTOS

Atualmente, comportamentos são muitas vezes descritos por interfaces, e a herança direta é frequentemente usada para compartilhar estado ou atributos. Dessa forma, o comportamento pode ser abstraído de seus dados.

SUPERCLASSES E SUBCLASSES

A superclasse, ou classe pai/base, contém atributos e comportamentos comuns às classes que herdam dela. No exemplo Mammal, mamíferos podem compartilhar eyeColor, hairColor, generateInternalHeat() e growHair(). Não é necessário duplicar essas definições em cada ramo da hierarquia. A duplicação aumenta o trabalho e, principalmente, o risco de erros e inconsistências.

A subclasse, ou classe filha/derivada, é uma extensão da superclasse. Dog e Cat herdam os elementos comuns de Mammal. Assim, Mammal é a superclasse e Dog e Cat são subclasses.

A herança oferece diversas vantagens de projeto. Ao projetar Cat, por exemplo, Mammal já fornece parte da funcionalidade necessária para representar um mamífero.

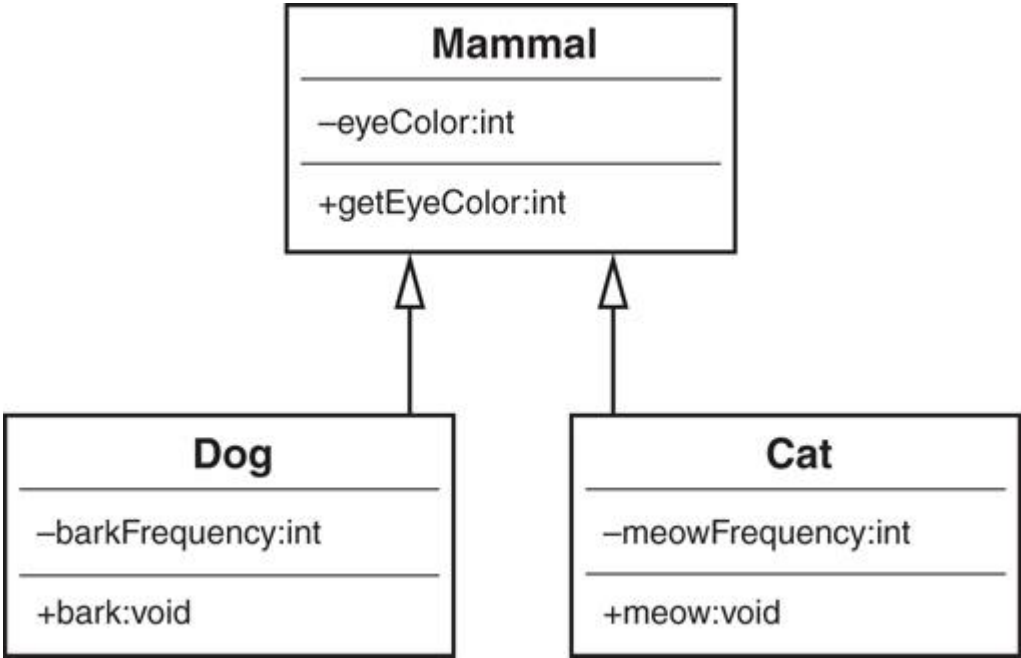


Figure 1.14 Mammal hierarchy.

The Dog and Cat classes both inherit from Mammal. This means that a Dog class has the following attributes:

```
eyeColor      // inherited from Mammal
barkFrequency // defined only for Dogs
```

In the same vein, the Dog object has the following methods:

```
getEyeColor // inherited from Mammal
bark        // defined only for Dogs
```

When the Dog or the Cat object is instantiated, it contains everything in its own class, as well as everything from the parent class. Thus, Dog has all the properties of its class definition, as well as the properties inherited from the Mammal class.

Behaviors

It is worth noting that behaviors today tend to be described in interfaces and that inheritance of attributes is the most common use of direct inheritance. In this way, the behaviors are abstracted away from their data.

Superclasses and Subclasses

The superclass, or parent class (sometimes called base class), contains all the attributes and behaviors that are common to classes that inherit from it. For example, in the case of the Mammal class, all mammals have similar attributes, such as eyeColor and hairColor, as well as behaviors, such as generateInternalHeat and growHair. All mammals have these attributes and behaviors, so it is not necessary to duplicate them down the inheritance tree for each type of mammal. Duplication requires a lot more work, and perhaps more worrisome, it can introduce errors and inconsistencies.

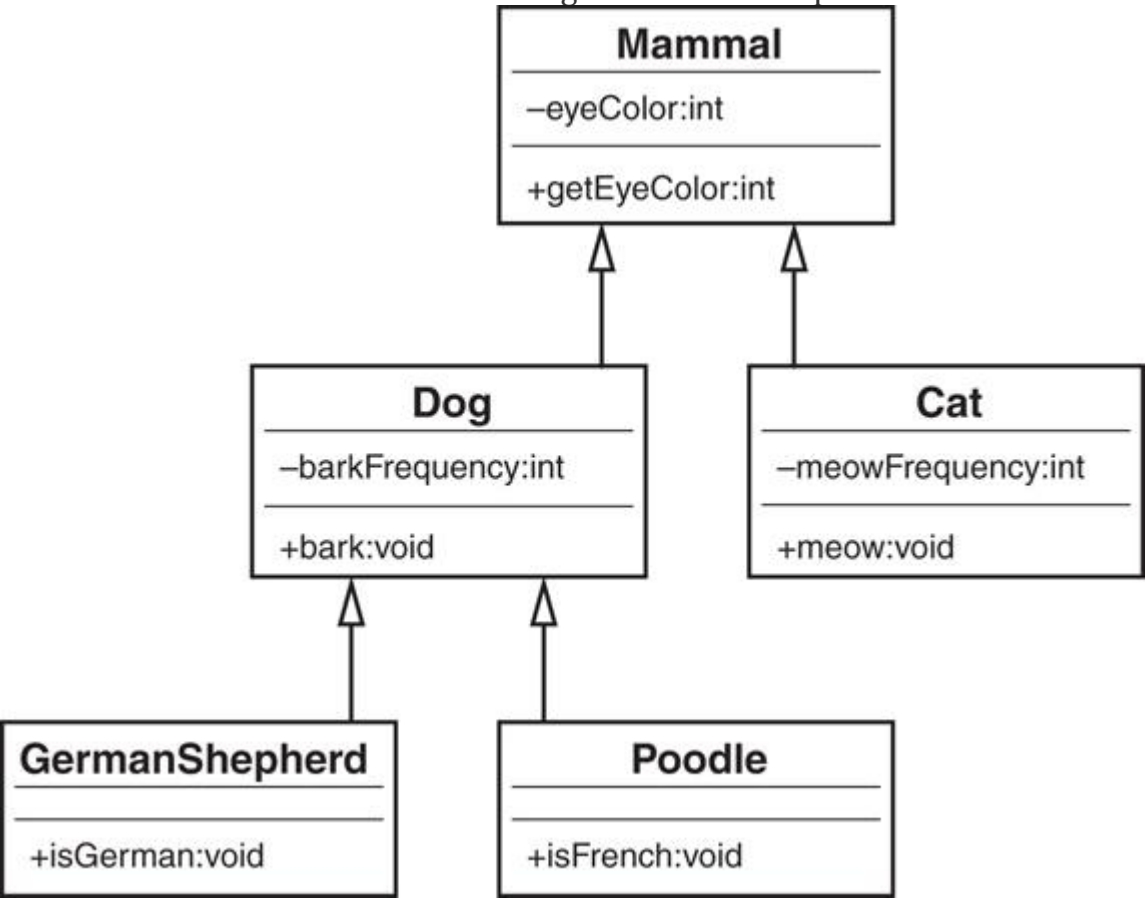
The subclass, or child class (sometimes called derived class) is an extension of the superclass. Thus, the Dog and Cat classes inherit all those common attributes and behaviors from the Mammal class. The Mammal class is considered the superclass of the Dog and the Cat subclasses, or child classes.

Inheritance provides a rich set of design advantages. When you’re designing a Cat class, the Mammal class provides much of the functionality needed. By inheriting from the Mammal object, Cat already has all the attributes and behaviors that make it a true mammal.

To make it more specifically a cat type of mammal, the `Cat` class must include any attributes or behaviors that pertain solely to a cat.

Abstraction

An inheritance tree can grow quite large. When the `Mammal` and `Cat` classes are complete, other mammals, such as dogs (or lions, tigers, and bears), can be added quite easily. The `Cat` class can also be a superclass to other classes. For example, it might be necessary to abstract the `Cat` class further, to provide classes for Persian cats, Siamese cats, and so on. Just as with `Cat`, the `Dog` class can be the parent for `GermanShepherd` and `Poodle` (see [Figure 1.15](#)). The power of inheritance lies in its abstraction and organization techniques.



Figure

1.15 Mammal UML diagram.

These multiple levels of abstraction are one of the reasons why many developers are wary of using inheritance at all. As we will see often, it is difficult to decide how much abstraction is required. For example, if a penguin is a bird and a hawk is a bird, should they both inherit from a class called `Bird`—a class that has a `fly` method?

In most recent OO languages (such as Java, .NET, and Swift), a class can have only a single parent class; however, a class can have many child classes. Some languages, such as C++, can have multiple parents. The former case is called *single inheritance*, and the latter is called *multiple inheritance*.

Multiple Inheritance

Consider a child that inherits from both parents. Which pair of eyes does the child inherit? This is a significant problem when it comes to writing a compiler. C++ allows multiple inheritance; many languages do not.

Para transformar essa representação genérica em um tipo específico de mamífero - um gato -, `Cat` precisa acrescentar apenas os atributos e comportamentos próprios dos gatos.

ABSTRAÇÃO

Uma árvore de herança pode crescer bastante. Depois que `Mammal` e `Cat` estiverem definidas, outros mamíferos, como cães, leões, tigres e ursos, podem ser adicionados com facilidade. `Cat` também pode se tornar superclasse de outras classes, como `PersianCat` e `SiameseCat`. Da mesma forma, `Dog` pode ser pai de `GermanShepherd` e `Poodle`, como mostra a [Figura 1.15](#). O poder da herança está em suas técnicas de abstração e organização.

Figura 1.15 - Diagrama UML de Mammal.

Esses vários níveis de abstração também explicam por que muitos desenvolvedores são cautelosos com herança. Nem sempre é fácil decidir quanta abstração é adequada. Se `Penguin` e `Hawk` são aves, por exemplo, ambas deveriam herdar de `Bird` se `Bird` possui um método `fly()`? Um pinguim é uma ave, mas não voa.

Na maioria das linguagens OO modernas, como Java, .NET e Swift, uma classe possui apenas uma classe pai, embora possa ter várias classes filhas. Algumas linguagens, como C++, permitem múltiplas classes pai. O primeiro caso é chamado de herança simples; o segundo, herança múltipla.

HERANÇA MÚLTIPLA

Pense em uma criança que herda características de dois pais. Qual par de olhos ela “herda”? Esse tipo de ambiguidade se torna um problema relevante na implementação de compiladores. C++ permite herança múltipla; muitas outras linguagens não.

Note that the classes `GermanShepherd` and `Poodle` both inherit from `Dog`—each contains only a single method. However, because they inherit from `Dog`, they also inherit from `Mammal`. Thus, the `GermanShepherd` and `Poodle` classes contain all the attributes and methods included in `Dog` and `Mammal`, as well as their own (see [Figure 1.16](#)).

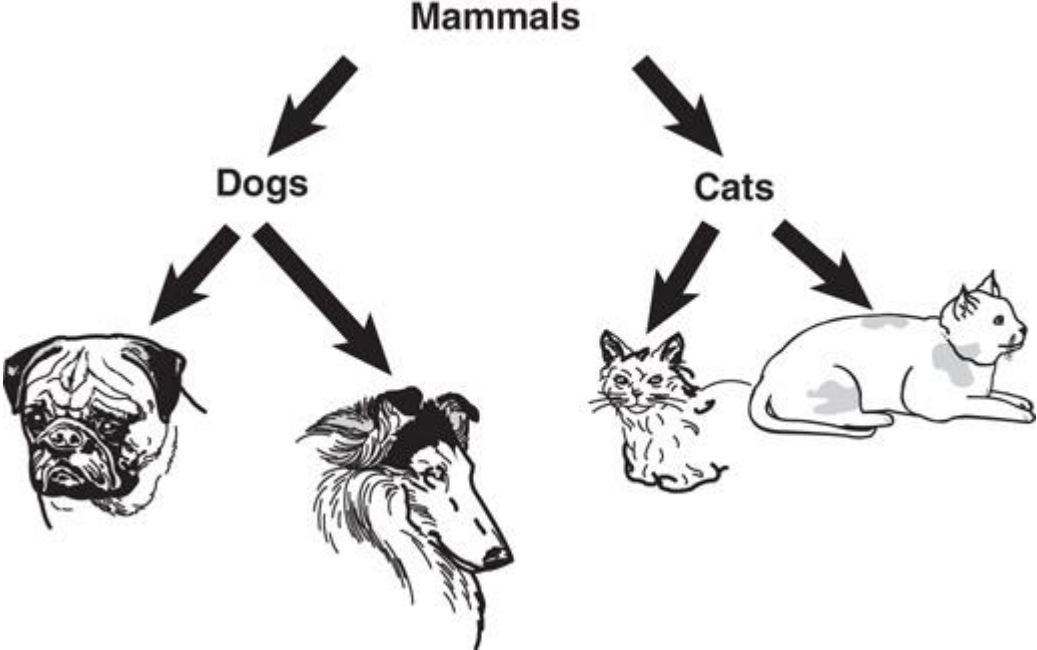


Figure 1.16 Mammal

hierarchy.

Is-a Relationships

Consider a `Shape` example where `Circle`, `Square`, and `Star` all inherit directly from `Shape`. This relationship is often referred to as an *is-a relationship* because a circle is a shape, and a square is a shape. When a subclass inherits from a superclass, it can do anything that the superclass can do. Thus, `Circle`, `Square`, and `Star` are all extensions of `Shape`.

In [Figure 1.17](#), the name on each of the objects represents the `draw` method for the `Circle`, `Star`, and `Square` objects, respectively. When we design this `Shape` system, it would be very helpful to standardize how we use the various shapes. Thus, we could decide that if we want to draw a shape, no matter what shape, we will invoke a method called `draw`. If we adhere to this decision, whenever we want to draw a shape, only the `draw` method needs to be called, regardless of what the shape is. Here lies the fundamental concept of polymorphism—it is the individual object’s responsibility, be it a `Circle`, `Star`, or `Square`, to draw itself. This is a common concept in many current software applications, such as drawing and word processing applications.

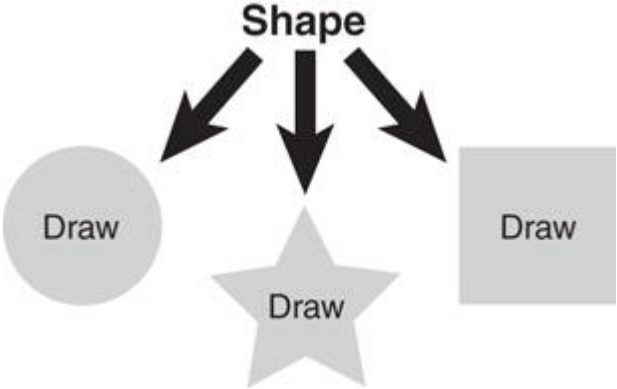


Figure 1.17 The shape hierarchy.

`GermanShepherd` e `Poodle` herdam de `Dog`. Mesmo que cada uma tenha apenas um método específico, também recebem, por meio de `Dog`, os elementos de `Mammal`. Portanto, essas classes incluem atributos e métodos de `Dog` e `Mammal`, além de suas próprias definições.

Figura 1.16 - Hierarquia Mammal.

RELAÇÕES “É-UM” (IS-A)

Considere uma hierarquia em que `Circle`, `Square` e `Star` herdam diretamente de `Shape`. Essa relação é chamada de is-a porque um círculo é uma forma e um quadrado é uma forma. Quando uma subclasse herda de uma superclasse, ela pode ser usada nos contextos previstos para a superclasse. `Circle`, `Square` e `Star` são extensões de `Shape`.

Na [Figura 1.17](#), cada tipo possui um método `draw()`. Ao projetar esse sistema, é útil padronizar o uso das formas: sempre que quisermos desenhar qualquer `Shape`, invocamos `draw()`. Não precisamos escolher um nome de método diferente para cada figura.

Aqui aparece a ideia fundamental de polimorfismo: é responsabilidade do objeto concreto - `Circle`, `Star` ou `Square` - saber como desenhar a si próprio. O cliente apenas envia a mesma mensagem `draw()`. Esse conceito é comum em aplicações atuais, como programas de desenho e processadores de texto.

Figura 1.17 - Hierarquia de formas.

POLYMORPHISM

Polymorphism is a Greek word that literally means many shapes. Although polymorphism is tightly coupled to inheritance, it is often cited separately as one of the most powerful advantages to object-oriented technologies. When a message is sent to an object, the object must have a method defined to respond to that message. In an inheritance hierarchy, all subclasses inherit the interfaces from their superclass. However, because each subclass is a separate entity, each might require a separate response to the same message.

For example, consider the `Shape` class and the behavior called `draw`. When you tell somebody to draw a shape, the first question asked is, “What shape?” No one can draw a shape, because it is an abstract concept (in fact, the `draw` method in the `Shape` code following contains no implementation). You must specify a concrete shape. To do this, you provide the actual implementation in `Circle`. Even though `Shape` has a `draw` method, `Circle` overrides this method and provides its own `draw` method. Overriding basically means replacing an implementation of a parent with one from a child.

For example, suppose you have an array of three shapes—`Circle`, `Square`, and `Star`. Even though you treat them all as `Shape` objects, and send a `draw` message to each `Shape` object, the end result is different for each because `Circle`, `Square`, and `Star` provide the actual implementations. In short, each class is able to respond differently to the same `draw` method and draw itself. This is what is meant by polymorphism.

Consider the following `Shape` class:

```
public abstract class Shape{

    private double area;

    public abstract double getArea();

}
```

The `Shape` class has an attribute called `area` that holds the value for the area of the shape. The method `getArea()` includes an identifier called `abstract`. When a method is defined as `abstract`, a subclass must provide the implementation for this method; in this case, `Shape` is requiring subclasses to provide a `getArea()` implementation. Now let’s create a class called `Circle` that inherits from `Shape` (the `extends` keyword specifies that `Circle` inherits from `Shape`):

[Click here to view code image](#)

```
public class Circle extends Shape{
    double radius;
    public Circle(double r) {
        radius = r;
    }

    public double getArea() {
        area = 3.14*(radius*radius);
        return (area);
    }

}
```

We introduce a new concept here called a *constructor*. The `Circle` class has a method with the same name, `Circle`. When a method name is the same as the class and no return type is provided, the method is a special method, called a constructor. Consider a constructor as the entry point for the class, where the object is built; the constructor is a good place to perform initializations and start-up tasks.

POLIMORFISMO

Polimorfismo é uma palavra de origem grega que significa literalmente “muitas formas”. Embora esteja fortemente relacionado à herança, costuma ser apresentado separadamente por ser uma das características mais poderosas das tecnologias OO.

Quando uma mensagem é enviada a um objeto, deve existir algum método capaz de responder a ela. Em uma hierarquia de herança, subclasses recebem a interface da superclasse, mas cada subclasse é uma entidade própria e pode precisar responder de maneira diferente à mesma mensagem.

Considere `Shape` e o comportamento `draw()`. Ao pedir que alguém “desenhe uma forma”, a primeira pergunta seria “qual forma?”. `Shape` é um conceito abstrato; para desenhar, precisamos de uma forma concreta. `Circle` pode substituir ou implementar o comportamento `draw()` com sua própria lógica. Sobrescrever (`override`) significa fornecer, na classe filha, uma implementação específica para um comportamento definido ou previsto pela classe pai.

Imagine um conjunto com `Circle`, `Square` e `Star`. Mesmo tratando todos como `Shape` e enviando a mensagem `draw()` a cada um, o resultado é diferente porque cada classe concreta fornece sua implementação. Essa capacidade de responder de maneiras diferentes à mesma mensagem é o polimorfismo.

A página original mostra uma classe abstrata `Shape` com um atributo `area` e um método abstrato `getArea()`. Um método `abstract` obriga as subclasses concretas a fornecerem sua implementação. Em seguida, a classe `Circle` herda de `Shape`, armazena `radius` e implementa `getArea()`.

Também surge o conceito de construtor. Quando um método especial possui o mesmo nome da classe e não declara tipo de retorno, ele é usado para construir e inicializar o objeto. O construtor é um local apropriado para realizar inicializações e tarefas de preparação.

The `Circle` constructor accepts a single parameter, representing the radius, and assigns it to the `radius` attribute of the `Circle` class. The `Circle` class also provides the implementation for the `getArea` method, originally defined as abstract in the `Shape` class. We can create a similar class, called `Rectangle`:

[Click here to view code image](#)

```
public class Rectangle extends Shape{

    double length;
    double width;

    public Rectangle(double l, double w){
        length = l;
        width = w;
    }

    public double getArea() {
        area = length*width;
        return (area);
    }

}
```

Now we can create any number of rectangles, circles, and so on and invoke their `getArea()` method. This is because we know that all rectangles and circles inherit from `Shape`, and all `Shape` classes have a `getArea()` method. If a subclass inherits an abstract method from a superclass, it must provide a concrete implementation of that method, or else it will be an abstract class itself (see [Figure 1.18](#) for a UML diagram). This approach also provides the mechanism to create other, new classes quite easily.

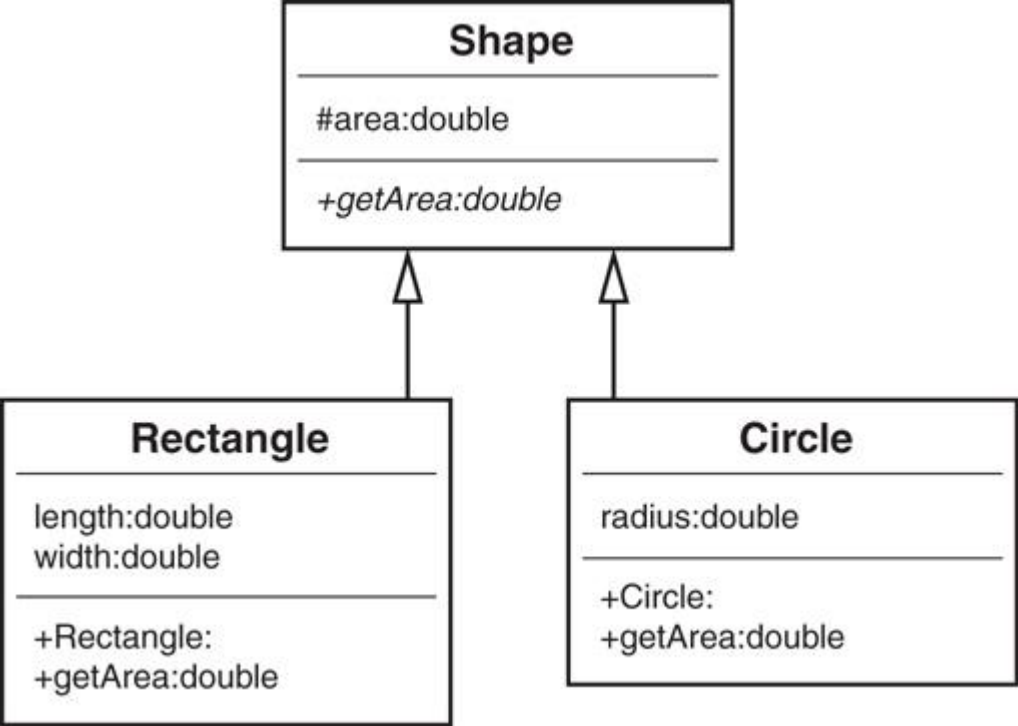


diagram.

Thus, we can instantiate the `Shape` classes in this way:

```
Circle circle = new Circle(5);
Rectangle rectangle = new Rectangle(4,5);
```

O construtor de `Circle` recebe um parâmetro que representa o raio e o associa ao atributo `radius`. `Circle` também fornece uma implementação concreta para `getArea()`, que havia sido declarado abstrato em `Shape`.

A página apresenta ainda uma classe `Rectangle` que herda de `Shape`, recebe `length` e `width` em seu construtor e implementa `getArea()` calculando `length * width`. O código é mantido no original à esquerda.

Podemos criar quantos círculos e retângulos forem necessários e chamar `getArea()` em qualquer um deles. Sabemos que todos herdam de `Shape` e que todo tipo concreto de `Shape` deve oferecer `getArea()`. Se uma subclasse herdar um método abstrato e não o implementar, ela própria também deverá permanecer abstrata.

Figura 1.18 - Diagrama UML de `Shape`.

Assim, objetos concretos podem ser instanciados, por exemplo, com `Circle(5)` e `Rectangle(4,5)`. A vantagem é que novas formas podem ser adicionadas ao sistema seguindo o mesmo contrato.

Then, using a construct such as a stack, we can add these `Shape` classes to the stack:

```
stack.push(circle);
stack.push(rectangle);
```

What Is a Stack?

A stack is a data structure that is a last-in, first-out system. It is like a coin changer, where you insert coins at the top of the cylinder and, when you need a coin, you take one off the top, which is the last one you inserted. Pushing an item onto the stack means that you are adding an item to the top (like inserting another coin into the changer). Popping an item off the stack means that you are taking the last item off the stack (like taking the coin off the top).

Now comes the fun part. We can empty the stack, and we do not have to worry about what kind of `Shape` classes are in it (we just know they are shapes):

Click here to view code image

```
while ( !stack.empty()) {
    Shape shape = (Shape) stack.pop();
    System.out.println ("Area = " + shape.getArea());
}
```

In reality, we are sending the same message to all the shapes:

```
shape.getArea()
```

However, the actual behavior that takes place depends on the type of shape. For example, `Circle` calculates the area for a circle, and `Rectangle` calculates the area of a rectangle. In effect (and here is the key concept), we are sending a message to the `Shape` classes and experiencing different behavior depending on what subclass of `Shape` is being used.

This approach is meant to provide standardization for the interface across classes, as well as applications. Consider an office suite application that includes a word processing and a spreadsheet application. Let’s assume that both have a class called `Office` which contains an interface called `print()`. This `print()` interface is required for all classes that are part of the office suite. The interesting thing here is that although both the word processor and the spreadsheet invoke the `print()` interface, they do different things: one prints a word processing document and the other a spreadsheet document.

Polymorphism by Composition

In “classical” OO, polymorphism is traditionally implemented with inheritance; however, there is a way to implement polymorphism using composition. We discuss this in [Chapter 12](#), “[The SOLID Principles of Object-Oriented Design](#).”

COMPOSITION

It is natural to think of objects as containing other objects. A television set contains a tuner and video display. A computer contains video cards, keyboards, and drives. Although the computer can be considered an object unto itself, the drive is also considered a valid object. In fact, you could open up the computer and remove the drive and hold it in your hand. Both the computer and the drive are considered objects. It is just that the computer contains other objects—such as drives.

In this way, objects are often built, or composed, from other objects: This is composition.

Depois de criar `Circle` e `Rectangle`, podemos armazená-los em uma estrutura como uma pilha (stack), tratando ambos como objetos `Shape`.

O QUE É UMA PILHA?

Uma pilha é uma estrutura de dados do tipo último a entrar, primeiro a sair (LIFO). Pense em um tubo para moedas: novas moedas são colocadas no topo e a primeira moeda retirada é a última que foi inserida. “Push” adiciona um item ao topo da pilha; “pop” remove o item que está no topo.

Ao esvaziar a pilha, não precisamos nos preocupar com o tipo concreto de cada forma; sabemos apenas que são objetos `Shape`. O código original mostra um laço que retira um `Shape` da pilha e chama `shape.getArea()`.

Na prática, enviamos a mesma mensagem a todas as formas: `shape.getArea()`

O comportamento real depende do tipo concreto. `Circle` calcula a área de um círculo; `Rectangle` calcula a área de um retângulo. A ideia central é enviar uma mensagem uniforme para objetos `Shape` e obter comportamentos diferentes de acordo com a subclasse real.

Esse mecanismo padroniza interfaces entre classes e aplicações. Imagine um pacote de escritório com processador de texto e planilha, ambos oferecendo uma interface `print()`. As duas aplicações recebem a mesma chamada, mas cada uma imprime um tipo diferente de documento.

POLIMORFISMO POR COMPOSIÇÃO

Na OO “clássica”, polimorfismo é tradicionalmente associado à herança. Entretanto, também pode ser implementado por composição. O assunto é retomado no Capítulo 12, sobre princípios SOLID.

COMPOSIÇÃO

É natural pensar em objetos que contêm outros objetos. Uma televisão contém sintonizador e tela; um computador contém placa de vídeo, teclado e unidades de armazenamento. O computador é um objeto, mas uma unidade de disco também é um objeto independente que pode ser removido fisicamente.

Quando um objeto é construído a partir de outros objetos, temos composição.

Abstraction

Just as with inheritance, composition provides a mechanism for building objects. In fact, I would argue that there are only two ways to build classes from other classes: *inheritance* and *composition*. As we have seen, inheritance allows one class to inherit from another class. We can thus abstract out attributes and behaviors for common classes. For example, dogs and cats are both mammals because a dog *is-a* mammal and a cat *is-a* mammal. With composition, we can also build classes by embedding classes in other classes.

Consider the relationship between a car and an engine. The benefits of separating the engine from the car are evident. By building the engine separately, we can use the engine in various cars—not to mention other advantages. But we can’t say that an engine *is-a* car. This just doesn’t sound right when it rolls off the tongue (and because we are modeling real-world systems, this is the effect we want). Rather, we use the term *has-a* to describe composition relationships. A car *has-a(n)* engine.

Has-a Relationships

While inheritance is considered an *is-a* relationship, a composition relationship is termed a *has-a relationship*. Using the example in the previous section, a television *has-a* tuner and *has-a* video display. A television is obviously not a tuner, so there is no inheritance relationship. In the same vein, a computer *has-a* video card, *has-a* keyboard, and *has-a* disk drive. The topics of inheritance, composition, and how they relate to each other are covered in great detail in [Chapter 7](#), “[Mastering Inheritance and Composition](#).”

CONCLUSION

There is a lot to cover when discussing OO technologies. However, you should leave this chapter with a good understanding of the following topics:

- **Encapsulation**—Encapsulating the data and behavior into a single object is of primary importance in OO development. A single object contains both its data and behaviors and can hide what it wants from other objects.
- **Inheritance**—A class can inherit from another class and take advantage of the attributes and methods defined by the superclass.
- **Polymorphism**—Polymorphism means that similar objects can respond to the same message in different ways. For example, you might have a system with many shapes. However, a circle, a square, and a star are each drawn differently. Using polymorphism, you can send each of these shapes the same message (for example, `Draw`), and each shape is responsible for drawing itself.
- **Composition**—Composition means that an object is built from other objects.

This chapter covers the fundamental OO concepts, of which by now you should have a good grasp.

ABSTRAÇÃO

Assim como a herança, a composição oferece um mecanismo para construir classes a partir de outras classes. Em termos gerais, há duas maneiras centrais de realizar isso: herança e composição.

Com herança, uma classe recebe elementos de outra, permitindo abstrair atributos e comportamentos comuns. Um cão é um mamífero e um gato é um mamífero: essa é uma relação “é-um” (is-a).

Com composição, classes são construídas incorporando objetos de outras classes. Considere um carro e um motor. É vantajoso construir o motor separadamente, pois o mesmo tipo de motor pode ser usado em diferentes carros. Entretanto, não faz sentido dizer que “um motor é um carro”. A relação correta é: um carro tem um motor.

RELAÇÕES “TEM-UM” (HAS-A)

Enquanto herança representa uma relação is-a, composição representa uma relação has-a. Uma televisão tem um sintonizador e tem uma tela; ela não é um sintonizador. Um computador tem uma placa de vídeo, tem um teclado e tem uma unidade de disco. Herança, composição e a relação entre ambas são discutidas em profundidade no Capítulo 7.

CONCLUSÃO

Há muito conteúdo envolvido em tecnologias OO, mas ao terminar este capítulo você deve compreender os seguintes conceitos:

- Encapsulamento - dados e comportamentos são reunidos em um único objeto. O objeto contém seu estado e suas operações e pode ocultar detalhes de outros objetos.
- Herança - uma classe pode herdar de outra e aproveitar atributos e métodos definidos na superclasse.
- Polimorfismo - objetos relacionados podem responder de maneiras diferentes à mesma mensagem. Em um sistema de formas, por exemplo, Circle, Square e Star podem receber Draw/draw e cada um desenhar a si próprio de maneira apropriada.
- Composição - um objeto pode ser construído a partir de outros objetos.

Este capítulo apresentou os conceitos fundamentais de orientação a objetos, que servirão de base para o restante do livro.

2. How to Think in Terms of Objects

In [Chapter 1](#), “[Introduction to Object-Oriented Concepts](#),” you learned the fundamental object-oriented (OO) concepts. The rest of the book delves more deeply into these concepts and introduces several others. Many factors go into a good design, whether it is an OO design or not. The fundamental unit of OO design is the class. The desired end result of OO design is a robust and functional object model—in other words, a complete system.

As with most things in life, there is no single right or wrong way to approach a problem. There are usually many ways to tackle the same problem. So when attempting to design an OO solution, don’t get hung up in trying to do a perfect design the first time (there will always be room for improvement). What you really need to do is brainstorm and let your thought process go in different directions. Do not try to conform to any standards or conventions when trying to solve a problem because the whole idea is to be creative.

In fact, at the start of the process, don’t even begin to consider a specific programming language. The first order of business is to identify and solve business problems. Work on the conceptual analysis and design first. Think about specific technologies only when they are fundamental to the business problem. For example, you can’t design a wireless network without wireless technology. However, it is often the case that you will have more than one software solution to consider.

Thus, before you start to design a system, or even a class, think the problem through and have some fun! In this chapter we explore the fine art and science of OO thinking.

Any fundamental change in thinking is not trivial. As a case in point, a lot has been mentioned about the move from structured to OO development. As was mentioned earlier, one side effect of this debate is the misconception that structured and object-oriented development are mutually exclusive. This is not the case. As we know from our discussion on wrappers, structured and object-oriented development coexist. In fact, when you write an OO application, you are using structured constructs everywhere. I have never seen a program, OO or otherwise, that does not use loops, if-statements, and so on. Yet making the switch to OO design does require a different type of investment.

Changing from FORTRAN to COBOL, or even to C, requires you to learn a new language; however, making the move from COBOL to C++, C# .NET, Visual Basic .NET, Objective-C, Swift, or Java requires you to learn a new thought process. This is where the overused phrase *OO paradigm* rears its ugly head. When moving to an OO language, you must first go through the investment of learning OO concepts and the corresponding thought process. If this paradigm shift does not take place, one of two things will happen: Either the project will not truly be OO in nature (for example, it will use C++ without using OO constructs) or the project will be a complete object-disoriented mess.

Three important things you can do to develop a good sense of the OO thought process are covered in this chapter:

- Knowing the difference between the interface and implementation
- Thinking more abstractly
- Giving the user the minimal interface possible

2. COMO PENSAR EM TERMOS DE OBJETOS

No Capítulo 1, “[Introdução aos Conceitos Orientados a Objetos](#)”, você conheceu os conceitos fundamentais de orientação a objetos. O restante do livro aprofunda esses conceitos e apresenta vários outros. Muitos fatores contribuem para um bom projeto, seja ele OO ou não. A unidade fundamental do projeto OO é a classe; o resultado desejado é um modelo de objetos robusto e funcional - em outras palavras, um sistema completo.

Como acontece com a maioria dos problemas da vida, raramente existe uma única maneira absolutamente certa ou errada de chegar a uma solução. Normalmente há diversas alternativas. Portanto, ao projetar uma solução OO, não fique preso à ideia de obter um projeto perfeito na primeira tentativa. Sempre haverá espaço para melhoria. Faça brainstorming, permita que seu raciocínio siga diferentes caminhos e, durante a exploração inicial, não se limite prematuramente a padrões ou convenções. A ideia é ser criativo.

No começo do processo, nem sequer escolha uma linguagem de programação específica. A primeira tarefa é identificar e resolver os problemas do negócio. Faça primeiro a análise e o projeto conceituais. Tecnologias específicas só devem entrar cedo quando forem parte fundamental do próprio problema - por exemplo, não se projeta uma rede sem fio ignorando a tecnologia sem fio. Em muitos problemas de software, porém, existem diversas soluções tecnológicas possíveis.

Assim, antes de projetar um sistema ou até uma classe, pense cuidadosamente sobre o problema e aproveite o processo. Este capítulo explora a arte e a ciência de pensar de forma orientada a objetos.

Uma mudança fundamental de maneira de pensar não é trivial. Muito se falou sobre a passagem do desenvolvimento estruturado para OO, e um efeito colateral desse debate foi a falsa ideia de que as duas abordagens são mutuamente excludentes. Não são. Como vimos ao falar de wrappers, desenvolvimento estruturado e orientado a objetos convivem. Ao escrever uma aplicação OO, usamos construções estruturadas o tempo todo: laços, ifs e assim por diante. Mesmo assim, adotar projeto OO exige um investimento mental diferente.

Trocar FORTRAN por COBOL, ou até por C, exige aprender uma nova linguagem. Passar de COBOL para C++, C# .NET, Visual Basic .NET, Objective-C, Swift ou Java também exige aprender uma nova forma de pensar. É aí que aparece a expressão “paradigma OO”. Se essa mudança de paradigma não ocorrer, uma de duas coisas costuma acontecer: o projeto não será realmente OO, ou se transformará em uma confusão de objetos mal projetados.

Três práticas importantes para desenvolver o pensamento OO são tratadas neste capítulo:

- conhecer a diferença entre interface e implementação;
- pensar de forma mais abstrata;
- fornecer ao usuário a menor interface possível.

We have already touched on some of these concepts in [Chapter 1](#), “[Introduction to Object-Oriented Concepts](#),” and we now go into much more detail.

KNOWING THE DIFFERENCE BETWEEN THE INTERFACE AND THE IMPLEMENTATION

As we saw in [Chapter 1](#), one of the keys to building a strong OO design is to understand the difference between the interface and the implementation. Thus, when designing a class, what the user needs to know and, perhaps of more importance, what the user does not need to know are of vital importance. The data hiding mechanism inherent with encapsulation is the means by which nonessential data is hidden from the user.

Caution

Do not confuse the concept of the interface with terms like *graphical user interface* (GUI). Although a GUI is, as its name implies, an interface, the term *interfaces*, as used here, is more general in nature and is not restricted to a graphical interface.

Remember the toaster example in [Chapter 1](#)? The toaster, or any appliance for that matter, is plugged into the interface, which is the electrical outlet—see [Figure 2.1](#). All appliances gain access to the required electricity by complying with the correct interface: the electrical outlet. The toaster doesn’t need to know anything about the implementation or how the electricity is produced. For all the toaster cares, a coal plant or a nuclear plant could produce the electricity—the appliance does not care which, as long as the interface works as specified, correctly and safely.

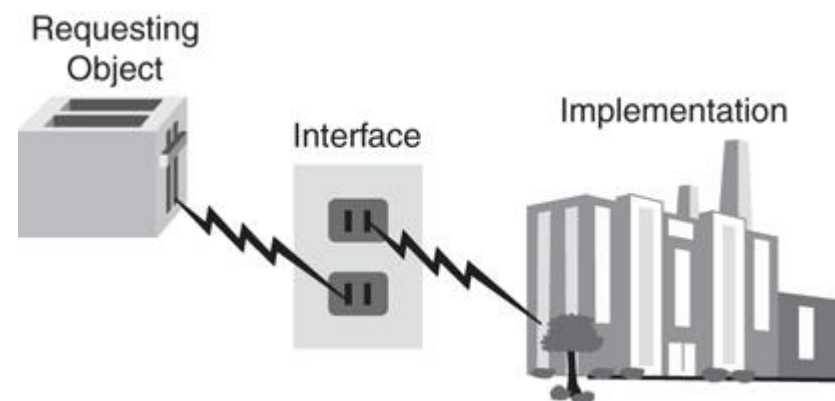


Figure 2.1 Power plant revisited.

As another example, consider an automobile. The interface between you and the car includes components such as the steering wheel, gas pedal, brake, and ignition switch. For most people, aesthetic issues aside, the main concern when driving a car is that the car starts, accelerates, stops, steers, and so on. The implementation, basically the stuff that you don’t see, is of little concern to the average driver. In fact, most people would not even be able to identify certain components, such as the catalytic converter and gasket. However, any driver would recognize and know how to use the steering wheel because this is a common interface. By installing a standard steering wheel in the car, manufacturers are assured that the people in their target market will be able to use the system.

If, however, a manufacturer decided to install a joystick in place of the steering wheel, most drivers would balk at this, and the automobile might not be a big seller (except possibly gamers). On the other hand, as long as the performance and aesthetics didn’t change, the average driver

Alguns desses conceitos já foram introduzidos no Capítulo 1; agora serão examinados com muito mais profundidade.

CONHECENDO A DIFERENÇA ENTRE INTERFACE E IMPLEMENTAÇÃO

Uma das chaves para construir um bom projeto OO é entender a diferença entre interface e implementação. Ao projetar uma classe, é essencial decidir o que o usuário precisa saber e, talvez ainda mais importante, o que ele não precisa saber. O mecanismo de ocultação de dados oferecido pelo encapsulamento permite esconder informações não essenciais.

CUIDADO

Não confunda o conceito de interface usado aqui com uma interface gráfica do usuário (GUI). Uma GUI é de fato uma interface, mas o termo empregado neste capítulo é mais geral e se refere principalmente à forma como classes e objetos expõem seus serviços.

Lembre-se do exemplo da torradeira do Capítulo 1. A torradeira, como qualquer aparelho, conecta-se a uma interface: a tomada elétrica. Todos os aparelhos obtêm eletricidade obedecendo à mesma interface. A torradeira não precisa saber como a eletricidade é produzida. Pode vir de uma usina a carvão, nuclear ou de outra fonte; o aparelho só exige que a interface se comporte de acordo com a especificação, de forma correta e segura.

Figura 2.1 - O exemplo da usina revisitado.

Considere também um automóvel. A interface entre motorista e carro inclui volante, acelerador, freio e ignição. Para a maioria dos motoristas, o importante é que o carro dê partida, acelere, pare e faça curvas. Os detalhes internos - grande parte da implementação - são pouco relevantes. Muitos motoristas nem saberiam identificar componentes como conversor catalítico ou certas juntas. Em contraste, todos reconhecem o volante porque ele é uma interface comum e padronizada.

Se um fabricante substituísse o volante por um joystick, grande parte dos usuários provavelmente rejeitaria a mudança. Já uma alteração interna no motor poderia passar despercebida, desde que o comportamento visível do automóvel permanecesse equivalente.

would not notice whether the manufacturer changed the engine (part of the implementation) of the automobile.

It must be stressed that the interchangeable engines must be identical in every way—as far as the interface goes. Replacing a four-cylinder engine with an eight-cylinder engine would change the rules and likely would not work with other components that interface with the engine, just as changing the current from AC to DC would affect the rules in the power plant example.

The engine is part of the implementation, and the steering wheel is part of the interface. A change in the implementation should have no impact on the driver, whereas a change to the interface might. The driver would notice an aesthetic change to the steering wheel, even if it performs in a similar manner. It must be stressed that a change to the engine that *is*noticeable by the driver breaks this rule. For example, a change that would result in noticeable loss of power is actually impacting the interface.

What Users See

When we talk about users in this chapter, we primarily mean designers and developers—not necessarily end users. Thus, when we talk about interfaces in this context, we are talking about class interfaces, not GUIs.

Properly constructed classes are designed in two parts—the interface and the implementation.

The Interface

The services presented to an end user constitute the interface. In the best case, *only* the services the end user needs are presented. Of course, which services the user needs might be a matter of opinion. If you put 10 people in a room and ask each of them to do an independent design, you might receive 10 totally different designs—and there is nothing wrong with that. However, as a general rule, the interface to a class should contain only what the user needs to know. In the toaster example, the user needs to know only that the toaster must be plugged into the interface (which in this case is the electrical outlet) and how to operate the toaster itself.

Identifying the User

Perhaps the most important consideration when designing a class is identifying the audience, or users, of the class.

The Implementation

The implementation details are hidden from the user. One goal regarding the implementation should be kept in mind: A change to the implementation *should not* require a change to the user’s code. This might seem a bit confusing, but this goal is at the heart of the design issue.

Good Interfaces

If the interface is designed properly, a change to the implementation should not require a change to the user’s code.

Remember that the interface includes the syntax to call a method and return a value. If this interface does not change, the user does not care whether the implementation is changed. As long as the programmer can use the same syntax and retrieve the same value, that’s all that matters.

We see this all the time when using a cell phone. To make a call, the interface is simple—we either dial a number or select an entry in the contact list. Yet, if the provider updates the

É importante enfatizar que componentes intercambiáveis de implementação precisam obedecer exatamente às mesmas regras da interface. Substituir um motor de quatro cilindros por um de oito pode mudar requisitos e interações com outros componentes. Da mesma forma, no exemplo elétrico, trocar corrente alternada por contínua mudaria o contrato da interface.

O motor pertence à implementação; o volante pertence à interface. Uma alteração de implementação idealmente não afeta o motorista, enquanto uma alteração de interface pode exigir adaptação. Até uma mudança estética do volante pode ser percebida. Da mesma forma, se uma alteração no motor provocar perda de potência perceptível, ela deixa de ser invisível ao usuário e passa a afetar a interface observada.

O QUE OS USUÁRIOS VEEM

Quando falamos em “usuários” neste capítulo, estamos pensando principalmente em projetistas e desenvolvedores que utilizarão uma classe, e não necessariamente no usuário final de uma aplicação. Portanto, “interface” aqui significa sobretudo interface de classe, não GUI.

Classes bem construídas podem ser vistas em duas partes: interface e implementação.

A INTERFACE

Os serviços apresentados ao usuário constituem a interface. Idealmente, somente os serviços de que ele realmente necessita devem ser oferecidos. Naturalmente, quais serviços são “necessários” pode ser questão de projeto; dez projetistas independentes podem propor dez soluções diferentes e várias delas podem ser válidas. Como regra geral, porém, a interface de uma classe deve conter apenas o que o usuário precisa saber.

No exemplo da torradeira, basta ao usuário saber como ligar o aparelho à tomada e como operá-lo.

IDENTIFICANDO O USUÁRIO

Talvez a consideração mais importante ao projetar uma classe seja identificar corretamente seu público, isto é, quem usará a classe.

A IMPLEMENTAÇÃO

Os detalhes de implementação ficam ocultos. Um objetivo fundamental é que uma mudança interna de implementação não obrigue o usuário da classe a mudar seu código. Esse princípio está no centro do projeto de boas interfaces.

BOAS INTERFACES

Se a interface foi projetada adequadamente, uma mudança de implementação não deve exigir mudança no código do cliente.

A interface inclui, por exemplo, a sintaxe usada para chamar um método e o tipo de valor devolvido. Se esse contrato permanece igual, o cliente não precisa saber que o algoritmo interno mudou.

O celular oferece um exemplo cotidiano: para fazer uma chamada, discamos um número ou selecionamos um contato. O provedor pode atualizar o software interno sem alterar esse procedimento.

software, it doesn’t change the way you make a call. The interface stays the same regardless of how the implementation changes. However, I can think of one situation when the provider did change the interface—when my area code changed. Fundamental interface changes, like an area code change, do require the users to change behavior. Businesses try to keep these types of changes to a minimum, for some customers will not like the change or perhaps not put up with the hassle.

Recall that in the toaster example, although the interface is always the electric outlet, the implementation could change from a coal power plant to a nuclear power plant without affecting the toaster. One very important caveat should be made here: The coal or nuclear plant must also conform to the interface specification. If the coal plant produces AC power but the nuclear plant produces DC power, a problem exists. The bottom line is that both the user and the implementation must conform to the interface specification.

An Interface/Implementation Example

Let’s create a simple (if not very functional) database reader class. We’ll write some Java code that will retrieve records from the database. As we’ve discussed, knowing your end users is always the most important issue when doing any kind of design. You should do some analysis of the situation and conduct interviews with end users, and then list the requirements for the project. The following are some requirements we might want to use for the database reader:

- We must be able to open a connection to the database.
- We must be able to close the connection to the database.
- We must be able to position the cursor on the first record in the database.
- We must be able to position the cursor on the last record in the database.
- We must be able to find the number of records in the database.
- We must be able to determine whether there are more records in the database (that is, if we are at the end).
- We must be able to position the cursor at a specific record by supplying the key.
- We must be able to retrieve a record by supplying a key.
- We must be able to get the next record, based on the position of the cursor.

With these requirements in mind, we can make an initial attempt to design the database reader class by creating possible interfaces for these end users.

In this case, the database reader class is intended for programmers who require use of a database. Thus, the interface is essentially the application-programming interface (API) that the programmer will use. These methods are, in effect, wrappers that enclose the functionality provided by the database system. Why would we do this? We explore this question in much greater detail later in the chapter; the short answer is that we might need to customize some database functionality. For example, we might need to process the objects so that we can write them to a relational database. Writing this *middleware* is not trivial as far as design and coding go, but it is a real-life example of wrapping functionality. More important, we may want to change the database engine itself without having to change the code.

Figure 2.2 shows a class diagram representing a possible interface to the `DataBaseReader` class.

A interface para fazer uma ligação permanece a mesma apesar de mudanças na implementação. Uma exceção seria uma alteração visível, como a mudança do código de área: nesse caso o usuário precisa adaptar seu comportamento. Empresas tentam minimizar mudanças desse tipo porque elas geram custo, incômodo e resistência.

No exemplo da torradeira, a tomada continua sendo a mesma interface mesmo que a geração de energia mude de uma usina a carvão para uma nuclear. Porém, ambas as implementações precisam cumprir a especificação da interface. Se uma produzisse corrente alternada e outra corrente contínua sem adaptação, haveria incompatibilidade. Em resumo, tanto o usuário quanto a implementação precisam obedecer ao mesmo contrato.

UM EXEMPLO DE INTERFACE/IMPLEMENTAÇÃO

Vamos imaginar uma classe simples para leitura de banco de dados. O código Java deverá recuperar registros, mas antes de programar é necessário conhecer os usuários e levantar requisitos. Alguns requisitos possíveis são:

- abrir uma conexão com o banco;
- fechar a conexão;
- posicionar o cursor no primeiro registro;
- posicionar o cursor no último registro;
- obter a quantidade de registros;
- verificar se ainda existem registros a percorrer;
- posicionar o cursor em um registro específico por meio de uma chave;
- recuperar um registro fornecendo uma chave;
- obter o próximo registro com base na posição atual.

Com esses requisitos, podemos começar a projetar interfaces para a classe de leitura.

Neste caso, os usuários são programadores que precisam acessar um banco de dados. A interface é essencialmente uma API. Seus métodos funcionam como wrappers para funcionalidades fornecidas pelo sistema de banco. Isso pode parecer trabalho extra, mas permite personalizar o acesso, adaptar objetos ao modelo relacional e, principalmente, substituir o mecanismo de banco de dados sem obrigar o código da aplicação a mudar.

A Figura 2.2 mostra um possível diagrama de classe para `DataBaseReader`.

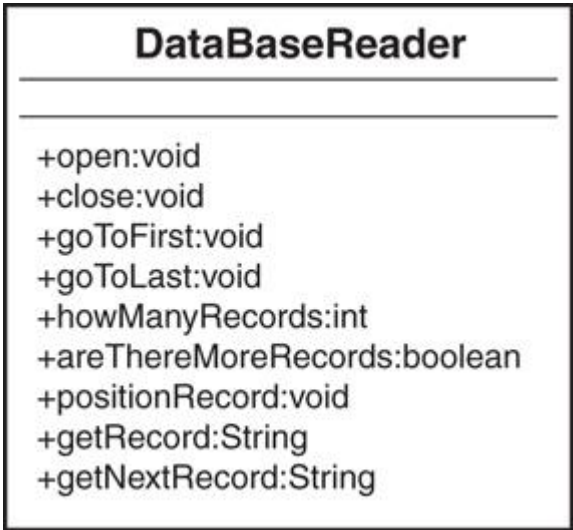


Figure 2.2 A Unified Modeling Language class diagram for

the `DataReader` class.

Note that the methods in this class are all public (remember that there are plus signs next to the names of methods that are public interfaces). Also note that only the interface is represented; the implementation is not shown. Take a minute to determine whether this class diagram generally satisfies the requirements outlined earlier for the project. If you find out later that the diagram does not meet all the requirements, that’s okay; remember that OO design is an iterative process, so you do not have to get it exactly right the first time.

Public Interface

Remember, an application programmer can access it, and thus, it is considered part of the class interface. Do not confuse the term *interface* with the keyword `interface` used in Java and .NET—which is discussed in later chapters.

For each of the requirements we listed, we need a corresponding method that provides the functionality we want. Now you need to ask a few questions:

- To effectively use this class, do you, as a programmer, need to know anything else about it?
- Do you need to know how the internal database code opens the database?
- Do you need to know how the internal database code physically positions itself over a specific record?
- Do you need to know how the internal database code determines whether any more records are left?

On all counts the answer is a resounding *no!* You don’t need to know any of this information. All you care about is that you get the proper return values and that the operations are performed correctly. In fact, the application programmer will most likely be at least one more abstract level away from the implementation. The application will use your classes to open the database, which in turn will invoke the proper database API.

Minimal Interface

Although perhaps extreme, one way to determine the minimalist interface is to initially provide the user no public interfaces. Of course, the class will be useless; however, this forces the user to come back to you and say, “Hey, I need this functionality.” Then you can negotiate. Thus, you add interfaces only when it is requested. Never assume that the user needs something.

Figura 2.2 - Diagrama UML de uma possível classe `DataReader`.

Observe que os métodos do diagrama são públicos, indicados pelo sinal +. Somente a interface é representada; a implementação interna não aparece. Verifique se o conjunto de métodos atende, de maneira geral, aos requisitos listados anteriormente. Se depois descobrirmos alguma lacuna, isso não representa fracasso: projeto OO é iterativo e pode ser refinado.

INTERFACE PÚBLICA

Um programador de aplicação pode acessar os membros públicos e, por isso, eles fazem parte da interface da classe. Não confunda esse uso da palavra interface com a palavra-chave interface de Java ou .NET, discutida em capítulos posteriores.

Para cada requisito, precisamos de algum método que ofereça a funcionalidade correspondente. Então devemos perguntar:

- para usar a classe, o programador precisa conhecer algo além da interface?
- precisa saber como o código interno abre o banco?
- precisa saber como o cursor é posicionado fisicamente sobre um registro?
- precisa saber como a implementação determina se existem registros restantes?

Em todos esses casos, a resposta é não. O cliente se preocupa com os valores retornados e com o fato de as operações funcionarem corretamente. Na prática, o programador da aplicação estará em um nível de abstração ainda mais alto: sua aplicação chama a nossa classe, e a nossa classe utiliza a API apropriada do banco de dados.

INTERFACE MÍNIMA

Uma forma extrema de descobrir a menor interface seria começar oferecendo nenhum método público. A classe ficaria inútil, mas o usuário teria de pedir explicitamente as funcionalidades de que realmente precisa. A ideia por trás da provocação é importante: adicione interfaces quando existir uma necessidade concreta, em vez de presumir que o usuário precisa de tudo.

Creating wrappers might seem like overkill, but there are many advantages to writing them. To illustrate, there are many middleware products on the market today. Consider the problem of mapping objects to a relational database. OO databases have never caught on; however, theoretically they may be perfect for OO applications. However, one small problem exists: Most companies have years of data in legacy relational database systems. How can a company embrace OO technologies and stay on the cutting edge while retaining its data in a relational database?

First, you can convert all your legacy, relational data to a brand-new OO database. However, anyone who has suffered the acute (and chronic) pain of any data conversion knows that this is to be avoided at all costs. Although these conversions can take large amounts of time and effort, all too often they never work properly.

Second, you can use a middleware product to seamlessly map the objects in your application code to a relational model. This is a much better solution since relational databases are so prevalent. Some might argue that OO databases are much more efficient for object persistence than relational databases. In fact, many development systems seamlessly provide this service.

Object Persistence

Object persistence refers to the concept of saving the state of an object so that it can be restored and used at a later time. An object that does not persist basically dies when it goes out of scope. For example, the state of an object can be saved in a database.

However, in the current business environment, relational-to-object mapping is a great solution. Many companies have integrated these technologies. It is common for a company to have a website front-end interface with data on a mainframe.

If you create a totally OO system, an OO database might be a viable (and better performing) option; however, OO databases have not experienced anywhere near the growth that OO languages have.

Standalone Application

Even when creating a new OO application from scratch, it might not be easy to avoid legacy data. Even a newly created OO application is most likely not a standalone application and might need to exchange information stored in relational databases (or any other data storage device, for that matter).

Let’s return to the database example. **Figure 2.2** shows the public interface to the class, and nothing else. When this class is complete, it will probably contain more methods, and it will certainly contain attributes. However, as a programmer using this class, you do not need to know anything about these private methods and attributes. You certainly don’t need to know what the code looks like within the public methods. You simply need to know how to interact with the interfaces.

What would the code for this public interface look like (assume that we start with a Oracle database example)? Let’s look at the `open()` method:

Click here to view code image

```
public void open(String Name){

    /* Some application-specific processing */

    /* call the Oracle API to open the database */

    /* Some more application-specific processing */
```

Criar wrappers pode parecer exagero, mas há muitas vantagens. Um exemplo é o uso de middleware para mapear objetos de uma aplicação OO para um banco de dados relacional. Bancos de dados OO nunca se popularizaram na mesma proporção das linguagens OO, enquanto a maioria das empresas acumulou anos de dados em sistemas relacionais legados. Como adotar tecnologias OO sem abandonar esse patrimônio?

Uma possibilidade seria converter todos os dados relacionais para um novo banco orientado a objetos. Quem já enfrentou uma conversão de dados sabe que ela pode consumir enorme tempo e esforço e ainda produzir resultados problemáticos.

Uma alternativa mais prática é usar middleware para mapear, de modo transparente, objetos da aplicação para um modelo relacional. Como bancos relacionais são extremamente difundidos, essa solução costuma ser muito mais conveniente. Há sistemas de desenvolvimento que oferecem esse mapeamento de forma integrada.

PERSISTÊNCIA DE OBJETOS

Persistência significa salvar o estado de um objeto para que ele possa ser restaurado e utilizado posteriormente. Um objeto não persistente desaparece, conceitualmente, quando deixa de existir no contexto de execução. Seu estado pode, por exemplo, ser armazenado em um banco de dados.

No ambiente empresarial atual, o mapeamento objeto-relacional é uma solução bastante útil. É comum uma empresa possuir uma interface web moderna enquanto os dados permanecem em um mainframe.

Em um sistema completamente OO, um banco de dados OO poderia ser uma alternativa viável e talvez eficiente, mas esse tipo de banco não teve crescimento comparável ao das linguagens orientadas a objetos.

APLICAÇÃO INDEPENDENTE

Mesmo ao criar uma aplicação OO do zero, pode ser difícil evitar dados legados. Uma aplicação nova raramente vive completamente isolada e pode precisar trocar informações com bancos relacionais ou outros mecanismos de armazenamento.

Voltando ao exemplo, a Figura 2.2 mostra apenas a interface pública. A classe completa provavelmente terá mais métodos e atributos, mas o programador cliente não precisa conhecer esses elementos privados nem o código existente dentro dos métodos públicos. Ele precisa apenas saber como usar a interface.

A página original mostra então o método `open(String Name)`, inicialmente implementado como um wrapper sobre a API de um banco Oracle. O código é preservado na página à esquerda.

```
};
```

In this case, you, wearing your programmer’s hat, realize that the `open` method requires `String` as a parameter. `Name`, which represents a database file, is passed in, but it’s not important to explain how `Name` is mapped to a specific database for this example. That’s all we need to know. Now comes the fun stuff—what really makes interfaces so great!

Just to annoy our users, let’s change the database implementation. Last night we translated all the data from an Oracle database to an SQLAnywhere database (we endured the acute and chronic pain). It took us hours—but we did it.

Now the code looks like this:

[Click here to view code image](#)

```
public void open(String Name){

    /* Some application-specific processing

    /* call the SQLAnywhere API to open the database */

    /* Some more application-specific processing */

};
```

To our great chagrin, this morning not one user complained. This is because even though the implementation changed, the interface did not! As far as the user is concerned, the calls are still the same. The code change for the implementation might have required quite a bit of work (and the module with the one-line code change would have to be rebuilt), but not one line of application code that uses this `DataBaseReader` class needed to change.

Code Recompilation

Dynamically loaded classes are loaded at runtime—not statically linked into an executable file. When using dynamically loaded classes, like Java and .NET do, no user classes would have to be recompiled. However, in statically linked languages such as C++, a link is required to bring in the new class.

By separating the user interface from the implementation, we can save a lot of headaches down the road. In [Figure 2.3](#), the database implementations are transparent to the end users, who see only the interface.

Como programador cliente, basta saber que `open()` recebe uma `String` que representa o nome do banco ou recurso a abrir. Para este exemplo, não é necessário saber como esse nome é mapeado para um banco específico.

Agora imagine que a implementação do banco seja substituída: os dados são migrados de Oracle para SQLAnywhere. A implementação interna de `open()` muda para chamar uma API diferente, mas sua assinatura pública continua igual.

No dia seguinte, nenhum usuário da classe precisa reclamar ou alterar seu código, porque a interface não mudou. A migração pode ter exigido muito trabalho internamente e o módulo que contém a classe pode precisar ser reconstruído, mas o código das aplicações que usam `DataBaseReader` continua chamando `open()` da mesma maneira.

RECOMPILAÇÃO DE CÓDIGO

Classes carregadas dinamicamente são incorporadas em tempo de execução, em vez de serem ligadas estaticamente a um executável. Em ambientes de carregamento dinâmico, como Java e .NET, classes clientes podem não precisar ser recompiladas após uma substituição compatível. Em linguagens com ligação estática, como ocorre em muitos cenários C++, pode ser necessário realizar uma nova etapa de linkedição.

Separar interface e implementação evita muitos problemas futuros. Na Figura 2.3, as diferentes implementações de banco de dados ficam transparentes para os usuários, que enxergam somente a interface.

Figura 2.3 - A interface.

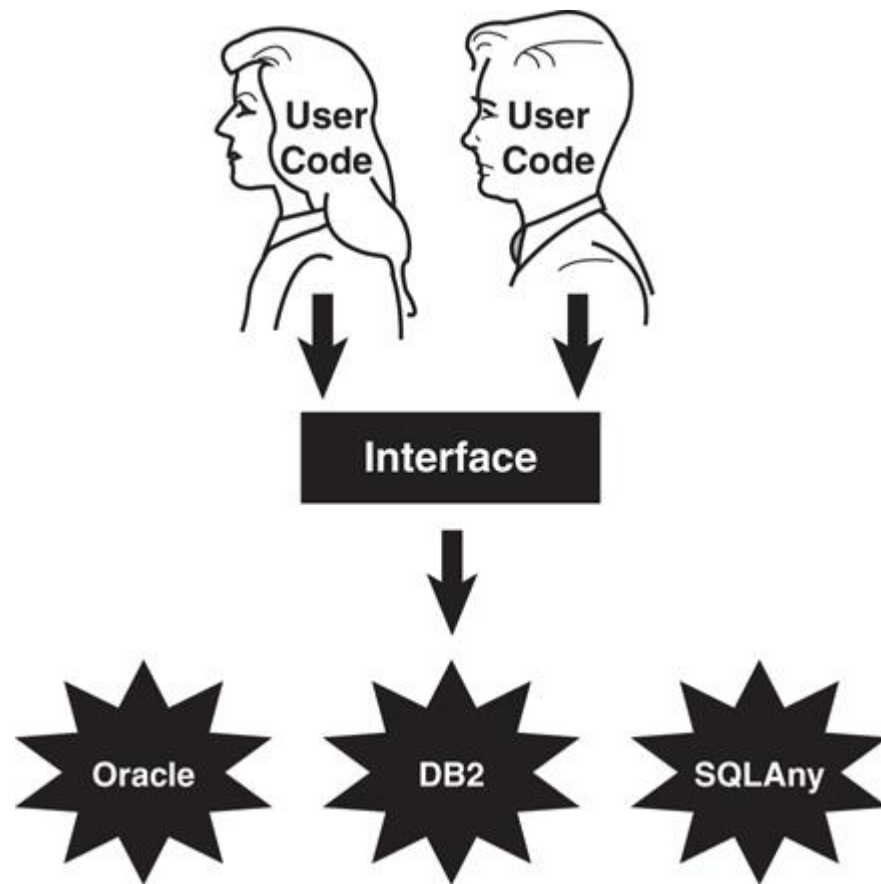


Figure 2.3 The interface.

USING ABSTRACT THINKING WHEN DESIGNING INTERFACES

One of the main advantages of OO programming is that classes can be reused. In general, reusable classes tend to have interfaces that are more abstract than concrete. Concrete interfaces tend to be very specific, whereas abstract interfaces are more general. However, simply stating that a highly abstract interface is more useful than a highly concrete interface, although often true, is not always the case.

It is possible to write a very useful, concrete class that is not at all reusable. This happens all the time, and nothing is wrong with it in some situations. However, we are now in the design business and want to take advantage of what OO offers us. So our goal is to design abstract, highly reusable classes—and to do this we will design highly abstract user interfaces. To illustrate the difference between an abstract and a concrete interface, let's create a taxi object. It is much more useful to have an interface such as “drive me to the airport” than to have separate interfaces such as “turn right,” “turn left,” “start,” “stop,” and so on, because as illustrated in [Figure 2.4](#), all the user wants to do is get to the airport.

Uma das grandes vantagens da programação OO é a possibilidade de reutilizar classes. Em geral, classes reutilizáveis tendem a possuir interfaces mais abstratas do que concretas. Interfaces concretas são muito específicas; interfaces abstratas expressam intenções mais gerais. Entretanto, dizer que uma interface abstrata é sempre melhor seria incorreto.

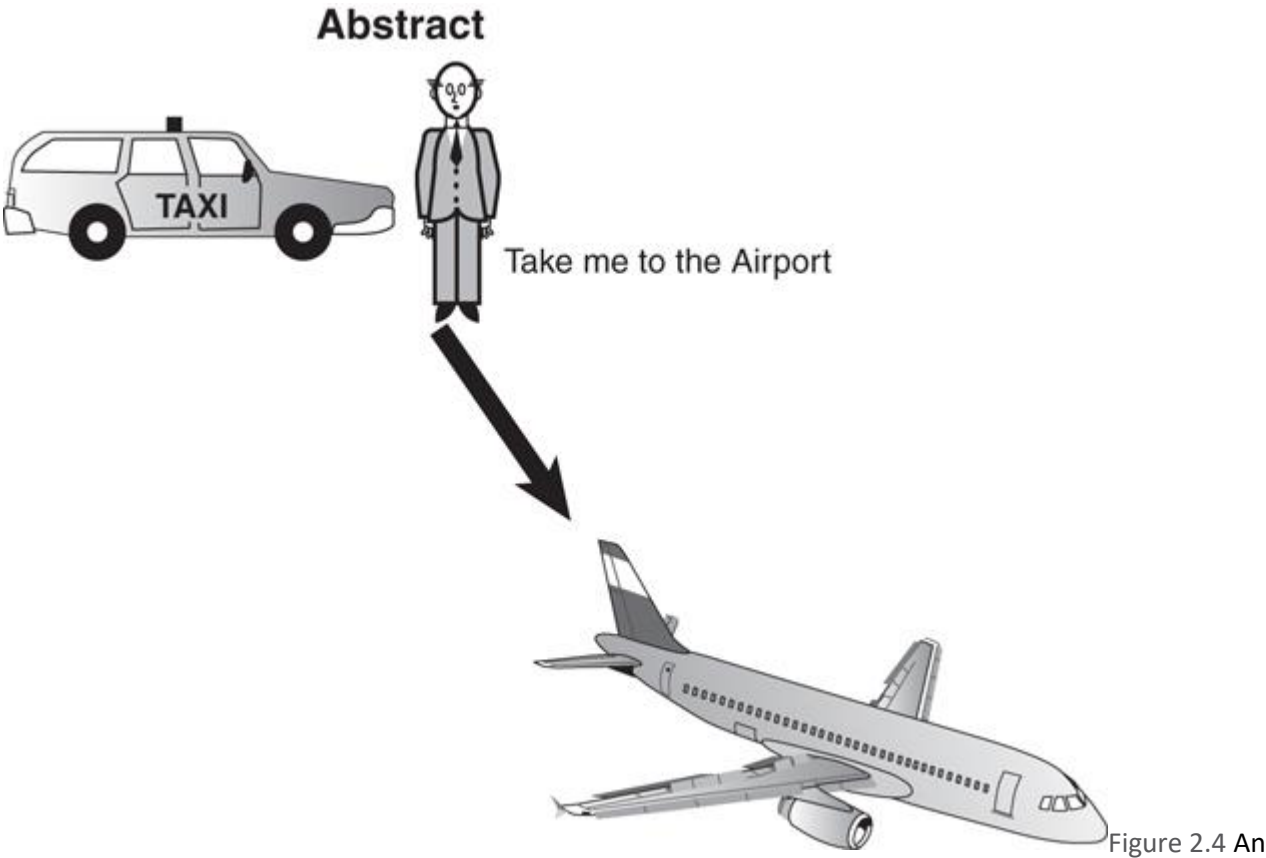
É perfeitamente possível escrever uma classe concreta extremamente útil e praticamente não reutilizável, e isso pode ser adequado em muitas situações. Porém, quando o objetivo é aproveitar a reutilização oferecida por OO, procuramos projetar classes e interfaces com um bom nível de abstração.

Considere um objeto Taxi. É muito mais útil oferecer uma operação como “leve-me ao aeroporto” do que obrigar o passageiro a usar operações separadas como “vire à direita”, “vire à esquerda”, “ande”, “pare” e assim por diante. Como mostra a Figura 2.4, o passageiro quer chegar ao aeroporto; não quer controlar cada detalhe do trajeto.

Figura 2.4 - Uma interface abstrata.

Ao sair do hotel, colocar as malas no táxi e entrar no carro, o motorista pergunta para onde você quer ir. Você responde: “Por favor, leve-me ao aeroporto”. Se houver mais de um aeroporto importante, é claro que será preciso especificar qual deles.

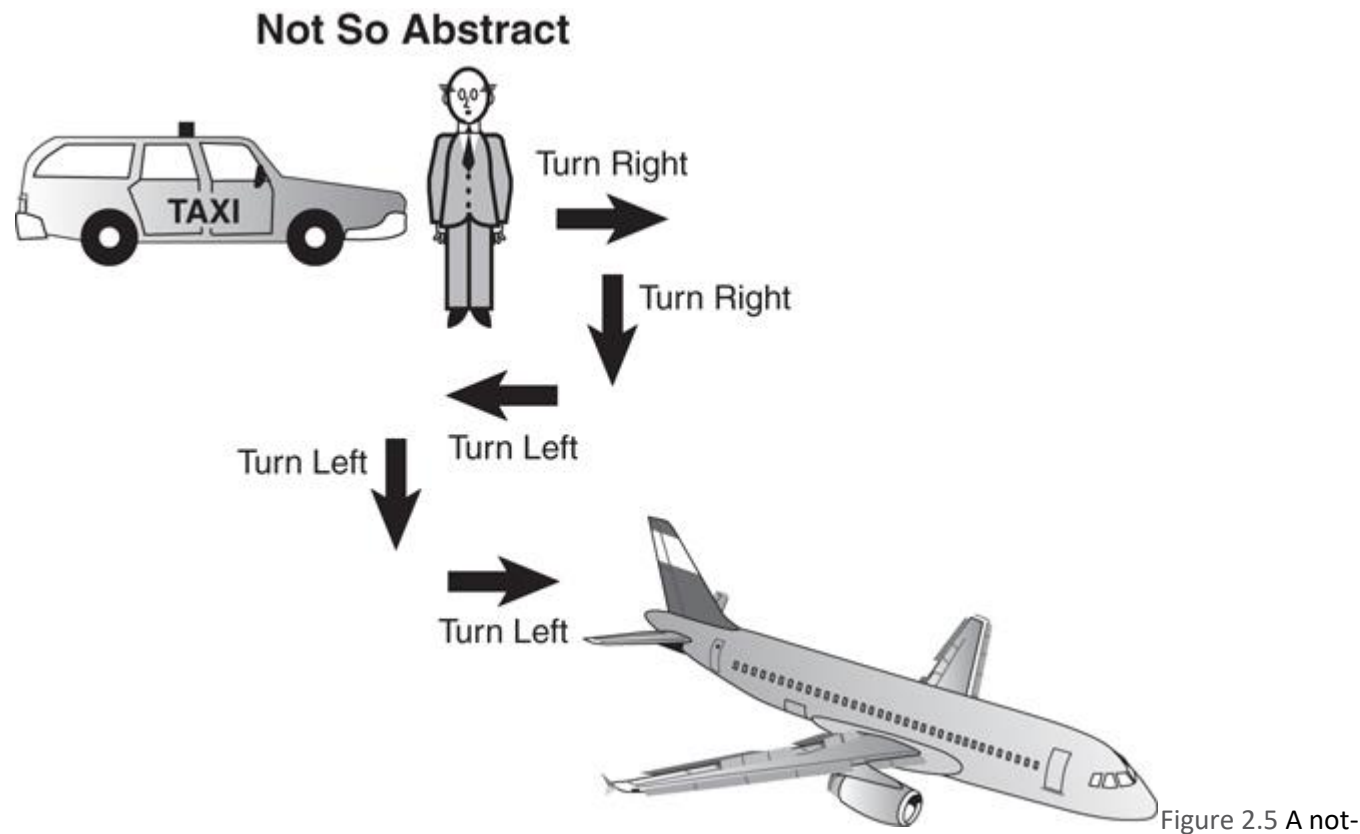
Talvez você nem saiba o caminho e, mesmo que saiba, não quer dizer ao motorista cada momento em que deve virar e para qual direção, como na Figura 2.5. A maneira como o motorista implementa o trajeto não é responsabilidade do passageiro. O passageiro se preocupa com o destino - e talvez com a tarifa, se o motorista escolher propositalmente um caminho muito longo.



abstract interface.

When you emerge from your hotel, throw your bags into the back seat of the taxi, and get in, the cabbie will turn to you and ask, “Where do you want to go?” You reply, “Please take me to the airport.” (This assumes, of course, that there is only one major airport in the city. In Chicago you would have to say, “Please take me to Midway Airport” or “Please take me to O’Hare.”) You might not even know how to get to the airport yourself, and even if you did, you wouldn’t want to have to tell the cabbie when to turn and which direction to turn, as illustrated in [Figure 2.5](#). How the cabbie implements the actual drive is of no concern to you, the passenger. (However, the fare might become an issue at some point, if the cabbie cheats and takes you the long way to the airport.)

Figura 2.5 - Uma interface pouco abstrata.



so-abstract interface.

Now, where does the connection between abstract and reuse come in? Ask yourself which of these two scenarios is more reusable, the abstract or the not-so-abstract? To put it more simply, which phrase is more reusable: “Take me to the airport,” or “Turn right, then right, then left, then left, then left”? Obviously, the first phrase is more reusable. You can use it in any city, whenever you get into a taxi and want to go to the airport. The second phrase will work only in a specific case. Thus, the abstract interface “Take me to the airport” is generally the way to go for a good, reusable OO design whose implementation would be different in Chicago, New York, or Cleveland.

PROVIDING THE ABSOLUTE MINIMAL USER INTERFACE POSSIBLE

When designing a class, the general rule is to always provide the user with as little knowledge of the inner workings of the class as possible. To accomplish this, follow these simple rules:

- Give the users only what they absolutely need. In effect, this means the class has as few interfaces as possible. When you start designing a class, start with a minimal interface. The design of a class is iterative, so you will soon discover that the minimal set of interfaces might not suffice. This is fine.
- It is better to have to add interfaces because users really need it than to give the users more interfaces than they need. At times it is highly problematic for the user to have access to certain interfaces. For example, you don’t want an interface that provides salary information to all users—only the ones who need to know.
- For the moment, let’s use a hardware example to illustrate our software example. Imagine handing a user a PC box without a monitor or a keyboard. Obviously, the PC would be of little use. You have just provided the user with the minimal set of interfaces to

Onde aparece a ligação entre abstração e reutilização? Compare as frases “Leve-me ao aeroporto” e “Vire à direita, depois à direita, depois à esquerda...”. A primeira é muito mais reutilizável. Você pode usá-la em praticamente qualquer cidade. A segunda funciona apenas para um trajeto específico.

Por isso, uma interface abstrata como “Leve-me ao aeroporto” costuma ser uma boa escolha em um projeto OO reutilizável: a implementação concreta pode ser diferente em Chicago, Nova York ou Cleveland, mas a intenção expressa pelo cliente permanece igual.

FORNECENDO A MENOR INTERFACE DE USUÁRIO POSSÍVEL

Ao projetar uma classe, uma regra geral é fornecer ao usuário o mínimo possível de conhecimento sobre seu funcionamento interno. Algumas orientações:

- Dê aos usuários apenas o que eles realmente precisam. Isso significa manter o menor número razoável de interfaces públicas. Comece com uma interface mínima e amplie-a conforme o projeto iterativo revelar necessidades reais.
- É melhor adicionar uma interface porque o usuário realmente precisa dela do que disponibilizar antecipadamente operações desnecessárias. Em alguns casos, oferecer acesso demais é perigoso. Informações salariais, por exemplo, não deveriam estar disponíveis a todos.
- Um exemplo de hardware ajuda a ilustrar a ideia: imagine entregar ao usuário apenas o gabinete de um computador, sem monitor nem teclado. O sistema seria pouco útil; isso mostra que “mínimo” não significa “insuficiente”, e sim fornecer exatamente as interfaces necessárias à tarefa.

the PC. However, this minimal set is insufficient, and it immediately becomes necessary to add interfaces.

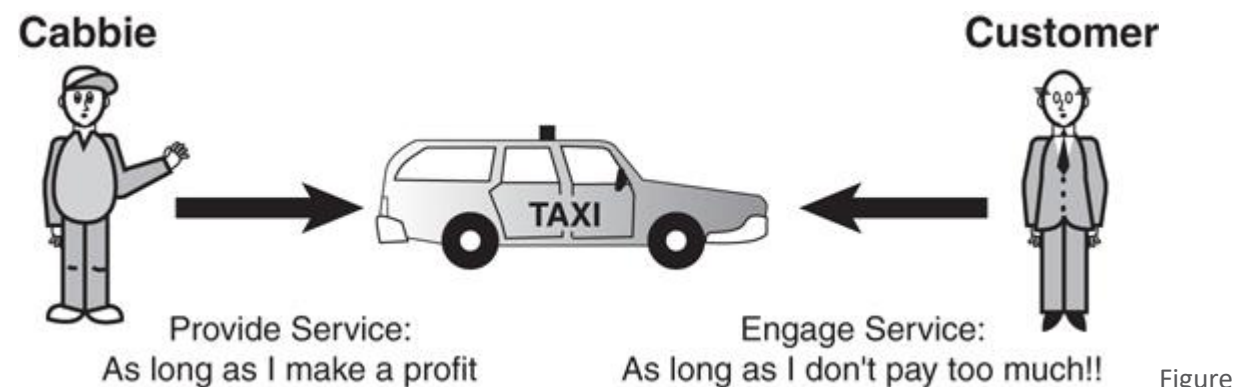
- Public interfaces define what the users can access. If you initially hide the entire class from the user by making the interfaces private, when programmers start using the class, you will be forced to make certain methods public—these methods thus become the public interface.
- It is vital to design classes from a user's perspective and not from an information systems viewpoint. Too often designers of classes (not to mention any other kind of software) design the class to make it fit into a specific technological model. Even if the designer takes a user's perspective, it is still probably a technician user's perspective, and the class is designed with an eye on getting it to work from a technology standpoint and not from ease of use for the user.
- Make sure when you are designing a class that you go over the requirements and the design with the people who will actually use it—not just developers (this includes all levels of testing). The class will most likely evolve and need to be updated when a prototype of the system is built.

Determining the Users

Let's look again at the taxi example. We have already decided that the users are the ones who will actually use the system. This said, the obvious question is, who are the users?

The first impulse is to say the *customers*. This is only about half right. Although the customers are certainly users, the cabbie must be able to successfully provide the service to the customers. In other words, providing an interface that would, no doubt, please the customer, such as "Take me to the airport for free," is not going to go over well with the cabbie. Thus, in reality, to build a realistic and usable interface, *both* the customer and the cabbie must be considered users.

In short, any object that sends a message to the taxi object is considered a user (and yes, the users are objects, too). **Figure 2.6** shows how the cabbie provides a service.



2.6 Providing services.

Looking Ahead

The cabbie is most likely an object as well.

Object Behavior

Identifying the users is only a part of the exercise. After the users are identified, you must determine the behaviors of the objects. From the viewpoint of all the users, begin identifying the

O conjunto mínimo do exemplo anterior era insuficiente, portanto novas interfaces precisariam ser acrescentadas.

- Interfaces públicas definem aquilo que os usuários podem acessar. Se toda a classe começa oculta, com métodos `private`, à medida que programadores tentam utilizá-la você será obrigado a tornar alguns métodos `public`; esses métodos passam a compor a interface pública.
- É fundamental projetar classes da perspectiva do usuário, e não apenas da perspectiva do sistema ou da tecnologia. Projetistas frequentemente moldam a classe para encaixá-la em um modelo tecnológico específico. Mesmo quando consideram "o usuário", às vezes pensam apenas no usuário técnico e acabam priorizando a facilidade de implementação em vez da facilidade de uso.
- Ao projetar uma classe, revise requisitos e projeto com as pessoas que realmente a utilizarão, não apenas com desenvolvedores. Isso inclui diferentes níveis de teste. A classe provavelmente evoluirá e precisará ser atualizada quando um protótipo for construído.

DETERMINANDO OS USUÁRIOS

Voltemos ao exemplo do táxi. Já decidimos que usuários são aqueles que realmente utilizam o sistema. Mas quem são eles?

A primeira resposta costuma ser "os passageiros". Isso está apenas parcialmente correto. O passageiro certamente é usuário, mas o motorista também precisa conseguir prestar o serviço. Uma interface que agradaria ao passageiro, como "leve-me ao aeroporto de graça", provavelmente não seria aceitável para o motorista. Portanto, para construir uma interface realista, devemos considerar ambos.

Em resumo, qualquer objeto que envie uma mensagem ao objeto Taxi pode ser considerado usuário. E, naturalmente, esses usuários também podem ser objetos.

Figura 2.6 - Prestação de serviços.

OLHANDO ADIANTE

O motorista provavelmente também será modelado como um objeto.

COMPORTAMENTO DO OBJETO

Identificar os usuários é apenas parte da tarefa. Depois disso, é preciso determinar os comportamentos dos objetos. A partir do ponto de vista de todos os usuários, devemos identificar o propósito de cada objeto e o que ele precisa fazer para funcionar corretamente.

purpose of each object and what it must do to perform properly. Note that many of the initial choices will not survive the final cut of the public interface. These choices are identified by gathering requirements using various methods such as UML Use Cases.

Environmental Constraints

In their book *Object-Oriented Design in Java*, Gilbert and McCarty point out that the environment often imposes limitations on what an object can do. In fact, environmental constraints are almost always a factor. Computer hardware might limit software functionality. For example, a system might not be connected to a network, or a company might use a specific type of printer. In the taxi example, the cab cannot drive on a road if a bridge is out, even if it provides a quicker way to the airport.

Identifying the Public Interfaces

With all the information gathered about the users, the object behaviors, and the environment, you need to determine the public interfaces for each user object. So think about how you would use the taxi object:

- Get into the taxi.
- Tell the cabbie where you want to go.
- Pay the cabbie.
- Give the cabbie a tip.
- Get out of the taxi.

What do you need to do to use the taxi object?

- Have a place to go.
- Hail a taxi.
- Pay the cabbie money.

Initially, you think about how the object is used and not how it is built. You might discover that the object needs more interfaces, such as “Put luggage in the trunk” or “Enter into a mindless conversation with the cabbie.” **Figure 2.7** provides a class diagram that lists possible methods for the `Cabbie` class.

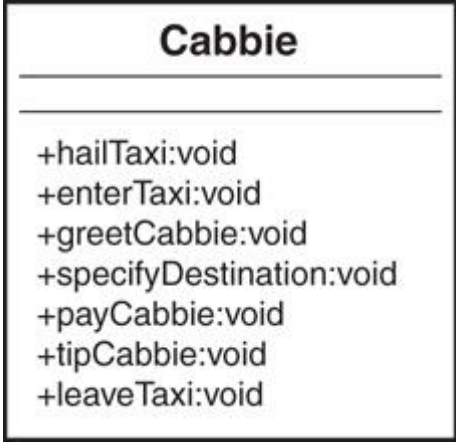


Figure 2.7 The methods in a `Cabbie` class.

As is always the case, nailing down the final interface is an iterative process. For each interface, you must determine whether the interface contributes to the operation of the object. If it does not, perhaps it is not necessary. Many OO texts recommend that each interface model only one

Muitas escolhas iniciais não permanecerão na versão final da interface pública. Elas são descobertas durante o levantamento de requisitos por diferentes técnicas, entre elas casos de uso UML.

RESTRIÇÕES DO AMBIENTE

O ambiente frequentemente impõe limites ao que um objeto pode fazer. Hardware pode restringir funcionalidades de software; um sistema pode não ter conexão de rede; uma empresa pode exigir determinado tipo de impressora. No exemplo do táxi, o carro não pode seguir por uma estrada se uma ponte estiver interditada, mesmo que esse caminho fosse o mais curto para o aeroporto.

IDENTIFICANDO AS INTERFACES PÚBLICAS

Com as informações sobre usuários, comportamentos e ambiente, determine as interfaces públicas necessárias. Pense em como um passageiro usa um táxi:

- entra no táxi;
- informa ao motorista o destino;
- paga a corrida;
- pode dar gorjeta;
- sai do táxi.

Para usar o serviço, o passageiro precisa ainda:

- ter um destino;
- conseguir chamar um táxi;
- possuir uma forma de pagamento.

Primeiro pense em como o objeto será usado, e não em como será construído. Durante o refinamento, podem surgir interfaces adicionais, como guardar bagagem no porta-malas ou iniciar uma conversa com o motorista.

Figura 2.7 - Possíveis métodos de uma classe `Cabbie`.

Definir a interface final é um processo iterativo. Para cada operação pública, pergunte se ela realmente contribui para a utilização do objeto. Se não contribui, talvez não seja necessária. Muitos textos de OO recomendam que cada interface represente um único comportamento.

behavior. This returns us to the question of how abstract we want to get with the design. If we have an interface called `enterTaxi()`, we certainly do not want `enterTaxi()` to have logic in it to pay the cabbie. If we do this, not only is the design somewhat illogical, but there is virtually no way that a user of the class can tell what has to be done to pay the cabbie.

Identifying the Implementation

After the public interfaces are chosen, you need to identify the implementation. After the class is designed and all the methods required to operate the class properly are in place, the specifics of how to get the class to work are considered.

Technically, anything that is not a public interface can be considered the implementation. This means that the user will never see any of the methods that are considered part of the implementation, including the method’s signature (which includes the name of the method and the parameter list), as well as the actual code inside the method.

It is possible to have a private method that is used internally by the class. Any private method is considered part of the implementation given that the user will never see it and thus will not have access to it. For example, a class may have a `changePassword()` method; however, the same class may have a private method that encrypts the password. This method would be hidden from the user and called only from inside the `changePassword()` method.

The implementation is totally hidden from the user. The code within public methods is a part of the implementation because the user cannot see it. (The user should see only the calling structure of an interface—not the code inside it.)

This means that, theoretically, anything that is considered the implementation might change without affecting how the user interfaces with the class. This assumes, of course, that the implementation is providing the answers the user expects.

Whereas the interface represents how the user sees the object, the implementation is really the nuts and bolts of the object. The implementation contains the code that represents that state of an object.

CONCLUSION

In this chapter, we have explored three areas that can get you started on the path to thinking in an OO way. Remember that there is no firm list of issues pertaining to the OO thought process. Doing things in an OO way is more of an art than a science. Try to think of your own ways to describe OO thinking.

In [Chapter 3](#), “[More Object-Oriented Concepts](#),” we discuss the object life cycle: it is born, it lives, and it dies. While it is alive, it might transition through many states. For example, a `DataBaseReader` object is in one state if the database is open and another state if the database is closed. How this is represented depends on the design of the class.

REFERENCES

Fowler, Martin. 2003. *UML Distilled*, Third Edition. Boston, MA: Addison-Wesley Professional.

Gilbert, Stephen, and Bill McCarty. 1998. *Object-Oriented Design in Java*. Berkeley, CA: The Waite Group Press (Pearson Education).

Meyers, Scott. 2005. *Effective C++*, Third Edition. Boston, MA: Addison-Wesley Professional.

Isso nos devolve à pergunta sobre o nível adequado de abstração. Se existir uma operação `enterTaxi()`, não faria sentido esconder dentro dela também a lógica de pagamento. Além de produzir um projeto pouco intuitivo, o usuário da classe não teria uma forma clara de descobrir o que precisa fazer para pagar o motorista.

IDENTIFICANDO A IMPLEMENTAÇÃO

Depois de escolher as interfaces públicas, passamos à implementação. Com a classe projetada e os métodos necessários definidos, consideramos os detalhes específicos de como fazê-la funcionar.

Tecnicamente, tudo o que não pertence à interface pública pode ser considerado implementação. Isso inclui métodos privados, suas assinaturas e o código que executam. O usuário da classe não precisa enxergar esses detalhes.

É perfeitamente possível ter métodos privados usados apenas internamente. Uma classe pode oferecer publicamente `changePassword()`, por exemplo, e utilizar internamente um método `private` para criptografar a senha. O método de criptografia fica oculto e é chamado apenas pelo código da própria classe.

A implementação é escondida do usuário. Até o código dentro de métodos públicos faz parte da implementação; o cliente precisa conhecer apenas a estrutura necessária para chamar a operação, não o código que existe dentro dela.

Em teoria, qualquer detalhe de implementação pode mudar sem alterar a forma como o cliente utiliza a classe, desde que os resultados e o contrato permaneçam corretos.

A interface representa como o usuário enxerga o objeto; a implementação representa seus mecanismos internos, os “parafusos e engrenagens” que mantêm e transformam seu estado.

CONCLUSÃO

Neste capítulo foram exploradas três ideias importantes para desenvolver o pensamento OO: distinguir interface de implementação, pensar de forma abstrata e oferecer interfaces mínimas. Não existe uma lista rígida que defina o pensamento OO; projetar dessa forma é tanto uma arte quanto uma ciência. Procure também criar suas próprias maneiras de raciocinar sobre objetos.

No Capítulo 3 será discutido o ciclo de vida de um objeto: ele nasce, vive e morre. Durante sua existência, pode passar por diferentes estados. Um `DataBaseReader`, por exemplo, está em um estado quando o banco está aberto e em outro quando está fechado. A representação desses estados depende do projeto da classe.

REFERÊNCIAS

Fowler, Martin. UML Distilled, 3ª ed. Addison-Wesley Professional, 2003.
Gilbert, Stephen; McCarty, Bill. Object-Oriented Design in Java. Waite Group Press, 1998.
Meyers, Scott. Effective C++, 3ª ed. Addison-Wesley Professional, 2005.

3. More Object-Oriented Concepts

Chapter 1, “Introduction to Object-Oriented Concepts,” and **Chapter 2, “How to Think in Terms of Objects,”** cover the basics of object-oriented (OO) concepts. Before we embark on our journey to learn some of the finer design issues relating to building an OO system, we need to cover a few more advanced OO concepts, such as constructors, operator overloading, and multiple inheritance. We also will consider error-handling techniques and how scope applies to object-oriented design.

Some of these concepts might not be vital to understanding an OO design at a higher level, but they are necessary to anyone involved in the design and implementation of an OO system.

CONSTRUCTORS

Constructors may be a new concept for structured programmers. Although constructors are not normally used in non-OO languages such as COBOL, C, and Basic, the *struct*, which is part of C/C++, does include constructors. In the first two chapters we alluded to these special methods that are used to *construct* objects. In some OO languages, such as Java and C#, constructors are methods that share the same name as the class. Visual Basic .NET uses the designation *New* and Swift uses the *init* keyword. As usual, we will focus on the concepts of constructors and not cover the specific syntax of all the languages. Let’s take a look at some Java code that implements a constructor.

For example, a constructor for the `Cabbie` class we covered in **Chapter 2** would look like this:

```
public Cabbie(){
    /* code to construct the object */
}
```

The compiler will recognize that the method name is identical to the class name and consider the method a constructor.

Caution

Note that in this Java code (as with C# and C++), a constructor does not have a return value. If you provide a return value, the compiler will not treat the method as a constructor.

For example, if you include the following code in the class, the compiler will not consider this a constructor because it has a return value—in this case an integer.

```
public int Cabbie(){
    /* code to construct the object */
}
```

This syntax requirement can cause problems because this code will compile but will not behave as expected.

When Is a Constructor Called?

When a new object is created, one of the first things that happens is that the constructor is called. Check out the following code:

```
Cabbie myCabbie = new Cabbie();
```

3. MAIS CONCEITOS ORIENTADOS A OBJETOS

Os Capítulos 1 e 2 apresentaram os fundamentos de orientação a objetos. Antes de avançar para questões mais refinadas de projeto de sistemas OO, precisamos abordar alguns conceitos adicionais, como construtores, sobrecarga de operadores e herança múltipla. Também serão consideradas técnicas de tratamento de erros e a maneira como escopo se aplica ao projeto orientado a objetos.

Alguns desses assuntos talvez não sejam essenciais para compreender um modelo OO em nível muito alto, mas são importantes para quem participa do projeto e da implementação de um sistema.

CONSTRUTORES

Construtores podem ser novidade para programadores vindos de linguagens estruturadas. Eles não são normalmente usados da mesma maneira em linguagens não OO, como COBOL, C e BASIC, embora estruturas e extensões de C/C++ ofereçam mecanismos relacionados. Nos capítulos anteriores já mencionamos esses métodos especiais usados para construir objetos.

Em algumas linguagens OO, como Java e C#, o construtor possui o mesmo nome da classe. Visual Basic .NET usa a designação `New` e Swift usa `init`. Como de costume, o foco aqui é o conceito, não a sintaxe específica de todas as linguagens.

Para a classe `Cabbie` do Capítulo 2, um construtor Java teria a forma mostrada no código original: `public Cabbie() { ... }`. O compilador reconhece que o nome do método coincide com o nome da classe e o trata como construtor.

CUIDADO

Em Java, C# e C++, um construtor não declara tipo de retorno. Se você acrescentar um tipo, o compilador deixa de tratá-lo como construtor. Um método como `public int Cabbie()` pode até compilar, mas não terá o comportamento esperado de construção do objeto.

QUANDO UM CONSTRUTOR É CHAMADO?

Quando um novo objeto é criado, uma das primeiras ações é a execução do construtor. A expressão `Cabbie myCabbie = new Cabbie();` cria uma nova instância e aciona o construtor correspondente.

The `new` keyword creates a new instance of the `Cabbie` class, thus allocating the required memory. Then the constructor itself is called, passing the arguments in the parameter list. The constructor provides the developer the opportunity to attend to the appropriate initialization. Thus, the code `new Cabbie()` will instantiate a `Cabbie` object and call the `Cabbie` method, which is the constructor.

What’s Inside a Constructor?

Perhaps the most important function of a constructor is to initialize the memory allocated when the `new` keyword is encountered. In short, code included inside a constructor should set the newly created object to its initial, stable, safe state.

For example, if you have a counter object with an attribute called `count`, you need to set `count` to zero in the constructor:

```
count = 0;
```

Initializing Attributes

In structured programming, a routine named housekeeping (or initialization) is often used for initialization purposes. Initializing attributes is a common function performed within a constructor. Regardless, don't rely on the system defaults.

The Default Constructor

If you write a class and do not include a constructor, the class will still compile, and you can still use it. If the class provides no explicit constructor, a default constructor will be provided. It is important to understand that at least one constructor always exists, regardless of whether you write a constructor yourself. If you do not provide a constructor, the system will provide a default constructor for you.

Besides the creation of the object itself, the only action that a default constructor takes is to call the constructor of its superclass. In many cases, the superclass will be part of the language framework, like the `Object` class in Java. For example, if a constructor is not provided for the `Cabbie` class, the following default constructor is inserted:

```
public Cabbie(){
    super();
}
```

If you were to decompile the bytecode produced by the compiler, you would see this code. The compiler actually inserts it.

In this case, if `Cabbie` does not explicitly inherit from another class, the `Object` class will be the parent class. Perhaps the default constructor might be sufficient in some cases; however, in most cases some sort of memory initialization should be performed. Regardless of the situation, it is good programming practice to always include at least one constructor in a class. If there are attributes in the class, it is always good practice to initialize them. Moreover, initializing variables is always a good practice when writing code, object-oriented or not.

Providing a Constructor

The general rule is that you should always provide a constructor, even if you do not plan to do anything inside it. You can provide a constructor with nothing in it and then add to it later. Although there is technically nothing wrong with using the default constructor provided by the compiler, for documentation and maintenance purposes, it is always nice to know exactly what your code looks like.

It is not surprising that maintenance becomes an issue here. If you depend on the default constructor and then subsequent maintenance adds another constructor, the default constructor

A palavra-chave `new` cria uma nova instância da classe `Cabbie` e aloca a memória necessária. Em seguida, o construtor é chamado, recebendo os argumentos existentes na lista de parâmetros. O construtor oferece ao desenvolvedor a oportunidade de realizar a inicialização apropriada. Portanto, `new Cabbie()` instancia um objeto `Cabbie` e executa seu construtor.

O QUE HÁ DENTRO DE UM CONSTRUTOR?

Talvez a função mais importante do construtor seja inicializar a memória e o estado associados ao objeto recém-criado. Em resumo, o código do construtor deve colocar o objeto em um estado inicial estável e seguro.

Se existir, por exemplo, um objeto contador com um atributo `count`, o construtor pode definir `count = 0`.

INICIALIZAÇÃO DE ATRIBUTOS

Na programação estruturada, uma rotina de housekeeping ou initialization costuma cumprir funções semelhantes. Em OO, inicializar atributos é uma tarefa comum do construtor. Não dependa cegamente dos valores padrão fornecidos pelo sistema.

O CONSTRUTOR PADRÃO

Se uma classe for escrita sem declarar explicitamente um construtor, ela ainda poderá compilar e ser usada: o compilador ou ambiente fornece um construtor padrão. Portanto, existe sempre pelo menos um mecanismo de construção, mesmo quando nenhum construtor foi escrito manualmente.

Além de criar o objeto, o construtor padrão normalmente chama o construtor da superclasse. Em Java, se `Cabbie` não declarar outro pai, `Object` é a superclasse e um construtor equivalente a `Cabbie() { super(); }` é inserido pelo compilador.

Em alguns casos esse comportamento pode ser suficiente, mas frequentemente é necessário inicializar atributos explicitamente. Como prática de programação, é recomendável fornecer pelo menos um construtor e inicializar o estado da classe de maneira deliberada. Inicializar variáveis é boa prática em código OO ou não OO.

FORNECENDO UM CONSTRUTOR

Como regra geral, forneça um construtor mesmo que inicialmente ele não faça nada. É possível começar com um corpo vazio e ampliá-lo depois. Embora usar o construtor padrão do compilador não seja tecnicamente errado, declarar o construtor torna o código e a intenção mais claros para documentação e manutenção.

Esse detalhe também importa na manutenção: se o código depende de um construtor padrão e, mais tarde, outro construtor explícito é acrescentado, o comportamento disponível pode mudar. A continuação é discutida na próxima página.

is no longer created. This may actually break code that was assuming the presence of a default constructor.

Always remember that the default constructor is added only if you don't include any constructors. As soon as you include just one, the default constructor is not provided.

Using Multiple Constructors

In many cases, an object can be constructed in more than one way. To accommodate this situation, you need to provide more than one constructor. For example, let's consider the `Count` class presented here:

```
public class Count {  
  
    int count;  
  
    public Count() {  
        count = 0;  
    }  
}
```

On the one hand, we want to initialize the attribute `count` to count to zero: We can easily accomplish this by having a constructor initialize `count` to zero as follows:

```
public Count() {  
    count = 0;  
}
```

On the other hand, we might want to pass an initialization parameter that allows `count` to be set to various numbers:

```
public Count (int number) {  
    count = number;  
}
```

This is called *overloading a method* (overloading pertains to all methods, not just constructors). Most OO languages provide functionality for overloading a method.

Overloading Methods

Overloading allows a programmer to use the same method name over and over, as long as the signature of the method is different each time. The signature consists of the method name and a parameter list (see [Figure 3.1](#)).

Signature

```
public String getRecord(int key)
```

Signature = `getRecord` `(int key)`
 method name + parameter list

Figure 3.1 The components of a signature.

Thus, the following methods *all* have different signatures:

```
public void getCab();
```

```
// different parameter list  
public void getCab (String cabbieName);
```

```
// different parameter list  
public void getCab (int numberOfPassengers);
```

Signatures

O construtor padrão deixa de ser criado assim que você declara qualquer construtor explicitamente. Isso pode quebrar código que dependia da existência automática de um construtor sem parâmetros.

Lembre-se: o construtor padrão é acrescentado somente quando a classe não declara nenhum construtor. Ao declarar apenas um, ele não será mais fornecido automaticamente.

USANDO MÚLTIPLOS CONSTRUTORES

Em muitos casos, um objeto pode ser construído de mais de uma maneira. Para acomodar essas situações, a classe pode oferecer vários construtores.

Considere uma classe `Count` com um atributo `count`. Um construtor sem parâmetros pode inicializar `count` com 0. Outro construtor pode receber um número e inicializar `count` com esse valor. Os códigos correspondentes permanecem preservados no original.

Esse recurso é chamado de sobrecarga de método (overloading), e não se limita a construtores. Muitas linguagens OO permitem que vários métodos compartilhem o mesmo nome desde que possuam assinaturas diferentes.

SOBRECARGA DE MÉTODOS

Sobrecarga permite reutilizar um mesmo nome de método várias vezes, desde que a assinatura seja diferente em cada versão. A assinatura é formada pelo nome do método e por sua lista de parâmetros, como ilustra a Figura 3.1.

Figura 3.1 - Componentes de uma assinatura.

Assim, `getCab()`, `getCab(String cabbieName)` e `getCab(int numberOfPassengers)` possuem assinaturas distintas porque suas listas de parâmetros são diferentes.

Depending on the language, the signature may or may not include the return type. In Java and C#, the return type is not part of the signature. For example, the following methods would conflict even though the return types are different:

```
public void getCab (String cabbieName);
public int  getCab (String cabbieName);
```

The best way to understand signatures is to write some code and run it through the compiler.

By using different signatures, you can construct objects differently depending on the constructor used. This functionality is very helpful when you don’t always know ahead of time how much information you have available. For example, when creating a shopping cart, customers may already be logged in to their account (and you will have all of their information). On the other hand, a totally new customer may be placing items in the cart with no account information available at all. In each case the constructor would initialize differently.

Using UML to Model Classes

Let’s return to the database reader example we used earlier in [Chapter 2](#). Consider that we have two ways we can construct a database reader:

- Pass the name of the database and position the cursor at the beginning of the database.
- Pass the name of the database and the position within the database where we want the cursor to position itself.

[Figure 3.2](#) shows a class diagram for the `DataBaseReader` class. Note that the diagram lists two constructors for the class. Although the diagram shows the two constructors, without the parameter list, there is no way to know which constructor is which. To distinguish the constructors, you can look at the corresponding code in the `DataBaseReader` class listed next.

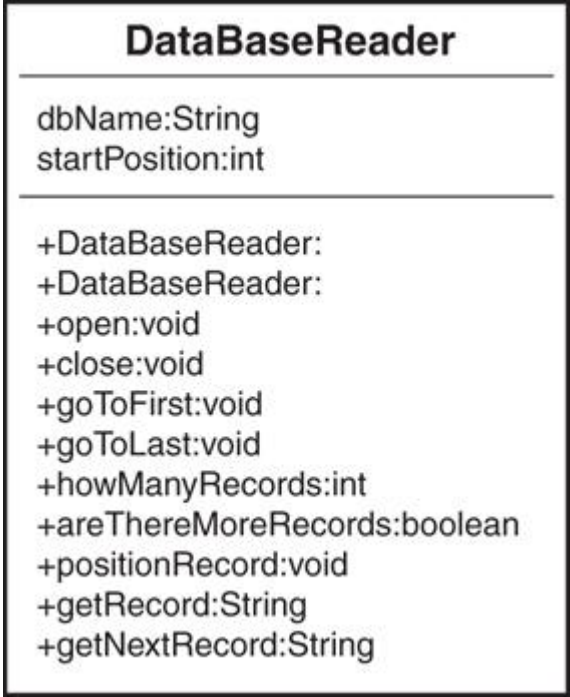


Figure 3.2 The `DataBaseReader` class diagram.

No Return Type

Notice that in this class diagram the constructors do not have a return type. All other methods besides constructors must have return types.

ASSINATURAS

Dependendo da linguagem, o tipo de retorno pode ou não fazer parte da assinatura. Em Java e C#, o tipo de retorno não participa da assinatura. Por isso, dois métodos com o mesmo nome e os mesmos parâmetros entram em conflito mesmo que devolvam tipos diferentes. A melhor maneira de compreender as regras é experimentar código e observar o compilador.

Com assinaturas diferentes, objetos podem ser construídos de formas distintas conforme o construtor chamado. Isso é útil quando nem sempre se sabe antecipadamente quanta informação estará disponível. Um carrinho de compras, por exemplo, pode ser criado para um cliente já autenticado, cujos dados estão disponíveis, ou para um visitante ainda sem conta. Cada construtor pode realizar inicializações diferentes.

USANDO UML PARA MODELAR CLASSES

Retomemos o exemplo `DataBaseReader`. Podemos imaginar duas formas de construí-lo:

- informar o nome do banco e iniciar o cursor no começo;
- informar o nome do banco e também uma posição inicial específica.

A [Figura 3.2](#) apresenta um diagrama da classe com dois construtores. Somente pelo nome não seria possível distingui-los; é a lista de parâmetros que identifica qual versão será chamada.

Figura 3.2 - Diagrama da classe `DataBaseReader`.

SEM TIPO DE RETORNO

Observe que, no diagrama, construtores não possuem tipo de retorno. Os demais métodos devem declarar o tipo de resultado conforme as regras da linguagem.

A página apresenta um trecho da classe DataBaseReader com dois construtores e os atributos que eles inicializam.

Figura 3.3 - Criando um novo objeto.

O primeiro construtor recebe apenas o nome do banco, define dbName e inicializa startPosition com 0. O segundo recebe nome e posição e inicializa ambos com os valores fornecidos.

Observe que startPosition recebe um valor nos dois casos. Quando a informação não é fornecida pela lista de parâmetros, o construtor escolhe um valor padrão apropriado, como 0.

O código Java completo é mantido na página original à esquerda.

Here is a code segment of the class that shows its constructors and the attributes that the constructors initialize (see [Figure 3.3](#)):

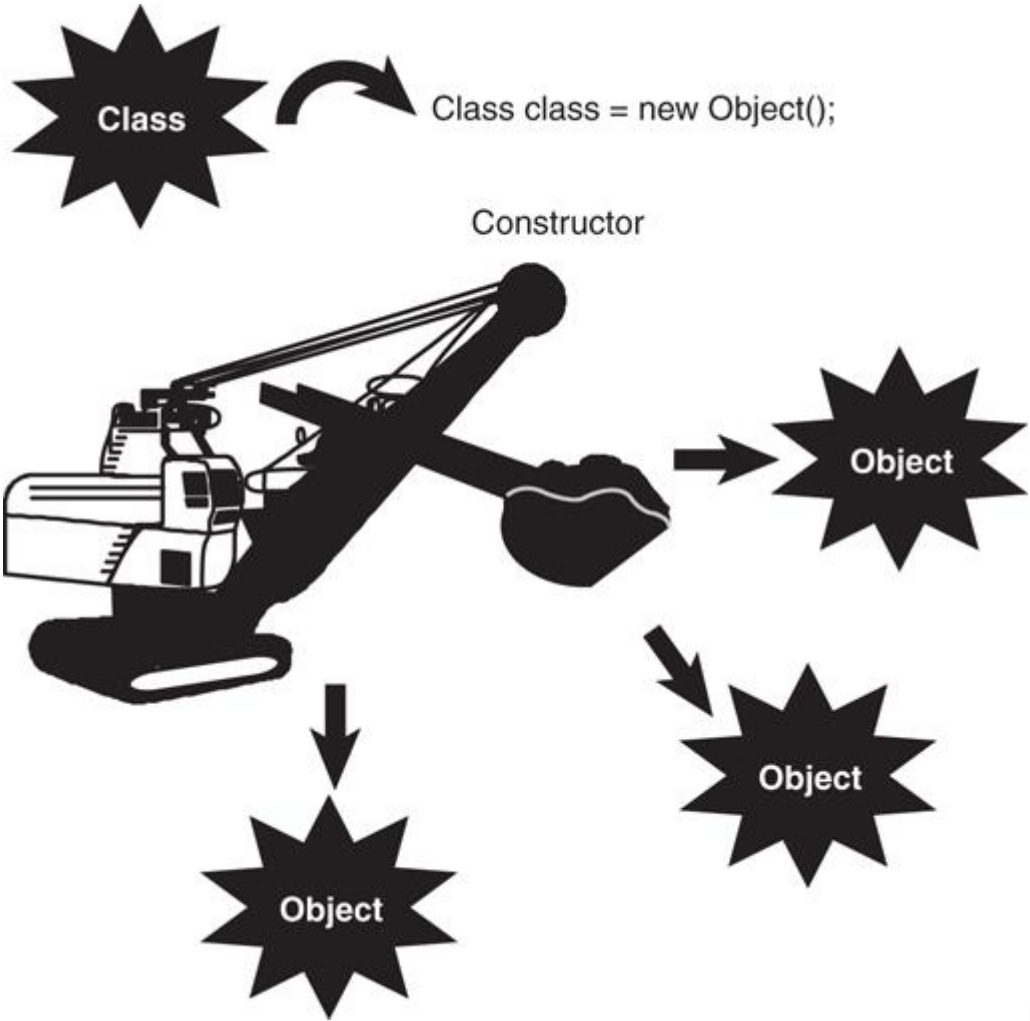


Figure 3.3 Creating a new

object.

[Click here to view code image](#)

```
public class DataBaseReader {  
  
    String dbName;  
    int startPosition;  
  
    // initialize just the name  
    public DataBaseReader (String name){  
        dbName = name;  
        startPosition = 0;  
    };  
  
    // initialize the name and the position  
    public DataBaseReader (String name, int pos){  
        dbName = name;  
        startPosition = pos;  
    };  
  
    .. // rest of class  
}
```

Note how startPosition is initialized in both cases. If the constructor is not passed the information via the parameter list, it is initialized to a default value, such as 0.

How the Superclass Is Constructed

When using inheritance, you must know how the parent class is constructed. Remember that when you use inheritance, you are inheriting everything about the parent. Thus, you must become intimately aware of all the parent's data and behavior. The inheritance of an attribute is fairly obvious; however, how a constructor is inherited is not as obvious. After the `new` keyword is encountered and the object is allocated, the following steps occur (see [Figure 3.4](#)):

1. Inside the constructor, the constructor of the class's superclass is called. If there is no explicit call to the superclass constructor, the default is called automatically; however, you can see the code in the bytecodes.
2. Each class attribute of the object is initialized. These are the attributes that are part of the class definition (instance variables), not the attributes inside the constructor or any other method (local variables). In the `DataBaseReader` code presented earlier, the `integer startPosition` is an instance variable of the class.
3. The rest of the code in the constructor executes.

Constructing an Object

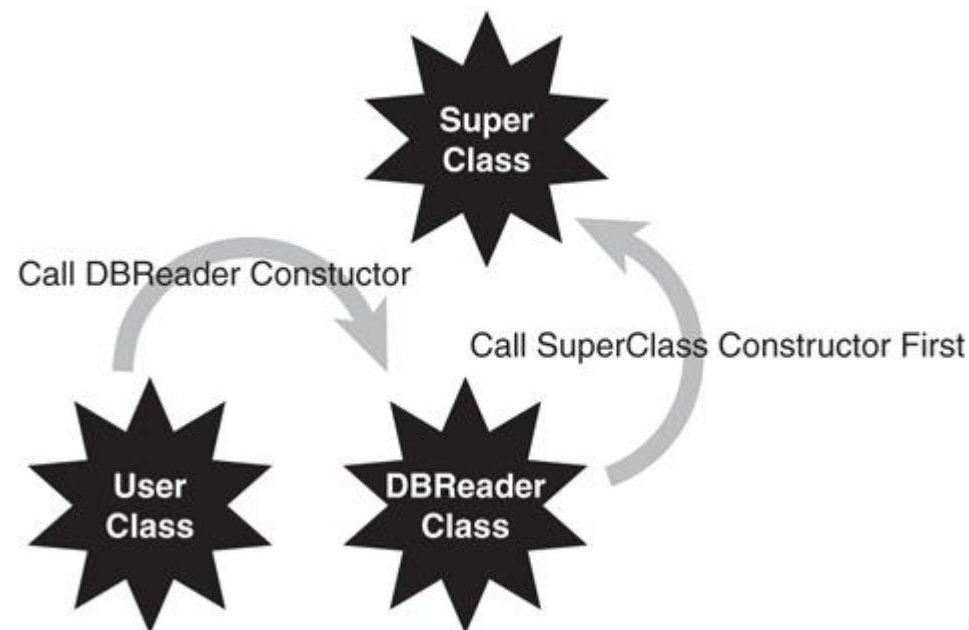


Figure 3.4 Constructing an object.

The Design of Constructors

As we have already seen, when designing a class, it is good practice to initialize all the attributes. In some languages, the compiler provides some sort of initialization. As always, don't count on the compiler to initialize attributes! In Java, you cannot use an attribute until it is initialized. If the attribute is first set in the code, make sure that you initialize the attribute to some valid condition—for example, set an integer to zero.

Constructors are used to ensure that the application is in a stable state (I like to call it a “safe” state). For example, initializing an attribute to zero, when it is intended for use as a denominator in a division operation, might lead to an unstable application. You must take into consideration that a division by zero is an illegal operation. Initializing to zero is not always the best policy.

During the design, it is good practice to identify a stable state for all attributes and then initialize them to this stable state in the constructor.

COMO A SUPERCLASSE É CONSTRUÍDA

Ao usar herança, é necessário entender como a classe pai é construída. Herança significa receber características da superclasse; por isso, seus dados, comportamentos e processo de inicialização influenciam a subclasse.

Depois que `new` é encontrado e a memória do objeto é alocada, o processo segue, de forma geral, estas etapas:

1. Dentro do construtor, chama-se o construtor da superclasse. Se não houver chamada explícita, o construtor padrão do pai pode ser chamado automaticamente.
2. Os atributos de instância da classe são inicializados. São os atributos da definição da classe, não variáveis locais existentes dentro de métodos.
3. O restante do código do construtor é executado.

Figura 3.4 - Construção de um objeto.

PROJETO DE CONSTRUTORES

É boa prática inicializar todos os atributos. Algumas linguagens fornecem inicializações automáticas, mas o projeto não deve depender cegamente disso. Cada atributo deve começar em um estado válido.

Construtores ajudam a garantir que a aplicação fique em um estado estável e seguro. Inicializar um inteiro com zero, por exemplo, pode ser inadequado se ele for usado como denominador, porque dividir por zero é uma operação inválida. Portanto, “zero” não é sempre o melhor valor padrão.

Durante o projeto, identifique um estado inicial estável para cada atributo e faça o construtor colocá-lo nesse estado.

ERROR HANDLING

It is extremely rare for a class to be written perfectly the first time. In most, if not all, situations, things *will* go wrong. Any developer who does not plan for problems is inviting disaster.

Assuming that your code has the capability to detect and trap an error condition, you can handle the error in several ways: In **Chapter 11** of their book *Java Primer Plus*, Tyma, Torok, and Downing (ISBN: 9781571690623) state that there are three basic solutions to handling problems that are detected in a program: fix it, ignore the problem by squelching it, or exit the runtime in some graceful manner. In **Chapter 4** of their book *Object-Oriented Design in Java* (ISBN: 9781571691347), Gilbert and McCarty expand on this theme by adding the choice of throwing an exception:

- Ignore the problem—not a good idea!
- Check for potential problems and abort the program when you find a problem.
- Check for potential problems, catch the mistake, and attempt to fix the problem.
- Throw an exception. (Often this is the preferred way to handle the situation.)

These strategies are discussed in the following sections.

Ignoring the Problem

Simply ignoring a potential problem is a recipe for disaster. And if you are going to ignore the problem, why bother detecting it in the first place? It is obvious that you should not ignore any known problem. The primary directive for all applications is that the application should never crash. If you do not handle your errors, the application will eventually terminate ungracefully or continue in a mode that can be considered an unstable state—possibly with corrupted data. In the latter case, you might not even know you are getting incorrect results, and that can be much worse than a program crash.

Checking for Problems and Aborting the Application

If you choose to check for potential problems and abort the application when a problem is detected, the application can display a message indicating that a problem exists. In this case the application gracefully exits, and the user is left staring at the computer screen, shaking her head and wondering what just happened. Although this is a far superior option to ignoring the problem, it is by no means optimal. However, this does allow the system to clean up things and put itself in a more stable state, such as closing files and forcing a system restart.

Checking for Problems and Attempting to Recover

Checking for potential problems, catching the mistake, and attempting to recover is a far superior solution than simply checking for problems and aborting. In this case, the problem is detected by the code, and the application attempts to fix itself. This works well in certain situations. For example, consider the following code:

```
if (a == 0)
    a=1;

c = b/a;
```

It is obvious that if the conditional statement is not included in the code, and a zero makes its way to the divide statement, you will get a system exception because you cannot divide by zero. By catching the exception and setting the variable `a` to `1`, at least the system will not crash.

TRATAMENTO DE ERROS

É extremamente raro escrever uma classe perfeita na primeira tentativa. Em praticamente todo projeto, algo dará errado. Um desenvolvedor que não planeja como lidar com problemas está convidando o desastre.

Quando o código consegue detectar uma condição de erro, existem várias estratégias possíveis. Entre as opções discutidas por autores citados no livro estão:

- ignorar o problema - geralmente uma péssima ideia;
- detectar o problema e encerrar o programa de forma controlada;
- detectar o problema, capturá-lo e tentar corrigi-lo;
- lançar uma exceção, frequentemente a alternativa preferida.

IGNORANDO O PROBLEMA

Ignorar um problema conhecido é uma receita para desastre. Se o erro não for tratado, a aplicação pode terminar abruptamente ou continuar em um estado instável, possivelmente corrompendo dados. Pior ainda, pode produzir resultados incorretos sem que ninguém perceba.

VERIFICANDO PROBLEMAS E ENCERRANDO A APLICAÇÃO

Uma solução melhor é detectar o problema, informar o usuário e encerrar de maneira controlada. Isso permite realizar limpeza, fechar arquivos e colocar o sistema em um estado mais seguro. Ainda assim, não é uma solução ideal porque a tarefa do usuário não é concluída.

VERIFICANDO PROBLEMAS E TENTANDO RECUPERAR

Uma abordagem ainda melhor é detectar o erro e tentar recuperar a execução. O livro mostra um exemplo simples em que, antes de uma divisão, o código verifica se o denominador é zero. Isso evita uma exceção de divisão por zero.

However, setting `a` to `1` might not be a proper solution because the result would be incorrect. The better solution would be to prompt the user to reenter the proper input value.

A Mix of Error-Handling Techniques

Despite the fact that this type of error handling is not necessarily object-oriented in nature, I believe that it has a valid place in OO design. Throwing an exception (discussed in the next section) can be expensive in terms of overhead. Thus, although exceptions may be a valid design choice, you will still want to consider other error-handling techniques (even tried-and-true structured techniques), depending on your design and performance needs.

Although the error checking techniques mentioned previously are preferable to doing nothing, they still have a few problems. It is not always easy to determine where a problem first appears. And it might take a while for the problem to be detected. In any event, it is beyond the scope of this book to explain error handling in great detail. However, it is important to design error handling into the class right from the start, and often the operating system itself can alert you to problems that it detects.

Throwing an Exception

Most OO languages provide a feature called *exceptions*. In the most basic sense, exceptions are unexpected events that occur within a system. Exceptions provide a way to detect problems and then handle them. In Java, C#, C++, Swift, and Visual Basic, exceptions are handled by the keywords `catch` and `throw`. This might sound like a baseball game, but the key concept here is that a specific block of code is written to handle a specific exception. This solves the problem of trying to figure out where the problem started and unwinding the code to the proper point. Here is the structure for a Java `try/catch` block:

```
try {  
  
    // possible nasty code  
  
} catch (Exception e) {  
  
    // code to handle the exception  
}
```

If an exception is thrown within the `try` block, the `catch` block will handle it. When an exception is thrown while the block is executing, the following occurs:

1. The execution of the `try` block is terminated.
2. The `catch` clauses are checked to determine whether an appropriate `catch` block for the offending exception was included. (There might be more than one `catch` clause per `try` block.)
3. If none of the `catch` clauses handles the offending exception, it is passed to the next higher-level `try` block. (If the exception is not caught in the code, the system ultimately catches it, and the results are unpredictable—that is, an application crash.)
4. If a `catch` clause is matched (the first match encountered), the statements in the `catch` clause are executed.
5. Execution then resumes with the statement following the `try` block.

Suffice it to say that exceptions are an important advantage for OO programming languages. Here is an example of how an exception is caught in Java:

[Click here to view code image](#)

```
try {
```

Entretanto, simplesmente trocar um valor inválido por `1` pode produzir um resultado matematicamente incorreto. Uma solução melhor talvez seja pedir ao usuário que informe um valor válido.

UMA COMBINAÇÃO DE TÉCNICAS DE TRATAMENTO DE ERROS

Esse tipo de verificação não é exclusivamente orientado a objetos, mas continua válido em um projeto OO. Lançar exceções pode ter custo de processamento, portanto diferentes técnicas - inclusive abordagens estruturadas tradicionais - podem ser combinadas conforme as necessidades de projeto e desempenho.

Mesmo técnicas de verificação comuns têm limitações. Nem sempre é fácil descobrir onde um problema surgiu originalmente, e o erro pode ser detectado muito depois de sua causa. O tratamento detalhado de erros está fora do escopo do livro, mas é importante projetá-lo desde o início da classe. O próprio sistema operacional também pode sinalizar problemas detectados.

LANÇANDO UMA EXCEÇÃO

A maioria das linguagens OO possui um mecanismo de exceções. Em termos simples, exceções representam eventos inesperados durante a execução e oferecem uma forma organizada de detectar e tratar problemas. Java, C#, C++, Swift e Visual Basic possuem mecanismos equivalentes com palavras como `try`, `catch` e `throw`.

A estrutura Java mostrada no original usa um bloco `try` para envolver código que pode falhar e um bloco `catch` para tratar a exceção. Quando uma exceção é lançada dentro de `try`:

1. a execução normal do bloco `try` é interrompida;
2. os blocos `catch` são examinados em busca de um tipo compatível;
3. se nenhum deles tratar a exceção, ela é propagada para um nível superior;
4. quando um `catch` apropriado é encontrado, seu código é executado;
5. depois do tratamento, a execução continua conforme as regras da linguagem, normalmente após a estrutura `try/catch`.

Exceções são um recurso importante das linguagens OO.

```
// possible nasty code
count = 0;
count = 5/count;

} catch(ArithmeticException e) {

    // code to handle the exception
    System.out.println(e.getMessage());
    count = 1;

}
System.out.println("The exception is handled.");
```

Exception Granularity

You can catch exceptions at various levels of granularity. You can catch all exceptions or check for specific exceptions, such as arithmetic exceptions. If your code does not catch an exception, the Java runtime will—and it won't be happy about it!

In this example, the division by zero (because `count` is equal to 0) within the `try` block will cause an arithmetic exception. If the exception was generated (thrown) outside a `try` block, the program would most likely have been terminated (crashed). However, because the exception was thrown within a `try` block, the `catch` block is checked to see whether the specific exception (in this case, an arithmetic exception) was planned for. Because the `catch` block contains a check for the arithmetic exception, the code within the `catch` block is executed, thus setting `count` to 1. After the `catch` block executes, the `try/catch` block is exited, and the message `The exception is handled.` appears on the Java console. The logical flow of this process is illustrated in [Figure 3.5](#).



Figure

3.5 Catching an exception.

If you had not put `ArithmeticException` in the `catch` block, the program would likely have crashed. You can catch all exceptions by using the following code:

```
try {

    // possible nasty code

} catch(Exception e) {

    // code to handle the exception

}
```

The `Exception` parameter in the `catch` block is used to catch any exception that might be generated within the scope of a `try` block.

Bulletproof Code

It's a good idea to use a combination of the methods described here to make your program as bulletproof to your user as possible.

A página original apresenta um exemplo em Java em que `count` recebe 0 e uma divisão por zero gera `ArithmeticException`. Como a operação está dentro de um bloco `try` e existe um `catch` específico para `ArithmeticException`, o erro é capturado, uma mensagem pode ser exibida e `count` é ajustado para um valor seguro antes de a execução continuar.

Figura 3.5 - Capturando uma exceção.

Se não houvesse tratamento apropriado, o programa poderia terminar. Também é possível escrever um `catch` mais genérico usando `Exception` para capturar diferentes tipos de exceção, embora isso reduza a especificidade do tratamento.

GRANULARIDADE DAS EXCEÇÕES

Exceções podem ser tratadas em diferentes níveis de detalhe. É possível capturar uma categoria ampla ou tratar tipos específicos, como erros aritméticos. Se o código não capturar uma exceção, o ambiente de execução acabará lidando com ela, frequentemente encerrando a aplicação.

CÓDIGO À PROVA DE FALHAS

É boa prática combinar várias técnicas de prevenção, validação e tratamento para tornar o programa o mais robusto possível para seus usuários.

THE IMPORTANCE OF SCOPE

Multiple objects can be instantiated from a single class. Each of these objects has a unique identity and state. This is an important point. Each object is constructed separately and is allocated its own separate memory. However, some attributes and methods may, if properly declared, be shared by all the objects instantiated from the same class, thus sharing the memory allocated for these class attributes and methods.

A Shared Method

A constructor is a good example of a method that is shared by all instances of a class.

Methods represent the behaviors of an object; the state of the object is represented by attributes. There are three types of attributes:

- Local attributes
- Object attributes
- Class attributes

Local Attributes

Local attributes are owned by a specific method. Consider the following code:

```
public class Number {  
  
    public method1() {  
        int count;  
  
    }  
  
    public method2() {  
  
    }  
  
}
```

The method `method1` contains a local variable called `count`. This integer is accessible only inside `method1`. The method `method2` has no idea that the integer `count` even exists.

At this point, we introduce a very important concept: scope. Attributes (and methods) exist within a particular scope. In this case, the integer `count` exists within the scope of `method1`. In Java, C#, C++, and Swift, scope is delineated by curly braces (`{ }`). In the `Number` class, there are several possible scopes—just start matching the curly braces.

The class itself has its own scope. Each instance of the class (that is, each object) has its own scope. Both `method1` and `method2` have their own scopes as well. Because `count` lives within `method1`’s curly braces, when `method1` is invoked, a copy of `count` is created. When `method1` terminates, the copy of `count` is removed.

For some more fun, look at this code:

```
public class Number {  
  
    public method1() {  
        int count;  
    }  
  
    public method2() {  
        int count;  
    }  
  
}
```

A IMPORTÂNCIA DO ESCOPO

Vários objetos podem ser instanciados a partir de uma única classe. Cada objeto possui identidade e estado próprios. Cada instância é construída separadamente e recebe sua própria memória. Entretanto, quando declarados de maneira apropriada, certos atributos e métodos podem ser compartilhados por todas as instâncias de uma classe.

UM MÉTODO COMPARTILHADO

O texto usa construtores como exemplo de comportamento definido uma única vez na classe e usado na criação de todas as instâncias.

Métodos representam comportamentos; atributos representam o estado. O livro distingue três categorias de atributos/variáveis segundo o escopo:

- locais;
- de objeto ou instância;
- de classe.

ATRIBUTOS LOCAIS

Variáveis locais pertencem a um método específico. No exemplo da classe `Number`, `method1()` declara um inteiro `count`. Esse `count` só é acessível dentro de `method1()`; `method2()` não sabe que ele existe.

Isso introduz o conceito de escopo. Atributos e métodos existem dentro de determinados limites. Em Java, C#, C++ e Swift, blocos delimitados por chaves ajudam a definir esses escopos. A própria classe possui um escopo, e cada método possui o seu.

Quando `method1()` é executado, sua variável local `count` passa a existir. Quando o método termina, essa variável local deixa de existir.


```
}
This example has two copies of an integer count in this class. Remember
that method1 and method2 each has its own scope. Thus, the compiler can tell which copy
of count to access simply by recognizing which method it is in. You can think of it in these terms:
method1.count;
```

```
method2.count;
```

As far as the compiler is concerned, the two attributes are easily differentiated, even though they have the same name. It is almost like two people having the same last name, but based on the context of their first names, you know that they are two separate individuals.

Object Attributes

In many design situations, an attribute must be shared by several methods within the same object. In **Figure 3.6**, for example, three objects have been constructed from a single class. Consider the following code:

[Click here to view code image](#)

```
public class Number {

    int count;    // available to both method1 and method2

    public method1() {
        count = 1;
    }

    public method2() {
        count = 2;
    }

}
```

É possível ter duas variáveis locais com o mesmo nome em métodos diferentes. `method1()` pode declarar `count` e `method2()` também pode declarar `count` porque cada método possui seu próprio escopo. O compilador distingue os dois valores pelo contexto em que são usados.

Podemos pensar conceitualmente em `method1.count` e `method2.count`: embora o nome simples seja igual, são variáveis independentes. É como duas pessoas com o mesmo sobrenome; o restante do contexto permite saber de quem estamos falando.

ATRIBUTOS DE OBJETO

Em muitas situações, um atributo precisa ser compartilhado por vários métodos do mesmo objeto. Nesse caso, ele é declarado no escopo da classe, fora do corpo dos métodos.

No exemplo Number mostrado no original, `count` é declarado como atributo da classe e tanto `method1()` quanto `method2()` podem modificá-lo. Há uma única cópia de `count` por objeto, e os dois métodos trabalham sobre essa mesma cópia.

Figura 3.6 - Atributos de objeto.

O atributo `count` está fora do escopo individual de `method1()` e `method2()`, mas dentro do escopo da classe. Portanto, os dois métodos podem acessá-lo. Para uma determinada instância de `Number`, ambos alteram a mesma posição de estado.

Porém, esse `count` não é compartilhado entre objetos diferentes. Se criarmos `number1`, `number2` e `number3`, cada instância é construída separadamente e possui sua própria cópia do atributo `count`. Alterar `count` em `number1` não modifica `count` em `number2` nem em `number3`. Nesse caso, `count` é um atributo de objeto/instância.

A página começa ainda um exemplo sobre sobreposição de escopos: a classe pode ter um atributo `count` e um método também declarar uma variável local `count`. O escopo determina a qual deles cada referência se aplica; a continuação aparece na próxima página.

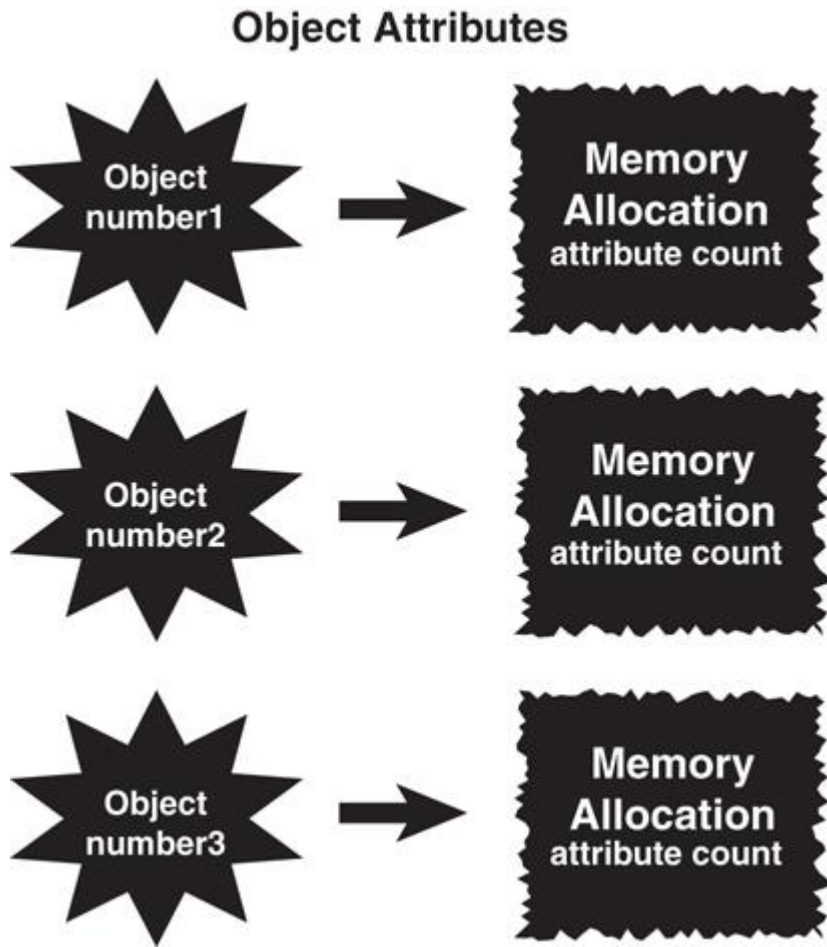


Figure 3.6 Object attributes.

Note here that the class attribute `count` is declared outside the scope of both `method1` and `method2`. However, it is within the scope of the class. Thus, `count` is available to both `method1` and `method2`. (Basically, all methods in the class have access to this attribute.) Notice that the code for both methods is setting `count` to a specific value. There is only one copy of `count` for the entire object, so both assignments operate on the same copy in memory. However, this copy of `count` is not shared between different objects.

To illustrate, let's create three copies of the `Number` class:

```
Number number1 = new Number();
Number number2 = new Number();
Number number3 = new Number();
```

Each of these objects—`number1`, `number2`, and `number3`—is constructed separately and is allocated its own resources. There are three separate instances of the integer `count`.

When `number1` changes its attribute `count`, this in no way affects the copy of `count` in object `number2` or object `number3`. In this case, integer `count` is an *object attribute*.

You can play some interesting games with scope. Consider the following code:

[Click here to view code image](#)

```
public class Number {

    int count;

    public method1() {
        int count;
    }

    public method2() {
```

```
        int count;
    }

}
```

In this case, three totally separate memory locations have the name of `count` for each object. The object owns one copy, and `method1()` and `method2()` each have their own copy.

To access the object variable from within one of the methods, say `method1()`, you can use a pointer called `this` in the C-based languages:

```
public method1() {
    int count;

    this.count = 1;
}
```

Notice that some code looks a bit curious:

```
this.count = 1;
```

The selection of the word `this` as a keyword is perhaps unfortunate. However, we must live with it. The use of the `this` keyword directs the compiler to access the object variable `count` and not the local variables within the method bodies.

Note

The keyword `this` is a reference to the current object.

Class Attributes

As mentioned earlier, it is possible for two or more objects of the same class to share attributes. In Java, C#, C++, and Swift, you do this by making the attribute *static*:

```
public class Number {

    static int count;

    public method1() {
    }

}
```

By declaring `count` as static, this attribute is allocated a single piece of memory for all objects instantiated from the class. Thus, all objects of the class use the same memory location for `count`. Essentially, each class has a single copy, which is shared by all objects of that class (see [Figure 3.7](#)). This is about as close to global data as we get in OO design.

Neste caso, para cada objeto podem existir três posições de memória diferentes chamadas `count`: uma pertencente ao objeto e uma variável local em cada método.

Para acessar explicitamente o atributo do objeto a partir de um método que também possui uma variável local com o mesmo nome, linguagens da família C usam a referência `this`. O exemplo original mostra `this.count = 1;`.

A palavra-chave `this` pode parecer curiosa, mas sua função é importante: ela direciona o compilador ao atributo `count` pertencente ao objeto atual, e não à variável local definida no corpo do método.

NOTA

A palavra-chave `this` é uma referência ao objeto atual.

ATRIBUTOS DE CLASSE

Dois ou mais objetos da mesma classe também podem compartilhar um atributo. Em Java, C#, C++ e Swift isso pode ser obtido, em diferentes formas, com membros de classe; em Java o exemplo usa `static`.

Ao declarar `count` como `static`, existe uma única posição de memória compartilhada por todas as instâncias da classe. Cada objeto acessa o mesmo `count`. Em termos conceituais, a classe possui uma única cópia compartilhada por todos os objetos. Esse é um dos mecanismos que mais se aproximam de dados globais em um projeto OO.

Figura 3.7 - Atributos de classe.

Atributos de classe possuem vários usos válidos, mas exigem cuidado com sincronização e interferência entre objetos. Imagine dois objetos `Count`, `Count1` e `Count2`, que compartilham o mesmo atributo `static count`. Se `Count1` usa `count` para contar pixels e `Count2` usa o mesmo atributo para contar ovelhas, assim que `Count2` alterar `count` o valor esperado por `Count1` será perdido.

A lição é que membros `static` devem ser usados conscientemente. Como representam estado compartilhado, seu uso muda a independência normal entre instâncias.

SOBRECARGA DE OPERADORES

Algumas linguagens OO, como C++, permitem sobrecarregar operadores, isto é, alterar ou estender o significado de símbolos como `+` para tipos definidos pelo programador.

Ao ver `X = 5 + 6`, esperamos que `X` receba 11. Mas em outros contextos o sinal `+` pode representar outra operação, como concatenação de strings. A continuação mostra exemplos desse comportamento.

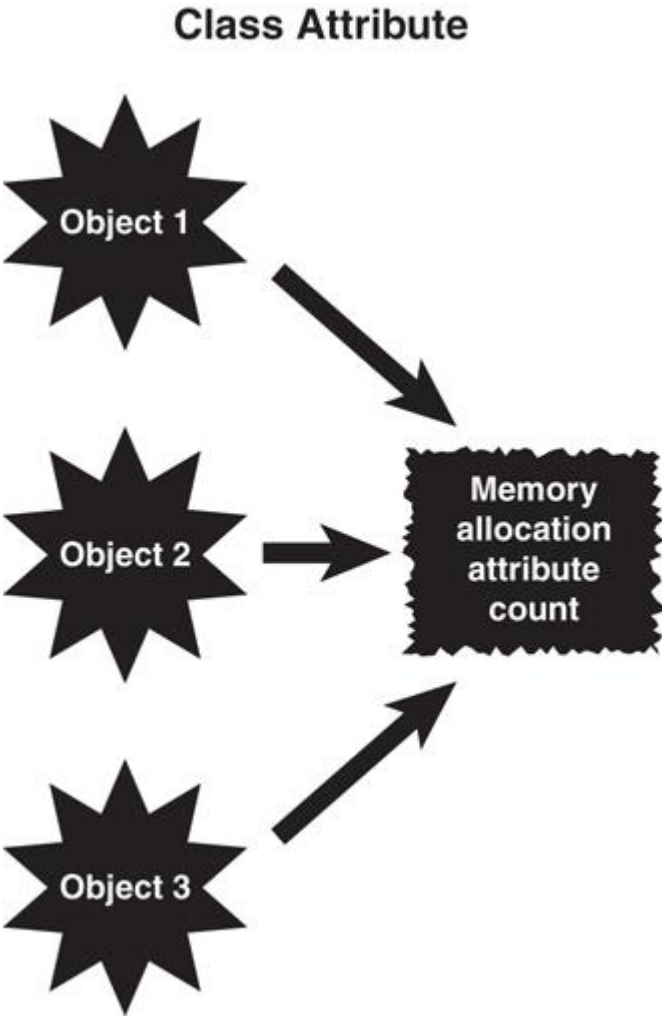


Figure 3.7 Class attributes.

There are many valid uses for class attributes; however, you must be aware of potential synchronization problems. Let's instantiate two `Count` objects:

```
Count Count1 = new Count();
Count Count2 = new Count();
```

For the sake of argument, let's say that the object `Count1` is going merrily about its way and is using `count` as a means of keeping track of the pixels on a computer screen. This is not a problem until the object `Count2` decides to use attribute `count` to keep track of sheep. The instant that `Count2` records its first sheep, the data that `Count1` was saving is lost. In practice, there might not be a lot of uses for static methods. Make sure you are confident in their use before incorporating them in designs.

OPERATOR OVERLOADING

Some OO languages enable you to overload an operator. C++ is an example of one such language. Operator overloading enables you to change the meaning of an operator. For example, when most people see a plus sign, they assume it represents addition. If you see the equation

```
X = 5 + 6;
```

you expect that `x` would contain the value 11. And in this case, you would be correct.

However, at times a plus sign could represent something else. For example, in the following code:

```
String firstName = "Joe", lastName = "Smith";
```

```
String Name = firstName + " " + lastName;
```

You would expect that `Name` would contain `Joe Smith`. The plus sign here has been overloaded to perform string concatenation.

String Concatenation

String concatenation occurs when two separate strings are combined to create a new, single string.

In the context of strings, the plus sign does not mean addition of integers or floats but concatenation of strings.

What about matrix addition? You could have code like this:

```
Matrix a, b, c;
```

```
c = a + b;
```

Thus, the plus sign now performs matrix addition, not addition of integers or floats.

Overloading is a powerful mechanism. However, it can be downright confusing for people who read and maintain code. In fact, developers can confuse themselves. To take this to an extreme, it would be possible to change the operation of addition to perform subtraction. Why not? Operator overloading allows you to change the meaning of an operator. Thus, if the plus sign were changed to perform subtraction, the following code would result in an `x` value of -1 .

```
x = 5 + 6;
```

OO languages such as Java and .NET do not allow operator overloading.

Although these languages do not allow the option of overloading operators, the languages themselves do overload the plus sign for string concatenation, but that’s about it. The designers of Java must have decided that operator overloading was more of a problem than it was worth. If you must use operator overloading in C++, take care by documenting and commenting properly so the people who will use the class are not confused.

MULTIPLE INHERITANCE

We cover inheritance in much more detail in [Chapter 7](#), “[Mastering Inheritance and Composition](#).” However, this is a good place to begin discussing multiple inheritance, which is one of the more powerful and challenging aspects of class design.

As the name implies, *multiple inheritance* allows a class to inherit from more than one class. In practice, this seems like a great idea. Objects are supposed to model the real world, are they not? And many real-world examples of multiple inheritance exist. Parents are a good example of multiple inheritance. Each child has two parents—that’s just the way it is. So it makes sense that you can design classes by using multiple inheritance. In some OO languages, such as C++, you can.

However, this situation falls into a category similar to operator overloading. Multiple inheritance is a very powerful technique, and in fact, some problems are quite difficult to solve without it. Multiple inheritance can even solve some problems quite elegantly. However, multiple inheritance can significantly increase the complexity of a system, both for the programmer and the compiler writers.

Se `firstName` contém “Joe” e `lastName` contém “Smith”, a expressão `firstName + " " + lastName` pode produzir “Joe Smith”. Nesse contexto, `+` realiza concatenação de strings, não soma numérica.

CONCATENAÇÃO DE STRINGS

Concatenação ocorre quando duas ou mais strings separadas são combinadas para formar uma nova string.

O mesmo princípio poderia ser aplicado, em uma linguagem que permita sobrecarga definida pelo usuário, à soma de matrizes: `c = a + b` poderia representar uma operação específica entre objetos `Matrix`.

Sobrecarga é poderosa, mas pode tornar o código confuso. Em um caso extremo, seria tecnicamente possível redefinir `+` para realizar subtração. Isso ilustra por que o recurso deve ser usado com cuidado e bem documentado.

Java não permite sobrecarga arbitrária de operadores pelo programador, embora a própria linguagem defina `+` tanto para soma quanto para concatenação de strings. Em C++, se a sobrecarga for usada, documente claramente seu significado para evitar surpresas na manutenção.

HERANÇA MÚLTIPLA

Herança será discutida em profundidade no Capítulo 7. Aqui começa a discussão sobre herança múltipla, um dos aspectos mais poderosos e desafiadores do projeto de classes.

Como o nome sugere, herança múltipla permite que uma classe herde de mais de uma classe pai. No mundo real, há exemplos que parecem sugerir essa modelagem, como uma criança que possui dois pais. C++ permite esse recurso.

Entretanto, a técnica aumenta significativamente a complexidade tanto para o programador quanto para quem implementa a linguagem e o compilador. Ela pode resolver certos problemas de modo elegante, mas deve ser tratada com cautela.

The designers of Java, .NET, and Swift decided that the increased complexity of allowing multiple inheritance far outweighed its advantages, so they simply did not implement it. In some ways, Java, .NET, and Swift compensate for this; however, the bottom line is that Java, .NET, and Swift do not allow conventional multiple inheritance.

The modern concept of inheritance is that you can only inherit attributes from a single parent (single inheritance). Even though you can use multiple interfaces or protocols, this is not truly multiple inheritance.

Behavioral and Implementation Inheritance

Interfaces are a mechanism for behavioral inheritance, whereas abstract classes are used for implementation inheritance. The bottom line is that interface language constructs provide behavioral interfaces but no implementation, whereas abstract classes may provide both interfaces and implementation. This topic is covered in great detail in [Chapter 8, “Frameworks and Reuse: Designing with Interfaces and Abstract Classes.”](#)

OBJECT OPERATIONS

Some of the most basic operations in programming become more complicated when you’re dealing with complex data structures and objects. For example, when you want to copy or compare primitive data types, the process is quite straightforward. However, copying and comparing objects is not quite as simple. In his book *Effective C++*, Scott Meyers devotes an entire section to copying and assigning objects.

Classes and References

The problem with complex data structures and objects is that they might contain references. Simply making a copy of the reference does not copy the data structures or the object that it references. In the same vein, when comparing objects, simply comparing a pointer to another pointer only compares the references—not what they point to.

The problems arise when comparisons and copies are performed on objects. Specifically, the question boils down to whether you follow the pointers. Regardless, there should be a way to copy an object. Again, this is not as simple as it might seem. Because objects can contain references, these reference trees must be followed to do a valid copy (if you truly want to do a deep copy).

Deep Versus Shallow Copies

A *deep copy* occurs when all the references are followed and new copies are created for all referenced objects. Many levels might be involved in a deep copy. For objects with references to many objects, which in turn might have references to even more objects, the copy itself can create significant overhead. A shallow copy would simply copy the reference and not follow the levels. Gilbert and McCarty have a good discussion about what shallow and deep hierarchies are in *Object-Oriented Design in Java* in a section called “Prefer a Tree to a Forest.”

To illustrate, in [Figure 3.8](#), if you do a simple copy of the object (called a *bitwise copy*), only the references are copied—not any of the actual objects. Thus, both objects (the original and the copy) will reference (point to) the same objects. To perform a complete copy, in which all reference objects are copied, you must write code to create all the subobjects.

Os projetistas de Java, .NET e Swift concluíram que a complexidade da herança múltipla clássica superava seus benefícios e não a incluíram da mesma forma que C++.

Em muitos projetos modernos, uma classe herda implementação/estado de apenas um pai direto, mas pode adotar múltiplas interfaces ou protocolos. Isso fornece múltiplos contratos de comportamento sem ser herança múltipla convencional de implementação.

HERANÇA DE COMPORTAMENTO E DE IMPLEMENTAÇÃO

Interfaces são mecanismos para herança de comportamento, enquanto classes abstratas podem oferecer tanto contratos quanto parte da implementação. Esse assunto é aprofundado no Capítulo 8, sobre frameworks, reutilização, interfaces e classes abstratas.

OPERAÇÕES COM OBJETOS

Algumas operações básicas ficam mais complexas quando lidamos com objetos e estruturas de dados compostas. Copiar ou comparar tipos primitivos é relativamente simples; copiar e comparar objetos pode exigir decisões importantes.

CLASSES E REFERÊNCIAS

O problema é que objetos podem conter referências para outros objetos. Copiar apenas uma referência não copia o objeto apontado. Da mesma forma, comparar duas referências pode dizer apenas se elas apontam para o mesmo lugar, não se os objetos possuem conteúdo equivalente.

Assim, ao copiar ou comparar objetos precisamos decidir se devemos “seguir” as referências. Uma cópia completa pode exigir percorrer uma árvore inteira de objetos relacionados.

CÓPIA PROFUNDA VERSUS CÓPIA SUPERFICIAL

Uma cópia profunda (deep copy) segue as referências e cria novas cópias dos objetos referenciados. Pode haver muitos níveis e, em estruturas grandes, isso gera custo significativo. Uma cópia superficial (shallow copy) copia apenas as referências, de modo que o original e a cópia continuam apontando para os mesmos subobjetos.

Na Figura 3.8, uma cópia simples copia referências, não todos os objetos. Para criar uma cópia realmente independente, é necessário construir também os subobjetos.

Figura 3.8 - Seguindo referências de objetos.

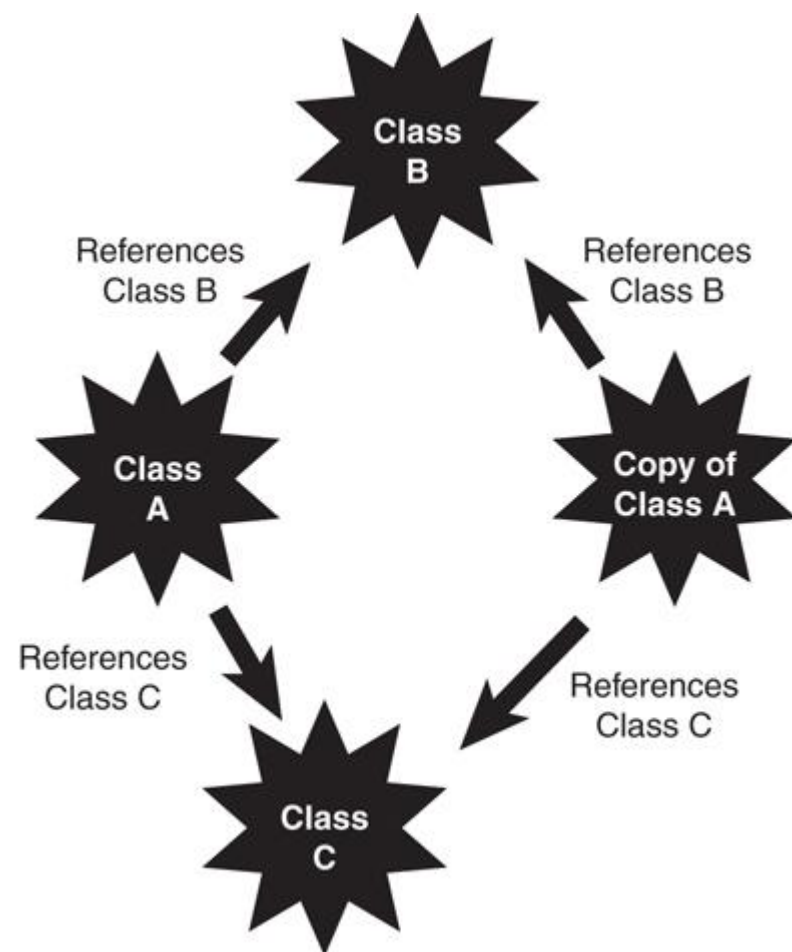


Figure 3.8 Following object references.

This problem also manifests itself when comparing objects. As with the copy function, this is not as simple as it might seem. Because objects contain references, these reference trees must be followed to do a valid comparison of objects. In most cases, languages provide a default mechanism to compare objects. As is usually the case, do not count on the default mechanism. When designing a class, you should consider providing a comparison function in your class that you know will behave as you want it to.

CONCLUSION

This chapter covered a number of advanced OO concepts that, although perhaps not vital to a general understanding of OO concepts, are quite necessary in higher-level OO tasks, such as designing a class. In [Chapter 4](#), “[The Anatomy of a Class](#),” we start looking specifically at how to design and build a class.

REFERENCES

- Gilbert, Stephen, and Bill McCarty. 1998. *Object-Oriented Design in Java*. Berkeley, CA: The Waite Group Press.
- Meyers, Scott. 2005. *Effective C++*, Third Edition. Boston, MA: Addison-Wesley Professional.
- Tyma, Paul, Gabriel Torok, and Troy Downing. 1996. *Java Primer Plus*. Berkeley, CA: The Waite Group.

O mesmo problema aparece nas comparações. Como objetos podem conter referências, uma comparação significativa talvez precise percorrer toda a estrutura referenciada. Muitas linguagens fornecem um comportamento padrão, mas não se deve presumir que ele representa exatamente a semântica desejada pela aplicação.

Ao projetar uma classe, considere oferecer uma operação de comparação que expresse explicitamente o que significa dois objetos serem equivalentes naquele domínio.

CONCLUSÃO

Este capítulo apresentou conceitos OO mais avançados. Alguns talvez não sejam indispensáveis para uma visão introdutória, mas são importantes em tarefas de nível mais alto, como projeto de classes. O próximo capítulo, “A Anatomia de uma Classe”, examina diretamente como projetar e construir uma classe.

REFERÊNCIAS

- Gilbert, Stephen; McCarty, Bill. *Object-Oriented Design in Java*. Waite Group Press, 1998.
- Meyers, Scott. *Effective C++*, 3ª ed. Addison-Wesley Professional, 2005.
- Tyma, Paul; Torok, Gabriel; Downing, Troy. *Java Primer Plus*. Waite Group, 1996.

4. The Anatomy of a Class

In previous chapters we have covered the fundamental object-oriented (OO) concepts and determined the difference between the interface and the implementation. No matter how well you think out the problem of what should be part of the interface and what should be part of the implementation, the bottom line always comes down to how useful the class is and how it interacts with other classes. A class should never be designed in a vacuum, for as might be said, no class is an island. When objects are instantiated, they almost always interact with other objects. An object can also be part of another object or be part of an inheritance hierarchy.

This chapter examines a simple class and then takes it apart piece by piece along with guidelines that you should consider when designing classes. We will continue using the cabbie example presented in [Chapter 2, “How to Think in Terms of Objects.”](#)

Each of the following sections covers a particular aspect of a class. Although not all components are necessary in every class, it is important to understand how a class is designed and constructed.

note

This example class is meant for illustration purposes only. Some of the methods are not fleshed out (meaning that there is no implementation) and simply present the interface—primarily to emphasize that the interface is the focus of the initial design.

THE NAME OF THE CLASS

The name of the class is important for several reasons. The obvious reason is to identify the class itself. Beyond simple identification, the name must be descriptive. The choice of a name is important because it provides information about what the class does and how it interacts within larger systems.

The name is also important when considering language constraints. For example, in Java, the public class name must be the same as the filename. If these names do not match, the application won’t compile.

Figure 4.1 shows the class that will be examined. Plain and simple, the name of the class in our example, `Cabbie`, is the name located after the keyword `class`:

```
public class Cabbie {  
  
}
```

4. A ANATOMIA DE UMA CLASSE

Nos capítulos anteriores foram apresentados os conceitos fundamentais de orientação a objetos e a diferença entre interface e implementação. Por melhor que seja a separação conceitual entre essas duas partes, no fim a qualidade de uma classe depende de quão útil ela é e de como interage com outras classes.

Uma classe não deve ser projetada isoladamente. Objetos instanciados quase sempre interagem com outros objetos. Um objeto também pode ser parte de outro objeto, por composição, ou participar de uma hierarquia de herança.

Este capítulo examina uma classe simples peça por peça e apresenta diretrizes úteis para o projeto. O exemplo do motorista de táxi (Cabbie), introduzido no Capítulo 2, continuará sendo usado.

Cada seção cobre um aspecto específico de uma classe. Nem todos os componentes aparecem em todas as classes, mas é importante compreender como uma classe é organizada e construída.

NOTA

A classe de exemplo serve apenas para fins didáticos. Alguns métodos não possuem implementação completa e mostram somente a interface, justamente para reforçar que a interface é o foco inicial do projeto.

O NOME DA CLASSE

O nome é importante não apenas para identificar a classe, mas também para comunicar sua finalidade e seu papel em sistemas maiores. Bons nomes devem ser descritivos.

Restrições da linguagem também podem influenciar. Em Java, por exemplo, uma classe `public` precisa normalmente estar em um arquivo cujo nome corresponda ao nome da classe.

A Figura 4.1 apresenta a classe de exemplo. Seu nome, `Cabbie`, aparece após a palavra-chave `class`.

Figura 4.1 - Nossa classe de exemplo.

USANDO SINTAXE JAVA

A convenção deste livro é apresentar exemplos principalmente em sintaxe Java. Outras linguagens possuem conceitos equivalentes, embora a sintaxe possa ser diferente.

```

/*
 * This class defines a cabbie and assigns a cab
 */
public class Cabbie{

    //Place name of Company Here
    private static String companyName = "Blue Cab Company";

    //Name of the Cabbie
    private String Name;

    //Car assigned to Cabbie
    private Cab myCab;

    // Default Constructor for the Cabbie
    public Cabbie() {

        name = null;
        myCab = null;

    }

    // Name Initializing Constructor for the Cabbie
    public Cabbie(String iName, String serialNumber){

        Name = iName;
        myCab = new Cab(serialNumber);

    }

    // Set the Name of the Cabbie
    public void setName(String iName) {
        Name = iName;
    }

    // Get the Name of the Company
    public static String getName(){
        return Name;
    }

    // Get the Name of the Cabbie
    public static String getCompanyName(){
        return companyName;
    }

    public void giveDestination(){

    }

    private void turnRight(){

    }

    private void turnLeft(){

    }

}

```

Comments →

Class Name →

Attributes

Constructors

Accessor Methods (Public Interfaces)

A Public Interface →

Private Implementation

Figure 4.1 Our sample class.

Using Java Syntax

O nome `Cabbie` é utilizado sempre que uma instância dessa classe é criada.

COMENTÁRIOS

Independentemente da sintaxe, comentários são essenciais para compreender a função de uma classe. Em Java e várias outras linguagens há diferentes estilos de comentários.

ESTILOS DE COMENTÁRIO EM JAVA E C#

Java possui comentários de bloco `/* ... */`, comentários de linha `//` e comentários de documentação `/** ... */`. C# possui mecanismos semelhantes, inclusive `///` para documentação. O livro não aprofunda os comentários de documentação aqui.

Comentários de bloco começam com `/*` e terminam com `*/` e podem ocupar várias linhas. É importante fechar corretamente o bloco; caso contrário, trechos de código podem ser interpretados como comentário e ignorados pelo compilador.

Comentários de linha começam com `//` e se estendem até o fim da linha. Não exigem marcador de fechamento, mas código colocado depois de `//` na mesma linha também será tratado como comentário.

ATRIBUTOS

Atributos representam o estado do objeto porque armazenam suas informações. A classe `Cabbie` possui atributos para o nome da empresa, o nome do motorista e o táxi atribuído a ele.

O primeiro exemplo declara `companyName` como `private static String`. A palavra `private` indica que o membro só pode ser acessado diretamente dentro da própria classe/objeto, conforme as regras da linguagem.

OCULTANDO O MÁXIMO POSSÍVEL DE DADOS

Todos os atributos deste exemplo são privados, seguindo o princípio de manter a interface pública mínima. O acesso externo deve ocorrer por meio dos métodos de interface apropriados.

Remember that the convention for this book is to use Java syntax. The syntax will be similar but, perhaps, somewhat different in other languages.

The class `Cabbie` name is used whenever this class is instantiated.

COMMENTS

Regardless of the syntax of the comments used, they are vital to understanding the function of a class. In Java and other languages, two kinds of comments are common.

The Extra Java and C# Comment Style

In Java and C#, there are three types of comments. In Java, the third comment type (`/** */`) relates to a form of documentation that Java provides. We do not cover this type of comment in this book. In C#, the syntax to create documentation comments is `///`, much like the `/** */` Javadoc documentation comments.

The first comment is the old C-style comment, which uses `/*` (slash-asterisk) to open the comment and `*/` (asterisk-slash) to close the comment. This type of comment can span more than one line, and it’s important not to forget to use the pair of open and close comment symbols for each comment. If you miss the closing comment (`*/`), some of your code might be tagged as a comment and ignored by the compiler. Here is an example of this type of comment used with the `Cabbie` class:

Click here to view code image

```
/*
```

```
    This class defines a cabbie and assigns a cab
```

```
*/
```

The second type of comment is the `//` (slash-slash), which renders everything after it, to the end of the line, a comment. This type of comment spans only one line, so you don’t need to remember to use a close comment symbol, but you do need to remember to confine the comment to just one line and not include any live code after the comment. Here is an example of this type of comment used with the `Cabbie` class:

```
// Name of the cabbie
```

ATTRIBUTES

Attributes represent the state of the object because they store the information about the object. For our example, the `Cabbie` class has attributes that store the name of the company, the name of the cabbie, and the cab assigned to the cabbie. For example, the first attribute stores the name of the company:

Click here to view code image

```
private static String companyName = "Blue Cab Company";
```

Note here the two keywords `private` and `static`. The keyword `private` signifies that a method or variable can be accessed only within the declaring object.

Hiding as Much Data as Possible

All the attributes in this example are private. This is in keeping with the design principle of keeping the interface design as minimal as possible. The only way to access these attributes is through the method interfaces provided (which we explore later in this chapter).

The `static` keyword signifies that there will be only one copy of this attribute for all the objects instantiated by this class. Basically, this is a class attribute. (See [Chapter 3, “More Object-Oriented Concepts,”](#) for further discussion on class attributes.) Thus, even if 500 objects are instantiated from the `Cabbie` class, only one copy will be in memory of the `companyName` attribute (see [Figure 4.2](#)).

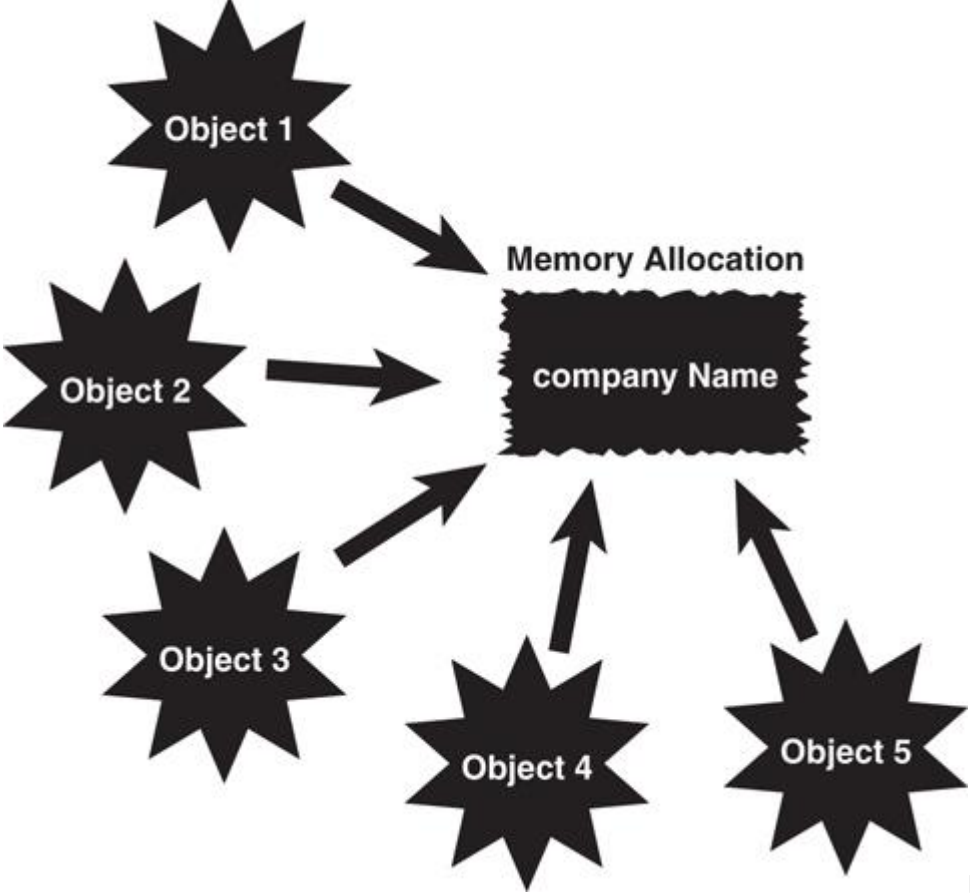


Figure 4.2 Object memory

allocation.

The second attribute, `name`, is a string that stores the name of the cabbie:

```
private String name;
```

This attribute is also private so that other objects cannot access it directly. They must use the interface methods.

The `myCab` attribute is a reference to another object. The class, called `Cab`, holds information about the cab, such as its serial number and maintenance records:

```
private Cab myCab;
```

Passing a Reference

It is likely that the `Cab` object was created by another object. Thus, the object reference would be passed to the `Cabbie` object. However, for the sake of this example, the `Cab` is created within the `Cabbie` object. As a result, we are not really interested in the internals of the `Cab` object. Note that at this point, only a reference to a `Cab` object is created; there is no memory allocated by this definition.

CONSTRUCTORS

This `Cabbie` class contains two constructors. We know they are constructors because they have the same name as the class: `Cabbie`. The first constructor is the default constructor:

A palavra-chave `static` indica que existe uma única cópia compartilhada desse atributo para todas as instâncias. Assim, mesmo que 500 objetos `Cabbie` sejam criados, haverá apenas uma instância de `companyName` associada à classe.

Figura 4.2 - Alocação de memória dos objetos.

O segundo atributo, `name`, é uma `String` que armazena o nome do motorista. Ele também é `private`; outros objetos devem usar os métodos públicos da classe para acessar ou modificar o valor.

O atributo `myCab` é uma referência para outro objeto, da classe `Cab`, que pode guardar informações como número de série e histórico de manutenção.

PASSANDO UMA REFERÊNCIA

Em um projeto real, o objeto `Cab` pode ter sido criado por outro objeto e sua referência passada ao `Cabbie`. Para simplificar o exemplo, porém, a página considera a criação de `Cab` dentro de `Cabbie`. O importante é perceber que declarar `private Cab myCab;` cria apenas uma referência; o objeto `Cab` propriamente dito ainda precisa ser instanciado.

CONSTRUTORES

A classe `Cabbie` contém dois construtores, reconhecidos porque possuem o mesmo nome da classe. O primeiro é um construtor sem argumentos, apresentado na próxima página.

```
public Cabbie() {  
  
    name = null;  
    myCab = null;  
  
}
```

Technically, this is not a default constructor provided by the system. Recall that the compiler will provide an empty default constructor if you do not code any constructor for a class. By definition, the reason it is called a default constructor here is because it is a constructor with no arguments, which, in effect, overrides the compiler’s default constructor.

If you provide a constructor with arguments, the system will not provide a default constructor. Although this can seem complicated, the rule is that the compiler’s default constructor is included only if you provide *no* constructors in your code.

No Constructor

Coding no constructor and allowing the default constructor to be in play can cause maintenance headaches down the road. If the code relies on a default constructor, and another constructor is added later, the default constructor will not be included by the system.

In this constructor, the attributes `name` and `myCab` are set to `null`:

```
name = null;  
myCab = null;
```

The Nothingness of `Null`

In many programming languages, the value `null` represents a value of nothing. This might seem like an esoteric concept, but setting an attribute to `nothing` is a useful programming tech-nique. Checking a variable for `null` can identify whether a value has been properly initialized. For example, you might want to declare an attribute that will later require user input. Thus, you can initialize the attribute to `null` before the user is given the opportunity to enter the data. By setting the attribute to `null` (which is a valid condition), you can check whether an attribute has been properly set. Note that in some languages this is not allowed with the `string` type. In .NET for example, it is required to use `name = string.empty;`.

As we know, it is always a good coding practice to initialize attributes in the constructors. In the same vein, it’s a good programming practice to then test the value of an attribute to see whether it is `null`. This can save you a lot of headaches later if the attribute or object was not set properly. For example, if you use the `myCab` reference before a real object is assigned to it, you will most likely have a problem. If you set the `myCab` reference to `null` in the constructor, you can later check to see whether `myCab` is still `null` when you attempt to use it. An exception might be generated if you treat an uninitialized reference as if it were properly initialized. Consider another example: If you have an `Employee` class that includes a `spouse` attribute (perhaps for insurance purposes), you’d better make provisions for the situation when an employee is not married. By initially setting the attribute to `null`, you can then check for this status.

The second constructor provides a way for the user of the class to initialize the `Name` and `myCab` attributes:

[Click here to view code image](#)

```
public Cabbie(String iName, String serialNumber) {  
  
    name = iName;  
    myCab = new Cab(serialNumber);  
  
}
```

O primeiro construtor `Cabbie()` inicializa `name` e `myCab` com `null`.

Tecnicamente, ele não é o construtor padrão inserido automaticamente pelo compilador, pois foi escrito explicitamente. Ainda assim, costuma ser chamado informalmente de construtor padrão ou sem argumentos por não receber parâmetros. Ao fornecê-lo, ele substitui a necessidade do construtor que o compilador forneceria.

Se você declarar apenas um construtor com argumentos e não declarar um construtor sem argumentos, o sistema não acrescentará automaticamente o construtor padrão. A regra é simples: o construtor padrão do compilador existe somente quando você não fornece nenhum construtor.

SEM CONSTRUTOR EXPLÍCITO

Depender do construtor padrão pode criar problemas de manutenção. Se o código cliente usa esse construtor e, mais tarde, alguém acrescenta um construtor explícito com parâmetros, o construtor padrão automático pode deixar de existir.

O construtor do exemplo define `name = null` e `myCab = null`.

O “NADA” REPRESENTADO POR `NULL`

Em muitas linguagens, `null` representa ausência de uma referência ou valor. Inicializar um atributo com `null` pode ser útil para indicar que ele ainda não recebeu um objeto válido. Posteriormente, o código pode testar esse estado e decidir como agir. Algumas linguagens ou tipos possuem regras diferentes, como `string.Empty` em certos contextos .NET.

É boa prática inicializar atributos e depois validar se continuam em estados não definidos antes de usá-los. Se `myCab` ainda for `null` e o código tentar utilizá-lo como um objeto válido, ocorrerá um erro ou exceção.

O mesmo princípio aparece em domínios reais: uma classe `Employee` pode possuir um atributo `spouse` que precisa representar corretamente a situação de uma pessoa sem cônjuge.

O segundo construtor da página recebe `iName` e `serialNumber`, atribui o nome e cria um novo objeto `Cab` com o número de série. O código Java permanece preservado à esquerda.

In this case, the user would provide two strings in the parameter list of the constructor to properly initialize attributes. Notice that the `myCab` object is instantiated in this constructor:
`myCab = new Cab(serialNumber);`
As a result of executing this line of code, the storage for a `Cab` object is allocated. **Figure 4.3** illustrates how a new instance of a `Cab` object is referenced by the attribute `myCab`. Using two constructors in this example demonstrates a common use of method overloading. Notice that the constructors are all defined as `public`. This makes sense because in this case, the constructors are obvious members of the class interface. If the constructors were `private`, other objects couldn't access them—thus, other objects could not instantiate a `Cab` object.

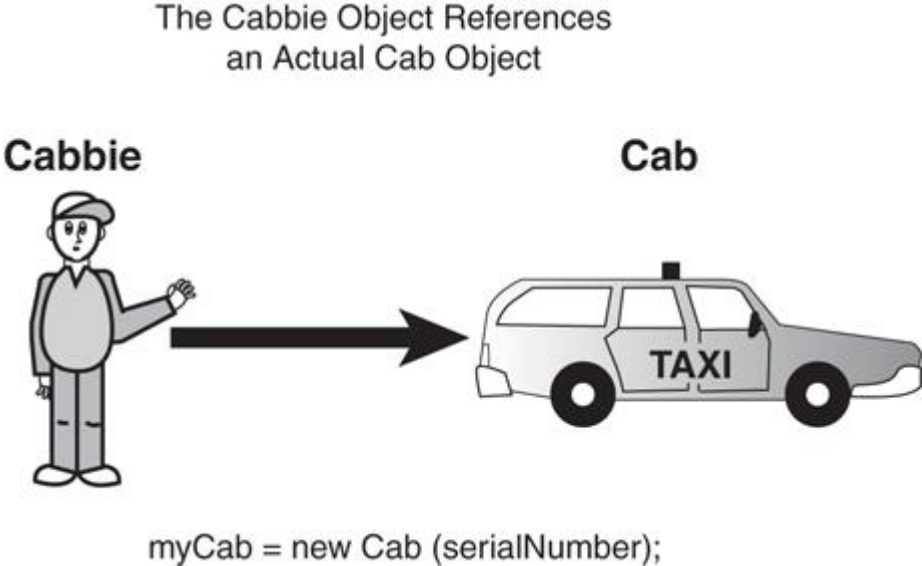


Figure 4.3 The Cabbie object

Multiple Constructors

It's worth noting that it is sometimes considered a less than ideal practice to use more than one constructor nowadays. With the prevalence of Inversion of Control (IoC) containers and the like, it's frowned upon, and even unsupported, for a number of frameworks without special configuration.

ACCESSORS

In most, if not all, examples in this book, the attributes are defined as `private` so that any other objects cannot access the attributes directly. It would be ridiculous to create an object in isolation that does not interact with other objects—for we want to share appropriate information. That said, isn't it sometimes necessary to inspect and, perhaps, change another class's attribute? The answer is, of course, yes. There are many times when an object needs to access another object's attributes; however, it does not need to do it directly. A class should be very protective of its attributes. For example, you do not want object `A` to have the capability to inspect or change the attributes of object `B` without object `B` having control. There are several reasons for this; the most important reasons boil down to data integrity and efficient debugging. Assume that a bug exists in the `Cab` class. You have tracked the problem to the `Name` attribute. Somehow it is getting overwritten, and garbage is turning up in some name queries. If `Name` were `public` and any class could change it, you would have to go searching through all the possible code, trying to find places that reference and change `Name`. However, if you let only a `Cabbie` object change `Name`, you'd have to look only in a method of the `Cabbie` class. This access is provided by a type of method called an *accessor*. Sometimes accessors are referred to as

Nesse construtor, o usuário fornece duas strings para inicializar corretamente os atributos. Observe que o objeto `myCab` é realmente instanciado pela expressão `new Cab(serialNumber);`. Ao executar essa linha, memória é alocada para um objeto `Cab`, e `myCab` passa a referenciá-lo.

Figura 4.3 - O objeto Cabbie referenciando um objeto Cab.

O uso de dois construtores é um exemplo comum de sobrecarga. Ambos são `public` porque fazem parte da interface usada para criar objetos. Se fossem `private`, código externo não poderia chamá-los e, salvo mecanismos internos especiais, não conseguiria instanciar Cabbie dessa forma.

MÚLTIPLOS CONSTRUTORES

O livro observa que, em alguns estilos e frameworks modernos, oferecer muitos construtores pode ser considerado menos desejável. Contêineres de Inversão de Controle (IoC) e mecanismos de injeção de dependência às vezes esperam um padrão específico de construção ou exigem configuração especial quando existem múltiplas opções.

MÉTODOS DE ACESSO

Nos exemplos do livro, atributos são normalmente `private` para impedir acesso direto por outros objetos. Isso não significa que objetos devam viver isolados: eles precisam compartilhar informações adequadas, mas de maneira controlada.

Uma classe deve proteger seu próprio estado. O objeto `A` não deveria inspecionar ou alterar livremente atributos internos do objeto `B` sem que `B` controle o processo. As razões incluem integridade dos dados, segurança e depuração.

Imagine um erro no atributo `name` de Cabbie. Se o campo fosse público e qualquer classe pudesse alterá-lo, seria necessário procurar por todo o sistema para descobrir quem escreveu o valor incorreto. Se apenas os métodos de Cabbie puderem modificá-lo, a investigação fica concentrada. Esse acesso controlado é oferecido por métodos normalmente chamados de *accessors*, *getters* e *setters*.

getters and setters, and sometimes they’re simply called `get()` and `set()`. By convention, in this book we name the methods with the `set` and `get` prefixes, as in the following:

```
// Set the Name of the Cabbie
public void setName(String iName) {
    name = iName;
}

// Get the Name of the Cabbie
public String getName() {
    return name;
}
```

In this code snippet, a `Supervisor` object must ask the `Cabbie` object to return its name (see [Figure 4.4](#)). The important point here is that the `Supervisor` object can’t retrieve the information on its own; it must ask the `Cabbie` object for the information. This concept is important at many levels. For example, you might have a `setAge()` method that checks to see whether the age entered was 0 or below. If the age is less than 0, the `setAge()` method can refuse to set this obviously incorrect value. In general, the setters are used to ensure a level of data integrity.

The Supervisor Object Must Ask
The Cabbie Object to Return Its Name

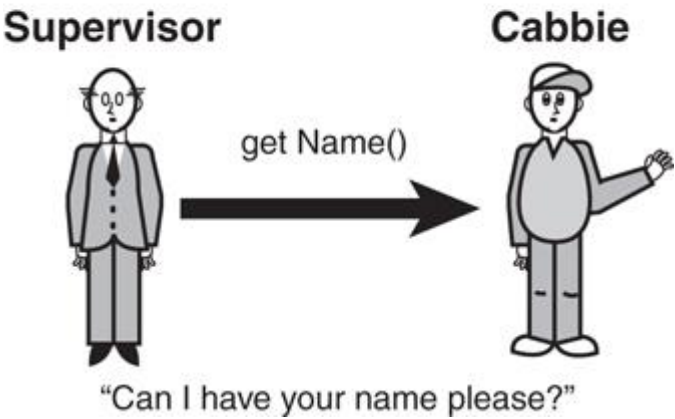


Figure 4.4 Asking for information.

This is also an issue of security. You may have sensitive data, such as passwords or payroll information, that you want to control access to. Thus, accessing data via getters and setters provides the capability to use mechanisms like password checks and other validation techniques. This greatly increases the integrity of the data.

Objects

Actually, there isn't a physical copy of each nonstatic method for each object. Each object would point to the same physical code. However, from a conceptual level, you can think of objects as being wholly independent and having their own attributes and methods.

The following code fragment illustrates how to define a static method, and [Figure 4.5](#) shows how more than one object points to the same code.

Tradução PT-BR - página original 72

Por convenção, este livro usa os prefixos `set` e `get`. O código original mostra `setName(String iName)`, que altera `name`, e `getName()`, que devolve o valor.

Um objeto `Supervisor`, portanto, não lê diretamente o campo `name`. Ele pede ao objeto `Cabbie` que informe seu nome chamando `getName()`.

Figura 4.4 - Solicitando informação a outro objeto.

Esse controle permite validar dados. Um método `setAge()`, por exemplo, pode rejeitar uma idade negativa. Setters podem proteger invariantes e aumentar a integridade do estado.

Também é uma questão de segurança. Dados sensíveis, como senhas e informações salariais, podem exigir verificação de autorização antes de serem devolvidos ou modificados. Getters e setters fornecem um ponto apropriado para validação, autenticação e outras políticas.

OBJETOS

Fisicamente, normalmente não existe uma cópia separada de cada método não estático para cada objeto; as instâncias podem compartilhar o mesmo código compilado. Conceitualmente, porém, é útil pensar em cada objeto como independente, com seu próprio estado e seus comportamentos disponíveis.

A continuação apresenta um exemplo de método `static` e mostra como várias instâncias podem utilizar o mesmo código.

Figura 4.5 - Alocação de memória para métodos.

ATRIBUTOS STATIC

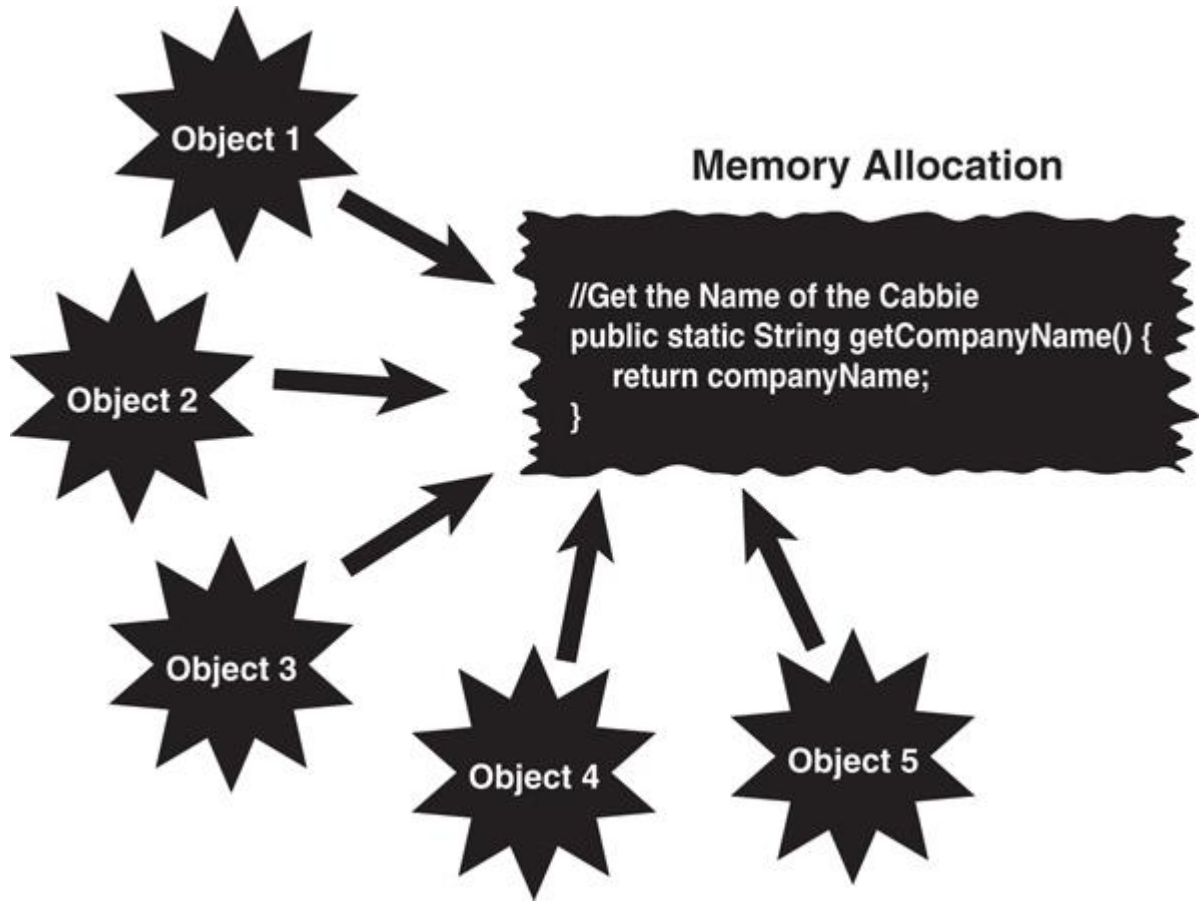
Se um atributo é static e existe um setter para ele, qualquer objeto que chamar esse setter altera a única cópia compartilhada. Portanto, o novo valor será observado por todas as instâncias da classe.

O exemplo `getCompanyName()` também é declarado static. Assim como atributos de classe, métodos static pertencem ao contexto da classe, e não ao estado de uma instância específica.

MÉTODOS DA INTERFACE PÚBLICA

Construtores e accessors são public e fazem parte da interface, mas não são os únicos métodos públicos. Grande parte do trabalho real da classe é oferecida por outras operações. Como discutido no Capítulo 2, a interface pública tende a expressar ações mais abstratas, enquanto a implementação concreta fica escondida.

Na classe Cabbie, um método chamado `giveDestination()` pode constituir a interface usada pelo passageiro para informar seu destino. O conteúdo interno do método não importa neste momento; o ponto central é que ele é público e faz parte do contrato da classe.



Figure

4.5 Method memory allocation.

Static Attributes

If an attribute is static, and the class provides a setter for that attribute, any object that invokes the setter will change the single copy. Thus, the value for the attribute will change for all objects.

[Click here to view code image](#)

```
// Get the Name of the Cabbie
public static String getCompanyName() {
    return companyName;
}
```

Notice that the `getCompanyName` method is declared as `static`, as a class method; class methods are described in more detail in [Chapter 3](#). Remember that the attribute `companyName` is also declared as `static`. A method, like an attribute, can be declared `static` to indicate that there is only one copy of the method for the entire class.

PUBLIC INTERFACE METHODS

Both the constructors and the accessor methods are declared as public and are part of the public interface. They are singled out because of their specific importance to the construction of the class. However, much of the *real* work is provided in other methods. As mentioned in [Chapter 2](#), the public interface methods tend to be very abstract, and the implementation tends to be more concrete. For this class, we provide a method called `giveDestination` that is the public interface for the user to describe where she wants to go:

```
public void giveDestination() {
    ...
}
```


What is inside this method is not important at this time. The main point is that this is a public method, and it is part of the public interface to the class.

PRIVATE IMPLEMENTATION METHODS

Although all the methods discussed so far in this chapter are defined as `public`, not all the methods in a class are part of the public interface. It is common for methods in a class to be hidden from other classes. These methods are declared as `private`:

```
private void turnRight(){
}
```

```
private void turnLeft() {
}
```

These private methods are meant to be part of the implementation and not the public interface. You might ask who invokes these methods, if no other class can. The answer is simple: You might have already surmised that these methods are called internally from the class itself. For example, these methods could be called from within the method `giveDestination`:

```
public void giveDestination(){
```

```
.. some code
```

```
    turnRight();
    turnLeft();
```

```
.. some more code
```

```
}
```

As another example, you may have an internal method that provides encryption that you will use only from within the class itself. In short, this encryption method can't be called from outside the instantiated object itself.

The point here is that private methods are strictly part of the implementation and are not accessible by other classes.

CONCLUSION

In this chapter we have gotten inside a class and described the fundamental concepts necessary for understanding how a class is built. Although this chapter takes a practical approach to discussing classes, [Chapter 5, “Class Design Guidelines,”](#) covers the class from a general design perspective.

REFERENCES

Fowler, Martin. 2003. *UML Distilled*, Third Edition. Boston, MA: Addison-Wesley Professional.

Gilbert, Stephen, and Bill McCarty. 1998. *Object-Oriented Design in Java*. Berkeley, CA: The Waite Group Press.

Tyma, Paul, Gabriel Torok, and Troy Downing. 1996. *Java Primer Plus*. Berkeley, CA: The Waite Group.

MÉTODOS PRIVADOS DE IMPLEMENTAÇÃO

Nem todos os métodos de uma classe pertencem à interface pública. É comum ocultar operações auxiliares declarando-as `private`. O original mostra `turnRight()` e `turnLeft()` como métodos privados.

Esses métodos fazem parte da implementação. Se nenhum outro objeto pode chamá-los, quem os usa? A própria classe. `giveDestination()`, por exemplo, pode executar internamente `turnRight()` e `turnLeft()` durante sua implementação.

Outro exemplo seria um método privado de criptografia usado apenas por operações públicas da classe. Código externo não pode invocá-lo diretamente.

A ideia é simples: métodos `private` são detalhes estritamente internos e não pertencem ao contrato oferecido a outras classes.

CONCLUSÃO

Neste capítulo entramos dentro de uma classe e examinamos os conceitos fundamentais de sua construção: nome, comentários, atributos, construtores, métodos de acesso, interfaces públicas e métodos privados de implementação. O Capítulo 5 examina a classe a partir de uma perspectiva mais ampla de projeto.

REFERÊNCIAS

Fowler, Martin. UML Distilled, 3ª ed., 2003.

Gilbert, Stephen; McCarty, Bill. Object-Oriented Design in Java, 1998.

Tyma, Paul; Torok, Gabriel; Downing, Troy. Java Primer Plus, 1996.

5. Class Design Guidelines

As we have already discussed, OO programming supports the idea of creating classes that are complete packages, encapsulating the data and behavior of a single entity. So, a class should represent a logical component, such as a taxicab.

This chapter presents several suggestions for designing classes. Obviously, no list such as this can be considered complete. You will undoubtedly add many guidelines to your personal list and incorporate useful guidelines from other developers.

MODELING REAL-WORLD SYSTEMS

One of the primary goals of object-oriented (OO) programming is to model real-world systems in ways similar to the ways in which people actually think. Designing classes is the object-oriented way to create these models. Rather than using a structured, or *top-down*, approach, where data and behavior are logically separate entities, the OO approach encapsulates the data and behavior into objects that interact with each other. We no longer think of a problem as a sequence of events or routines operating on separate data files. The elegance of this mindset is that classes literally model real-world objects and how these objects interact with other real-world objects.

These interactions occur in a way similar to the interactions between real-world objects, such as people. Thus, when creating classes, you should design them in a way that represents the true behavior of the object. Let's use the cabbie example from previous chapters. The `Cab` class and the `Cabbie` class model a real-world entity. As illustrated in [Figure 5.1](#), the `Cab` and the `Cabbie` objects encapsulate their data and behavior, and they interact through each other's public interfaces.

Cabbie

Cab

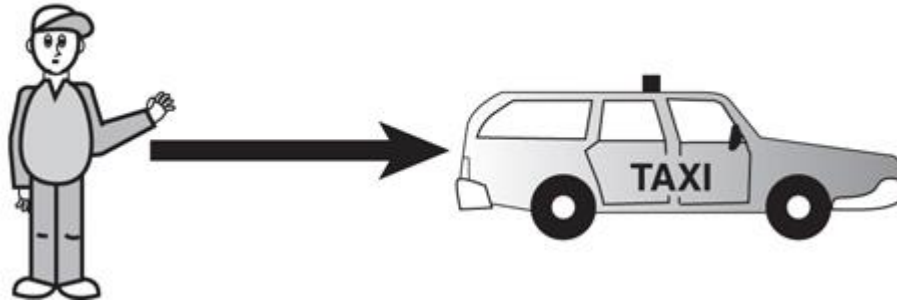


Figure 5.1 A cabbie and a cab are

real-world objects.

When OO programming was first becoming popular, it was difficult for many structured programmers to make the transition. One primary mistake structured programmers made was to create a class that had behavior but no class data, in effect creating a set of functions or subroutines in the structured model. This was not desirable because it didn't take advantage of the power of encapsulation.

This is only partially true now. Currently, much development is done with anemic domain models, a.k.a. data transfer objects (DTOs) and view models that have just enough data to populate a view or just the right amount of data that is needed by a consumer. Much more focus has been placed on behaviors and operating on the data, and that is handled via interfaces. Encapsulating the behaviors into single-responsibility interfaces and coding to the interfaces keeps code flexible and modular and far easier to maintain.

5. DIRETRIZES PARA O PROJETO DE CLASSES

Como já discutimos, POO apoia a criação de classes que funcionam como pacotes completos, encapsulando dados e comportamentos de uma entidade. Uma classe deve representar um componente lógico coerente, como um táxi.

Este capítulo apresenta várias sugestões para projetar classes. Nenhuma lista pode ser considerada definitiva. Com experiência, você provavelmente acrescentará suas próprias diretrizes e incorporará boas práticas aprendidas com outros desenvolvedores.

MODELAGEM DE SISTEMAS DO MUNDO REAL

Um dos principais objetivos da orientação a objetos é modelar sistemas do mundo real de maneira semelhante à forma como as pessoas pensam sobre eles. Projetar classes é o mecanismo OO para construir esses modelos.

Em vez de uma abordagem estritamente estruturada/top-down, em que dados e comportamentos aparecem como entidades separadas, OO reúne estado e comportamento em objetos que interagem. O problema deixa de ser visto apenas como uma sequência de rotinas operando sobre arquivos e passa a ser visto como uma comunidade de entidades.

A elegância dessa abordagem está em permitir que classes modelem objetos do mundo real e suas interações. O exemplo Cab/Cabbie ilustra isso: ambos encapsulam dados e comportamentos e cooperam através de suas interfaces públicas.

Figura 5.1 - Um motorista e um táxi como objetos do mundo real.

Quando OO começou a se popularizar, muitos programadores estruturados tiveram dificuldade de transição. Um erro comum era criar classes com comportamento, mas sem estado, reproduzindo apenas conjuntos de funções e sub-rotinas. Isso desperdiçava parte do potencial do encapsulamento.

Hoje essa afirmação precisa de nuance. Muitos sistemas utilizam modelos de domínio "anêmicos", DTOs e view models que carregam apenas os dados necessários a um consumidor. Ao mesmo tempo, há forte ênfase em separar comportamentos em interfaces de responsabilidade única. Programar contra interfaces pode manter o código flexível, modular e mais fácil de manter.

Note

One of my favorite books pertaining to class design guidelines and suggestions remains *Effective C++: 50 Specific Ways to Improve Your Programs and Designs*, by Scott Meyers. It offers important information about program design in a very concise manner.

One of the reasons why *Effective C++* interests me so much is that, because C++ is backward compatible with C, you can write structured code in C++ without using OO design principles. As I mentioned earlier, during interviews, some people claim that they are OO programmers simply because they program in C++. This indicates a total misunderstanding of what OO design is all about. Thus, you may have to pay more attention to the OO design issues in languages such as C++ as opposed to Java, Swift, or .NET.

IDENTIFYING THE PUBLIC INTERFACES

It should be clear by now that perhaps the most important issue when designing a class is to keep the public interface to a minimum. The entire purpose of building a class is to provide something useful and concise. In their book *Object-Oriented Design in Java*, Gilbert and McCarty state that “the interface of a well-designed object describes the services that the client wants accomplished.” If a class does not provide a useful service to a user, it should not have been built in the first place.

The Minimum Public Interface

Providing the minimum public interface makes the class as concise as possible. The goal is to provide the user with the exact interface to do the job right. If the public interface is incomplete (that is, there is missing behavior), the user will not be able to do the complete job. If the public interface is not properly restricted (that is, the user has access to behavior that is unnecessary or even dangerous), problems can result in the need for debugging, and even trouble with system integrity and security can surface.

Creating a class is a business proposition, and as with all steps in the design process, it is very important that the users are involved with the design right from the start and throughout the testing phase. In this way, the utility of the class, as well as the proper interfaces, will be assured.

Extending the Interface

Even if the public interface of a class is insufficient for a certain application, object technology easily allows the capability to extend and adapt this interface. In short, if properly designed, a new class can utilize an existing class and create a new class with an extended interface.

This is the point where, if you're adding behaviors, the developers should not be using inheritance,

To illustrate, consider the cabbie example again. If other objects in the system need to get the name of a cabbie, the `Cabbie` class must provide a public interface to return its name; this is the `getName()` method. Thus, if a `Supervisor` object needs a name from a `Cabbie` object, it must invoke the `getName()` method from the `Cabbie` object. In effect, the supervisor is asking the cabbie for its name (see [Figure 5.2](#)).

NOTA

O autor recomenda *Effective C++*, de Scott Meyers, como uma de suas obras favoritas sobre diretrizes de projeto. O livro apresenta recomendações de forma concisa.

C++ é compatível retroativamente com C, portanto é possível escrever código estruturado em C++ sem usar princípios OO. Isso explica por que simplesmente programar em uma linguagem que possui recursos de objetos não transforma alguém automaticamente em projetista OO. Em linguagens que não impõem uma estrutura OO, é necessário atenção especial às decisões de projeto.

IDENTIFICANDO AS INTERFACES PÚBLICAS

Talvez a decisão mais importante ao projetar uma classe seja manter a interface pública mínima. O objetivo de uma classe é oferecer um conjunto útil e conciso de serviços. Como citado no original, a interface de um objeto bem projetado descreve os serviços que o cliente deseja que sejam realizados. Se a classe não oferece um serviço útil, sua existência deve ser questionada.

A INTERFACE PÚBLICA MÍNIMA

Uma interface mínima torna a classe mais concisa. O objetivo é oferecer exatamente o que o usuário precisa para realizar a tarefa. Se faltar comportamento, o cliente não conseguirá completar o trabalho. Se houver acesso excessivo a operações desnecessárias ou perigosas, surgem riscos de erros, integridade e segurança.

Criar uma classe também é uma decisão de negócio. Usuários devem participar do projeto desde o levantamento inicial até os testes, para validar a utilidade da classe e a adequação das interfaces.

ESTENDENDO A INTERFACE

Mesmo que a interface pública de uma classe seja insuficiente para uma aplicação específica, tecnologias OO oferecem mecanismos para adaptação e extensão. Um novo componente pode reutilizar uma classe existente e expor novos comportamentos sem alterar desnecessariamente a classe original.

No exemplo Cabbie, se um Supervisor precisa saber o nome do motorista, Cabbie deve oferecer uma interface pública como `getName()`. O supervisor pede a informação ao motorista por meio dessa operação, em vez de acessar diretamente seu estado.

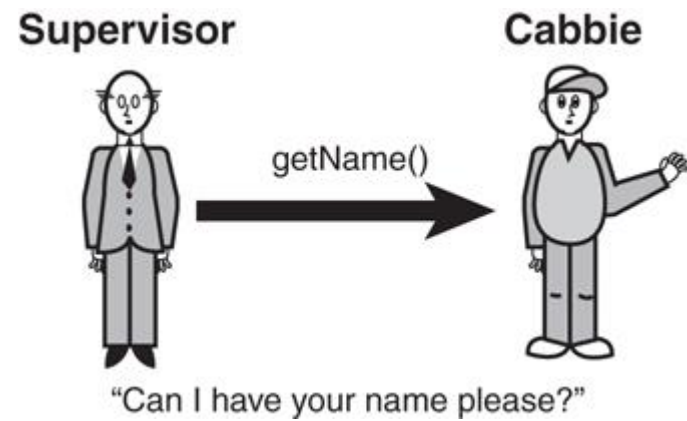


Figure 5.2 The public interface specifies how the objects

interact.

Users of your code need to know nothing about its internal workings. All they need to know is how to instantiate and use the object. In short, provide users a way to get there but hide the details.

Hiding the Implementation

The need for hiding the implementation has already been covered in great detail. Whereas identifying the public interface is a design issue that revolves around the users of the class, the implementation should not involve the users at all. The implementation must provide the services that the user needs, but how these services are actually performed should not be made apparent to the user. A class is most useful if the implementation can change without affecting the users. Basically, a change to the implementation should not necessitate a change in the user’s application code. Again, the best way to enable change of behaviors is via the use of interfaces and composition.

Customer Versus User

Sometimes I use the term *customer* rather than *user* when referring to the people who will actually be using the software. Users of the system may, in fact, be customers. In the same vein, users who are part of your organization can be called *internal customers*. This may seem like a trivial point, but I think it is important to think of all end users as actual customers—and you must satisfy their requirements.

In the cabbie example, the `Cabbie` class might contain behavior pertaining to how, or where, he eats lunch. However, the cabbie’s supervisor does not need to know what the cabbie has for lunch. Thus, this behavior is part of the implementation of the `Cabbie` object and should not be available to other objects in this system (see [Figure 5.3](#)). Gilbert and McCarty state that the prime directive of encapsulation is that “all fields shall be private.” In this way, none of the fields in a class are accessible from other objects.

Figura 5.2 - A interface pública especifica como os objetos interagem.

Usuários do código não precisam conhecer seu funcionamento interno. Precisam apenas saber como criar e usar o objeto. Forneça um caminho claro para realizar a tarefa e esconda os detalhes.

OCULTANDO A IMPLEMENTAÇÃO

A necessidade de esconder implementação já foi discutida em profundidade. A interface pública é projetada em torno das necessidades dos usuários; a implementação não deveria exigir conhecimento do usuário. Ela deve entregar os serviços prometidos, mas a forma de realizá-los deve permanecer oculta.

Uma classe é especialmente útil quando sua implementação pode mudar sem afetar o código cliente. Para permitir mudanças de comportamento com baixo acoplamento, interfaces e composição são ferramentas importantes.

CLIENTE VERSUS USUÁRIO

O autor às vezes usa “cliente” em vez de “usuário” para enfatizar que quem utiliza o software deve ser tratado como alguém cujos requisitos precisam ser satisfeitos. Usuários internos de uma organização também podem ser vistos como clientes internos.

No exemplo, Cabbie pode possuir comportamentos sobre onde ou como o motorista almoça. O Supervisor não precisa conhecer esse detalhe. Portanto, ele pertence à implementação do objeto e não deve ser exposto àquele cliente.

Figura 5.3 - Objetos não precisam conhecer todos os detalhes de implementação.

Uma diretriz forte de encapsulamento é manter campos privados, impedindo que outros objetos manipulem diretamente o estado interno da classe.

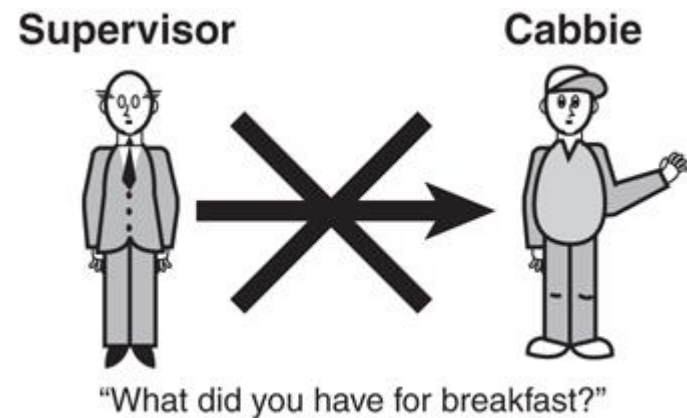


Figure 5.3 Objects don't need to know some implementation details.

DESIGNING ROBUST CONSTRUCTORS (AND PERHAPS DESTRUCTORS)

When designing a class, one of the most important design issues involves how the class will be constructed. Constructors are discussed in [Chapter 3](#), “[More Object-Oriented Concepts](#).” Revisit this discussion if you need a refresher on guidelines for designing constructors.

First and foremost, a constructor should put an object into an initial, safe state. This includes issues such as attribute initialization and memory management. You also need to make sure the object is constructed properly in the default condition. It is normally a good idea to provide a constructor to handle this default situation.

In languages that include destructors, it is of vital importance that the destructors include proper clean-up functions. In most cases, this clean-up pertains to releasing system memory that the object acquired at some point. Java and .NET reclaim memory automatically via a garbage collection mechanism. In languages such as C++, the developer must include code in the destructor to properly free up the memory that the object acquired during its existence. If this function is ignored, a memory leak will result.

Constructor Injection

This is a good point at which to introduce the concept of constructor injection, where service classes are injected on object creation (via a constructor) instead of within the class (using the `new` keyword). For example, the cabbie can get his license object, his radio information object (frequency, call sign, etc.), and the key that starts his cab passes into the object via a constructor.

Memory Leaks

When an object fails to properly release the memory that it acquired during an object's life cycle, the memory is lost to the entire operating system as long as the application that created the object is executing. For example, suppose multiple objects of the same class are created and then destroyed, perhaps in some sort of loop. If these objects fail to release their memory when they go out of scope, this memory leak slowly depletes the available pool of system memory. At some point, it is possible that enough memory will be consumed that the system will have no available memory left to allocate. This means that any application executing in the system would be unable to acquire any memory. This could put the application in an unsafe state and even lock up the system.

DESIGNING ERROR HANDLING INTO A CLASS

As with the design of constructors, designing how a class handles errors is of vital importance. Error handling is discussed in detail in [Chapter 3](#).

PROJETANDO CONSTRUTORES ROBUSTOS - E TALVEZ DESTRUTORES

A forma como uma classe é construída é uma das decisões mais importantes de projeto. O construtor deve colocar o objeto em um estado inicial seguro, incluindo inicialização de atributos e tratamento adequado de recursos. Também deve ser considerada a construção no caso padrão.

Em linguagens que possuem destrutores explícitos, eles devem realizar a limpeza correta, especialmente liberando memória ou outros recursos adquiridos pelo objeto. Java e .NET utilizam coleta de lixo para recuperar memória gerenciada. Em C++, o desenvolvedor frequentemente precisa assegurar explicitamente a liberação de recursos; ignorar isso pode causar vazamentos de memória.

INJEÇÃO PELO CONSTRUTOR

Este é um bom ponto para introduzir constructor injection. Em vez de uma classe criar internamente todas as dependências com `new`, serviços podem ser fornecidos ao objeto no momento da construção. Um Cabbie, por exemplo, poderia receber através do construtor objetos que representam licença, informações de rádio e chave do veículo.

VAZAMENTOS DE MEMÓRIA

Quando um objeto não libera corretamente recursos de memória que adquiriu, essa memória pode permanecer indisponível enquanto a aplicação continua executando. Se muitos objetos forem criados e descartados repetidamente sem liberação adequada, o vazamento pode consumir progressivamente a memória disponível e colocar o sistema em estado inseguro.

PROJETANDO TRATAMENTO DE ERROS NA CLASSE

Assim como construtores, mecanismos de erro precisam ser considerados desde o projeto. O assunto foi introduzido no Capítulo 3.

It is virtually certain that every system will encounter unforeseen problems. Thus, it is not a good idea to ignore potential errors. The developer of a good class (or any code, for that matter) anticipates potential errors and includes code to handle these conditions when they are encountered.

The general rule is that the application should never crash. When an error is encountered, the system should either fix itself and continue, or at minimum, exit gracefully without losing any data that’s important to the user.

Documenting a Class and Using Comments

The topic of comments and documentation comes up in most programming books and articles, in every code review, and in every discussion you have about good design. Unfortunately, comments and good documentation are often not taken seriously, or even worse, they are ignored.

Most developers know that they should thoroughly document their code, but they don’t usually want to take the time to do it. However, a good design is practically impossible without good documentation practices. At the class level, the scope might be small enough that a developer can get away with shoddy documentation. However, when the class gets passed to someone else to extend and/or maintain, or it becomes part of a larger system (which is what should happen), a lack of proper documentation and comments can undermine the entire system.

Many people have said all this before. One of the most crucial aspects of a good design, whether it’s a design for a class or something else, is to carefully document the process. Implementations such as Java and .NET provide special comment syntax to facilitate the documentation process. Check out [Chapter 4](#), “[The Anatomy of a Class](#),” for the appropriate syntax.

Too Much Documentation

Be aware that over-commenting can be a problem as well. Too much documentation and/or commenting can become background noise and may actually defeat the purpose of the documentation. Just like in good class design, make the documentation and comments straightforward and to the point. Well-written code is, in itself, the best documentation.

Building Objects with the Intent to Cooperate

We can safely say that almost no class lives in isolation. In most cases, there is little reason to build a class if it is not going to interact with other classes, unless the class will be used only once. This is a fact in the life of a class. A class will service other classes; it will request the services of other classes, or both. In later chapters we discuss various ways that classes interact with each other.

In the cabbie example, the cabbie and the supervisor are not standalone entities; they interact with each other at various levels (see [Figure 5.4](#)).

É praticamente certo que todo sistema encontrará problemas não previstos. Ignorar possíveis erros não é uma boa estratégia. Um bom desenvolvedor antecipa condições de falha e inclui maneiras de tratá-las.

A regra geral é que a aplicação não deveria simplesmente “cair”. Quando um erro ocorre, o sistema deve, quando possível, recuperar-se e continuar ou, no mínimo, encerrar de forma controlada sem perder dados importantes do usuário.

DOCUMENTANDO UMA CLASSE E USANDO COMENTÁRIOS

Comentários e documentação aparecem em praticamente todos os livros, revisões de código e discussões sobre bom projeto. Apesar disso, frequentemente são tratados com pouca seriedade ou ignorados.

A maioria dos desenvolvedores sabe que deveria documentar o código, mas nem sempre dedica tempo a isso. Uma boa arquitetura, porém, torna-se muito difícil de sustentar sem documentação adequada. Em uma classe pequena, um único autor talvez consiga sobreviver com documentação fraca; quando a classe é entregue a outra pessoa, estendida, mantida ou integrada a um sistema maior, a ausência de documentação pode comprometer o projeto.

Linguagens e plataformas como Java e .NET oferecem sintaxes especiais para gerar documentação a partir de comentários.

DOCUMENTAÇÃO EM EXCESSO

Comentar demais também pode ser um problema. Documentação excessiva vira ruído e pode esconder aquilo que realmente importa. Como no projeto de classes, comentários devem ser claros, diretos e úteis. Código bem escrito é, por si só, uma importante forma de documentação.

CONSTRUINDO OBJETOS COM A INTENÇÃO DE COOPERAR

Quase nenhuma classe vive isoladamente. Em geral, uma classe presta serviços a outras, solicita serviços ou faz as duas coisas. Nos próximos capítulos serão apresentadas várias formas de interação.

No exemplo, Cabbie e Supervisor não são entidades independentes; eles se comunicam em diferentes níveis.

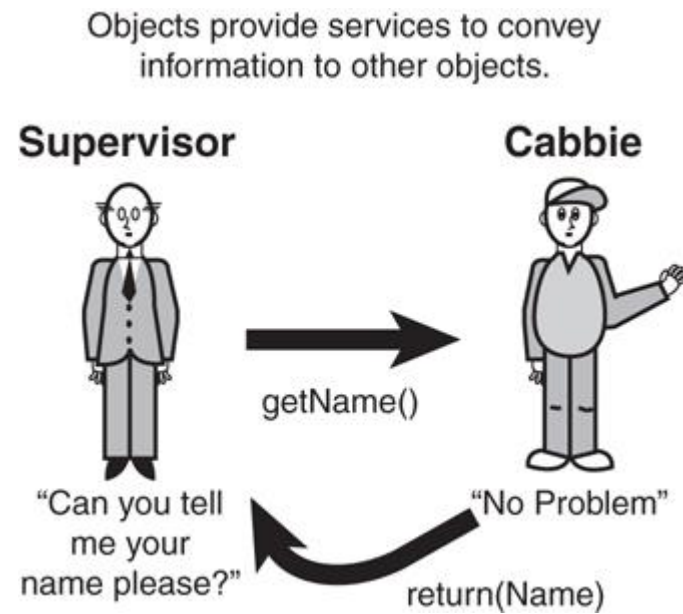


Figure 5.4 Objects should request information.

When designing a class, make sure you are aware of how other objects will interact with it.

DESIGNING WITH REUSE IN MIND

Objects can be reused in different systems, and code should be written with reuse in mind. For example, when a `Cabbie` class is developed and tested, it can be used anywhere you need a cabbie. To make a class usable in various systems, the class must be designed with reuse in mind. This is where much of the thought is required in the design process. Attempting to predict all the possible scenarios in which a `Cabbie` object must operate is not a trivial task—in fact, it is virtually impossible.

DESIGNING WITH EXTENSIBILITY IN MIND

Adding new features to a class might be as simple as extending an existing class, adding a few new methods, and modifying the behavior of others. It is not necessary to rewrite everything. This is where inheritance comes into play. If you have just written a `Person` class, you must consider the fact that you might later want to write an `Employee` class or a `Customer` class. Thus, having `Employee` inherit from `Person` might be the best strategy; in this case, the `Person` class is said to be *extensible*. You do not want to design `Person` so that it contains behavior that prevents it from being extended by classes such as `Employee` or `Customer` (assuming that in your design you really intend for other classes to extend `Person`). For example, you would not want to code functionality into an `Employee` class that is specific to supervisory functions. If you did, and then a class that does not require supervisory functionality inherited from `Employee`, you would have a problem.

This point touches on the abstraction guideline discussed earlier. `Person` should contain only the data and behaviors that are specific to a person. Other classes can then subclass it and inherit appropriate data and behaviors.

As we will cover in the SOLID discussion in [Chapter 11, “Avoiding Dependencies and Highly Coupled Classes,”](#) and [Chapter 12, “The SOLID Principles of Object-Oriented Design,”](#) classes should be open for extension but closed for modification. By using interfaces first and coding to them, you can use all sorts of patterns like Decorator to extend things without touching the code that’s been tested and deployed live, for example.

Figura 5.4 - Objetos devem solicitar informações uns aos outros.

Ao projetar uma classe, tenha consciência de como outros objetos interagem com ela.

PROJETANDO COM REUTILIZAÇÃO EM MENTE

Objetos podem ser reutilizados em sistemas diferentes, portanto vale a pena escrever código pensando em reuso. Depois que `Cabbie` foi desenvolvido e testado, por exemplo, ele pode ser empregado em outros contextos que precisem representar um motorista de táxi.

Projetar para reutilização exige pensamento cuidadoso, porque é praticamente impossível prever todos os cenários futuros em que uma classe será utilizada. O objetivo não é adivinhar tudo, mas evitar acoplamentos desnecessários e responsabilidades excessivamente específicas.

PROJETANDO COM EXTENSIBILIDADE EM MENTE

Adicionar recursos a uma classe pode ser tão simples quanto criar uma extensão apropriada, adicionar novos comportamentos ou adaptar outros. Não deve ser necessário reescrever tudo.

Herança pode participar desse processo quando existe uma relação conceitual adequada. Se já temos uma classe `Person` e depois surgem `Employee` e `Customer`, pode fazer sentido que essas classes aproveitem características gerais de `Person`. Isso exige que `Person` contenha somente dados e comportamentos realmente próprios de uma pessoa.

Não devemos colocar em `Employee` comportamentos exclusivos de supervisores se também esperamos criar subclasses de empregados que não sejam supervisores. A classe base precisa manter uma abstração coerente.

Nos capítulos sobre SOLID, essa ideia aparece na máxima de que classes devem estar abertas para extensão e fechadas para modificação. Projetar primeiro em torno de interfaces e contratos permite utilizar padrões como Decorator para acrescentar comportamento sem alterar código já testado e implantado.

QUE ATRIBUTOS E MÉTODOS PODEM SER STATIC?

Métodos static aumentam o acoplamento direto à classe. Eles não participam da mesma forma de abstrações de instância, são difíceis de substituir em testes e não implementam um contrato polimórfico do mesmo modo que objetos comuns. Em código de aplicação, classes static costumam ser mais adequadas para funções auxiliares puras, sem efeitos colaterais. Uma classe estática que apenas soma números é simples; uma classe estática que acessa diretamente banco de dados ou serviço web cria dependências muito mais rígidas.

CRIANDO NOMES DESCRITIVOS

Já discutimos documentação e comentários. Convenções de nomes para classes, atributos e métodos seguem a mesma lógica. Existem muitas convenções; mais importante do que escolher uma específica é adotar uma que faça sentido e usá-la consistentemente. Além disso, os nomes precisam ser descritivos. Quem lê o código deveria conseguir inferir o papel de um elemento pelo nome. Organizações frequentemente formalizam essas regras em padrões de codificação.

BONS NOMES

Uma convenção deve ser compreensível para toda a equipe. Regras excessivamente artificiais podem fazer sentido apenas para quem as criou. Variáveis devem descrever seu propósito, em vez de codificar informações redundantes ou obscuras sobre seu tipo.

Usar nomes descritivos é uma boa prática que transcende paradigmas de desenvolvimento.

ABSTRAINDO CÓDIGO NÃO PORTÁVEL

Se um sistema precisa usar código nativo ou dependente de uma plataforma específica, isole esse código em sua própria classe ou, pelo menos, em um método substituível. Se for necessário acessar diretamente uma porta serial de determinado hardware, por exemplo, crie uma classe wrapper responsável por essa comunicação. A classe principal pede ao wrapper as informações ou serviços de que necessita. Evite espalhar código dependente do sistema pela lógica principal.

Figura 5.5 - Wrapper para uma porta serial.

What Attributes and Methods Can Be Static?

Static methods promote strong coupling to classes. You cannot abstract a static method. You cannot mock a static method or static class. You cannot provide a static interface. The only time it is reasonable to use static classes (within application development—framework development is a bit different) is if you're working with some sort of helper class or extension method that does not produce side effects. For example, a static class to add numbers is fine. A static class that interacts with a database or a web service is not.

Making Names Descriptive

Earlier we discussed the use of proper documentation and comments. Following a naming convention for your classes, attributes, and methods is a similar subject. There are many naming conventions, and the convention you choose is not as important as choosing one and sticking to it. However, when you choose a convention, make sure that when you create classes, attributes, and method names, you not only follow the convention but also make the names descriptive. When someone reads the name, he should be able to tell from the name what the object represents. These naming conventions are often dictated by the coding standards at various organizations.

Good Naming

Make sure that a naming convention makes sense. Often, people go overboard and create con-ventions that might make sense to them but are totally incomprehensible to others. Take care when forcing others to conform to a convention. Make sure that the conventions are sensible and that everyone involved understands the intent behind them. Make variables descriptive of their use, not encoded based on their type.

Making names descriptive is a good development practice that transcends the various development paradigms.

Abstracting Out Nonportable Code

If you are designing a system that must use nonportable (native) code (that is, the code will run only on a specific hardware platform), you should abstract this code out of the class. By abstracting out, we mean isolating the nonportable code in its own class or at least its own method (a method that can be overridden). For example, if you are writing code to access a serial port of particular hardware, you should create a wrapper class to deal with it. Your class should then send a message to the wrapper class to get the information or services it needs. Do not put the system-dependent code into your primary class (see [Figure 5.5](#)).

Ao interagir diretamente com hardware, implementações para plataformas diferentes provavelmente terão código de baixo nível diferente. Se essa variação ficar encapsulada em uma classe wrapper, o cliente utiliza uma interface uniforme e o próprio wrapper decide qual implementação específica deve executar.

FORNECENDO UMA FORMA DE COPIAR E COMPARAR OBJETOS

O Capítulo 3 discutiu cópia e comparação. Uma cópia bit a bit ou uma comparação padrão pode não representar a semântica desejada. Ao projetar uma classe, defina claramente como objetos devem ser copiados e quando dois objetos devem ser considerados equivalentes.

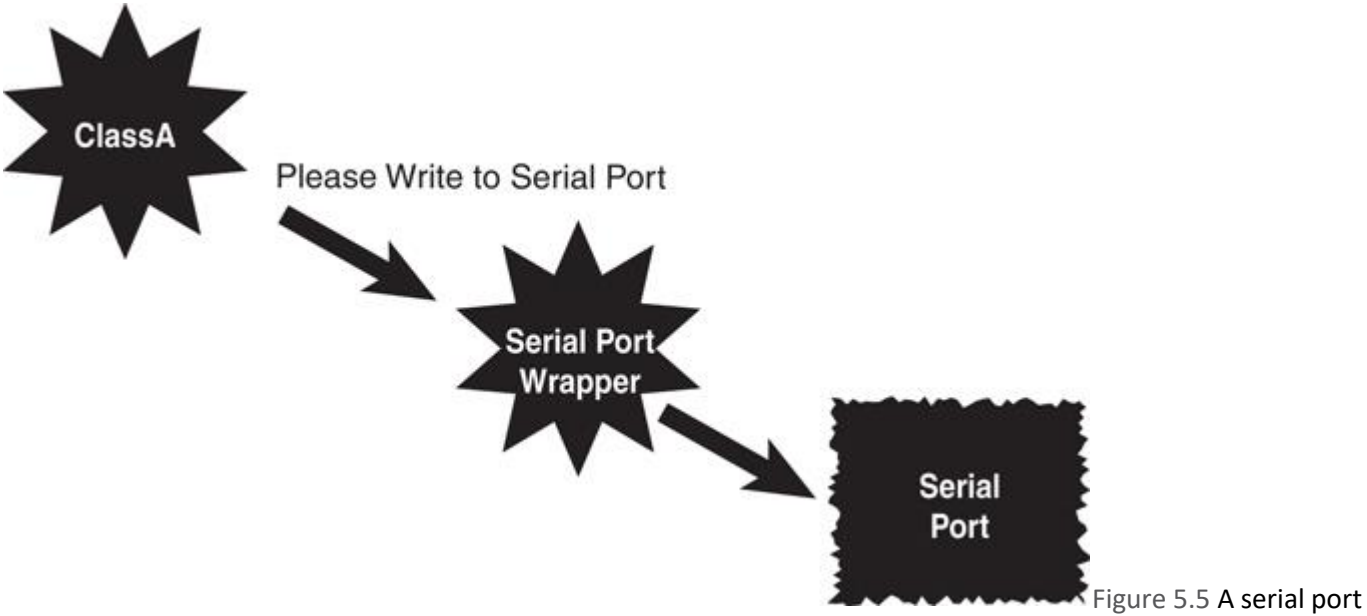
MANTENDO O ESCOPO O MENOR POSSÍVEL

Manter escopo pequeno combina bem com abstração e ocultação da implementação. Localize atributos e comportamentos tanto quanto possível. Isso facilita manutenção, teste e extensão. Interfaces também ajudam a limitar dependências e a exposição de detalhes.

ESCOPO E DADOS GLOBAIS

Minimizar dados globais é uma boa prática geral, não exclusiva de OO. Em projeto OO, membros static podem ser compartilhados entre instâncias de uma mesma classe, mas continuam associados a um tipo e não constituem, conceitualmente, estado público universal. Dados também podem ser compartilhados por arquivos, bancos de dados e outros serviços, mas esses mecanismos devem ser controlados.

Se uma variável é necessária apenas temporariamente em um método, mantenha-a local. O exemplo seguinte mostra uma variável temp inicialmente declarada no nível da classe, embora seja usada somente por swap().



wrapper.

For example, consider the situation when a programmer is interfacing directly with hardware. In these cases, the object code of the various platforms will most likely be quite different, and thus code must be written for each platform. However, if the functionality is placed in a wrapper class, then a user of the class can interface directly with the wrapper and not have to worry about the various low-level code. The wrapper class will deal with the differences in these platforms and decide which code to invoke.

Providing a Way to Copy and Compare Objects

Chapter 3 discussed the issue of copying and comparing objects. It is important to understand how objects are copied and compared. You might not want, or expect, a simple bitwise copy or compare operation. You must make sure that your class behaves as expected, and this means you have to spend some time designing how objects are copied and compared.

Keeping the Scope as Small as Possible

Keeping the scope as small as possible goes hand-in-hand with abstraction and hiding the implementation. The idea is to localize attributes and behaviors as much as possible. In this way, maintaining, testing, and extending a class are much easier. Using interfaces is a great way to enforce this.

Scope and Global Data

Minimizing the scope of global variables is a good programming style and is not specific to OO programming. Global variables are allowed in structured development, yet they can get dicey. In fact, there is no global data in OO development. Static attributes and methods are shared among objects of the same class; however, they are not available to objects not of the class. You could also share data via a file or database.

For example, if you have a method that requires a temporary attribute, keep it local. Consider the following code:

[Click here to view code image](#)

```
public class Math {
```



```
int temp=0;

public int swap (int a, int b) {

    temp = a;
    a=b;
    b=temp;

    return temp;

}
```

What is wrong with this class? The problem is that the attribute `temp` is needed only within the scope of the `swap()` method. There is no reason for it to be at the class level. Thus, you should move `temp` within the scope of the `swap()` method:

[Click here to view code image](#)

```
public class Math {

    public int swap (int a, int b) {

        int temp=0;

        temp = a;
        a=b;
        b=temp;

        return temp;

    }

}
```

This is what is meant by keeping the scope as small as possible].

DESIGNING WITH MAINTAINABILITY IN MIND

Designing useful and concise classes promotes a high level of maintainability. Just as you design a class with extensibility in mind, you should also design with future maintenance in mind.

The process of designing classes forces you to organize your code into many (ideally) manageable pieces. Separate pieces of code tend to be more maintainable than larger pieces of code (at least that’s the idea). One of the best ways to promote maintainability is to reduce interdependent code—that is, changes in one class have no impact or minimal impact on other classes.

Highly Coupled Classes

Classes that are highly dependent on one another are considered highly coupled. Thus, if a change made to one class forces a change to another class, these two classes are considered highly coupled. Classes that have no such dependencies have a very low degree of coupling. For more information on this topic, refer to *The Object Primer*, by Scott Ambler.

If the classes are designed properly in the first place, any changes to the system should be made only to the implementation of an object. Changes to the public interface should be avoided at all costs. Any changes to the public interface will cause ripple effects throughout all the systems that use the interface.

No exemplo original, `temp` é usada apenas dentro de `swap()`. Não há motivo para armazená-la como atributo da classe. A solução é declará-la dentro do próprio método, reduzindo seu escopo.

Esse é exatamente o princípio de manter o escopo tão pequeno quanto possível.

PROJETANDO COM MANUTENIBILIDADE EM MENTE

Classes úteis, concisas e com responsabilidades claras tendem a ser mais fáceis de manter. Assim como projetamos pensando em extensibilidade, devemos considerar desde o início a manutenção futura.

Projetar classes organiza o código em partes menores e, idealmente, gerenciáveis. Partes separadas são mais fáceis de modificar quando dependem pouco umas das outras. Uma das melhores estratégias de manutenção é reduzir código interdependente: uma mudança em uma classe deve ter impacto mínimo nas demais.

CLASSES ALTAMENTE ACOPLADAS

Classes que dependem fortemente umas das outras são altamente acopladas. Se alterar uma classe obriga a alterar outra, há acoplamento significativo. Classes com poucas dependências apresentam acoplamento baixo.

Se o projeto for bem feito, muitas mudanças futuras podem ficar restritas à implementação de um objeto. Alterações na interface pública devem ser evitadas porque geram efeitos em cascata em todos os clientes que dependem dela.

For example, if a change were made to the `getName()` method of the `Cabbie` class, every single place in all systems that use this interface must be changed and recompiled. Finding all these method calls is a daunting task, and the likelihood of missing one is pretty high.

To promote a high level of maintainability, keep the coupling level of your classes as low as possible.

Using Iteration in the Development Process

As in most design and programming functions, using an iterative process is recommended. This dovetails well with the concept of providing minimal interfaces. Basically, this means *don't write all the code at once!* Create the code in small increments and then build and test it at each step. A good testing plan quickly uncovers any areas where insufficient interfaces are provided. In this way, the process can iterate until the class has the appropriate interfaces. This testing process is not simply confined to coding. Testing the design with walkthroughs and other design review techniques is very helpful. Testers' lives are more pleasant when iterative processes are used, because they are involved in the process early and are not simply handed a system that is thrown over the wall at the end of the development process.

Testing the Interface

The minimal implementations of the interface are often called *stubs*. (Gilbert and McCarty have a good discussion on stubs in *Object-Oriented Design in Java*.) By using stubs, you can test the interfaces without writing any *real* code. In the following example, rather than connecting to an actual database, stubs are used to verify that the interfaces are working properly (from the user's perspective—remember that interfaces are meant for the user). Thus, the implementation is not necessary at this point. In fact, it might cost valuable time and energy to complete the implementation yet because the design of the interface will affect the implementation, and the interface is not yet complete.

In **Figure 5.6**, note that when a user class sends a message to the `DataBaseReader` class, the information returned to the user class is provided by code stubs and not by the actual database. (In fact, the database most likely does not exist yet.) When the interface is complete and the implementation is under development, the database can then be connected and the stubs disconnected.

Se o contrato de `getName()` em `Cabbie` mudar, por exemplo, cada chamada desse método em todos os sistemas clientes poderá precisar ser revisada e recompilada. Localizar todas essas dependências é trabalhoso e propenso a erros.

Para melhorar a manutenibilidade, mantenha o acoplamento entre classes o mais baixo possível.

USANDO ITERAÇÃO NO PROCESSO DE DESENVOLVIMENTO

Como em grande parte das atividades de projeto e programação, recomenda-se um processo iterativo. Isso combina com a ideia de interface mínima: não escreva todo o sistema de uma vez. Construa pequenos incrementos, compile, teste e aprenda com o resultado.

Um bom plano de testes revela rapidamente onde faltam interfaces ou onde o contrato ainda não está adequado. O processo se repete até que a classe ofereça o conjunto correto de operações. Essa validação não precisa esperar o código: walkthroughs e revisões de projeto também podem testar decisões cedo.

Testadores se beneficiam quando participam desde o início, em vez de receber um sistema completo apenas no final.

TESTANDO A INTERFACE

Implementações mínimas usadas apenas para testar contratos são frequentemente chamadas de stubs. Com stubs, é possível verificar a interface antes de desenvolver a lógica real.

No exemplo `DataBaseReader`, em vez de conectar imediatamente a um banco verdadeiro, stubs retornam dados simulados. Isso permite verificar se a interface funciona da perspectiva do usuário. Concluir toda a implementação cedo demais pode desperdiçar esforço, pois mudanças posteriores no contrato poderiam obrigar a refazê-la.

Figura 5.6 - Uso de stubs.

A página original mostra uma implementação de DataBaseReader que usa um array interno de strings para simular registros de banco de dados. Há métodos open(), close(), goToFirst(), goToLast(), howManyRecords(), getRecord() e getNextRecord().

Em vez de acessar um banco real, os métodos trabalham sobre o array, permitindo testar o contrato externo antes da integração com a infraestrutura definitiva. O código completo é preservado à esquerda.

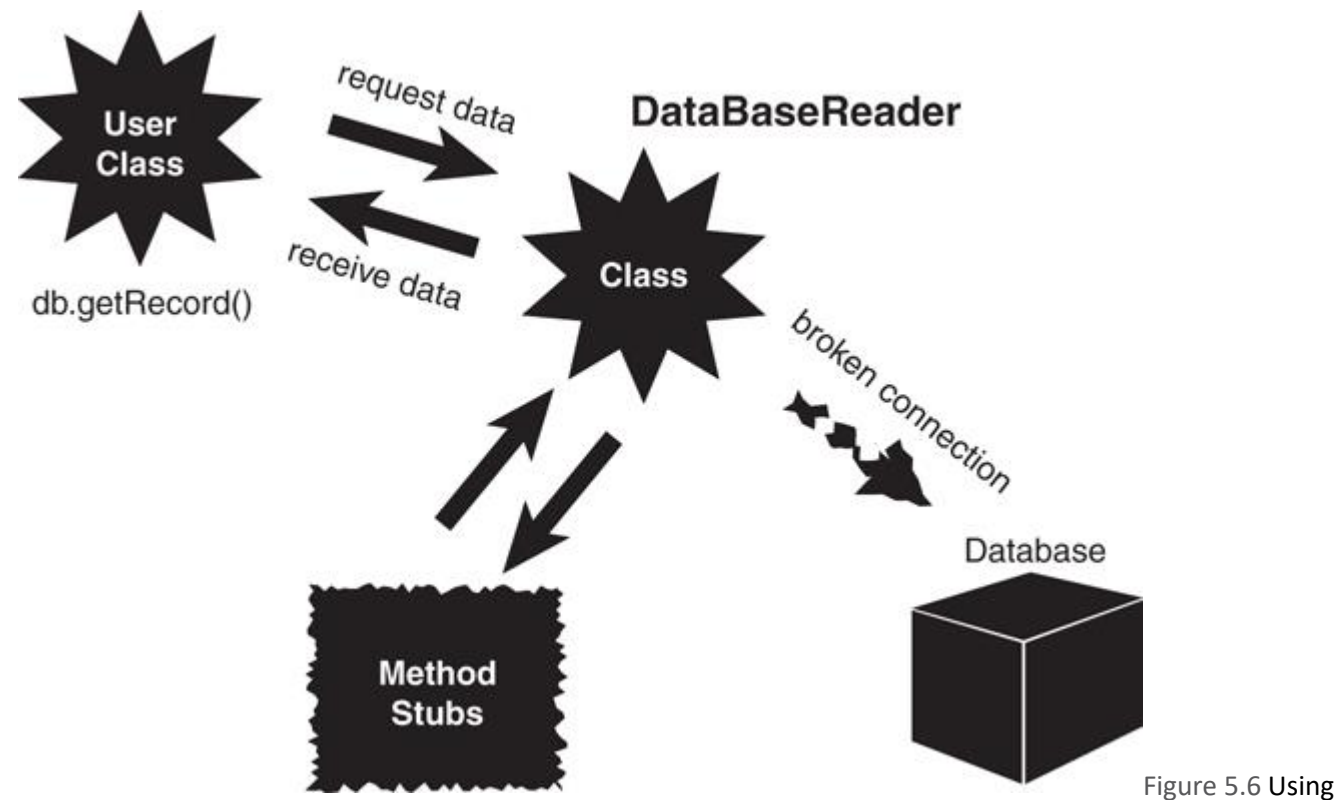


Figure 5.6 Using

stubs.

Here is a code example that uses an internal array to simulate a working database (albeit a simple one):

[Click here to view code image](#)

```
public class DataBaseReader {  
  
    private String db[] = { "Record1","Record2","Record3","Record4","Record5"};  
    private booleanDBOpen = false;  
    private int pos;  
  
    public void open(String Name){  
        DBOpen = true;  
    }  
  
    public void close(){  
        DBOpen = false;  
    }  
  
    public void goToFirst(){  
        pos = 0;  
    }  
  
    public void goToLast(){  
        pos = 4;  
    }  
  
    public int howManyRecords(){  
        int numOfRecords = 5;  
        return numOfRecords;  
    }  
  
    public String getRecord(int key){  
        /* DB Specific Implementation */  
        return db[key];  
    }  
}
```

```
    }

    public String getNextRecord(){
        /* DB Specific Implementation */
        return db[pos++];
    }
}
```

Notice how the methods simulate the database calls. The strings within the array represent the records that will be written to the database. When the database is successfully integrated into the system, it will be substituted for the array.

Keeping the Stubs Around

When you are done with the stubs, don't delete them. Keep them in the code for possible use later—just make sure the users can't see them and the other team members know that they are there. In fact, in a well-designed program, your test stubs should be integrated into the design and kept in the program for later use. In short, design the testing right into the class! Perhaps even better, create stubs with mock data and coded to interfaces, and then you can swap them out with the actual implementation when the time comes.

As you find problems with the interface design, make changes and repeat the process until you are satisfied with the result.

USING OBJECT PERSISTENCE

Object persistence is another issue that must be addressed in many OO systems. *Persistence* is the concept of maintaining the state of an object. When you run a program, if you don't save the object in some manner, the object dies, never to be recovered. These transient objects might work in some applications, but in most business systems, the state of the object must be saved for later use.

Object Persistence

Although the topic of object persistence and the topics in the next section might not be considered true design guidelines, I believe that they must be addressed when designing classes. I introduce them here to stress that they must be addressed early on when designing classes.

In its simplest form, an object can persist by being serialized and written to a flat file. The state-of-the-art technology is now XML-based. Although it is true that an object theoretically can persist in memory as long as it is not destroyed, we will concentrate on storing persistent objects on some sort of storage device. There are three primary storage devices to consider:

- **Flat file system**—You can store an object in a flat file by serializing the object. This is definitely outdated. More often than not, objects are serialized to XML and/or JSON and written to some sort of file system or data store or web endpoint. They can be put into a database or written to disk, which is the most common practice nowadays.
- **Relational database**—Some sort of middleware is necessary to convert an object to a relational model.
- **NoSQL database**—This may be a more efficient way to make objects persistent, but most companies have all their data in legacy systems and at this point in time are unlikely to convert their relational databases to OO databases. This is the most common form of a flexible structure database. MongoDB or Cosmos DB are two of the bigger names in this space.

Os métodos simulam chamadas ao banco. As strings do array representam registros que futuramente seriam obtidos do armazenamento real. Quando o banco estiver integrado ao sistema, a implementação simulada pode ser substituída sem alterar o contrato público.

MANTENDO OS STUBS

Quando os stubs deixam de ser necessários para a execução normal, não é obrigatório apagá-los. Eles podem permanecer como suporte a testes futuros, desde que não fiquem expostos indevidamente aos usuários e que a equipe saiba de sua existência. Melhor ainda, os stubs podem ser modelados como implementações de interfaces, usando dados simulados, permitindo trocar facilmente a implementação de teste pela real.

À medida que problemas na interface forem descobertos, o projeto deve ser ajustado e testado novamente até atingir um resultado satisfatório.

USANDO PERSISTÊNCIA DE OBJETOS

Persistência é o conceito de manter o estado de um objeto para uso posterior. Se um programa termina e nenhum estado é armazenado, objetos transitórios desaparecem. Em muitas aplicações empresariais, porém, as informações precisam sobreviver à execução.

PERSISTÊNCIA DE OBJETOS

Embora persistência não seja apenas uma “diretriz de projeto de classe”, ela precisa ser considerada cedo porque influencia como o objeto é modelado e integrado ao sistema.

Em sua forma mais simples, um objeto pode ser serializado e gravado em arquivo. Hoje é comum representar dados em formatos como XML e JSON e armazená-los em arquivos, bancos, data stores ou serviços web.

Três categorias discutidas no livro são:

- arquivos/armazenamento simples, frequentemente com serialização;
- bancos relacionais, normalmente exigindo algum mapeamento entre objetos e tabelas;
- bancos NoSQL, com estruturas mais flexíveis, como MongoDB ou Cosmos DB.

Serializing and Marshaling Objects

We have already discussed the problem of using objects in environments that were originally designed for structured programming. The middleware example, where we wrote objects to a relational database, is one good example. We also touched on the problem of writing an object to a flat file or sending it over a network.

To send an object over a wire (for example, to a file, over a network), the system must deconstruct the object (flatten it out), send it over the wire, and then reconstruct it on the other end of the wire. This process is called *serializing* an object. The act of sending the object across a wire is called *marshaling* an object. A serialized object, in theory, can be written to a flat file and retrieved later, in the same state in which it was written.

The major issue here is that the serialization and deserialization must use the same specifications. It is sort of like an encryption algorithm. If one object encrypts a string, the object that wants to decrypt it must use the same encryption algorithm. Java provides an interface called `Serializable` that provides this translation.

This is another reason why data is separated from behaviors nowadays. It’s far simpler to create an interface for a data contract and push that out to a web service than it is to make sure people have the same code on both sides.

CONCLUSION

This chapter presents many guidelines that can help you in designing classes. This is by no means a complete list of guidelines. You will undoubtedly come across additional guidelines as you go about your travels in OO design.

This chapter deals with design issues as they pertain to individual classes. However, we have already seen that a class does not live in isolation. Classes must be designed to interact with other classes. A group of classes that interact with each other is part of a system. Ultimately, these systems provide value to end users. Chapter 6, “Designing with Objects,” covers the topic of designing complete systems.

REFERENCES

Ambler, Scott. 2004. *The Object Primer*, Third Edition. Cambridge, United Kingdom: Cambridge University Press.

Gilbert, Stephen, and Bill McCarty. 1998. *Object-Oriented Design in Java*. Berkeley, CA: The Waite Group Press.

Jaworski, Jamie. 1997. *Java 1.1 Developers Guide*. Indianapolis, IN: Sams Publishing.

Jaworski, Jamie. 1999. *Java 2 Platform Unleashed*. Indianapolis, IN: Sams Publishing.

Meyers, Scott. 2005. *Effective C++*, Third Edition. Boston, MA: Addison-Wesley Professional.

Tyma, Paul, Gabriel Torok, and Troy Downing. 1996. *Java Primer Plus*. Berkeley, CA: The Waite Group.

SERIALIZANDO E FAZENDO MARSHALING DE OBJETOS

Já discutimos a adaptação de objetos a ambientes originalmente projetados para programação estruturada, como o mapeamento para bancos relacionais. O mesmo problema aparece ao gravar um objeto em arquivo ou transmiti-lo por uma rede.

Para enviar um objeto por um canal, o sistema precisa representá-lo em uma forma transportável, transmitir essa representação e reconstruir o objeto no destino. Esse processo é chamado de serialização. O ato de preparar e transportar dados entre contextos também é frequentemente chamado de marshaling.

Um objeto serializado pode ser armazenado e, posteriormente, reconstruído em um estado equivalente. O ponto essencial é que serialização e desserialização precisam compartilhar a mesma especificação, assim como criptografia e descriptografia precisam concordar sobre o algoritmo. Java oferece, entre outros mecanismos, o marcador `Serializable` para participar de processos de serialização.

A separação moderna entre contratos de dados e comportamento também simplifica integração distribuída: pode ser mais simples publicar um contrato de dados em um serviço web do que garantir que os dois lados possuam exatamente o mesmo código de objetos.

CONCLUSÃO

Este capítulo apresentou diversas diretrizes para projeto de classes, sem pretender formar uma lista completa. A experiência em OO certamente acrescentará outras práticas.

O foco foi a classe individual, mas classes não vivem isoladas. Elas interagem e formam sistemas que entregam valor aos usuários. O Capítulo 6 aborda o projeto desses sistemas completos.

REFERÊNCIAS

Ambler, Scott. The Object Primer, 3ª ed., 2004.
Gilbert, Stephen; McCarty, Bill. Object-Oriented Design in Java, 1998.
Jaworski, Jamie. Java 1.1 Developers Guide, 1997; Java 2 Platform Unleashed, 1999.
Meyers, Scott. Effective C++, 3ª ed., 2005.
Tyma, Paul; Torok, Gabriel; Downing, Troy. Java Primer Plus, 1996.

6. Designing with Objects

When you use a software product, you expect it to behave as advertised. Unfortunately, not all products live up to expectations. The problem is that when many products are produced, the majority of time and effort go into the engineering phase and not into the design phase.

Object-oriented (OO) design has been touted as a robust and flexible software development approach. The truth is that you can create both good and bad OO designs just as easily as you can create both good and bad non-OO designs. Don't be lulled into a false sense of security just because you are using a state-of-the-art design methodology. You must pay attention to the overall design and invest the proper amount of time and effort to create the best possible product.

In [Chapter 5, “Class Design Guidelines,”](#) we concentrated on designing good classes. This chapter focuses on designing good systems. A *system* can be defined as classes that interact with each other. Proper design practices have evolved throughout the history of software development, and there is no reason you should not take advantage of the blood, sweat, and tears of your software predecessors, whether they used OO technologies or not.

Taking advantage of previous efforts is not limited to design practices; you can even incorporate existing legacy code in your object-oriented designs. In many cases, you can take code, which may have been working well for years, and literally wrap it in your objects. The *wrapping* is discussed later in the chapter.

DESIGN GUIDELINES

One fallacy is that there is one true (best) design methodology. This is certainly not the case. There is no right or wrong way to create a design. Many design methodologies are available today, and they all have their proponents. However, the primary issue is not which design method to use, but whether to use a method at all. This can be expanded beyond design to encompass the entire software development process. Some organizations do not follow a standard software development process, or they have one and don't adhere to it. The most important factor in creating a good design is to find a process that you and your organization feel comfortable with, stick to it, and keep refining it. It makes no sense to implement a design process that no one will follow.

Most books that deal with object-oriented technologies offer very similar strategies for designing systems. In fact, except for some of the object-oriented specific issues involved, much of the strategy is applicable to non-OO systems as well.

Generally, a solid OO design process includes the following steps:

1. Doing the proper analysis
2. Developing a statement of work that describes the system
3. Gathering the requirements from this statement of work
4. Developing a prototype for the user interface
5. Identifying the classes
6. Determining the responsibilities of each class
7. Determining how the various classes interact with each other
8. Creating a high-level model that describes the system to be built

6. PROJETANDO COM OBJETOS

Quando você usa um produto de software, espera que ele se comporte como prometido. Infelizmente, nem todos correspondem às expectativas. Em muitos projetos, grande parte do tempo e esforço é aplicada à fase de construção e pouco à fase de projeto.

Projeto orientado a objetos é frequentemente apresentado como uma abordagem robusta e flexível, mas OO não garante qualidade automaticamente. É tão possível criar um projeto OO ruim quanto um projeto não OO ruim. Não confunda tecnologia moderna com bom design; é necessário investir tempo e raciocínio na estrutura geral do produto.

O Capítulo 5 concentrou-se no projeto de boas classes. Este capítulo amplia o foco para bons sistemas. Um sistema pode ser visto como um conjunto de classes que interagem. Boas práticas de projeto evoluíram durante toda a história do desenvolvimento de software, e não há motivo para ignorar a experiência acumulada por gerações anteriores, tenham usado OO ou não.

Reutilização não se limita a ideias de projeto. Código legado que funciona bem há anos pode, em muitos casos, ser incorporado a um sistema OO por meio de wrappers.

DIRETRIZES DE PROJETO

Uma falácia comum é imaginar que exista uma única metodologia “verdadeira” ou melhor. Há diversas metodologias e todas possuem defensores. A questão principal não é escolher um nome específico, mas adotar um processo que a equipe realmente seguirá, refiná-lo e aplicá-lo com disciplina. Um processo que ninguém usa não tem valor.

Muitas estratégias de projeto OO compartilham passos semelhantes, vários deles aplicáveis também a sistemas não OO. Um processo sólido costuma incluir:

1. realizar análise adequada;
2. produzir uma descrição do trabalho/sistema;
3. levantar requisitos a partir dessa descrição;
4. desenvolver um protótipo da interface do usuário;
5. identificar classes;
6. determinar responsabilidades de cada classe;
7. determinar como as classes interagem;
8. criar um modelo de alto nível do sistema.

For object-oriented development, the high-level system model is of special interest. The system, or object model, is made up of class diagrams and class interactions. This model should represent the system faithfully and be easy to understand and modify. We also need a notation for the model. This is where the Unified Modeling Language (UML) comes in. As you know, UML is not a design process but a modeling tool. In this book, I only use class diagrams within UML. I like to utilize class diagrams as a visual tool to assist with the design process as well as with documentation—even if I don't use the other available UML tools.

The Ongoing Design Process

Despite the best intentions and planning, in all but the most trivial cases, the design is an ongoing process. Even after a product is in testing, design changes will pop up. It is up to the project manager to draw the line that says when to stop changing a product and adding features. I like to call this Version 1.

It is important to understand that many design methodologies are available. One early methodology, called the waterfall model, advocates strict boundaries between the various phases. In this case, the design phase is completed before the implementation phase, which is completed before the testing phase, and so on. In practice, the waterfall model has been found to be unrealistic. Currently, other design models, such as rapid prototyping, Extreme Programming, Agile, Scrum, and so on, promote a true iterative process. In these models, some implementation is attempted prior to completing the design phase as a type of proof-of-concept. Despite the recent aversion to the waterfall model, the goal behind the model is understandable. Coming up with a complete and thorough design before starting to code is a sound practice. You do not want to be in the release phase of the product and then decide to iterate through the design phase again. Iterating across phase boundaries is unavoidable; however, you should keep these iterations to a minimum (see [Figure 6.1](#)).

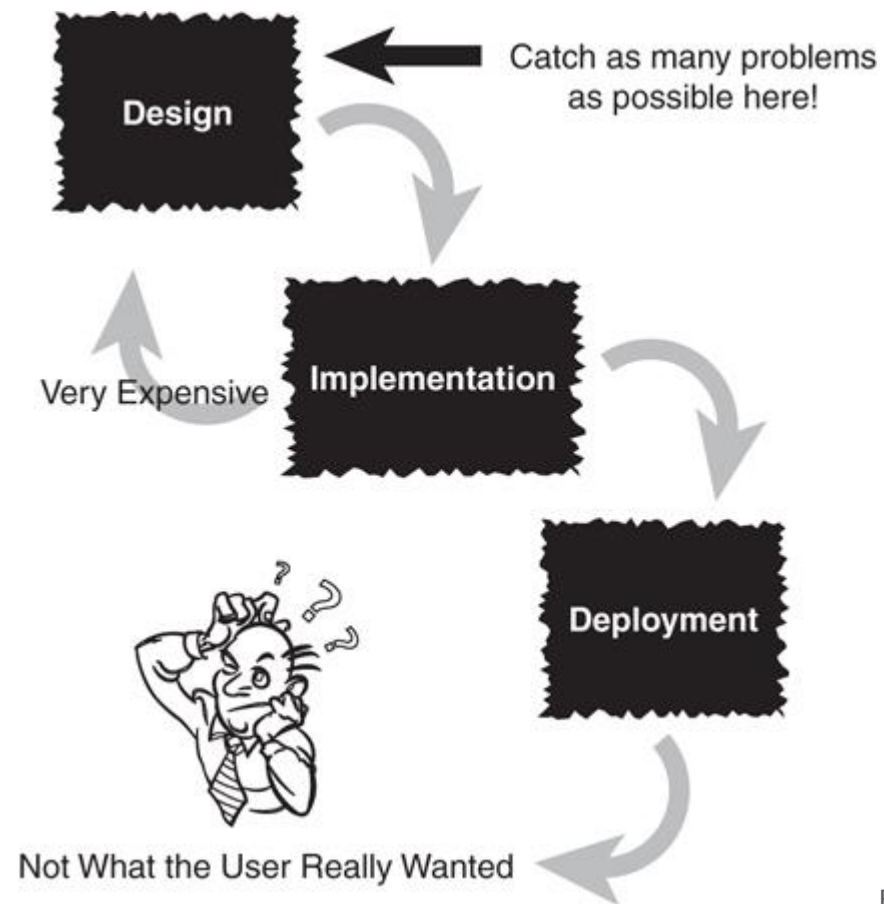


Figure 6.1 The waterfall method.

No desenvolvimento OO, o modelo de alto nível é especialmente importante. O modelo de objetos é composto por classes e suas interações e deve representar o sistema de forma fiel, compreensível e modificável. Também precisamos de uma notação; UML pode cumprir esse papel.

UML não é um processo de projeto, mas uma linguagem de modelagem. Neste livro, o autor usa principalmente diagramas de classes como ferramenta visual de apoio ao projeto e à documentação.

O PROCESSO CONTÍNUO DE PROJETO

Mesmo com bom planejamento, o projeto continua evoluindo em praticamente todos os sistemas não triviais. Mudanças podem surgir até durante testes. Em algum momento, a gestão do projeto precisa estabelecer um limite para uma versão e deixar novas ideias para versões posteriores.

Existem várias metodologias. O modelo cascata clássico separa rigidamente análise, projeto, implementação, teste e demais fases. Na prática, essa rigidez mostrou-se pouco realista. Modelos mais recentes - prototipagem rápida, Extreme Programming, Agile, Scrum e outros - promovem ciclos iterativos e provas de conceito antes do encerramento formal de todo o projeto.

Apesar das críticas à cascata, sua intenção de pensar seriamente no projeto antes de codificar continua válida. Não queremos chegar à fase de liberação e descobrir que precisamos refazer decisões fundamentais. Iterações entre fases são inevitáveis, mas devem ser administradas.

Figura 6.1 - O método cascata.

Simply put, the reasons to identify requirements early and keep design changes to a minimum are as follows:

- The cost of a requirement/design change in the design phase is relatively small.
- The cost of a design change in the implementation phase is significantly higher.
- The cost of a design change after the deployment phase is astronomical when compared to the first item.

Similarly, you would not want to start the construction of your dream house before the architectural design was complete. If I said that the Golden Gate Bridge or the Empire State Building was constructed without any consideration of design issues, you would consider the statement absolutely crazy. Yet, you would most likely not find it crazy if I told you that the software you were using might contain some design flaws, and in fact, might not have been thoroughly tested.

In reality, it might be impossible to thoroughly test software, in the sense that absolutely *nobugs* exist. However, in theory, that is always the goal. We should always attempt to weed out as many bugs as possible. Bridges and software might not be directly comparable; however, software must strive for the same level of engineering excellence as the “harder” engineering disciplines such as bridge building. Poor-quality software can be lethal—it’s not just wrong numbers on payroll checks. For example, inferior software in medical equipment can kill and maim people. Yet, you may be willing to live with having to reboot your computer every now and then. But the same cannot be said for a bridge failing.

Safety Versus Economics

Would you want to cross a bridge that has not been inspected and tested? Unfortunately, with many software packages, users are left with the responsibility of doing much of the testing. This is very costly for both the users and the software providers. Unfortunately, short-term economics often seem to be the primary factor in making project decisions.

Because customers seem to be willing to pay a limited price and put up with software of poor quality, some software providers find that it is cheaper in the long run to let the customers test the product rather than do it themselves. In the short term this might be true, but in the long run it costs far more than the software provider realizes. Ultimately, the software provider’s reputation will be damaged.

Some computer software companies are willing to use the beta test phase to let the customers do testing—testing that should, theoretically, have been done before the beta version ever reached the customer. Many customers are willing to take the risk of using prerelease software because they are anxious to get the functionality the product promises. Conversely, some customers resist new releases like the plague. If it works, don’t fix it. Upgrading can be a nightmare!

After the software is released, problems that have not been caught and fixed prior to release become much more expensive. To illustrate, consider the dilemma automobile companies face when they are confronted with a recall. If a defect in the automobile is identified and fixed before it is shipped (ideally before it is manufactured), it is much cheaper than if all delivered automobiles have to be recalled and fixed one at a time. Not only is this scenario very expensive, but it damages the reputation of the company. In an increasingly competitive market, high-quality software, support services, and reputation are *the* competitive advantage (see [Figure 6.2](#)).

Em termos simples, identificar requisitos cedo e minimizar mudanças tardias é importante porque:

- uma alteração durante o projeto costuma ser relativamente barata;
- a mesma alteração durante a implementação custa mais;
- depois da implantação, o custo pode ser enorme em comparação com as fases iniciais.

Ninguém começaria a construir uma casa dos sonhos antes de concluir um projeto arquitetônico minimamente sólido. Seria absurdo imaginar a Golden Gate Bridge ou o Empire State Building construídos sem preocupação com projeto. Curiosamente, as pessoas aceitam com muito mais naturalidade a ideia de software com falhas de projeto ou testes insuficientes.

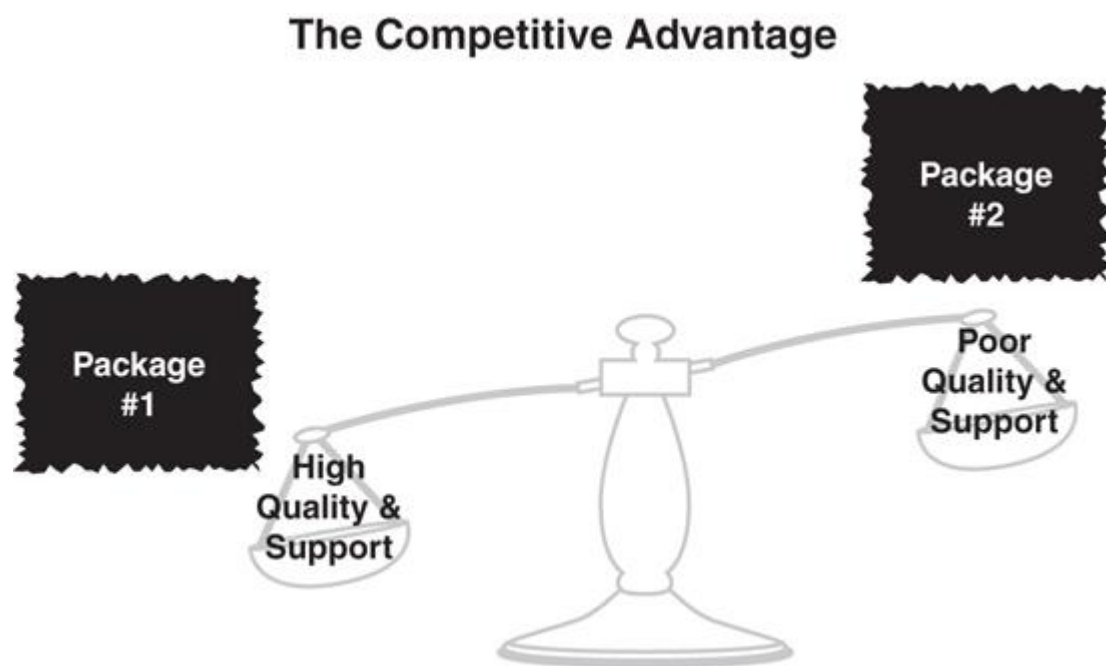
Talvez seja impossível provar que um software complexo não possui absolutamente nenhum bug, mas esse continua sendo o objetivo ideal: encontrar e eliminar o máximo possível. Software deve buscar o mesmo compromisso com engenharia de qualidade encontrado em outras disciplinas. Em sistemas médicos, automotivos ou de infraestrutura, software ruim pode ferir ou matar pessoas; não se trata apenas de números errados em uma folha de pagamento.

SEGURANÇA VERSUS ECONOMIA

Você atravessaria uma ponte que nunca foi inspecionada ou testada? Em muitos produtos de software, parte considerável dos testes acaba sendo feita pelos próprios usuários, o que transfere custos e riscos. Decisões de curto prazo frequentemente priorizam economia imediata.

Algumas empresas usam fases beta para receber feedback do mercado, mas uma versão beta não deveria substituir testes fundamentais do fornecedor. Alguns clientes aceitam o risco para obter recursos novos; outros evitam atualizações porque sabem que migrações podem ser problemáticas.

Depois do lançamento, falhas não detectadas ficam muito mais caras. A analogia com recall automotivo é clara: corrigir um defeito antes da fabricação ou entrega custa muito menos do que recolher milhares de veículos. Além do custo financeiro, erros comprometem a reputação. Em mercados competitivos, qualidade, suporte e confiança tornam-se vantagens importantes.



competitive advantage.

The following sections provide brief summaries of the items listed previously as being part of the design process. Later in the chapter, we work through an example that explains in greater detail each of these items.

Performing the Proper Analysis

A lot of variables are involved in building a design and producing a software product. The users must work hand in hand with the developers at all stages. In the analysis phase, the users and the developers must do the proper research and analysis to determine the statement of work, the requirements of the project, and whether to actually do the project. The last point might seem a bit surprising, but it is important. During the analysis phase, there must not be any hesitation to terminate the project if a valid reason exists to do so. Too many times, pet project status or some political inertia keeps a project going, regardless of the obvious warning signs that cry out for project cancellation. Assuming that the project is viable, the primary focus of the analysis phase is for everyone to learn the systems (both the old and the proposed new one) and determine the system requirements.

Generic Software Principles

Most of these practices are not specific to OO. They apply to software development in general.

Developing a Statement of Work

The *statement of work* (SOW) is a document that describes the system. Although determining the requirements is the ultimate goal of the analysis phase, at this point the requirements are not yet in a final format. The SOW should give anyone who reads it a complete, high level understanding of the system. Regardless of how it is written, the SOW must represent the complete system and be clear about how the system will look and feel.

The SOW contains everything that must be known about the system. Many customers create a *request for proposal* (RFP) for distribution, which is similar to the statement of work. A customer creates an RFP that completely describes the system the customer wants built and releases it to multiple vendors. The vendors then use this document, along with whatever

Figura 6.2 - A vantagem competitiva.

As próximas seções resumem os itens apresentados como partes do processo de projeto. Mais adiante, um exemplo mostra cada etapa em maior detalhe.

REALIZANDO A ANÁLISE ADEQUADA

Muitas variáveis participam da construção de um produto de software. Usuários e desenvolvedores precisam trabalhar juntos em todas as etapas. Durante a análise, ambos pesquisam e estudam o contexto para definir o escopo, os requisitos e até mesmo decidir se o projeto deve realmente ser realizado.

Esse último ponto é importante: se a análise revelar um motivo válido para cancelar o projeto, a equipe precisa ter liberdade para fazê-lo. Projetos mantidos apenas por interesse político, apego pessoal ou inércia podem continuar mesmo diante de sinais claros de inviabilidade.

Assumindo que o projeto seja viável, o objetivo da análise é compreender tanto os sistemas existentes quanto o sistema proposto e determinar suas necessidades.

PRINCÍPIOS GERAIS DE SOFTWARE

A maior parte dessas práticas não é exclusiva de orientação a objetos; aplica-se ao desenvolvimento de software em geral.

DESENVOLVENDO UMA DECLARAÇÃO DE TRABALHO

A declaração de trabalho (Statement of Work - SOW) é um documento que descreve o sistema em alto nível. Os requisitos finais ainda não precisam estar totalmente formalizados, mas o SOW deve fornecer a qualquer leitor uma visão completa e compreensível de como o sistema deverá funcionar e se apresentar.

O SOW reúne informações essenciais sobre o sistema. Muitos clientes publicam um Request for Proposal (RFP), semelhante a essa declaração, descrevendo o que desejam construir e distribuindo o documento a diversos fornecedores. Os fornecedores usam o RFP e suas próprias análises para decidir se participarão da concorrência e qual preço propor.

analysis they need to do, to determine whether they should bid on the project, and if so, what price to charge.

Gathering the Requirements

The *requirements document* describes what the users want the system to do. Even though the level of detail of the requirements document does not need to be of a highly technical nature, the requirements must be specific enough to represent the true nature of the user’s needs for the end product. The requirements document must be of sufficient detail for the user to make educated judgments about the completeness of the system. It must also be of specific detail for a design group to use the document to proceed with the design phase.

Whereas the SOW is a document written in paragraph (even narrative) form, the requirements are usually represented as a summary statement or presented as bulleted items. Each individual bulleted item represents one specific requirement of the system. The requirements are distilled from the statement of work. This process is shown later in the chapter.

In many ways, these requirements are the most important part of the system. The SOW might contain irrelevant material; however, the requirements are the final representation of the system that must be implemented. All future documents in the software development process will be based on the requirements.

Developing a System Prototype

One of the best ways to make sure users and developers understand the system is to create a *prototype*. A prototype can be just about anything; however, most people consider the prototype to be a simulated user interface. By creating actual screens and screen flows, it is easier for people to get an idea of what they will be working with and what the system will feel like. In any event, a prototype will almost certainly not contain all the functionality of the final system.

Most prototypes are created with an integrated development environment (IDE). However, in some basic cases, drawing the screens on a whiteboard or even on paper might be all that is needed. Remember that you are not necessarily creating business logic (the logic/code behind the interface that actually does the work) when you build the prototype, although it is possible to do so. The look and feel of the user interface is a major concern at this point. Having a good prototype can help immensely when identifying classes.

Identifying the Classes

After the requirements are documented, the process of identifying classes can begin. From the requirements, one straightforward way of identifying classes is to highlight all the nouns. These tend to represent objects, such as people, places, and things. Don’t be too fussy about getting all the classes right the first time. You might end up eliminating classes, adding classes, and changing classes at various stages throughout the design. It is important to get something down first. Take advantage of the fact that the design is an iterative process. As in other forms of brainstorming, get something down initially, with the understanding that the final result might look nothing like the initial pass.

Determining the Responsibilities of Each Class

You need to determine the responsibilities of each class you have identified. This includes the data that the class must store and what operations the class must perform. For example, an `Employee` object would be responsible for calculating payroll and transferring the money to

LEVANTANDO OS REQUISITOS

O documento de requisitos descreve o que os usuários desejam que o sistema faça. Ele não precisa ser escrito em linguagem altamente técnica, mas deve ser específico o suficiente para representar fielmente as necessidades do produto final. Usuários devem conseguir avaliar se o sistema está completo, e a equipe de projeto deve conseguir utilizar o documento como base para o design.

Enquanto o SOW costuma ser narrativo, requisitos são frequentemente apresentados como frases curtas ou itens de lista, cada um representando uma necessidade específica. Eles são extraídos e refinados a partir da declaração de trabalho.

De muitas maneiras, os requisitos são a parte mais importante do sistema. O SOW pode conter contexto e material adicional; os requisitos representam aquilo que efetivamente deverá ser implementado. Documentos posteriores do processo de desenvolvimento se basearão neles.

DESENVOLVENDO UM PROTÓTIPO DO SISTEMA

Uma das melhores formas de garantir entendimento comum entre usuários e desenvolvedores é criar um protótipo. Um protótipo pode assumir várias formas, mas muitas vezes representa uma interface de usuário simulada. Telas e fluxos permitem que as pessoas visualizem como será trabalhar com o sistema. O protótipo não precisa conter toda a funcionalidade final.

Pode ser construído em uma IDE, em uma ferramenta específica ou, em casos simples, até em papel ou quadro branco. A lógica de negócio pode não existir nessa etapa; o foco está principalmente na aparência, no fluxo e na experiência. Um bom protótipo também ajuda na identificação das classes.

IDENTIFICANDO AS CLASSES

Depois de documentar requisitos, inicia-se a identificação das classes. Uma técnica simples é destacar substantivos nos requisitos, pois eles frequentemente representam pessoas, lugares, coisas ou conceitos do domínio.

Não tente acertar tudo na primeira passagem. Classes podem ser removidas, acrescentadas ou alteradas durante as iterações. O importante é criar uma primeira proposta e refiná-la.

DETERMINANDO AS RESPONSABILIDADES DE CADA CLASSE

Para cada classe, identifique que dados ela precisa armazenar e quais operações precisa realizar. Um `Employee`, por exemplo, poderia participar do cálculo de pagamento, transferência de valores e armazenamento das informações necessárias para essas tarefas.

the appropriate account. It might also be responsible for storing the various payroll rates and the account numbers of various banks.

Determining How the Classes Collaborate with Each Other

Most classes do not exist in isolation. Although a class must fulfill certain responsibilities, many times it will have to interact with another class to get something it wants. This is where the messages between classes apply. One class can send a message to another class when it needs information from that class, or if it wants the other class to do something for it.

Creating a Class Model to Describe the System

When all the classes are determined and the class responsibilities and collaborations are listed, a class model that represents the complete system can be constructed. The class model shows how the various classes interact within the system.

In this book, we are using UML to model the system. Several tools on the market use UML and provide a good environment for creating and maintaining UML class models. As we develop the example in the next section, we will see how the class diagrams fit into the big picture and how modeling large systems would be virtually impossible without some sort of good modeling notation and modeling tool.

Prototyping the User Interface in Code

During the design process, we must create a prototype of our user interface. This prototype will provide invaluable information to help navigate through the iterations of the design process. As Gilbert and McCarty in *Object-Oriented Design in Java* aptly point out, “to a system user, the user interface is the system.” There are several ways to create a user interface prototype. You can sketch the user interface by drawing it on paper or a whiteboard. You also can use a special prototyping tool, or even a language environment like Visual Basic, which is often used for rapid prototyping. Or you can use the IDE from your favorite development tool to create the prototype. However, at this point they are basically facades; the business logic is not necessarily in place.

However you develop the user interface prototype, make sure that the users have the final say on the look and feel.

OBJECT WRAPPERS

Several times in the previous chapters I have indicated that one of my primary goals in this book is to dispel the fallacy that object-oriented programming is a separate paradigm from structured programming, and is even at odds with it. In fact, as I have already mentioned, I am often asked the following question: “Are you an object-oriented programmer or a structured programmer?” The answer is always the same—I am both!

In my mind, there is no way to write a program without using structures. Thus, when you write a program that uses an object-oriented programming language and are using sound object-oriented design techniques, you are also using structured programming techniques. There is no way around this.

For example, when you create a new object that contains attributes and methods, those methods will include structured code. In fact, I might even say that these methods will contain *mostly* structured code. This approach fits in well with the container concept that we

DETERMINANDO COMO AS CLASSES COLABORAM

A maioria das classes não existe isoladamente. Uma classe cumpre responsabilidades próprias, mas frequentemente precisa pedir informações ou serviços a outras. É aqui que entram as mensagens entre objetos: uma classe envia uma mensagem quando precisa que outra forneça dados ou execute alguma operação.

CRIANDO UM MODELO DE CLASSES PARA DESCREVER O SISTEMA

Depois de identificar classes, responsabilidades e colaborações, podemos construir um modelo que represente o sistema completo. Esse modelo mostra como as classes se relacionam e interagem.

Neste livro, UML é usada como notação visual. Ferramentas UML facilitam criar e manter modelos de classes. À medida que o exemplo evoluir, veremos como os diagramas se encaixam no quadro geral e por que sistemas grandes são difíceis de modelar sem uma notação e ferramentas adequadas.

PROTOTIPANDO A INTERFACE DO USUÁRIO EM CÓDIGO

Durante o projeto, o protótipo da interface fornece informações valiosas para as iterações. Para o usuário, a interface frequentemente é “o sistema”. Ela pode ser esboçada em papel, construída em uma ferramenta de prototipação ou criada diretamente em uma IDE. Nesse estágio, muitas telas são essencialmente fachadas: a lógica de negócio ainda pode não existir.

Independentemente da técnica, usuários reais devem ter a palavra final sobre a aparência e o comportamento da interface.

WRAPPERS DE OBJETOS

Um dos objetivos do livro é desfazer a ideia de que programação OO é um paradigma que se opõe à programação estruturada. A pergunta “você é programador OO ou estruturado?” tem como resposta: ambos.

Não há como escrever programas sem estruturas de controle. Mesmo em uma linguagem OO e com bom projeto orientado a objetos, os métodos contêm código estruturado. Ao criar um objeto com atributos e métodos, os corpos desses métodos utilizarão sequências, condições, laços e outras construções tradicionais.

have encountered in earlier chapters. In fact, when I get to the point where I am coding at the method level, my coding thought process hasn’t changed much since when I was programming in structured languages, such as Cobol, C, and the like. This is not to say that it is exactly the same, because I obviously have had to adjust to some object-oriented constructs; however, the fundamental approach to coding at the method level is virtually the same as programming has always been.

Now I’ll return to the question “Are you an object-oriented programmer or a structured programmer?” I often like to say that programming is programming. By this I contend that being a good programmer means understanding the basics of programming logic and having a passion for coding. Often you will see ads for a programmer with a specific skill set—let’s say a specific language like Java.

Although I totally understand that an organization may well need an experienced Java programmer in a pinch, over the long run I would prefer to focus on hiring a programmer who has a wide range of programming experience and who can learn and adjust quickly when new technologies emerge. Some of my colleagues do not always agree with this; however, I believe that when hiring, I look more at what a potential employee can learn than what they already know. The passion part is critical because it ensures that an employee will always be exploring new technologies and development methodologies.

Structured Code

Although the basics of programming logic may be debated, as I have stressed, the fundamental object-oriented constructs are *encapsulation*, *inheritance*, *polymorphism*, and *composition*. In most textbooks that I have seen, the basic constructs of structured programming are *sequence*, *conditions*, and *iterations*.

The sequence part is a given, because it seems logical to start at the top and proceed in a logical manner to the bottom. For me, the meat of structured programming resides in the conditions and iterations, which I call if-statements and loops, respectively.

Take a look at the following Java code that starts at 0 and loops 10 times, printing out the value if it equals 5:

[Click here to view code image](#)

```
class MainApplication {  
  
    public static void main(String args[]) {  
  
        int x = 0;  
  
        while (x <= 10) {  
  
            if (x==5) System.out.println("x = " + x);  
            x++;  
        }  
    }  
}
```

Now while this code is written in an object-oriented language, the code that resides inside the main method is structured code. All three basics of structured programming are present: *sequence*, *conditions*, and *iterations*.

The sequence part is easy to identify because the first line executed is

No nível do código dentro de métodos, o processo mental do autor mudou menos do que se poderia imaginar desde a época de linguagens estruturadas como COBOL e C. Existem construções OO adicionais, é claro, mas a lógica fundamental de programação continua presente.

Por isso, o autor costuma dizer que “programação é programação”. Ser um bom programador envolve compreender lógica, resolver problemas e ter interesse contínuo em codificar e aprender. Vagas muitas vezes pedem uma tecnologia específica, como Java, e há situações em que experiência imediata realmente importa.

No longo prazo, porém, o autor prefere desenvolvedores com experiência ampla e capacidade de aprender rapidamente novas tecnologias. Mais importante do que aquilo que alguém já sabe pode ser aquilo que é capaz e está disposto a aprender. A curiosidade e a paixão ajudam a acompanhar novas linguagens e metodologias.

CÓDIGO ESTRUTURADO

Podemos discutir detalhes, mas os fundamentos clássicos de OO apresentados no livro são encapsulamento, herança, polimorfismo e composição. Já a programação estruturada costuma ser descrita por três construções básicas: sequência, seleção/condição e iteração.

A sequência significa executar instruções em ordem lógica. As condições são normalmente representadas por if; iterações, por laços.

O exemplo Java da página começa com x = 0, executa um while até 10 e usa um if para imprimir quando x == 5. Embora esteja escrito em uma linguagem OO, o corpo de main utiliza claramente código estruturado.

```
int x = 0;
```

When that line completes, the next line is executed:

```
while (x <= 10) {
```

And so on. In short, this is tried and true top-down programming: start at the first line, execute it, and then go on to the next.

There is also a condition present in this code as part of the if-statement:

```
if (x==5)
```

Finally, there is a loop to complete the structured trio.

```
while (x <= 10) {  
}
```

Actually, the while loop also contains a condition:

```
(x <= 10)
```

You can pretty much code anything with just these three constructs. In fact, the concept of the wrapper is basically the same for structured programming as it is for object-oriented programming. In structured design you wrap the code in functions (such as the main method in this example), and in object-oriented design you wrap the code in objects and methods.

Wrapping Structured Code

Although defining attributes is considered coding (for example, creating an integer), the behavior of an object resides in the methods. And these methods are where the bulk of the code logic is found.

Consider **Figure 6.3**. As you can see, an object contains methods, and these methods contain code, which can be anything from variable declarations to conditions to loops.

A primeira instrução executada é `int x = 0;`. Em seguida, a execução continua para o `while` e assim por diante. Essa é a sequência tradicional de cima para baixo.

O código também contém uma condição `if (x == 5)` e um laço `while`. O próprio `while` possui uma condição `x <= 10`. Com sequência, seleção e iteração é possível expressar uma enorme variedade de algoritmos.

O conceito de wrapper aparece tanto em programação estruturada quanto em OO. No projeto estruturado, agrupamos código em funções e procedimentos; no projeto OO, agrupamos código em objetos e métodos.

ENVOLVENDO CÓDIGO ESTRUTURADO

Embora declarar atributos também seja parte da programação, o comportamento dos objetos reside principalmente nos métodos. É neles que encontramos a maior parte da lógica: declarações, condições, laços, chamadas e algoritmos.

Figura 6.3 - Encapsulando código estruturado em métodos.

Considere um exemplo simples que encapsula uma soma. Criamos uma classe SomeMath com um método add(int a, int b) que devolve a + b.

O código estruturado que realiza a soma fica dentro do método add(). É um exemplo trivial, mas ilustra exatamente o que significa envolver lógica estruturada em um componente OO.

O cliente não precisa conhecer o corpo do método. Ele instancia SomeMath, chama math.add(1,2) e recebe o resultado. O código completo permanece na página original.

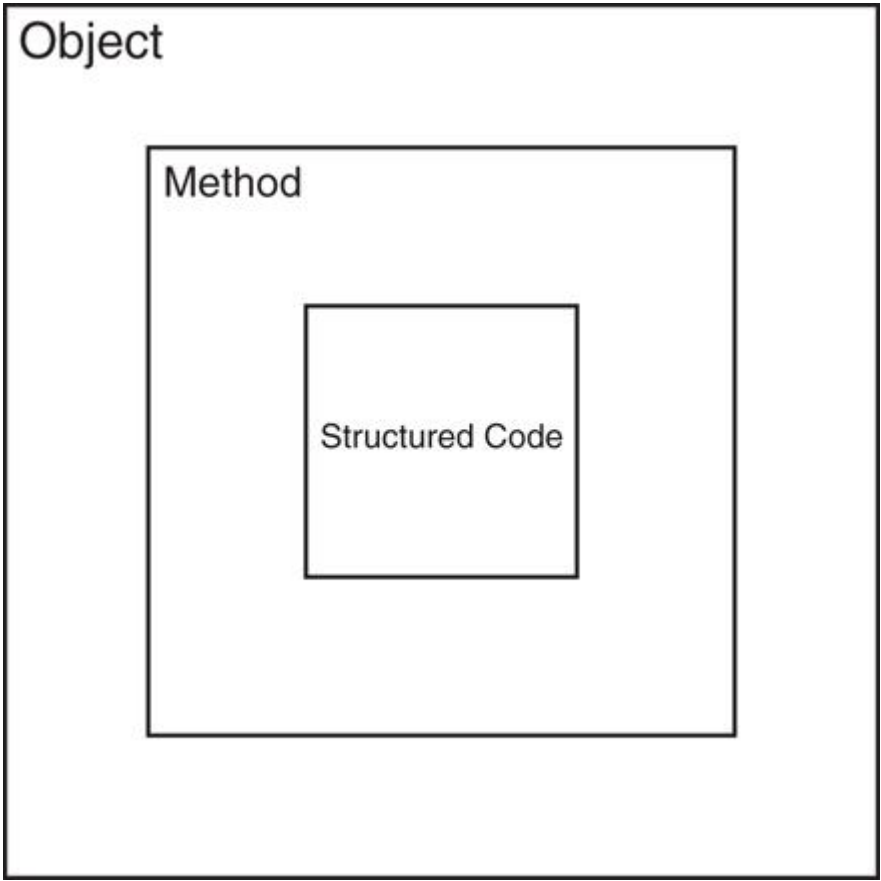


Figure 6.3 Wrapping structured code.

Let’s consider a simple example in which we are wrapping the functionality for addition. Here we create a method named add, which accepts two integer parameters and returns their sum.

```
class SomeMath {  
  
    public int add(int a, int b) {  
  
        return a + b;  
  
    }  
  
}
```

As you can see, the structured code used to perform the addition (a + b) is wrapped inside the add method. Although this is a trivial example, that is all there is to wrapping structured code. Thus, when the user wants to use this method, all that is needed is the signature of the method as seen next:

[Click here to view code image](#)

```
public class TestMath {  
  
    public static void main(String[] args) {  
  
        int x = 0;  
  
        SomeMath math = new SomeMath();  
        x = math.add(1,2);  
        System.out.println("x = " + x);  
  
    }  
  
}
```

Finally, we can add some more functionality that is a bit more interesting and complicated. Suppose that we wanted to include a method to calculate the Fibonacci value of a number. We can then add a method like this:

```
public static int fib(int n) {  
  
    if (n < 2) {  
        return n;  
    } else {  
        return fib(n-1)+fib(n-2);  
    }  
}
```

The whole point here is to show that we have an object-oriented method that contains (wraps) structured code, because the `fib` method contains conditions, recursion, and so on. And as mentioned in the introduction, it is possible to incorporate existing legacy code in wrappers as well.

Wrapping Nonportable Code

One other use of object wrappers is for the hiding of nonportable (or native) code. The concept is essentially the same; however, in this case the point is to take code that can be executed on only one platform (or a few platforms) and encapsulate it in a method providing a simple interface for the programmers using the code.

Consider the task of making the computer make a noise—in this case, a beep. On a Windows platform we can execute a beep with the following code:

```
System.out.println("\007");
```

Rather than making the programmer memorize the code (or look it up), you can provide a class called `Sound` that contains a method called `beep` as shown next:

```
class Sound {  
  
    public void beep() {  
        System.out.println("\007");  
    }  
}
```

Now, rather than having to know the code for making the sound, the programmer can use the class and call the `beep` method:

Click here to view code image

```
public class TestBeep {  
  
    public static void main(String[] args) {  
        Sound mySound = new Sound();  
        mySound.beep();  
    }  
}
```

Not only is this simpler for the programmer to use, but you can extend the functionality of the class to include other sounds. Perhaps more importantly, when the code is used on a non-Windows platform, the interface for the user remains the same. In short, the team that builds the code for the `Sound` class will have to deal with the change in platform. For the programmers who utilize the class in their applications, the change will be seamless because they will still call the `beep` method.

Podemos acrescentar um comportamento mais interessante, como o cálculo recursivo de Fibonacci. O método `fib(n)` mostrado no original contém uma condição e chamadas recursivas. Ele continua sendo um método de um contexto OO, mas sua lógica interna é código estruturado.

Isso reforça a ideia de que código legado estruturado também pode ser incorporado a wrappers quando apropriado.

ENVOLVENDO CÓDIGO NÃO PORTÁVEL

Outro uso importante de wrappers é esconder código nativo ou dependente de plataforma. O objetivo é encapsular uma implementação que funciona apenas em determinados ambientes dentro de um método que oferece uma interface simples e estável aos clientes.

O exemplo usa a tarefa de emitir um beep. Em determinado ambiente, o código de baixo nível pode ser `System.out.println("\007");`. Em vez de obrigar todos os programadores a conhecer esse detalhe, cria-se uma classe `Sound` com um método `beep()`.

O cliente instancia `Sound` e chama `mySound.beep()`. Além de ser mais simples, o design permite ampliar a classe com outros sons. Mais importante, se a implementação mudar em outra plataforma, o contrato `beep()` pode continuar igual. A equipe responsável por `Sound` trata a diferença de plataforma; os clientes continuam usando a mesma interface.

Wrapping Existing Classes

Although the need to wrap legacy structured code, or even nonportable code, into a new (object-oriented) class may seem reasonable, the need to wrap existing classes might not seem so obvious. However, there are also many reasons to create wrappers for existing (object-oriented) classes.

Software developers often utilize code written by someone else. Perhaps the code was purchased from a vendor or even written internally within the same organization. In many of these cases, the code cannot be changed. Perhaps the individual who wrote the code is no longer with the organization, or the vendor cannot perform maintenance updates, and so on. This is where the true power of wrappers emerges.

The idea is to take an existing class and alter its implementation or interface by wrapping it inside a new class—just like we did for the structured code and nonportable code. The difference in this case is that, rather than putting an object-oriented face to the code, we are altering its implementation or interface.

Why would we want to do this? Well, the answer lies with both the implementation and the interface.

Consider the database example that we used in [Chapter 2](#), “[How to Think in Terms of Objects](#).” Our goal was to provide the same interface for the developers regardless of which database they were using. In fact, if we need to support another database, our goal would remain the same—to make the transition to the new database transparent to the user (see [Figure 2.3](#) as shown in [Chapter 2](#)).

Also, remember our earlier discussion about creating middleware to provide an interface between objects and relational databases. As developers, we want to use objects. Thus, we want functionality that will allow us to persist objects to a database. What we don’t want to have to do is write SQL code for every single object transaction performed to a relational database. This is where we can consider middleware to be a wrapper, and many object-relational mapping products are available.

Conceptually, for me, the ultimate example of the interface and implementation paradigm is the discussion that we had regarding the power plant example in [Chapter 2](#) and shown in [Figure 2.1](#). In this case we can swap out (wrap) both: We can alter the interface by changing the outlet, and we can alter the implementation by changing the power generation facility.

The use of wrappers in software development is fairly extensive, not only from a developer’s perspective but also from a vendor’s. Wrappers are an important tool when developing software systems.

In this chapter, we have focused on various design considerations, including writing new code as well as utilizing previously written code, whether in house or from vendors. In some cases, wrappers are even design paradigms unto themselves. Design patterns, for example, utilize wrappers in various cases. As we will see later, the Decorator pattern focuses on wrapping the implementation, whereas the Adaptor pattern focuses on altering the interface. The discussion of design patterns is explored in more detail in [Chapter 10](#), “[Design Patterns](#).”

ENVOLVENDO CLASSES EXISTENTES

Pode parecer natural envolver código legado ou não portátil, mas também existem muitos motivos para criar wrappers em torno de classes OO já existentes.

Desenvolvedores utilizam frequentemente código de terceiros ou bibliotecas internas que não podem ser alteradas. O autor original pode não estar mais disponível, o fornecedor pode não fornecer a modificação necessária ou simplesmente queremos manter o componente original intacto. Um wrapper oferece uma camada de adaptação.

A ideia é colocar a classe existente dentro de uma nova classe e modificar a interface percebida ou acrescentar comportamento, de modo semelhante ao que fizemos com código estruturado e nativo.

Por que fazer isso? Tanto por razões de implementação quanto de interface.

No exemplo de banco de dados do Capítulo 2, o objetivo era fornecer a mesma interface aos desenvolvedores independentemente do banco usado. Se outro banco for adicionado, queremos que a transição seja transparente ao cliente.

O mesmo raciocínio aparece em middleware objeto-relacional. Desenvolvedores querem trabalhar com objetos persistentes, não escrever SQL manual para cada transação. Uma camada ORM pode ser vista como wrapper que traduz entre modelos.

O exemplo da usina resume bem o paradigma: podemos alterar a implementação por trás de uma interface estável e, em alguns casos, também envolver um componente para oferecer uma interface diferente.

Wrappers são amplamente usados tanto por desenvolvedores quanto por fornecedores. Padrões de projeto também exploram essa técnica: Decorator concentra-se em acrescentar ou envolver comportamento, enquanto Adapter altera a forma de interface com um componente. O assunto é retomado no Capítulo 10.

CONCLUSION

This chapter covers the design process for complete systems. It is important to note that object-oriented and structured code are not mutually exclusive. In fact, you can’t create objects without using structured code. Thus, while building object-oriented systems, you are also using structured techniques in the design.

Object wrappers are used to encapsulate many types of functionality, which can range from traditional structured (legacy) and object-oriented (classes) code to nonportable (native) code. The primary purpose of object wrappers is to provide consistent interfaces for the programmers who are using the code.

In the next several chapters, we explore in more detail the relationships between classes. Chapter 7, “Mastering Inheritance and Composition,” covers the concepts of inheritance and composition and how they relate to each other.

REFERENCES

Ambler, Scott. 2004. *The Object Primer*, Third Edition. Cambridge, United Kingdom: Cambridge University Press.

Gilbert, Stephen, and Bill McCarty. 1998. *Object-Oriented Design in Java*. Berkeley, CA: The Waite Group Press.

Jaworski, Jamie. 1999. *Java 2 Platform Unleashed*. Indianapolis, IN: Sams Publishing.

Jaworski, Jamie. 1997. *Java 1.1 Developers Guide*. Indianapolis, IN: Sams Publishing.

McConnell, Steve. 2004. *Code Complete: A Practical Handbook of Software Construction*, Second Edition. Redmond, WA: Microsoft Press.

Weisfeld, Matt, and John Ciccozzi. September, 1999. “Software by Committee,” *Project Management Journal* v5, number 1: 30–36.

Wirfs-Brock, R., B. Wilkerson, and L. Weiner. 1990. *Designing Object-Oriented Software*. Upper Saddle River, NJ: Prentice-Hall.

CONCLUSÃO

Este capítulo abordou o processo de projeto de sistemas completos. É importante lembrar que código orientado a objetos e código estruturado não são mutuamente exclusivos. Objetos são implementados por métodos, e métodos utilizam construções estruturadas.

Wrappers de objetos podem encapsular vários tipos de funcionalidade: código legado estruturado, classes OO existentes e código nativo/dependente de plataforma. Seu objetivo principal é oferecer interfaces consistentes aos programadores que utilizam o componente.

Os próximos capítulos aprofundam os relacionamentos entre classes. O Capítulo 7 trata de herança e composição e de como esses mecanismos se relacionam.

REFERÊNCIAS

Ambler, Scott. The Object Primer, 3ª ed., 2004.

Gilbert, Stephen; McCarty, Bill. Object-Oriented Design in Java, 1998.

Jaworski, Jamie. Java 2 Platform Unleashed, 1999; Java 1.1 Developers Guide, 1997.

McConnell, Steve. Code Complete, 2ª ed., 2004.

Weisfeld, Matt; Ciccozzi, John. “Software by Committee”, 1999.

Wirfs-Brock, R.; Wilkerson, B.; Weiner, L. Designing Object-Oriented Software, 1990.

7. Mastering Inheritance and Composition

Inheritance and composition play major roles in the design of object-oriented (OO) systems. In fact, many of the most difficult and interesting design decisions come down to deciding between inheritance and composition.

These decisions have become much more interesting over the years as object-oriented design practices have evolved. Perhaps one of the most interesting debates revolves around inheritance. Although inheritance is one of the fundamental constructs of object-oriented development (a language must support inheritance to be considered object-oriented), some developers are even turning away from inheritance by implementing designs solely with composition.

It is common to use interface inheritance rather than direct inheritance for behaviors (implementing versus inheriting). Inheritance tends to be used often for data/models whereas implementation tends to be used for behaviors.

Regardless, both inheritance and composition are mechanisms for reuse. *Inheritance*, as its name implies, involves inheriting attributes and behaviors from other classes, where there is a true parent/child relationship. The child (or subclass) inherits directly from the parent (or superclass).

Composition, also as its name implies, involves building objects by using other objects. In this chapter we explore the obvious and subtle differences between inheritance and composition. Primarily, we will consider the appropriate times to use one or the other.

REUSING OBJECTS

Perhaps the primary reason why inheritance and composition exist is object reuse. In short, you can build classes (which ultimately become objects) by utilizing other classes via inheritance and composition, which in effect are the only ways to reuse previously built classes.

Inheritance represents the is-a relationship that was introduced in [Chapter 1](#), “[Introduction to Object-Oriented Concepts](#).” For example, a dog *is a* mammal.

Composition involves using other classes to build more complex classes—a sort of assembly. No parent/child relationship exists in this case. Basically, complex objects are composed of other objects. Composition represents a has-a relationship. For example, a car *has an* engine. Both the engine and the car are separate, potentially standalone objects. However, the car is a complex object that contains (has an) engine object. In fact, a child object might itself be composed of other objects; for example, the engine might include cylinders. In this case an engine *has a* cylinder, actually several.

When OO technologies first entered the mainstream, inheritance was often the first example used in how to design an OO system. That you could design a class once and then inherit functionality from it was considered one of the foremost advantages to using OO technologies. Reuse was the name of the game, and inheritance was the ultimate expression of reuse.

However, over time the luster of inheritance has dulled a bit. In fact, in some discussions, the use of inheritance itself is questioned. In their book *Java Design*, Peter Coad and Mark Mayfield have a complete chapter titled “Design with Composition Rather Than Inheritance.” Many early object-based platforms did not even support true inheritance. As Visual Basic evolved into Visual Basic .NET, early object-based implementations did not include strict inheritance

7. DOMINANDO HERANÇA E COMPOSIÇÃO

Herança e composição desempenham papéis importantes no projeto de sistemas orientados a objetos. Muitas das decisões mais difíceis e interessantes surgem justamente ao escolher entre esses dois mecanismos.

Essa discussão ficou ainda mais interessante à medida que as práticas de OO evoluíram. Herança é uma construção fundamental das linguagens orientadas a objetos, mas vários desenvolvedores modernos procuram reduzir seu uso e preferir composição em muitos cenários.

É comum utilizar herança de interfaces para comportamentos - implementar um contrato em vez de herdar diretamente uma implementação. Herança direta tende a aparecer mais em modelos de dados ou estruturas em que existe uma relação conceitual realmente forte.

De qualquer forma, herança e composição são mecanismos de reutilização. Herança envolve receber atributos e comportamentos de uma classe pai quando existe uma relação genuína de generalização/especialização. A subclasse herda da superclasse.

Composição constrói objetos a partir de outros objetos. Não existe relação pai/filho; existe uma relação de montagem. Herança representa “é-um” (is-a): um cão é um mamífero. Composição representa “tem-um” (has-a): um carro tem um motor. O motor e o carro são objetos distintos, e o próprio motor pode ser composto por cilindros e outros componentes.

REUTILIZANDO OBJETOS

Uma das principais motivações para herança e composição é o reuso. Classes existentes podem ser incorporadas a novos projetos por esses mecanismos.

Quando OO se popularizou, herança era frequentemente apresentada como o exemplo máximo de reutilização: projetar uma classe uma vez e herdar sua funcionalidade em várias outras.

Com o tempo, porém, a visão tornou-se mais cautelosa. Alguns autores recomendam “projetar com composição em vez de herança” em muitos casos. Plataformas mais antigas orientadas a objetos também nem sempre ofereceram herança clássica completa. A discussão continua nas próximas páginas.

capabilities. Platforms such as the MS COM model were based on interface inheritance. Interface inheritance is covered in great detail in [Chapter 8](#), “[Frameworks and Reuse: Designing with Interfaces and Abstract Classes](#).”

Today, the use of inheritance is still a major topic of debate. Abstract classes, which are a form of inheritance, are not directly supported in some languages, such as Objective-C and Swift. Interfaces are used even though they don’t provide all the functionality that abstract classes do.

The good news is that the discussions about whether to use inheritance or composition are a natural progression toward some seasoned middle ground. As in all philosophical debates, there are passionate arguments on both sides. Fortunately, as is normally the case, these heated discussions have led to a more sensible understanding of how to utilize the technologies.

We will see later in this chapter why some people believe that inheritance should be avoided, and composition should be the design method of choice. The argument is fairly complex and subtle. In actuality, both inheritance and composition are valid class design techniques, and they each have a proper place in the OO developer’s toolkit. And, at least, you need to understand both to make the proper design choice—not to mention maintenance of legacy code.

The fact that inheritance is often misused and overused is more a result of a lack of understanding of what inheritance is all about than a fundamental flaw in using inheritance as a design strategy.

The bottom line is that inheritance and composition are both important techniques in building OO systems. Designers and developers need to take the time to understand the strengths and weaknesses of both and to use each in the proper contexts.

INHERITANCE

Inheritance was defined in [Chapter 1](#) as a system in which child classes inherit attributes and behaviors from a parent class. However, there is more to inheritance, and in this chapter we explore inheritance in greater detail.

[Chapter 1](#) states that you can determine an inheritance relationship by following a simple rule: If you can say that Class B *is a* Class A, then this relationship is a good candidate for inheritance.

Is-a

One of the primary rules of OO design is that public inheritance is represented by an is-a relationship. In the case of interfaces you might add “behaves like” (implements). The data (attributes) that are inherited are the “is,” the interfaces describing encapsulated behaviors are “acts like,” and composition is “has a.” The lines get pretty blurry, however.

Let’s revisit the mammal example used in [Chapter 1](#). Let’s consider a `Dog` class. A dog has several behaviors that make it distinctly a dog, as opposed to a cat. For this example, let’s specify two: A dog barks and a dog pants. So we can create a `Dog` class that has these two behaviors, along with two attributes (see [Figure 7.1](#)).

...capacidades. Plataformas como o modelo MS COM foram baseadas em herança de interfaces. A herança de interfaces é tratada em detalhes no Capítulo 8, “Frameworks e Reuso: Projetando com Interfaces e Classes Abstratas”.

Hoje, o uso de herança ainda é um tema importante de debate. Classes abstratas, que constituem uma forma de herança, não são diretamente suportadas por algumas linguagens, como Objective-C e Swift. Nesses casos, interfaces são utilizadas, embora não forneçam toda a funcionalidade de uma classe abstrata.

A boa notícia é que a discussão sobre usar herança ou composição vem amadurecendo para uma posição mais equilibrada. Como em todo debate filosófico, há argumentos apaixonados dos dois lados, mas isso também levou a uma compreensão mais sensata de como empregar cada tecnologia.

Mais adiante veremos por que alguns defendem evitar herança e preferir composição. O argumento é complexo e sutil. Na prática, porém, herança e composição são técnicas válidas de projeto de classes e cada uma tem seu lugar no conjunto de ferramentas do desenvolvedor OO. No mínimo, é preciso compreender ambas para tomar boas decisões de projeto e também para manter código legado.

O uso incorreto ou excessivo de herança decorre mais de uma compreensão inadequada do conceito do que de uma falha fundamental da técnica. Em resumo: herança e composição são importantes na construção de sistemas OO; projetistas e desenvolvedores devem conhecer suas vantagens e limitações e empregá-las no contexto apropriado.

HERANÇA

No Capítulo 1, herança foi definida como um sistema no qual classes filhas herdaram atributos e comportamentos de uma classe pai. Há, porém, muito mais a explorar.

Uma regra simples ajuda a reconhecer uma possível relação de herança: se podemos dizer que “Classe B é uma Classe A”, então a relação é uma boa candidata.

É-UM (IS-A)

Uma das regras centrais do projeto OO é representar herança pública por uma relação “é-um”. Para interfaces, pode-se acrescentar “comporta-se como” ou “implementa”. Os dados herdados representam o “é”; as interfaces descrevem os comportamentos do “age como”; e composição representa “tem-um”. Na prática, essas fronteiras podem ficar menos nítidas.

Retomemos o exemplo dos mamíferos. Considere uma classe `Dog`. Para diferenciá-la de um gato, vamos representar dois comportamentos: um cão late e ofega. Assim, podemos criar `Dog` com esses dois comportamentos e dois atributos, conforme a Figura 7.1.

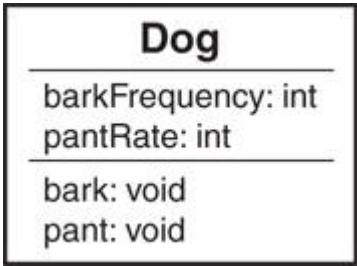


Figure 7.1 A class diagram for the Dog class.

Now let’s say that you want to create a `GoldenRetriever` class. You could create a brand-new class that contains the same behaviors that the `Dog` class has. However, we could make the following, and quite reasonable, conclusion: A Golden Retriever is-a dog. Because of this relationship, we can inherit the attributes and behaviors from `Dog` and use it in our new `GoldenRetriever` class (see [Figure 7.2](#)).

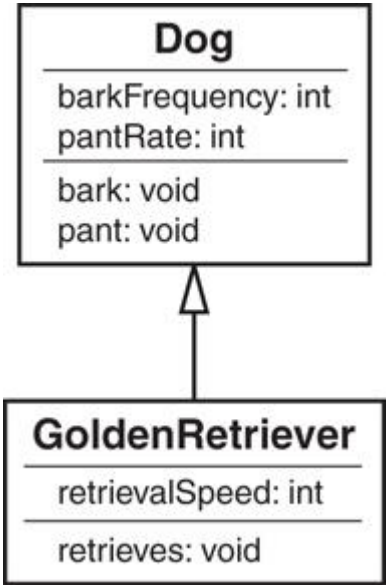


Figure 7.2 The `GoldenRetriever` class inherits from the `Dog` class.

The `GoldenRetriever` class now contains its own behaviors as well as all the more general behaviors of a dog. This provides us with some significant benefits. First, when we wrote the `GoldenRetriever` class, we did not have to reinvent part of the wheel by rewriting the `bark` and `pant` methods. Not only does this save some design and coding time, but it saves testing and maintenance time as well. The `bark` and `pant` methods are written only once and, assuming that they were properly tested when the `Dog` class was written, they do not need to be heavily tested again; but it does need to be retested because there are new interfaces, and so on. Now let’s take full advantage of our inheritance structure and create a second class under the `Dog` class: a class called `LhasaApso`. Whereas retrievers were bred for retrieving, Lhasa Apsos were bred for use as guard dogs. These dogs are not attack dogs; they have acute senses, and when they sense something unusual, they start barking. So we can create our `LhasaApso` class and inherit from the `Dog` class just as we did with the `GoldenRetriever` class (see [Figure 7.3](#)).

Figura 7.1 - Diagrama de classe para Dog.

Suponha agora que queiramos criar uma classe `GoldenRetriever`. Poderíamos escrever uma classe completamente nova repetindo os mesmos comportamentos de `Dog`, mas existe uma conclusão natural: um Golden Retriever é-um cão. Por isso, podemos herdar os atributos e comportamentos de `Dog` e reutilizá-los em `GoldenRetriever`, como mostra a Figura 7.2.

Figura 7.2 - `GoldenRetriever` herda de `Dog`.

`GoldenRetriever` passa a conter seus comportamentos específicos e também os comportamentos mais gerais de um cão. Isso oferece vantagens importantes. Ao escrever `GoldenRetriever`, não precisamos reimplementar os métodos `bark()` e `pant()`. Economizamos tempo de projeto, codificação, testes e manutenção.

Os métodos `bark()` e `pant()` são escritos uma única vez. Se foram testados corretamente quando `Dog` foi criado, sua implementação não precisa ser reconstruída. Ainda assim, a nova relação de herança deve ser retestada, pois ela cria um novo contexto, novas interfaces e novas interações.

Podemos aproveitar a estrutura de herança e criar outra classe derivada de `Dog`: `LhasaApso`. Enquanto retrievers foram criados historicamente para recuperar objetos, Lhasa Apsos foram criados como cães de guarda. Não são cães de ataque; possuem sentidos aguçados e, ao perceber algo incomum, começam a latir. Assim, `LhasaApso` pode herdar de `Dog` exatamente como `GoldenRetriever`, conforme a Figura 7.3.

Figura 7.3 - LhasaApso herda de Dog.

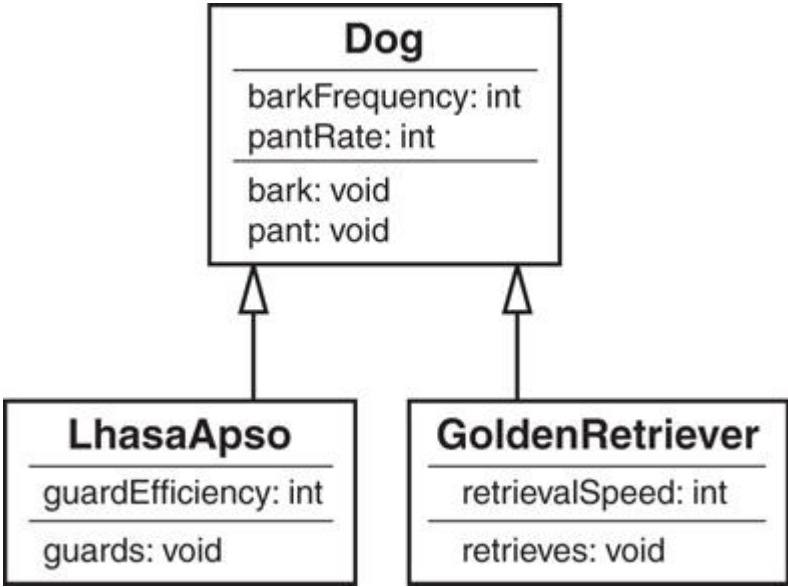


Figure 7.3 The LhasaApso class inherits from the Dog class.

Testing New Code

In our example with the GoldenRetriever class, the bark and pant methods should be writ-ten, tested, and debugged when the Dog class is written. Theoretically, this code is now robust and ready to reuse in other situations. However, the fact that you do not need to rewrite the codedoes not mean it should not be tested. However unlikely, there might be some specific characteristic of a retriever that somehow breaks the code. The bottom line is that you should always test new code. Each new inheritance relationship creates a new context for using inher-ited methods. A complete testing strategy should take into account each of these contexts.

Another primary advantage of inheritance is that the code for `bark()` and `pant()` is in a single place. Let’s say there is a need to change the code in the `bark()` method. When you change it in the `Dog` class, you do not need to change it in the `LhasaApso` class and the `GoldenRetriever` class. Do you see a problem here? At this level the inheritance model appears to work very well. However, can you be certain that all dogs have the behavior contained in the `Dog` class? In his book *Effective C++*, Scott Meyers gives a great example of a dilemma with design using inheritance. Consider a class for a bird. One of the most recognizable characteristics of a bird is, of course, that it can fly. So we create a class called `Bird` with a `fly` method. You should immediately understand the problem. What do we do with a penguin, or an ostrich? They are birds, yet they can’t fly. You could override the behavior locally, but the method would still be called `fly`. And it would not make sense to have a method called `fly` for a bird that does not fly but only waddles, runs, or swims. This is an example of the Liskov Substitution Principle of SOLID, which we discuss in [Chapter 12](#), “[The SOLID Principles of Object-Oriented Design](#).” This leads to some potentially significant problems. For example, if a penguin has a `fly` method, the penguin might understandably decide to test it out. However, if the `fly` method was in fact overridden and the behavior to fly did not exist, the penguin would be in for a major surprise when the `fly` method is invoked after jumping over a cliff. Imagine the penguin’s chagrin when the call to the `fly` method results in waddling instead of flight (or even a no-op, which means no operation, where nothing happens at all). In this situation, waddling doesn’t cut it. Just imagine if code such as this ever found its way into a spacecraft’s guidance system. In our dog example, we have designed the class so that all dogs have the ability to bark. However, some dogs do not bark. The Basenji breed is a barkless dog. Although these dogs do not bark, they do yodel. So should we reevaluate our design? What would this design look

No exemplo de GoldenRetriever, bark() e pant() devem ter sido escritos, testados e depurados quando Dog foi criado. Em teoria, esse código agora é robusto e pronto para reuso. Porém, não precisar reescrever o código não significa que ele não deva ser testado. Alguma característica específica de uma nova subclasse pode revelar um problema. Cada nova relação de herança cria um novo contexto para métodos herdados; uma estratégia completa de testes precisa considerar cada contexto.

Outra grande vantagem é manter bark() e pant() em um único lugar. Se bark() precisar mudar, alteramos Dog uma vez, e não cada subclasse. Porém, surge uma questão: podemos garantir que todos os cães possuem exatamente os comportamentos definidos em Dog?

Scott Meyers apresenta um exemplo clássico com aves. É natural pensar em uma classe Bird com um método fly(), mas pinguins e avestruzes são aves que não voam. Poderíamos sobrescrever fly(), porém ainda teríamos um método chamado “voar” em uma ave que não voa. Esse é um exemplo relacionado ao Princípio da Substituição de Liskov (LSP), discutido no Capítulo 12 sobre SOLID.

Isso pode produzir problemas importantes. Imagine um código que trate um pinguim como Bird e invoque fly(). Se o método tiver sido sobrescrito para apenas andar, nadar ou não fazer nada, o comportamento não corresponde ao contrato sugerido. Em sistemas críticos, uma inconsistência conceitual desse tipo pode ser desastrosa.

O mesmo ocorre com nosso modelo de cães. Projetamos Dog supondo que todo cão late, mas a raça Basenji é conhecida por não latir; ela produz um som semelhante a um iodelei. Portanto, o modelo precisa ser reconsiderado. A Figura 7.4 mostra uma hierarquia mais adequada.

Figura 7.4 - Hierarquia da classe Dog.

GENERALIZAÇÃO E ESPECIALIZAÇÃO

Considere o modelo da hierarquia de Dog. Começamos com uma única classe e extraímos aquilo que é comum entre diferentes raças. Esse processo, muitas vezes chamado de generalização-especialização, é essencial ao trabalhar com herança.

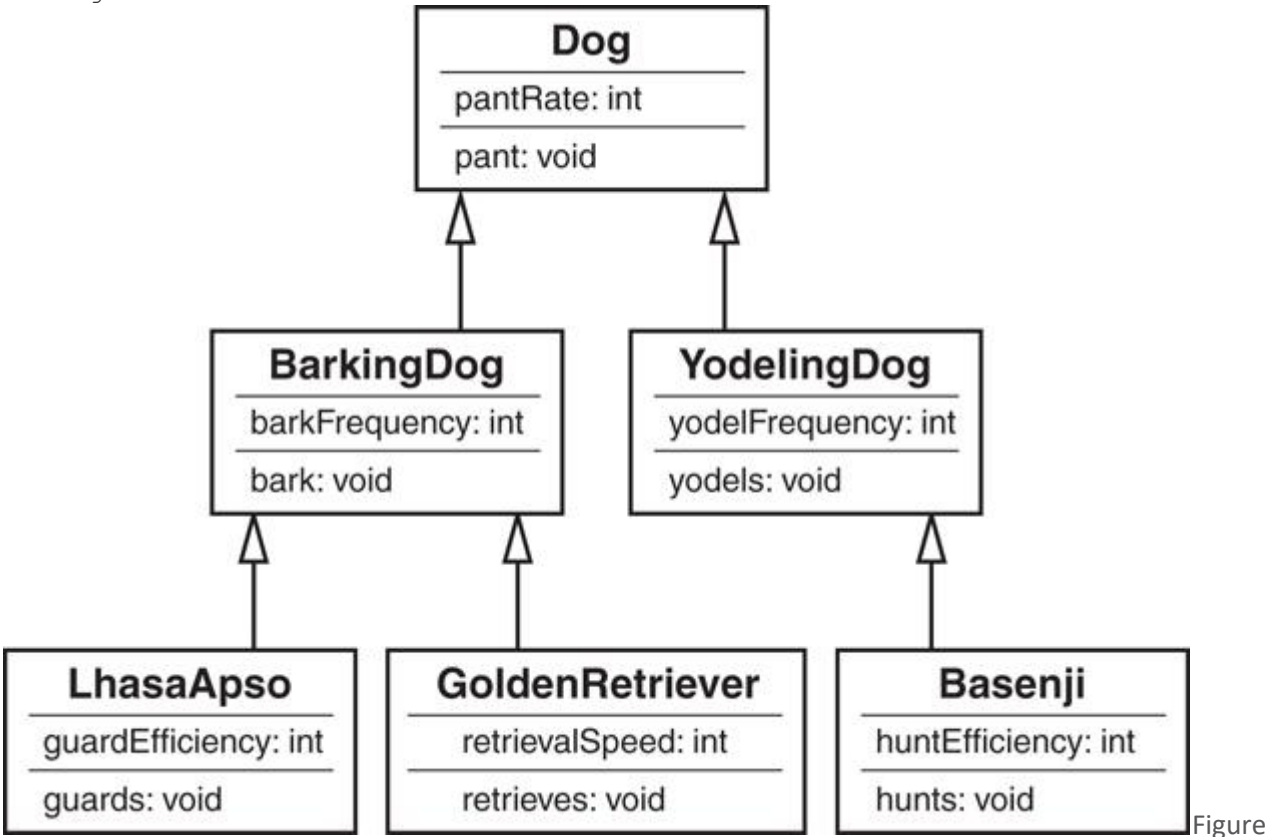
À medida que descemos na árvore de herança, as classes tornam-se mais específicas. O caso mais geral fica no topo. Em nossa árvore, Dog é a categoria mais geral; GoldenRetriever, LhasaApso e Basenji são categorias mais específicas. A ideia da herança é partir do geral para o específico, fatorando a funcionalidade comum.

Inicialmente percebemos que retrievers e Lhasa Apsos compartilham comportamentos como ofegar e latir. Depois, observamos que nem todos os cães latem; alguns vocalizam de outra forma. Por isso, o comportamento de latir foi separado em BarkingDog e o comportamento de “iodelei” em YodelingDog.

Mesmo assim, cães que latem e cães que não latem ainda compartilham um comportamento: todos ofegam. Mantemos, então, Dog como classe mais geral; BarkingDog e YodelingDog herdam de Dog. Basenji herda de YodelingDog, enquanto LhasaApso e GoldenRetriever herdam de BarkingDog.

Também seria possível não criar BarkingDog e YodelingDog e implementar os sons diretamente em cada raça, já que cada uma pode vocalizar de modo diferente. Essa é uma decisão de projeto. Outra solução possível é modelar os comportamentos de latir e iodelei por meio de interfaces, assunto do Capítulo 8.

like? **Figure 7.4** is an example that shows a more correct way to model the hierarchy of the `Dog` class.



7.4 The `Dog` class hierarchy.

Generalization and Specialization

Consider the object model of the `Dog` class hierarchy. We started with a single class, called `Dog`, and we factored out some of the commonality between various breeds of dogs. This concept, sometimes called *generalization-specialization*, is yet another important consideration when using inheritance. The idea is that as you make your way down the inheritance tree, things get more specific. The most general case is at the top of the tree. In our `Dog` inheritance tree, the class `Dog` is at the top and is the most general category. The various breeds—the `GoldenRetriever`, `LhasaApso`, and `Basenji` classes—are the most specific. The idea of inheritance is to go from the general to the specific by factoring out commonality.

In the `Dog` inheritance model, we started factoring out common behavior by understanding that although a retriever has some different behavior from that of a `LhasaApso`, the breeds do share some common behaviors—for example, they both pant and bark. Then we realized that all dogs do not bark—some yodel. Thus, we had to factor out the barking behavior into a separate `BarkingDog` class. The yodeling behavior went into a `YodelingDog` class. However, we realized that both barking dogs and barkless dogs still shared some common behavior—all dogs pant. Thus, we kept the `Dog` class and had the `BarkingDog` and the `YodelingDog` classes inherit from `Dog`. Now `Basenji` can inherit from `YodelingDog`, and `LhasaApso` and `GoldenRetriever` can inherit from `BarkingDog`.

We could have decided not to create two distinct classes for `BarkingDog` and `YodelingDog`. In this case we could implement all barking and yodeling as part of each individual breed’s class—since each dog would sound differently. This is just one example of some of the design decisions that have to be made. Perhaps the best solution is to implement the barking and yodeling as interfaces, which we discuss in [Chapter 8](#).

A design pattern, which is covered in [Chapter 10](#), “[Design Patterns](#),” might be a good option in this case. A developer might not typically create these variations of `Dog`; they would either use a `Dog` (which implements `IDog`) or use a decorator to add behaviors to a `Dog` object.

Design Decisions

In theory, factoring out as much commonality as possible is great. However, as in all design issues, sometimes it really is too much of a good thing. Although factoring out as much commonality as possible might represent real life as closely as possible, it might not represent your model as closely as possible. The more you factor out, the more complex your system gets. So you have a conundrum: Do you want to live with a more accurate model or a system with less complexity? You must make this choice based on your situation, for there are no hard guidelines to make the decision.

What Computers Are Not Good At

Obviously, a computer model can only approximate real-world situations. Computers are good at number crunching but are not as good at more abstract operations.

For example, breaking up the `Dog` class into `BarkingDog` and the `YodelingDog` models real life better than assuming that all dogs bark, but it does add a bit of complexity.

Model Complexity

At this level of our example, adding two more classes does not make things so complex that it makes the model untenable. However, in larger systems, when these kinds of decisions are made over and over, the complexity quickly adds up. In larger systems, keeping things as simple as possible is usually the best practice.

There will be instances in your design when the advantage of a more accurate model does not warrant the additional complexity. Let’s assume that you are a dog breeder and that you contract out for a system that tracks all your dogs. The system model that includes barking dogs and yodeling dogs works fine. However, suppose that you do not breed any yodeling dogs—never have and never will. Perhaps you do not need to include the complexity of differentiating between yodeling dogs and barking dogs. This will make your system less complex, and it will provide the functionality that you need.

Deciding whether to design for less complexity or more functionality is a balancing act. The primary goal is always to build a system that is flexible without adding so much complexity that the system collapses under its own weight. What happens if you need to add yodeling at a later point in the project?

Current and future costs are also a major factor in these decisions. Although it might seem appropriate to make a system more complete and flexible, this added functionality might barely add any benefit—the return on investment might not be there. For example, would you extend the design of your `Dog` system to include other canines, such as hyenas and foxes (see [Figure 7.5](#))?

Um padrão de projeto, tema do Capítulo 10, também pode ser uma boa solução. Em vez de criar várias subclasses de `Dog`, um desenvolvedor poderia trabalhar com um `Dog` que implementa `IDog` e usar um Decorator para acrescentar comportamentos quando necessário.

DECISÕES DE PROJETO

Em teoria, extrair o máximo possível de características comuns parece excelente. Na prática, porém, até uma boa ideia pode ser exagerada. Um modelo muito fiel ao mundo real pode não ser o melhor modelo para o seu sistema. Quanto mais detalhes são fatorados, maior tende a ser a complexidade.

Surge então um dilema: preferir um modelo mais preciso ou um sistema mais simples? Não existe uma regra rígida; a decisão depende do contexto.

NO QUE COMPUTADORES NÃO SÃO TÃO BONS

Um modelo computacional só consegue aproximar situações do mundo real. Computadores são excelentes para cálculos, mas menos naturais em operações abstratas. Separar `Dog` em `BarkingDog` e `YodelingDog` representa melhor a realidade do que assumir que todos os cães latem, mas adiciona complexidade.

COMPLEXIDADE DO MODELO

Neste pequeno exemplo, duas classes extras não tornam o sistema inviável. Em sistemas grandes, contudo, decisões semelhantes se acumulam rapidamente. Por isso, manter o projeto tão simples quanto possível costuma ser uma boa prática.

Haverá situações em que a precisão adicional não compensa a complexidade. Suponha que você seja um criador de cães e contrate um sistema para acompanhar todos os animais. Se você nunca cria e nunca pretende criar cães que “iodeleiam”, talvez não seja necessário distinguir `BarkingDog` de `YodelingDog`. O sistema fica mais simples e ainda atende às necessidades atuais.

Projetar para menor complexidade ou maior funcionalidade é um exercício de equilíbrio. O objetivo é construir algo flexível sem adicionar tanta complexidade que o próprio sistema se torne difícil de sustentar. Mas também é preciso perguntar: e se, no futuro, o requisito de cães que “iodeleiam” aparecer?

Custos atuais e futuros pesam nessa decisão. Tornar um sistema mais completo e flexível pode gerar pouco benefício e um retorno sobre investimento insuficiente. Por exemplo: valeria a pena estender o sistema de `Dog` para abranger outros canídeos, como hienas e raposas, conforme a Figura 7.5?

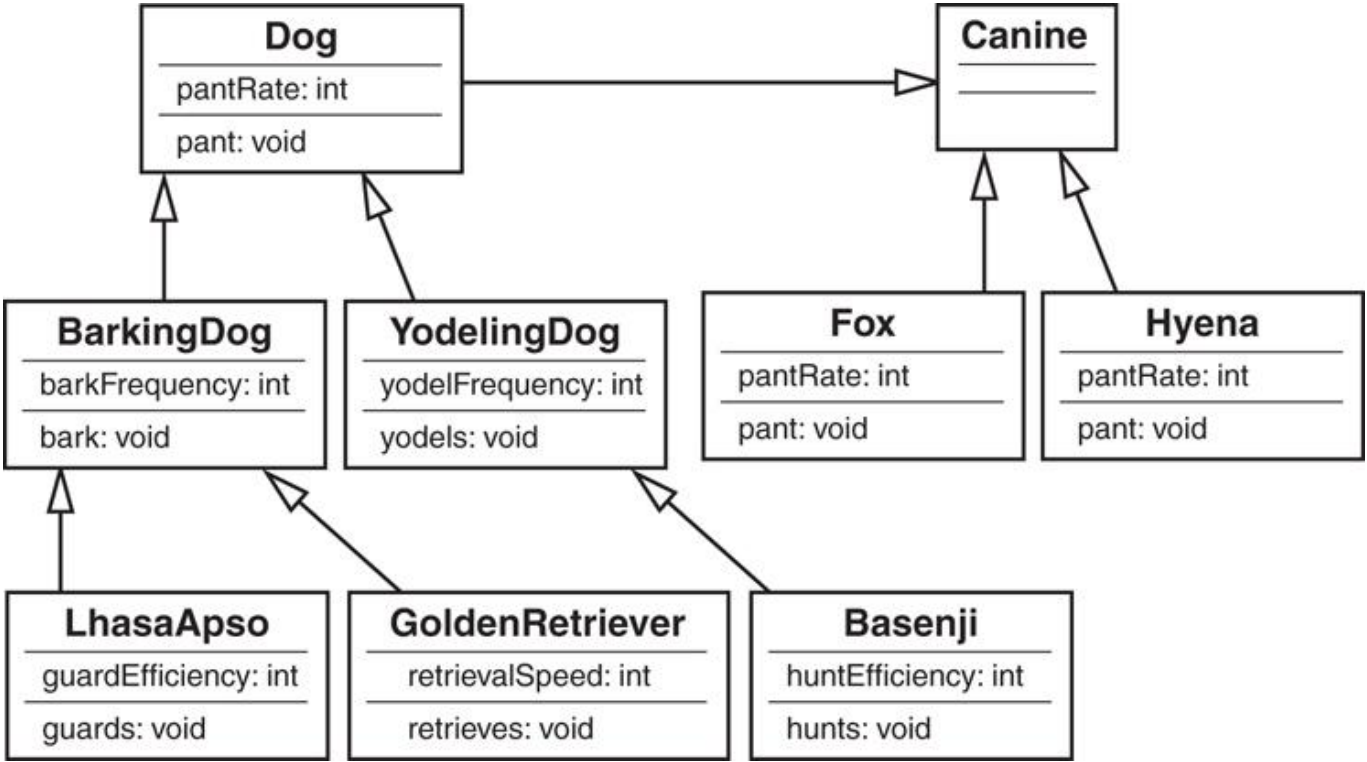


Figure 7.5 An expanded canine model.

Although this design might be prudent if you were a zookeeper, the extension of the `Canine` class is probably not necessary if you are breeding and selling domesticated dogs.

So as you can see, there are always trade-offs when creating a design.

Making Design Decisions with the Future in Mind

You might at this point say, “Never say never.” Although you might not breed yodeling dogs now, sometime in the future you might want to do so. If you do not design for the possibility of yodeling dogs now, it will be much more expensive to change the system later to include them. This is yet another of the many design decisions that you have to make. You could possibly override the `bark()` method to make it `yodel`; however, this is not intuitive, and some people will expect a method called `bark()` to actually bark.

COMPOSITION

It is natural to think of objects as containing other objects. A television set contains a tuner and video display. A computer contains video cards, keyboards, and drives. The computer can be considered an object unto itself, and a flash drive is also considered a valid object. You could open up the computer and remove the hard drive and hold it in your hand. In fact, you could take the hard drive to another computer and install it. The fact that it is a standalone object is reinforced because it works in multiple computers.

The classic example of object composition is the automobile. Many books, training classes, and articles seem to use the automobile as the classic example of object composition. Besides the original interchangeable manufacture of the rifle, most people think of the automobile assembly line created by Henry Ford as the quintessential example of interchangeable parts. Thus, it seems natural that the automobile has become a primary reference point for designing OO software systems.

Figura 7.5 - Um modelo de canídeos expandido.

Esse projeto pode ser prudente para um zoológico, mas provavelmente é desnecessário para uma empresa que cria e vende cães domésticos. Todo projeto envolve compromissos.

TOMANDO DECISÕES DE PROJETO PENSANDO NO FUTURO

É possível argumentar: “nunca diga nunca”. Talvez hoje não existam cães que “iodeleiam” no seu negócio, mas isso pode mudar. Se o sistema não tiver sido pensado para essa possibilidade, adaptar a estrutura depois pode custar mais. Essa é outra decisão de projeto. Poderíamos sobrescrever `bark()` para produzir outro som, mas isso seria pouco intuitivo: um método chamado `bark()` cria a expectativa de que o objeto realmente lata.

COMPOSIÇÃO

É natural imaginar objetos contendo outros objetos. Uma televisão contém sintonizador e tela. Um computador contém placas, teclado e unidades de armazenamento. O computador é um objeto, e uma unidade de armazenamento também é um objeto independente. Podemos removê-la de uma máquina e instalá-la em outra, reforçando sua identidade própria.

O exemplo clássico de composição de objetos é o automóvel. Assim como linhas de montagem usam peças intercambiáveis, um automóvel é construído a partir de diversos componentes que podem ser vistos como objetos independentes. Por isso, carros aparecem com frequência como referência em livros, cursos e exemplos de projeto OO.

Most people would think it natural for a car to contain an engine. However, a car contains many objects besides an engine, including wheels, a steering wheel, and a stereo. Whenever a particular object is composed of other objects, and those objects are included as object fields, the new object is known as a *compound*, an *aggregate*, or a *composite object* (see [Figure 7.6](#)).

A Car has a Steering Wheel

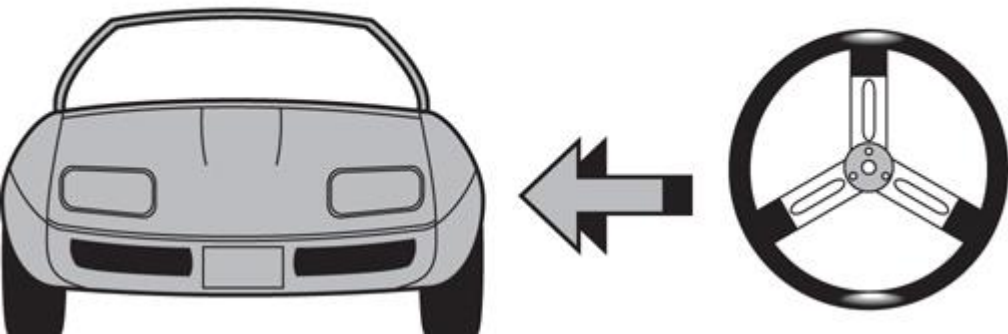


Figure 7.6 An example of

composition.

Aggregation, Association, and Composition

From my perspective, there are only two ways to reuse classes—with inheritance or with composition. In [Chapter 9, “Building Objects and Object-Oriented Design,”](#) we discuss composition in more detail—specifically, aggregation and association. In this book, I consider aggregation and association to be types of composition, although there are varied opinions on this.

Representing Composition with UML

To model the fact that the car object contains a steering wheel object, UML uses the notation shown in [Figure 7.7](#).

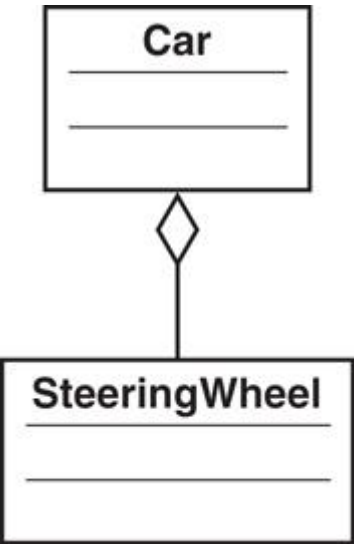


Figure 7.7 Representing composition in UML.

Aggregation, Association, and UML

In this book, aggregations are represented in UML by lines with a diamond, such as an engine as part of a car. Associations are represented by just the line (no diamond), such as a standalone keyboard servicing a separate computer box.

A maioria das pessoas considera natural que um carro contenha um motor. Mas ele também contém rodas, volante, sistema de som e muitos outros objetos. Quando um objeto é formado por outros objetos mantidos como seus campos, ele pode ser chamado de objeto composto, agregado ou composite, conforme a Figura 7.6.

Figura 7.6 - Exemplo de composição.

AGREGAÇÃO, ASSOCIAÇÃO E COMPOSIÇÃO

Neste livro, existem essencialmente duas formas de reutilizar classes: por herança ou por composição. O Capítulo 9 detalha agregação e associação. Aqui, agregação e associação são tratadas como tipos de composição, embora existam diferentes convenções na literatura.

REPRESENTANDO COMPOSIÇÃO COM UML

Para modelar que um objeto Car contém um objeto SteeringWheel, a UML usa a notação da Figura 7.7.

Figura 7.7 - Representação de composição em UML.

AGREGAÇÃO, ASSOCIAÇÃO E UML

Neste livro, agregações são representadas por uma linha com um losango, como um motor que faz parte de um carro. Associações aparecem apenas como uma linha, sem losango, como um teclado independente associado a um computador.

Note that the line connecting the `Car` class to the `SteeringWheel` class has a diamond shape on the `Car` side of the line. This signifies that a `Car` *contains* (has-a) `SteeringWheel`.

Let’s expand this example. Suppose that none of the objects in this design use inheritance in any way. All the object relationships are strictly composition, and there are multiple levels of composition. Of course, this is a simplistic example, and there are many, many more object and object relationships in designing a car. However, this design is meant to be a simple illustration of what composition is all about.

Let’s say that a car is composed of an engine, a stereo system, and a door.

How Many Doors and Stereos?

Note that a car normally has more than one door. Some have two, and some have four. You might even consider a hatchback a fifth door. In the same vein, it is not necessarily true that all cars have a stereo system. A car could have no stereo system or it could have one. I have even seen a car with two separate stereo systems. These situations are discussed in detail in [Chapter 9](#). For the sake of this example, just pretend that a car has only a single door (perhaps it's a special racing car) and a single stereo system.

That a car is made up of an engine, a stereo system, and a door is easy to understand because most people think of cars in this way. However, it is important to keep in mind when designing software systems, just like automobiles, that objects are made up of other objects. In fact, the number of nodes and branches that can be included in this tree structure of classes is virtually unlimited.

Figure 7.8 shows the object model for the car, with the engine, stereo system, and door included.

Observe que a linha entre `Car` e `SteeringWheel` possui um losango no lado de `Car`. Isso indica que `Car` contém - ou “tem-um” - `SteeringWheel`.

Vamos ampliar o exemplo. Suponha que nenhum objeto do projeto utilize herança: todas as relações são exclusivamente de composição e existem vários níveis de composição. É uma simplificação, pois um automóvel real possui muito mais componentes e relações, mas serve para ilustrar o conceito.

Considere que um carro seja composto por um motor, um sistema de som e uma porta.

QUANTAS PORTAS E SISTEMAS DE SOM?

Um carro normalmente possui mais de uma porta: pode ter duas, quatro ou até considerar o porta-malas/hatch como uma quinta. Também não é obrigatório existir sistema de som; poderia haver zero, um ou até mais. Essas multiplicidades serão discutidas no Capítulo 9. Para simplificar o exemplo, vamos fingir que nosso carro possui uma única porta - talvez seja um carro especial de corrida - e um único sistema de som.

É fácil entender que um carro seja formado por motor, som e porta. O ponto importante para o projeto de software é perceber que objetos podem ser construídos com outros objetos. A quantidade de níveis e ramificações nessa estrutura pode ser muito grande.

A Figura 7.8 mostra o modelo do carro incluindo motor, sistema de som e porta.

Figura 7.8 - Hierarquia/composição da classe Car.

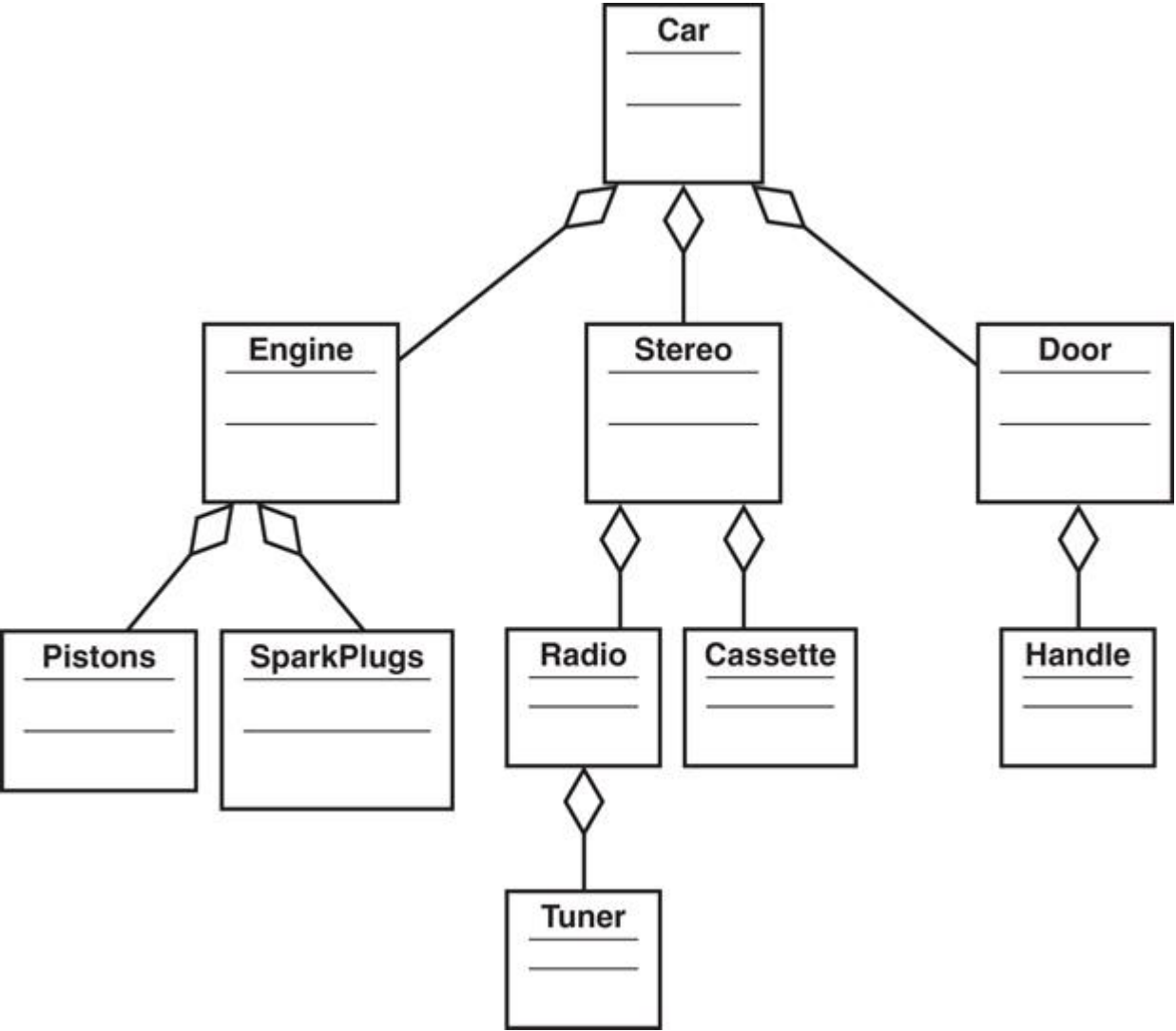
Os três objetos que compõem o carro também são formados por outros objetos. O motor contém pistões e velas; o sistema de som contém rádio e tocador de CD; a porta contém uma maçaneta. Há ainda mais um nível: o rádio contém um sintonizador. Poderíamos continuar: a maçaneta pode conter uma fechadura, o tocador de CD pode conter controles, o sintonizador pode conter um seletor, e assim por diante. O nível de detalhamento e a complexidade do modelo são decisões do projetista.

COMPLEXIDADE DO MODELO

Assim como no exemplo de herança com cães que latem ou “iodeliam”, composição em excesso também pode aumentar demais a complexidade. Existe uma linha tênue entre um modelo granular o bastante para expressar o problema e um modelo tão fragmentado que se torna difícil de compreender e manter.

POR QUE ENCAPSULAMENTO É FUNDAMENTAL EM OO

Encapsulamento é um conceito fundamental da orientação a objetos. Sempre que falamos do paradigma interface/implementação, estamos falando de encapsulamento. A pergunta básica é: o que uma classe deve expor e o que deve ocultar? Isso se aplica tanto aos dados quanto aos comportamentos.



Figure

7.8 The Car class hierarchy.

Note that all three objects that make up a car are themselves composed of other objects. The engine contains pistons and spark plugs. The stereo contains a radio and a CD player. The door contains a handle. Also note that there is yet another level. The radio contains a tuner. We could have also added the fact that a handle contains a lock; the CD player contains a fast forward button, and so on. Additionally, we could have gone one level beyond the tuner and created an object for a dial. The level and complexity of the object model is up to the designer.

Model Complexity

As with the inheritance problem of the barking and yodeling dogs, using too much composition can also lead to more complexity. A fine line exists between creating an object model that contains enough granularity to be sufficiently expressive and a model that is so granular that it is difficult to understand and maintain.

WHY ENCAPSULATION IS FUNDAMENTAL TO OO

Encapsulation is the fundamental concept of OO. Whenever the interface/implementation paradigm is covered, we are talking about encapsulation. The basic question is what in a class should be exposed and what should not be exposed. This encapsulation pertains equally to data

and behavior. When talking about a class, the primary design decision revolves around encapsulating both the data and the behavior into a well-written class.

Stephen Gilbert and Bill McCarty define encapsulation as “the process of packaging your program, dividing each of its classes into two distinct parts: the interface and the implementation.” This is the message that has been presented over and over in this book.

But what does encapsulation have to do with inheritance, and how does it apply with regard to this chapter? This has to do with an OO paradox. Encapsulation is so fundamental to OO that it is one of OO design’s cardinal rules. Inheritance is also considered one of the three primary OO concepts. However, in one way, inheritance actually breaks encapsulation! How can this be? Is it possible that two of the three primary concepts of OO are incompatible with each other? Let’s explore this possibility.

How Inheritance Weakens Encapsulation

As already stated, encapsulation is the process of packaging classes into the public interface and the private implementation. In essence, a class hides everything that is not necessary for other classes to know about.

Peter Coad and Mark Mayfield make a case that when using inheritance, encapsulation is inherently weakened within a class hierarchy. They talk about a specific risk: Inheritance connotes strong encapsulation with other classes but weak encapsulation between a superclass and its subclasses.

The problem is that if you inherit an implementation from a superclass and then change that implementation, the change from the superclass *ripples through* the class hierarchy. This rippling effect potentially affects all the subclasses. At first, this might not seem like a major problem; however, as we have seen, a rippling effect such as this can cause unanticipated problems. For example, testing can become a nightmare. In [Chapter 6](#), “[Designing with Objects](#),” we talked about how encapsulation makes testing systems easier. In theory, if you create a class called `Cabbie` (see [Figure 7.9](#)) with the appropriate public interfaces, any change to the implementation of `Cabbie` should be transparent to all other classes. However, in any design a change to a superclass is certainly not transparent to a subclass. Do you see the conundrum?

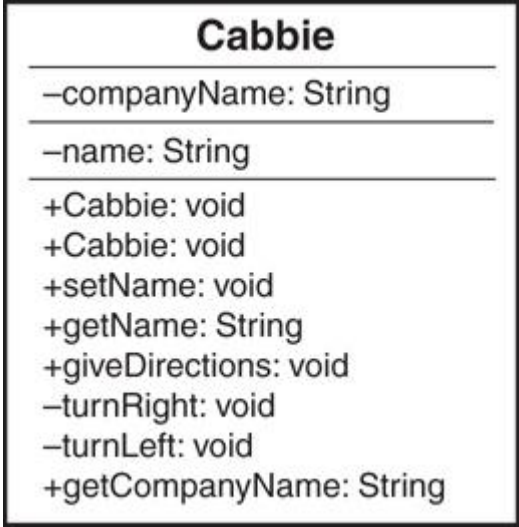


Figure 7.9 A UML diagram of the `Cabbie` class.

If the other classes were directly dependent on the implementation of the `Cabbie` class, testing would become more difficult, if not untenable. By using a different design approach, by abstracting out the behaviors and inheriting only attributes, these issues noted above go away.

Ao projetar uma classe, uma das principais decisões consiste em encapsular corretamente dados e comportamentos em uma unidade bem definida.

Stephen Gilbert e Bill McCarty definem encapsulamento como o processo de empacotar o programa, dividindo cada classe em duas partes distintas: interface e implementação. Essa ideia reaparece ao longo de todo o livro.

Mas qual é a relação entre encapsulamento e herança? Surge aqui um paradoxo OO. Encapsulamento é uma regra central de projeto. Herança também é considerada um dos três conceitos clássicos da orientação a objetos. Entretanto, sob certo ponto de vista, herança enfraquece o encapsulamento. Como dois conceitos tão importantes podem entrar em tensão?

COMO A HERANÇA ENFRAQUECE O ENCAPSULAMENTO

Encapsular significa separar uma interface pública de uma implementação privada: a classe oculta tudo o que outras classes não precisam conhecer.

Peter Coad e Mark Mayfield argumentam que, dentro de uma hierarquia de herança, esse encapsulamento é naturalmente enfraquecido. A herança pode preservar forte encapsulamento em relação a classes externas, mas criar um acoplamento mais forte entre uma superclasse e suas subclasses.

O problema aparece quando uma subclasse herda a implementação da superclasse. Se essa implementação muda, o efeito pode se propagar por toda a hierarquia e afetar diversas subclasses. Essa propagação pode causar problemas inesperados e tornar os testes mais difíceis.

No Capítulo 6 vimos que encapsulamento facilita testes. Em teoria, se criamos uma classe `Cabbie` com interfaces públicas adequadas, uma alteração interna em `Cabbie` deveria ser transparente para outras classes. Porém, uma alteração em uma superclasse não é necessariamente transparente para suas subclasses. Esse é o dilema.

Figura 7.9 - Diagrama UML da classe `Cabbie`.

Se outras classes dependessem diretamente da implementação de `Cabbie`, os testes seriam muito mais difíceis. Uma alternativa é abstrair os comportamentos e herdar apenas aquilo que realmente deve ser compartilhado, reduzindo essas dependências.

If you then create a subclass of `Cabbie` called `PartTimeCabbie`, and `PartTimeCabbie` inherits the implementation from `Cabbie`, changing the implementation of `Cabbie` directly affects the `PartTimeCabbie` class. For example, consider the UML diagram in [Figure 7.10](#). `PartTimeCabbie` is a subclass of `Cabbie`. Thus, `PartTimeCabbie` inherits the public implementation of `Cabbie`, including the method `giveDirections()`. If the method `giveDirections()` is changed in `Cabbie`, it will have a direct impact on `PartTimeCabbie` and any other classes that might later be subclasses of `Cabbie`. In this subtle way, changes to the implementation of `Cabbie` are not necessarily encapsulated within the `Cabbie` class.

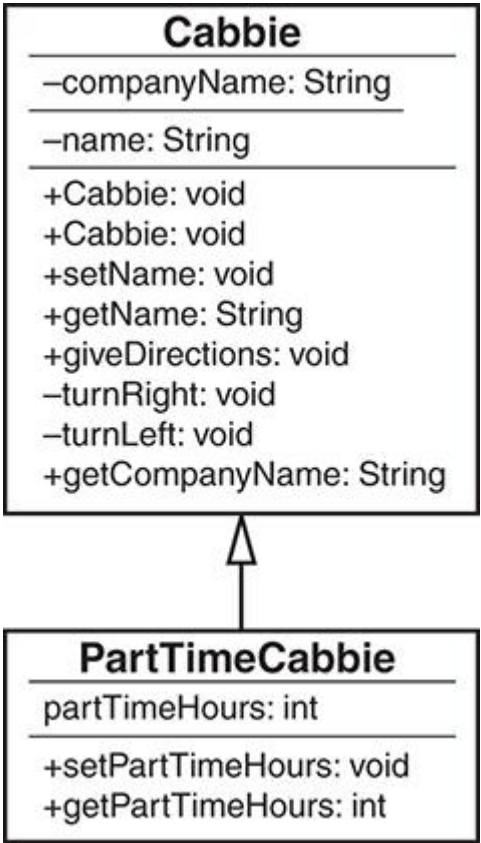


Figure 7.10 A UML diagram of the `Cabbie/PartTimeCabbie` classes.

To reduce the risk posed by this dilemma, it is important that you stick to the strict is-a condition when using inheritance. If the subclass were truly a specialization of the superclass, changes to the parent would likely affect the child in ways that are natural and expected. To illustrate, if a `Circle` class inherits implementation from a `Shape` class, and a change to the implementation of `Shape` breaks `Circle`, then `Circle` was not truly a `Shape` to begin with.

How can inheritance be used improperly? Consider a situation in which you want to create a window for the purposes of a graphical user interface (GUI). One impulse might be to create a window by making it a subclass of a rectangle class:

```
public class Rectangle {
    ...
}

public class Window extends Rectangle {
    ...
}
```

In reality a GUI window is much, much more than a rectangle. It is not a specialized version of a rectangle, as is a square. A true window might contain a rectangle (in fact, many rectangles);

Se criarmos uma subclasse de `Cabbie` chamada `PartTimeCabbie` e ela herdar a implementação de `Cabbie`, qualquer alteração nessa implementação afetará diretamente `PartTimeCabbie`.

Considere o diagrama UML da Figura 7.10. `PartTimeCabbie` é uma subclasse de `Cabbie` e, portanto, herda sua implementação pública, inclusive `giveDirections()`. Se `giveDirections()` for alterado em `Cabbie`, a mudança afetará `PartTimeCabbie` e qualquer outra subclasse futura. Dessa forma sutil, mudanças na implementação de `Cabbie` não ficam completamente encapsuladas dentro dela.

Figura 7.10 - Diagrama UML de `Cabbie/PartTimeCabbie`.

Para reduzir esse risco, é importante obedecer rigorosamente à condição “é-um” ao usar herança. Se a subclasse for realmente uma especialização da superclasse, mudanças no pai tenderão a afetar o filho de maneira natural e esperada. Por exemplo, se `Circle` herda de `Shape` e uma alteração legítima em `Shape` quebra `Circle`, talvez `Circle` não estivesse modelado corretamente como `Shape`.

COMO A HERANÇA PODE SER USADA INDEVIDAMENTE?

Imagine que você queira criar uma janela de interface gráfica. Uma reação inicial poderia ser fazê-la herdar de `Rectangle`:

```
public class Rectangle {
    ...
}

public class Window extends Rectangle {
    ...
}
```

Mas uma janela GUI é muito mais do que um retângulo. Ela não é uma especialização de `Rectangle` como um quadrado poderia ser. Uma janela pode conter retângulos - na verdade, vários -, mas não é, conceitualmente, um `Rectangle`.

however, it is not a true rectangle. In this approach, a `Window` class should not inherit from `Rectangle`, but it should contain `Rectangle` classes.

```
public class Window {  
  
    Rectangle menubar;  
    Rectangle statusbar;  
    Rectangle mainview;  
  
}
```

A Detailed Example of Polymorphism

Many people consider polymorphism a cornerstone of OO design. Designing a class for the purpose of creating totally independent objects is what OO is all about. In a well-designed system, an object should be able to answer all the important questions about it. As a rule, an object should be responsible for itself. This independence is one of the primary mechanisms of code reuse.

As stated in [Chapter 1](#), polymorphism literally means *many shapes*. When a message is sent to an object, the object must have a method defined to respond to that message. In an inheritance hierarchy, all subclasses inherit the interfaces from their superclass. However, because each subclass is a separate entity, each might require a separate response to the same message.

To review the example in [Chapter 1](#), consider a class called `Shape`. This class has a behavior called `Draw`. However, when you tell somebody to draw a shape, the first question is likely to be, “What shape?” Simply telling a person to draw a shape is too abstract (in fact, the `Draw` method in `Shape` contains no implementation). You must specify which shape you mean. To do this, you provide the actual implementation in `Circle` and other subclasses. Even though `Shape` has a `Draw` method, `Circle` overrides this method and provides its own `Draw` method. Overriding basically means replacing an implementation of a parent with your own.

Object Responsibility

Let’s revisit the `Shape` example from [Chapter 1](#) (see [Figure 7.11](#)).

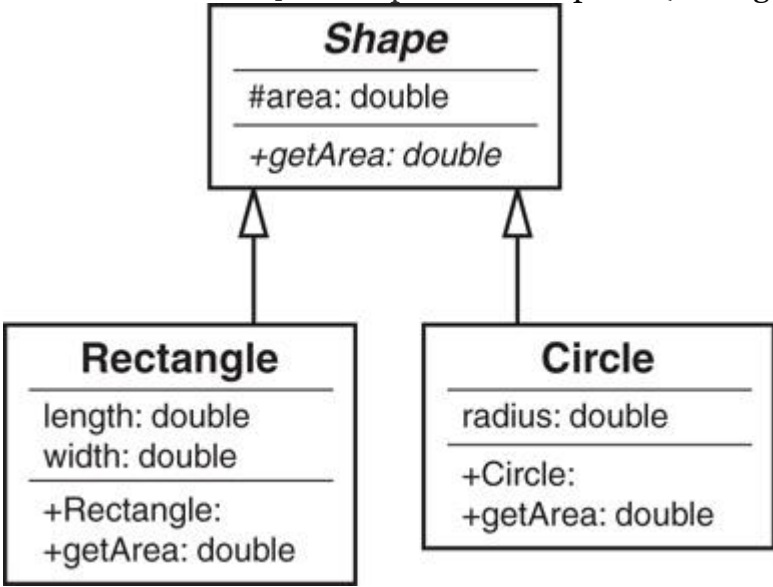


Figure 7.11 The `Shape` class hierarchy.

Polymorphism is one of the most elegant uses of inheritance. Remember that a `Shape` cannot be instantiated. It is an abstract class because it has an abstract method, `getArea()`. [Chapter 8](#) explains abstract classes in great detail.

Nesse caso, `Window` não deveria herdar de `Rectangle`; deveria conter objetos `Rectangle`:

```
public class Window {  
    Rectangle menubar;  
    Rectangle statusbar;  
    Rectangle mainview;  
}
```

UM EXEMPLO DETALHADO DE POLIMORFISMO

Muitos consideram polimorfismo uma pedra angular do projeto OO. Projetar classes capazes de produzir objetos independentes é parte essencial da orientação a objetos. Em um sistema bem projetado, um objeto deve conseguir responder às principais perguntas sobre si mesmo. Como regra, deve ser responsável por si próprio. Essa independência é um dos mecanismos fundamentais de reutilização de código.

Como vimos no Capítulo 1, polimorfismo significa literalmente “muitas formas”. Quando uma mensagem é enviada a um objeto, ele deve possuir um método capaz de responder. Em uma hierarquia de herança, subclasses recebem as interfaces da superclasse, mas cada subclasse é uma entidade própria e pode exigir uma resposta diferente à mesma mensagem.

Retomemos `Shape`. A classe possui um comportamento `Draw`. Porém, pedir simplesmente “desenhe uma forma” é abstrato demais. Precisamos saber qual forma. Por isso, a implementação concreta de `Draw` é fornecida por `Circle` e pelas demais subclasses. `Circle` sobrescreve o método herdado e fornece sua própria implementação. Sobrescrever significa substituir a implementação herdada por outra apropriada ao subtipo.

RESPONSABILIDADE DO OBJETO

A Figura 7.11 retoma a hierarquia `Shape`. `Shape` não pode ser instanciada: é uma classe abstrata porque possui um método abstrato, `getArea()`. Classes abstratas serão estudadas em detalhe no Capítulo 8.

Rectangle e Circle, por outro lado, podem ser instanciadas porque são classes concretas. Ambas são formas e ambas possuem uma área, mas a fórmula para calculá-la é diferente em cada caso. Por isso, a fórmula não pode ser implementada de maneira única em Shape.

É aqui que entra o polimorfismo. Podemos enviar a mesma mensagem a objetos diferentes e cada um responderá de acordo com seu tipo. Se enviarmos `getArea()` a Circle, a execução será diferente de enviar `getArea()` a Rectangle. Cada classe concreta é responsável por conhecer seu próprio comportamento.

Se pedirmos a Circle sua área, ela sabe calculá-la. Se pedirmos que se desenhe, ela sabe fazê-lo. Shape, mesmo se pudesse ser instanciada, não possuiria informação suficiente. No diagrama UML da Figura 7.11, `getArea()` aparece em itálico para indicar que é abstrato.

Como exemplo simples, considere quatro classes: Shape abstrata e as concretas Circle, Rectangle e Star. O código da página define em Shape apenas o contrato:

```
public abstract void draw();
```

Cada classe concreta sobrescreve `draw()` e imprime uma mensagem correspondente, como “I am drawing a Circle”, “I am drawing a Rectangle” ou “I am drawing a Star”.

O ponto principal é que cada classe concreta assume a responsabilidade por se desenhar. Shape não fornece o código de desenho; Circle, Rectangle e Star fornecem suas próprias implementações. O programa de teste que começa nesta página cria instâncias dessas classes para demonstrar esse comportamento.

However, `Rectangle` and `Circle` can be instantiated because they are concrete classes.

Although `Rectangle` and `Circle` are both shapes, they have some differences. As shapes, their area can be calculated. Yet the formula to calculate the area is different for each. Thus, the area formulas cannot be placed in the `Shape` class.

This is where polymorphism comes in. The premise of polymorphism is that you can send messages to various objects, and they will respond according to their object’s type. For example, if you send the message `getArea()` to a `Circle` class, you will invoke a different calculation than if you send the same `getArea()` message to a `Rectangle` class. This is because both `Circle` and `Rectangle` are responsible for themselves. If you ask `Circle` to return its area, it knows how to do this. If you want a circle to draw itself, it can do this as well. A `Shape` object could not do this even if it could be instantiated because it does not have enough information about itself. Notice that in the UML diagram (**Figure 7.11**), the `getArea()` method in the `Shape` class is italicized. This designates that the method is abstract.

As a very simple example, imagine that there are four classes: the abstract class `Shape`, and concrete classes `Circle`, `Rectangle`, and `Star`. Here is the code:

[Click here to view code image](#)

```
public abstract class Shape{

    public abstract void draw();

}

public class Circle extends Shape{

    public void draw() {

        System.out.println("I am drawing a Circle");

    }

}

public class Rectangle extends Shape{

    public void draw() {

        System.out.println("I am drawing a Rectangle");

    }

}

public class Star extends Shape{

    public void draw() {

        System.out.println("I am drawing a Star");

    }

}
```

Notice that only one method exists for each class: `draw()`. Here is the important point regarding polymorphism and an object being responsible for itself: The concrete classes themselves have responsibility for the drawing function. The `Shape` class does not provide the code for drawing; the `Circle`, `Rectangle`, and `Star` classes do this for themselves. Here is some code to prove it:

[Click here to view code image](#)

```
public class TestShape {

    public static void main(String args[]) {
```



```
Circle circle = new Circle();
Rectangle rectangle = new Rectangle();
Star star = new Star();

circle.draw();
rectangle.draw();
star.draw();

}

}

The test application TestShape creates three classes: Circle, Rectangle, and Star. To draw these
classes, TestShape asks the individual classes to draw themselves:
circle.draw();
rectangle.draw();
star.draw();
When you execute TestShape, you get the following results:
C:\>java TestShape
I am drawing a Circle
I am drawing a Rectangle
I am drawing a Star
This is polymorphism at work. What would happen if you wanted to create a new shape, such
as Triangle? Simply write the class, compile it, test it, and use it. The base class Shape does not
have to change—nor does any other code:

Click here to view code image
public class Triangle extends Shape{

    public void draw() {

        System.out.println("I am drawing a Triangle");

    }

}

A message can now be sent to Triangle. And even though Shape does not know how to draw a
triangle, the Triangle class does:

Click here to view code image
public class TestShape {

    public static void main(String args[]) {

        Circle circle = new Circle();
        Rectangle rectangle = new Rectangle();
        Star star = new Star();
        Triangle triangle = new Triangle ();

        circle.draw();
        rectangle.draw();
        star.draw();
        triangle.draw();

    }

}

C:\>java TestShape
I am drawing a Circle
I am drawing a Rectangle
I am drawing a Star
I am drawing a Triangle
```

O programa TestShape cria Circle, Rectangle e Star e pede a cada objeto que se desenhe:

```
circle.draw();
rectangle.draw();
star.draw();
```

A execução produz mensagens diferentes para cada objeto. Esse é o polimorfismo em funcionamento: a mesma operação conceitual, draw(), produz o comportamento apropriado ao tipo concreto.

E se quisermos acrescentar Triangle? Basta criar uma nova classe Triangle que estenda Shape, implementar draw(), compilar, testar e usar. Shape não precisa ser alterada, nem as outras classes existentes.

A nova classe pode implementar:

```
public void draw() {
    System.out.println("I am drawing a Triangle");
}
```

Agora uma mensagem draw() pode ser enviada a Triangle da mesma forma que às demais. Shape não sabe desenhar um triângulo, mas Triangle sabe.

A versão ampliada de TestShape cria Circle, Rectangle, Star e Triangle e chama draw() em cada uma. A saída mostra uma mensagem específica para cada forma. Assim, novas especializações podem ser adicionadas sem modificar a abstração central.

To see the real power of polymorphism, you can pass the shape to a method that has absolutely no idea what shape is coming. Take a look at the following code, which includes the specific shapes as parameters:

[Click here to view code image](#)

```
public class TestShape {

    public static void main(String args[]) {

        Circle circle = new Circle();
        Rectangle rectangle = new Rectangle();
        Star star = new Star();

        drawMe(circle);
        drawMe(rectangle);
        drawMe(star);

    }

    static void drawMe(Shape s) {
        s.draw();
    }

}
```

In this case, the `Shape` object can be passed to the method `drawMe()`, and the `drawMe()` method can handle any valid `Shape`—even one you add later. You can run this version of `TestShape` just like the previous one.

Abstract Classes, Virtual Methods, and Protocols

Abstract classes, as they are defined in Java, can be directly implemented in .NET and C++ as well. Not surprisingly, the C# .NET code looks similar to the Java code, as shown in the following:

[Click here to view code image](#)

```
public abstract class Shape{

    public abstract void draw();

}
```

The Visual Basic .NET code is written like this:

```
Public MustInherit Class Shape

    Public MustOverride Function draw()

End Class
```

The same functionality can be provided in C++ using virtual methods with the following code:

```
class Shape
{
    public:
        virtual void draw() = 0;
}
```

As mentioned in previous chapters, Objective-C and Swift do not fully implement the functionality of abstract classes.

A verdadeira força do polimorfismo aparece quando podemos passar qualquer forma a um método que não precisa saber qual tipo concreto recebeu.

O exemplo da página define:

```
static void drawMe(Shape s) {
    s.draw();
}
```

O programa pode então chamar `drawMe(circle)`, `drawMe(rectangle)` e `drawMe(star)`. O parâmetro é do tipo `Shape`, portanto `drawMe()` aceita qualquer objeto compatível com `Shape`, inclusive novas subclasses acrescentadas no futuro. Ao executar `s.draw()`, a implementação correta é selecionada de acordo com o objeto real.

CLASSES ABSTRATAS, MÉTODOS VIRTUAIS E PROTOCOLOS

Classes abstratas, como definidas em Java, também podem ser implementadas diretamente em .NET e C++. Em C# a sintaxe é muito semelhante:

```
public abstract class Shape {
    public abstract void draw();
}
```

Em Visual Basic .NET, usa-se `MustInherit` e `MustOverride`. Em C++, funcionalidade equivalente pode ser representada por um método virtual puro, por exemplo `virtual void draw() = 0;`.

Como mencionado anteriormente, Objective-C e Swift não implementam exatamente toda a funcionalidade de classes abstratas da mesma forma. Nesses ecossistemas, protocolos cumprem parte desse papel.

For example, consider the following Java interface code for the `Shape` class we have seen many times:

```
public abstract class Shape{

    public abstract void draw();

}
```

The corresponding Objective-C (Swift) protocol is shown in the following code. Note that in both the Java code and the Objective-C code, there is no implementation for the `draw()` method.

```
@protocol Shape

@required
- (void) draw;

@end // Shape
```

At this point, the functionality for the abstract class and the protocol are pretty much equivalent; however, here is where the Java-type interface and protocols diverge. Consider the following Java code:

```
public abstract class Shape{
    public abstract void draw();
    public void print() {
        System.out.println("I am printing");
    };
}
```

In the preceding Java code, the `print()` method provides code that can be inherited by a subclass. Although this is also the case with C# .NET, VB .NET, and C++,the same cannot be said for an Objective-C protocol, which would look like this:

```
@protocol Shape

@required
- (void) draw;
- (void) print;

@end // Shape
```

In this protocol, the `print()` method signature is provided, and thus must be implemented by a subclass; however, no code can be included. In short, subclasses cannot directly inherit any code from a protocol. Thus, the protocol cannot be used in the same way as an abstract class, and this has implications when designing an object model.

CONCLUSION

This chapter gives a basic overview of what inheritance and composition are and how they are different. Many well-respected OO designers have stated that composition should be used whenever possible, and inheritance should be used only when necessary.

However, this is a bit simplistic. I believe that the idea that composition should be used whenever possible hides the real issue, which might be that composition is more appropriate in more cases than inheritance—not that it should be used whenever possible. The fact that composition might be more appropriate in most cases does not mean that inheritance is evil. Use both composition and inheritance, but only in their proper contexts.

In earlier chapters, the concepts of abstract classes and Java interfaces arose several times. In [Chapter 8](#), we explore the concept of development contracts and how abstract classes and Java interfaces are used to satisfy these contracts.

Considere novamente o exemplo Java de `Shape` com um método abstrato `draw()`. O equivalente conceitual em Objective-C pode ser um protocolo:

```
@protocol Shape
@required
- (void) draw;
@end
```

Nos dois casos não há implementação de `draw()`; existe apenas uma exigência de interface.

A diferença aparece quando a classe abstrata também fornece uma implementação concreta. Em Java, `Shape` poderia declarar `draw()` como abstrato e, ao mesmo tempo, implementar `print()`:

```
public void print() {
    System.out.println("I am printing");
}
```

Subclasses podem herdar essa implementação. C# .NET, VB .NET e C++ permitem algo equivalente. Já um protocolo Objective-C pode declarar `print()`, mas não fornecer o corpo do método para ser herdado. A classe que adota o protocolo deve implementar o comportamento por conta própria.

Portanto, protocolos/interfaces e classes abstratas podem representar contratos semelhantes, mas não oferecem exatamente o mesmo mecanismo de reutilização de implementação. Isso tem consequências para o projeto de um modelo de objetos.

CONCLUSÃO

Este capítulo apresentou uma visão geral de herança e composição e de suas diferenças. Muitos projetistas OO respeitados recomendam preferir composição quando apropriado e usar herança apenas quando a relação realmente fizer sentido.

Dizer simplesmente “sempre use composição” é simplista. A questão é que composição pode ser adequada em mais situações, não que herança seja ruim. Ambas são ferramentas válidas, desde que usadas em seus contextos corretos.

Nos capítulos anteriores, classes abstratas e interfaces Java já apareceram diversas vezes. O Capítulo 8 aprofunda a ideia de contratos de desenvolvimento e mostra como classes abstratas e interfaces podem satisfazê-los.

REFERENCES

Booch, Grady and Robert A. Maksimchuk and Michael W. Engel and Bobbi J. Young, Jim Conallen, and Kelli A. Houston. 2007. *Object-Oriented Analysis and Design with Applications*, Third Edition. Boston, MA: Addison-Wesley.

Coad, Peter, and Mark Mayfield. 1997. *Java Design*. Upper Saddle River, NJ: Prentice Hall.

Gilbert, Stephen, and Bill McCarty. 1998. *Object-Oriented Design in Java*. Berkeley CA: The Waite Group Press.

Meyers, Scott. 2005. *Effective C++*, Third Edition. Boston, MA: Addison-Wesley Professional.

REFERÊNCIAS

Booch, Grady; Maksimchuk, Robert A.; Engel, Michael W.; Young, Bobbi J.; Conallen, Jim; Houston, Kelli A. 2007. *Object-Oriented Analysis and Design with Applications*, Third Edition. Boston, MA: Addison-Wesley.

Coad, Peter; Mayfield, Mark. 1997. *Java Design*. Upper Saddle River, NJ: Prentice Hall.

Gilbert, Stephen; McCarty, Bill. 1998. *Object-Oriented Design in Java*. Berkeley, CA: The Waite Group Press.

Meyers, Scott. 2005. *Effective C++*, Third Edition. Boston, MA: Addison-Wesley Professional.

[Os títulos bibliográficos foram preservados no idioma de publicação.]

8. Frameworks and Reuse: Designing with Interfaces and Abstract Classes

Chapter 7, “Mastering Inheritance and Composition,” explains how inheritance and composition play major roles in the design of object-oriented (OO) systems. This chapter expands upon the concepts of interfaces, protocols, and abstract classes.

Interfaces, protocols, and abstract classes are powerful mechanisms for code reuse, providing the foundation for a concept I call *contracts*. This chapter covers the topics of code reuse, frameworks, contracts, interfaces, protocols, and abstract classes (for the remainder of the chapter, unless otherwise indicated, I use the term *interface* to include the concept of protocols). At the end of the chapter, we’ll work through an example of how all these concepts can be applied to a real-world situation.

CODE: TO REUSE OR NOT TO REUSE?

Programmers have been dealing with the issue of code reuse ever since writing their first line of code. Many software development paradigms stress code reuse as a major part of the process. Since the dawn of computer software, the concept of reusing code has been reinvented several times. The OO paradigm is no different. One of the major advantages touted by OO proponents is that if you write code properly the first time, you can reuse it to your heart’s content.

This is true only to a certain degree. As with all design approaches, the utility and the reusability of code depend on how well it was designed and implemented. OO design does not hold the patent on code reuse. There is nothing stopping anyone from writing very robust and reusable code in a non-OO language. Certainly, countless numbers of routines and functions, written in structured languages such as COBOL, C, and traditional VB, are of high quality and are quite reusable.

Thus, it is clear that following the OO paradigm is not the only way to develop reusable code. However, the OO approach does provide several mechanisms for facilitating the development of reusable code. One way to create reusable code is to create frameworks. In this chapter, we focus on using interfaces and abstract classes to create frameworks and encourage reusable code.

WHAT IS A FRAMEWORK?

Hand in hand with the concept of code reuse is the concept of *standardization*, which is sometimes called *plug and play*. The idea of a framework revolves around these plug-and-play and reuse principles. One classic example of a framework is a desktop application. Let’s take an office suite application as an example. The document editor that I am currently using has a ribbon that includes multiple tab options. These options are similar to those in the presentation package and the spreadsheet software that I also have open. In fact, the first two menu items (Home, Insert) are the same in all three programs. Not only are the menu options similar, but many of the options look remarkably alike as well (New, Open, Save, and so on). Below the ribbon is the document area—whether it be for a document, a presentation, or a spreadsheet. The common framework makes it easier to learn various applications within the office suite. It also makes a developer’s life easier by allowing maximum code reuse, not to mention that we can reuse portions of the design as well.

8. FRAMEWORKS E REUSO: PROJETANDO COM INTERFACES E CLASSES ABSTRATAS

O Capítulo 7 explicou como herança e composição desempenham papéis importantes no projeto de sistemas orientados a objetos. Este capítulo amplia os conceitos de interfaces, protocolos e classes abstratas.

Interfaces, protocolos e classes abstratas são mecanismos poderosos de reutilização e oferecem a base para aquilo que o autor chama de contratos. O capítulo trata de reuso de código, frameworks, contratos, interfaces, protocolos e classes abstratas. Salvo indicação em contrário, o termo “interface” também abrangerá a ideia de protocolo. Ao final, os conceitos serão aplicados a uma situação real.

CÓDIGO: REUTILIZAR OU NÃO REUTILIZAR?

Programadores lidam com reuso desde que começaram a escrever software. Muitos paradigmas enfatizam esse objetivo e, ao longo da história, a ideia de reaproveitar código foi reinventada várias vezes. OO não é diferente. Um de seus benefícios frequentemente anunciados é que um código bem escrito pode ser reutilizado extensivamente.

Isso é verdade apenas até certo ponto. A utilidade e a reutilização dependem da qualidade do projeto e da implementação. OO não detém exclusividade sobre reuso: é perfeitamente possível produzir rotinas robustas e reutilizáveis em linguagens estruturadas como COBOL, C e Visual Basic tradicional.

Ainda assim, OO fornece mecanismos úteis para facilitar esse objetivo. Um deles é a criação de frameworks. Neste capítulo, o foco está em usar interfaces e classes abstratas para construir frameworks e estimular reuso.

O QUE É UM FRAMEWORK?

Reuso está ligado à padronização, às vezes descrita como “plug and play”. Um framework gira em torno desses princípios. Um exemplo clássico é uma aplicação desktop de uma suíte de escritório. Editor de texto, software de apresentações e planilha compartilham menus, abas, comandos e aparência semelhantes. Itens como Home, Insert, New, Open e Save seguem um padrão.

Essa estrutura comum facilita a aprendizagem para o usuário e permite ao desenvolvedor reaproveitar não apenas código, mas também partes do próprio projeto de interface.

The fact that all these menu bars have a similar look and feel is obviously not an accident. In fact, when you develop in most integrated development environments, on a certain platform like Microsoft Windows or Linux, for example, you get certain things without having to create them yourself. When you create a window in a Windows environment, you get elements like the main title bar and the file Close button in the top-right corner. Actions are standardized as well—when you double-click the main title bar, the screen always minimizes/maximizes. When you click the Close button in the top-right corner, the application always terminates. This is all part of the framework. **Figure 8.1** is a screenshot of a word processor. Note the menu bars, toolbars, and other elements that are part of the framework.

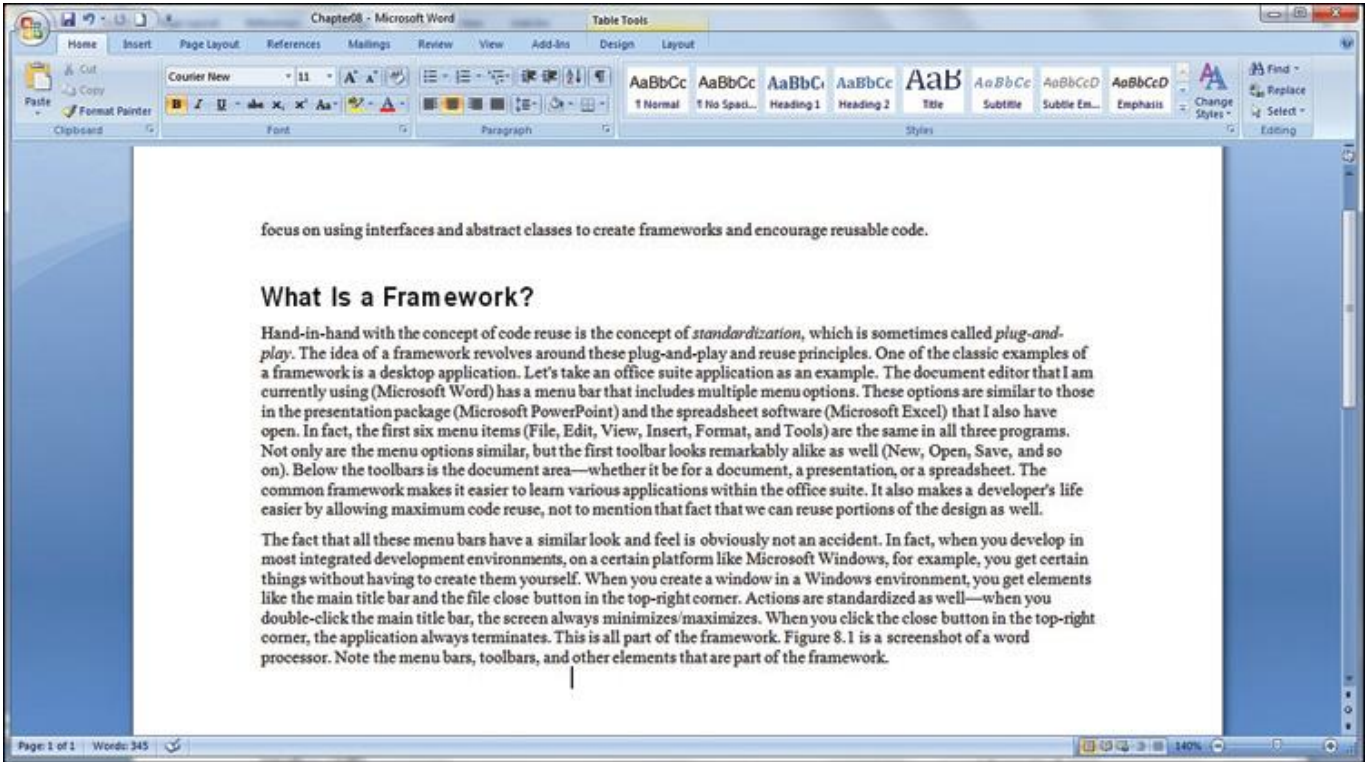


Figure 8.1 A word processing framework.

A word processing framework generally includes operations such as creating documents, opening documents, saving documents, cutting text, copying text, pasting text, searching through documents, and so on. To use this framework, a developer must use a predetermined interface to create an application. This predetermined interface conforms to the standard framework, which has two obvious advantages. First, as we have already seen, the look and feel are consistent, and the end users do not have to learn a new framework. Second, a developer can take advantage of code that has already been written and tested (and this testing issue is a huge advantage). Why write code to create a brand new Open dialog when one already exists and has been thoroughly tested? In a business setting, when time is critical, people do not want to have to learn new things unless it is absolutely necessary.

Code Reuse Revisited

In **Chapter 7**, we talked about code reuse as it pertains to inheritance—basically one class inheriting from another class. This chapter is about frameworks and reusing whole or partial systems.

The obvious question is this: If you need a dialog box, how do you use the dialog box provided by the framework? The answer is simple: Follow the rules that the framework provides. And where might you find these rules? The rules for the framework are found in the documentation. The person or persons who wrote the class, classes, or class libraries should have provided

A aparência semelhante entre aplicações de uma mesma plataforma não é acidente. Ao desenvolver em ambientes integrados para plataformas como Windows ou Linux, vários elementos já são fornecidos pelo framework. Uma janela pode receber automaticamente barra de título e botão de fechar. Ações também são padronizadas: clicar ou dar duplo clique em determinados elementos produz comportamentos esperados.

A Figura 8.1 mostra um processador de texto e destaca barras de menu, ferramentas e outros componentes pertencentes ao framework.

Figura 8.1 - Um framework de processamento de texto.

Um framework desse tipo normalmente inclui operações para criar, abrir e salvar documentos, recortar, copiar e colar texto, pesquisar conteúdo etc. Para utilizá-lo, o desenvolvedor trabalha com uma interface predefinida e segue os padrões estabelecidos.

Isso oferece duas vantagens evidentes. Primeiro, a aparência e o comportamento permanecem consistentes, reduzindo a curva de aprendizagem do usuário. Segundo, o desenvolvedor pode usar código já escrito e testado. Não há motivo para recriar do zero uma caixa de diálogo Open se uma implementação robusta já está disponível. Em ambientes de negócio, tempo e confiabilidade são fatores importantes.

REUSO DE CÓDIGO REVISITADO

No Capítulo 7, reuso foi tratado principalmente no contexto de herança - uma classe herdando de outra. Aqui, o assunto são frameworks e a reutilização de sistemas completos ou parciais.

Se precisamos de uma caixa de diálogo fornecida pelo framework, como utilizá-la? Seguimos as regras definidas pelo framework. Essas regras devem estar documentadas pelos autores das classes ou bibliotecas.

documentation on how to use the public interfaces of the class, classes, or class libraries (at least we hope). In many cases, this takes the form of the application-programming interface (API).

For example, to create a menu bar in Java, you would bring up the API documentation for the `JMenuBar` class and take a look at the public interfaces it presents. **Figure 8.2** shows a part of the Java API. By using these APIs, you can create a valid Java application and conform to required standards. If you follow these standards, your application will be set to run in Java-enabled browsers.

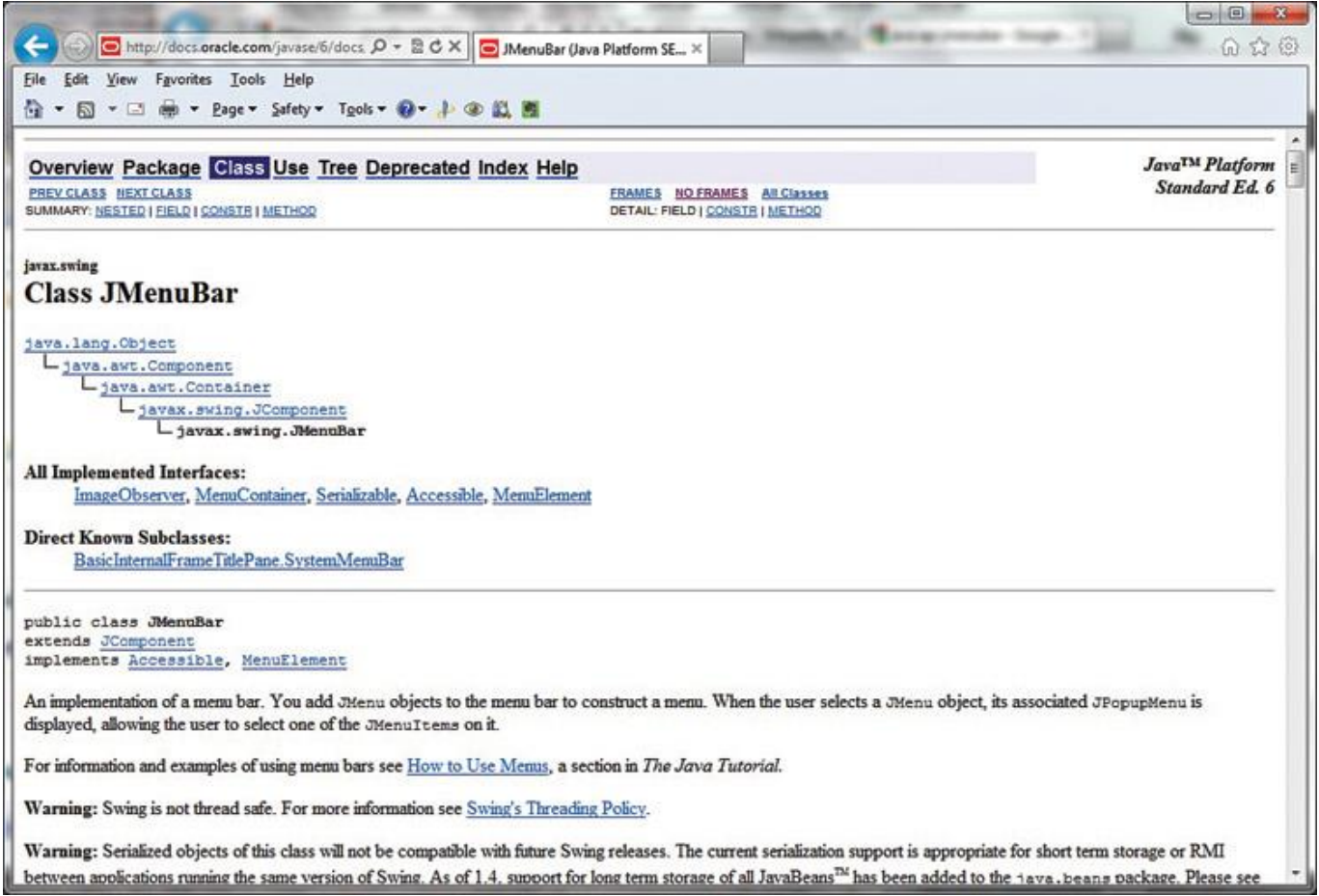


Figure 8.2 API documentation.

WHAT IS A CONTRACT?

In the context of this chapter, we will consider a *contract* to be any mechanism that requires a developer to comply with the specifications of an API. Often, an API is referred to as a framework. The online dictionary, Merriam-Webster (<https://www.merriam-webster.com>), defines a contract as a “binding agreement between two or more persons or parties, especially: one legally enforceable.”

This is exactly what happens when a developer uses an API—with the project manager, business owner, or industry standard providing the enforcement. When using contracts, the developer is required to comply with the rules defined in the framework. This includes issues such as method names, number of parameters, and so on (signatures, and the like). In short, standards are created to facilitate good development practices.

The Term Contract

A documentação deve explicar como utilizar as interfaces públicas das classes e bibliotecas. Muitas vezes ela assume a forma de uma API - Application Programming Interface.

Por exemplo, para criar uma barra de menu em Java, podemos consultar a documentação da classe `JMenuBar` e verificar quais interfaces públicas ela oferece. A Figura 8.2 mostra parte da API Java. Ao seguir essas APIs, desenvolvemos aplicações válidas e compatíveis com os padrões exigidos pelo ambiente.

Figura 8.2 - Documentação de API.

O QUE É UM CONTRATO?

Neste capítulo, “contrato” significa qualquer mecanismo que exija do desenvolvedor conformidade com as especificações de uma API. Uma API é frequentemente tratada como parte de um framework. No sentido comum, um contrato é um acordo vinculante entre partes; a analogia é apropriada ao desenvolvimento de software.

Ao utilizar uma API, o desenvolvedor precisa obedecer às regras definidas pelo framework, pelo gerente do projeto, pelo proprietário do negócio ou por um padrão da indústria. Isso inclui nomes de métodos, quantidade e tipos de parâmetros, assinaturas e outras convenções.

Em resumo, padrões são criados para facilitar boas práticas de desenvolvimento e permitir que componentes independentes trabalhem corretamente em conjunto.

O TERMO “CONTRATO”

A próxima seção aprofunda essa analogia e mostra como interfaces e classes abstratas podem funcionar como mecanismos formais para estabelecer as obrigações de um componente.

The term contract is widely used in many aspects of business, including software development. Do not confuse the concept presented here with other possible software design concepts called contracts.

Enforcement is vital because it is always possible for a developer to break a contract. Without enforcement, a rogue developer could decide to reinvent the wheel and write her own code rather than use the specification provided by the framework. There is little benefit to a standard if people routinely disregard or circumvent it. In Java and the .NET languages, the two ways to implement contracts are to use abstract classes and interfaces.

Abstract Classes

One way a contract is implemented is via an abstract class. An *abstract class* is a class that contains one or more methods that do not have any implementation provided. Suppose that you have an abstract class called `Shape`. It is abstract because you cannot instantiate it. If you ask someone to draw a shape, the first thing the person will most likely ask you is, “What kind of shape?” Thus, the concept of a shape is abstract. However, if someone asks you to draw a circle, this does not pose quite the same problem, because a circle is a concrete concept. You know what a circle looks like. You also know how to draw other shapes, such as rectangles.

How does this apply to a contract? Let’s assume that we want to create an application to draw shapes. Our goal is to draw every kind of shape represented in our current design, as well as ones that might be added later. There are two conditions we must adhere to.

First, we want all shapes to use the same syntax to draw themselves. For example, we want every shape implemented in our system to contain a method called `draw()`. Thus, seasoned developers implicitly know that to draw a shape, you invoke the `draw()` method, regardless of what the shape happens to be. Theoretically, this reduces the amount of time spent fumbling through manuals, and it cuts down on syntax errors.

Second, remember that it is important that every class be responsible for its own actions. Thus, even though a class is required to provide a method called `draw()`, that class must provide its own implementation of the code. For example, the classes `Circle` and `Rectangle` both have a `draw()` method; however, the `Circle` class obviously has code to draw a circle, and as expected, the `Rectangle` class has code to draw a rectangle. When we ultimately create classes called `Circle` and `Rectangle`, which are subclasses of `Shape`, these classes must implement their own version of `draw()` (see [Figure 8.3](#)).

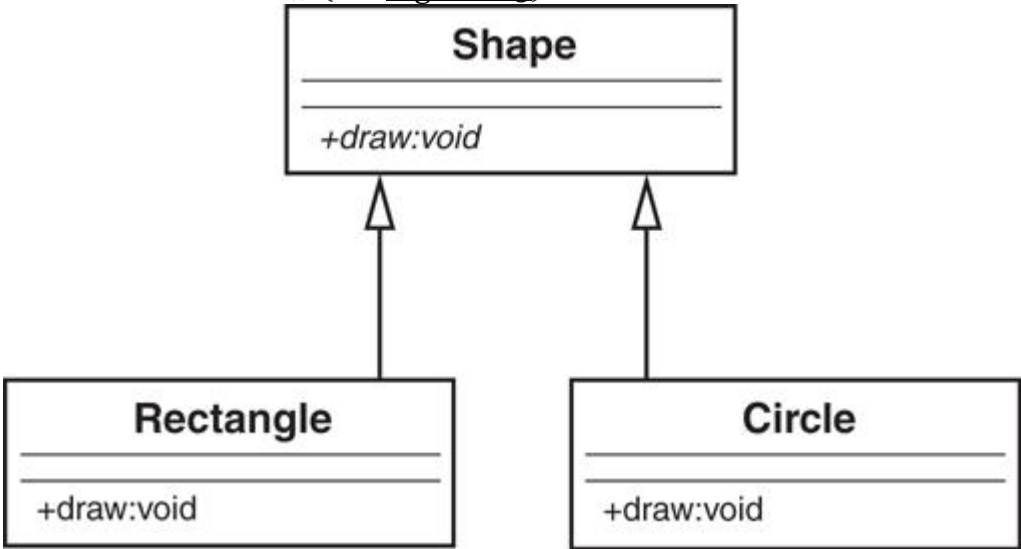


Figure 8.3 An abstract class hierarchy.

Abstract Methods

O termo “contrato” é amplamente usado em negócios e também em desenvolvimento de software. Aqui ele possui o significado específico apresentado neste capítulo e não deve ser confundido com outros conceitos de projeto que também recebem esse nome.

A aplicação das regras é essencial, porque sempre existe a possibilidade de um desenvolvedor violar o contrato. Sem algum mecanismo de imposição, alguém poderia ignorar o framework e reescrever tudo por conta própria. Um padrão perde valor se for rotineiramente desrespeitado. Em Java e nas linguagens .NET, duas formas importantes de implementar contratos são classes abstratas e interfaces.

CLASSES ABSTRATAS

Uma forma de implementar um contrato é por meio de uma classe abstrata. Uma classe abstrata contém um ou mais métodos para os quais não existe implementação concreta. Considere `Shape`. Não podemos instanciá-la diretamente: “forma” é um conceito abstrato. Se alguém disser “desenhe uma forma”, a pergunta natural é “qual forma?”. Já um círculo é concreto: sabemos como ele deve ser desenhado.

Como isso se relaciona a contratos? Suponha que construamos uma aplicação para desenhar todas as formas existentes e também as que forem acrescentadas no futuro. Queremos duas condições.

Primeiro, todas as formas devem usar a mesma sintaxe para se desenhar. Cada forma deverá oferecer um método `draw()`. Assim, um desenvolvedor experiente sabe que, independentemente da forma concreta, a operação para desenhá-la é `draw()`. Isso reduz consulta a documentação e erros de sintaxe.

Segundo, cada classe deve ser responsável por seu próprio comportamento. Embora todas sejam obrigadas a fornecer `draw()`, cada uma implementa seu próprio código. `Circle` sabe desenhar um círculo; `Rectangle` sabe desenhar um retângulo. Ao criar essas subclasses de `Shape`, cada uma precisa implementar sua versão de `draw()`, conforme a [Figura 8.3](#).

Figura 8.3 - Hierarquia com uma classe abstrata.

MÉTODOS ABSTRATOS

Nos diagramas UML, métodos abstratos aparecem em itálico.

Com esse desenho, temos um framework Shape realmente polimórfico. O mesmo método `draw()` pode ser invocado em qualquer forma, mas cada objeto produz um resultado diferente. `circle.draw()` desenha um círculo; `rectangle.draw()` desenha um retângulo. Enviar a mesma mensagem a objetos diferentes provoca respostas adequadas a cada tipo: essa é a essência do polimorfismo.

A classe Shape em Java pode ser definida como abstrata e declarar:

```
public abstract void draw();
```

Não existe implementação para `draw()`. Isso ocorre por dois motivos. Primeiro, Shape não sabe qual forma concreta deverá desenhar, portanto não possui informação suficiente para implementar o comportamento.

ANALOGIA ESTRUTURADA

Se tentássemos colocar em Shape o código para todas as formas presentes e futuras, precisaríamos de grandes estruturas condicionais, como `switch`. O código ficaria confuso e difícil de manter. Este é um ponto em que o projeto OO mostra sua força.

Segundo, queremos que as subclasses forneçam a implementação. Circle e Rectangle estendem Shape e implementam `draw()` de maneiras diferentes. A implementação do livro é simples, apenas imprimindo “Draw a Circle” ou “Draw a Rectangle”, mas o princípio é o mesmo.

É aqui que o contrato se torna efetivo. Se Circle herdar de Shape e não implementar `draw()`, não poderá ser usada como uma classe concreta válida. O compilador força o cumprimento da obrigação. Assim, um gerente pode exigir que toda nova forma derive de Shape; como consequência, todas terão um método `draw()` com a interface esperada.

Se Circle não implementar `draw()`, ela própria continuará abstrata. Será necessário criar uma subclasse concreta que implemente o método antes de poder instanciar objetos.

In the UML diagrams, note that the abstract methods are italicized.

In this way, we have a Shape framework that is truly polymorphic. The `draw()` method can be invoked for every single shape in the system, and invoking each shape produces a different result. Invoking the `draw()` method on a Circle object draws a circle, and invoking the `draw()` method on a Rectangle object draws a rectangle. In essence, sending a message to an object evokes a different response, depending on the object. This is the essence of polymorphism.

```
circle.draw();           // draws a circle
rectangle.draw();        // draws a rectangle
```

Let’s look at some code to illustrate how Rectangle and Circle conform to the Shapecontract.

Here is the code for the Shape class:

```
public abstract class Shape {

    public abstract void draw(); // no implementation

}
```

Note that the class does not provide any implementation for `draw()`; basically there is no code, and this is what makes the method abstract (providing any code would make the method concrete). There are two reasons why there is no implementation. First, Shape does not know what to draw, so we could not implement the `draw()` method even if we wanted to.

Structured Analogy

This is an interesting issue. If we did want the Shape class to contain the code for all possible shapes, present and future, conditional statements, such as a Java `switch` statement, would be required. This would be very messy and difficult to maintain. This is one example of where the strength of an object-oriented design comes into play.

Second, we want the subclasses to provide the implementation. Let’s look at the Circle and Rectangle classes:

Click here to view code image

```
public class Circle extends Shape {

    public void Draw() {System.out.println ("Draw a Circle");}

}

public class Rectangle extends Shape {

    public void Draw() {System.out.println ("Draw a Rectangle");}

}
```

Note that both Circle and Rectangle extend (that is, inherit from) Shape. Also notice that they provide the actual implementation (in this case, the implementation is trivial). Here is where the contract comes in. If Circle inherits from Shape and fails to provide a `draw()` method, Circle won’t even compile. Thus, Circle would fail to satisfy the contract with Shape. A project manager can require that programmers creating shapes for the application must inherit from Shape. By doing this, all shapes in the application will have a `draw()` method that performs in an expected manner.

Circle

If Circle does indeed fail to implement a `draw()` method, Circle will be considered abstract itself. Thus, yet another subclass must inherit from Circleand implement a `draw()` method. This subclass would then become the concrete implementation of both Shape and Circle.

Although the concept of abstract classes revolves around abstract methods, nothing is stopping `Shape` from providing some implementation. Remember that the definition for an abstract class is that it contains *one or more* abstract methods—this implies that an abstract class can also provide concrete methods. For example, although `Circle` and `Rectangle` implement the `draw()` method differently, they share the same mechanism for setting the color of the shape. So, the `Shape` class can have a color attribute and a method to set the color. This `setColor()` method is a concrete implementation and would be inherited by both `Circle` and `Rectangle`. The only methods that a subclass must implement are the ones that the superclass declares as abstract. These abstract methods are the contract.

Caution

Be aware that in the cases of `Shape`, `Circle`, and `Rectangle`, we are dealing with a strict inheritance relationship, as opposed to an interface, which we discuss in the next section. `Circle` *is-a* `Shape`, and `Rectangle` *is-a* `Shape`.

Some languages, such as C++, use only abstract classes to implement contracts; however, Java and .NET have another mechanism that implements a contract called an interface. In other cases, such as Objective-C and Swift, abstract classes are not provided by the language. Thus, to implement a contract in Objective-C or Swift, you need to use protocols.

Interfaces

Before defining an interface, it is interesting to note that C++ does not have a construct called an interface. When using C++, you can essentially create an interface by using a syntax subset of an abstract class. For example, the following C++ code is an abstract class. However, because the only method in the class is a virtual method, there is no implementation. As a result, this abstract class provides the same functionality as an interface.

```
class Shape
{
    public:
        virtual void draw() = 0;
}
```

Interface Terminology

This is another one of those times when software terminology gets confusing—very confusing. Be aware that you can use the term interface in several ways, so be sure to use each in the proper context.

First, the graphical user interface (GUI) is widely used when referring to the visual interface that a user interacts with—often on a monitor.

Second, the interface to a class is basically the signatures of its methods.

Third, in Objective-C and Swift, you break up the code into physically separate modules called the interface and implementation.

Fourth, an interface and a protocol are basically a contract between a parent class and a child class.Can you think of any others?

The obvious question is this: If an abstract class can provide the same functionality as an interface, why do Java and .NET bother to provide this construct called an interface? And why does Objective-C and Swift provide the protocol?

For one thing, C++ supports multiple inheritance, whereas Java, Objective-C, Swift, and .NET do not. Although Java, Objective-C, Swift, and .NET classes can inherit from only one parent

Embora classes abstratas sejam definidas pela presença de métodos abstratos, elas também podem fornecer implementações concretas. Por exemplo, `Circle` e `Rectangle` implementam `draw()` de maneiras diferentes, mas podem compartilhar o mesmo mecanismo para definir cor.

`Shape` pode então possuir um atributo `color` e um método concreto `setColor()`. Esse método é herdado pelas duas subclasses. A subclasse precisa implementar apenas os métodos que a superclasse declarou como abstratos. Esses métodos abstratos formam o contrato.

CUIDADO

Nos exemplos `Shape/Circle/Rectangle` existe uma relação rigorosa de herança: `Circle` é-um `Shape` e `Rectangle` é-um `Shape`. Isso é diferente de uma interface, assunto da próxima seção.

Algumas linguagens, como C++, usam classes abstratas para representar contratos. Java e .NET fornecem também uma construção específica chamada interface. Em Objective-C e Swift, onde classes abstratas não são oferecidas da mesma maneira, protocolos cumprem esse papel.

INTERFACES

C++ não possui uma construção chamada literalmente interface, mas é possível obter funcionalidade semelhante por meio de uma classe abstrata composta apenas por métodos virtuais puros. Se não há implementação concreta, essa classe funciona, na prática, como um contrato de interface.

TERMINOLOGIA DE “INTERFACE”

- A palavra interface aparece em vários contextos e pode ser confusa:
- GUI é a interface gráfica com a qual o usuário interage.
 - A interface de uma classe corresponde às assinaturas dos métodos expostos.
 - Em Objective-C/Swift, interface e implementation também podem designar módulos físicos de código.
 - Uma interface/protocolo pode representar um contrato comportamental entre componentes.

Se uma classe abstrata pode fornecer um contrato, por que Java e .NET têm interfaces e por que Objective-C/Swift têm protocolos? Uma razão importante está ligada à herança múltipla.

C++ suporta herança múltipla; Java, Objective-C, Swift e .NET não permitem que uma classe herde diretamente de várias classes. Entretanto, uma classe pode implementar várias interfaces.

class, they can implement many interfaces. Using more than one abstract class constitutes multiple inheritance; thus, Java and .NET cannot go this route. In short, when using an interface, you do not have to concern yourself with a formal inheritance structure—you can theoretically add an interface to any class if the design makes sense. However, an abstract class requires you to inherit from that abstract class and, by extension, all of its potential parents.

Circle

Because of these considerations, interfaces are often thought to be a workaround for the lack of multiple inheritance. This is not technically true. Interfaces are a separate design technique, and although they can be used to design applications that could be done with multiple inheri-tance, they do not replace or circumvent multiple inheritance.

As with abstract classes, interfaces are a powerful way to enforce contracts for a framework. Before we get into any conceptual definitions, it’s helpful to see an actual interface UML diagram and the corresponding code. Consider an interface called `Nameable`, as shown in [Figure 8.4](#).

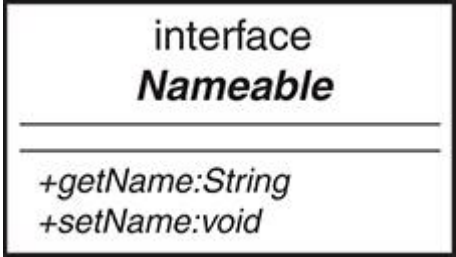


Figure 8.4 A UML diagram of a Java interface.

Note that `Nameable` is identified in the UML diagram as an interface, which distinguishes it from a regular class (abstract or not). Also note that the interface contains two methods, `getName()` and `setName()`. Here is the corresponding code:

```
public interface Nameable {

    String getName();
    void setName (String aName);

}
```

For comparison purposes, here is the code for the corresponding Objective-C protocol:

```
@protocol Nameable

@required
- (char *) getName;
- (void) setName: (char *) n;
@end // Nameable
```

In the code, notice that `Nameable` is not declared as a class but as an interface. Because of this, both methods, `getName()` and `setName()`, are considered abstract and no implementation is provided. An interface, unlike an abstract class, can provide *noimplementation* at all. As a result, any class that implements an interface must provide the implementation for all methods. For example, in Java, a class inherits from an abstract class, whereas a class implements an interface.

Implementation Versus Definition Inheritance

Sometimes inheritance is referred to as implementation inheritance, and interfaces are called *definition inheritance*.

Usar mais de uma classe abstrata como pais equivaleria a herança múltipla, o que Java e .NET evitam. Uma interface, por outro lado, não exige que a classe faça parte de uma hierarquia específica; em princípio, qualquer classe pode implementar uma interface se isso fizer sentido no projeto. Uma classe abstrata exige uma relação formal de herança e traz consigo sua cadeia de ancestrais.

Interfaces às vezes são descritas como uma solução para a ausência de herança múltipla, mas isso é uma simplificação. Elas constituem uma técnica de projeto própria. Podem resolver alguns problemas que também seriam modelados com herança múltipla, mas não a substituem tecnicamente.

Assim como classes abstratas, interfaces são poderosas para impor contratos em um framework. Considere a interface `Nameable` da Figura 8.4.

Figura 8.4 - Diagrama UML de uma interface Java.

`Nameable` é explicitamente marcada como interface e declara dois métodos: `getName()` e `setName()`. O código Java correspondente é:

```
public interface Nameable {
    String getName();
    void setName(String aName);
}
```

O protocolo equivalente em Objective-C declara as mesmas obrigações, sem implementação.

Como `Nameable` é interface, seus métodos representam requisitos de comportamento. Uma classe que implementa `Nameable` deve fornecer a implementação de todos eles. Em Java, dizemos que uma classe “extends” uma classe abstrata, mas “implements” uma interface.

HERANÇA DE IMPLEMENTAÇÃO VERSUS HERANÇA DE DEFINIÇÃO

Às vezes, herança entre classes é chamada de herança de implementação, enquanto o uso de interfaces é chamado de herança de definição.

Tying It All Together

If both abstract classes and interfaces provide abstract methods, what is the real difference between the two? As we saw before, an abstract class can provides both abstract and concrete methods, whereas an interface provides only abstract methods. Why is there such a difference?

Assume that we want to design a class that represents a dog, with the intent of adding more mammals later. The logical move would be to create an abstract class called `Mammal`:

[Click here to view code image](#)

```
public abstract class Mammal {

    public void generateHeat() {System.out.println("Generate heat");}

    public abstract void makeNoise();

}
```

This class has a concrete method called `generateHeat()` and an abstract method called `makeNoise()`. The method `generateHeat()` is concrete because all mammals generate heat. The method `makeNoise()` is abstract because each mammal will make noise differently. Let’s also create a class called `Head` that we will use in a composition relationship:

[Click here to view code image](#)

```
public class Head {

    String size;

    public String getSize() {

        return size;

    }

    public void setSize(String aSize) { size = aSize; }

}
```

`Head` has two methods: `getSize()` and `setSize()`. Although composition might not shed much light on the difference between abstract classes and interfaces, using composition in this example does illustrate how composition relates to abstract classes and interfaces in the overall design of an object-oriented system. I feel that this is important because the example is more complete. Remember that there are two ways to build object relationships: the *is-arelationship*, represented by inheritance, and the *has-a* relationship, represented by composition. The question is: Where does the interface fit in?

To answer this question and tie everything together, let’s create a class called `Dog` that is a subclass of `Mammal`, implements `Nameable`, and has a `Head` object (see [Figure 8.5](#)).

JUNTANDO TUDO

Se classes abstratas e interfaces podem declarar métodos abstratos, qual é a diferença prática? Uma classe abstrata pode combinar métodos abstratos e concretos; uma interface, no modelo apresentado aqui, define apenas o contrato de comportamento.

Suponha que desejemos modelar um cão e, futuramente, outros mamíferos. Uma escolha lógica é criar a classe abstrata `Mammal`:

```
public abstract class Mammal {
    public void generateHeat() { ... }
    public abstract void makeNoise();
}
```

`generateHeat()` é concreto porque todos os mamíferos geram calor. `makeNoise()` é abstrato porque cada mamífero produz sons de maneira diferente.

Também criamos uma classe `Head` para participar de uma relação de composição. Ela possui um atributo `size` e métodos `getSize()` e `setSize()`.

A composição por si só não explica toda a diferença entre classe abstrata e interface, mas torna o exemplo mais completo. Recorde as duas relações clássicas: herança representa “é-um”; composição representa “tem-um”. Então onde a interface se encaixa?

Para integrar os conceitos, criamos `Dog` como uma subclasse de `Mammal`, implementando `Nameable` e contendo um objeto `Head`, conforme a Figura 8.5.

Figura 8.5 - Diagrama UML do código de exemplo.

Em resumo, o exemplo combina três mecanismos de construção de relações em Java/.NET: herança, interfaces e composição. A linha tracejada do diagrama representa a interface.

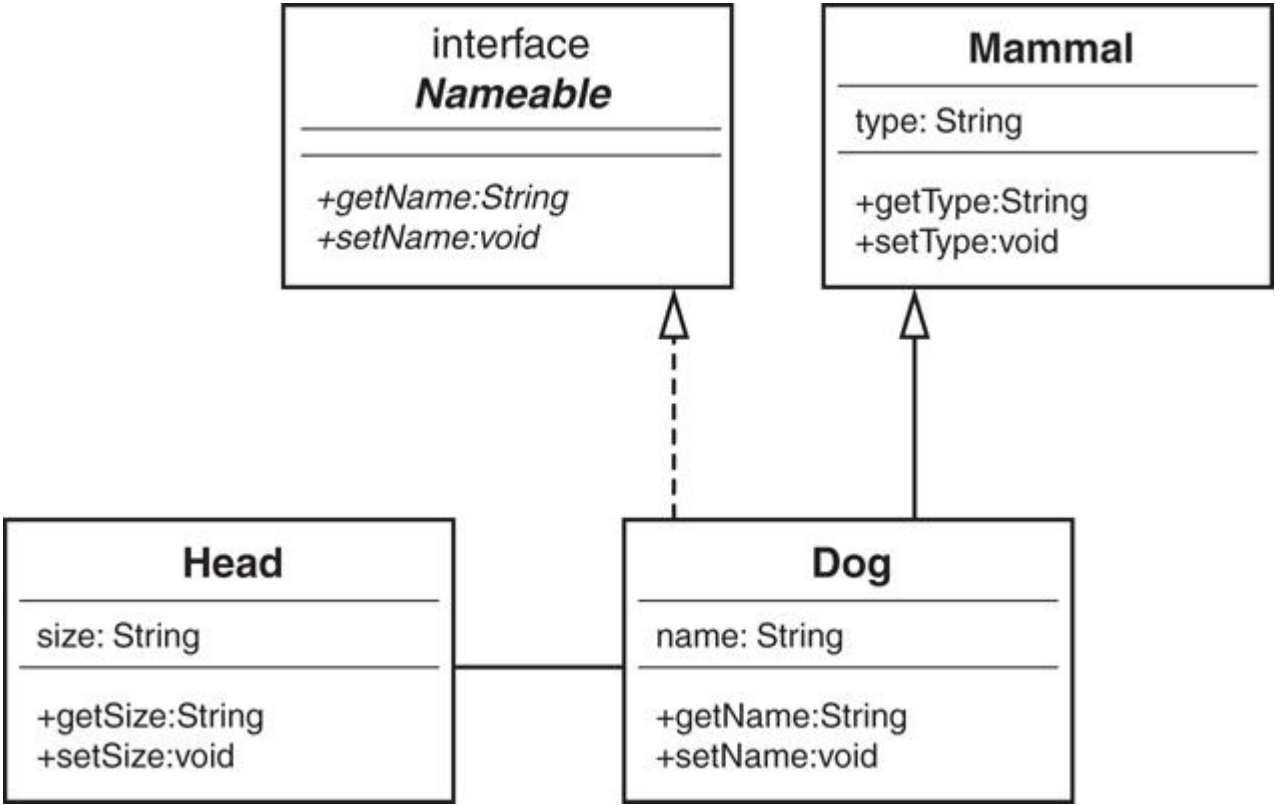
As relações são:
- Dog é-um Mammal -> herança.
- Dog implementa Nameable -> contrato/interface.
- Dog tem-um Head -> composição.

O código correspondente declara:

```
public class Dog extends Mammal implements Nameable { ... }
```

Dog mantém um campo name e um campo Head; implementa makeNoise() com o comportamento apropriado e também setName()/getName() exigidos por Nameable.

Depois de observar o diagrama, surge uma pergunta natural: se a linha entre Dog e Nameable representa uma interface, isso ainda não seria uma forma de herança? Em certo sentido, interfaces podem ser vistas como um tipo especial de relação de substituição, mas é essencial compreender o que “especial” significa. Essa diferença é importante para um bom projeto OO.



8.5 A UML diagram of the sample code.

Figure

In a nutshell, Java and .NET build objects in three ways: inheritance, interfaces, and composition. Note the dashed line in [Figure 8.5](#) that represents the interface. This example illustrates when you should use each of these constructs. When do you choose an abstract class? When do you choose an interface? When do you choose composition? Let’s explore further.

You should be familiar with the following concepts:

- Dog is a Mammal, so the relationship is inheritance.
- Dog implements Nameable, so the relationship is an interface.
- Dog has a Head, so the relationship is composition.

The following code shows how you would incorporate an abstract class and an interface in the same class:

[Click here to view code image](#)

```
public class Dog extends Mammal implements Nameable {

    String name;

    Head head;

    public void makeNoise() {System.out.println("Bark");}

    public void setName (String aName) {name = aName;}
    public String getName () {return (name);}

}
```

After looking at the UML diagram, you might come up with an obvious question: Even though the dashed line from Dog to Nameable represents an interface, isn’t it still inheritance? At first glance, the answer is not simple. Although interfaces may well be considered a special type of

inheritance, it is important to know what *special* means. Understanding these *special* differences is key to understanding a solid object-oriented design.

Although inheritance is a strict is-a relationship, an interface is not quite. For example:

- A dog is a mammal.
- A reptile is not a mammal.

Thus, a `Reptile` class could not inherit from the `Mammal` class. However, an interface transcends the various classes. For example:

- A dog is nameable.
- A lizard is nameable.

The key here is that classes in a strict inheritance relationship must be related. For example, in this design, the `Dog` class is directly related to the `Mammal` class. A dog is a mammal. Dogs and lizards are not related at the mammal level because you can't say that a lizard is a mammal. However, interfaces can be used for classes that are not related. You can name a dog just as well as you can name a lizard. This is the key difference between using an abstract class and using an interface.

The abstract class represents some sort of implementation. In fact, we saw that `Mammal` provided a concrete method called `generateHeat()`. Even though we do not know what kind of mammal we have, we know that all mammals generate heat. However, an interface models only behavior. An interface *never* provides any type of implementation, only behavior. The interface specifies behavior that is the same across classes that conceivably have no connection. Not only are dogs nameable, but so are cars, planets, and so on.

Some say that interfaces are a poor substitute for multiple inheritance. While it may be true that interfaces were part of the same Java design that eliminated multiple inheritance (and were adopted by many other languages), interfaces are used in different design situations than inheritance, as the `Nameable` example illustrates.

The Compiler Proof

Can we prove or disprove that interfaces have a true is-a relationship? In the case of Java (and this can also be done in C# or VB), we can let the compiler tell us. Consider the following code:

```
Dog D = new Dog();
Head H = D;
```

When this code is run through the compiler, the following error is produced:

Click here to view code image

```
Test.java:6: Incompatible type for Identifier. Can't convert Dog to Head. Head H = D;
```

Obviously, a dog is not a head. Not only do we know this, but the compiler agrees. However, as expected, the following code works just fine:

```
Dog D = new Dog();
Mammal M = D;
```

This is a true inheritance relationship, and it is not surprising that the compiler parses this code cleanly because a dog is a mammal.

Now we can perform the true test of the interface. Is an interface an actual is-a relationship? The compiler thinks so:

```
Dog D = new Dog();
```

A herança entre classes expressa uma relação “é-um” estrita. Por exemplo:
- um cão é um mamífero;
- um réptil não é um mamífero.

Logo, Reptile não deveria herdar de Mammal.

Interfaces atravessam hierarquias não relacionadas. Um cão pode ser “nomeável”; um lagarto também. Cães e lagartos não são parentes no nível Mammal, mas ambos podem oferecer o mesmo comportamento de atribuição de nome.

Essa é uma diferença central. A classe abstrata pode carregar implementação compartilhada. Mammal, por exemplo, fornece generateHeat(), comportamento comum a todos os mamíferos. Uma interface, por sua vez, modela um conjunto de comportamentos esperado de classes que podem não ter qualquer parentesco entre si. Cães, carros e planetas podem todos ser Nameable.

Por isso, interfaces não devem ser vistas apenas como substituto pobre para herança múltipla. Elas atendem a situações de projeto distintas.

A PROVA DO COMPILADOR

Podemos usar o compilador para testar as relações. O código:

```
Dog D = new Dog();
Head H = D;
```

falha porque Dog não é Head. Já:

```
Dog D = new Dog();
Mammal M = D;
```

é válido porque Dog é Mammal.

Agora podemos testar a interface. Se Dog implementa Nameable, uma referência Dog pode ser atribuída a uma variável Nameable. O compilador aceita essa relação, demonstrando que o objeto pode ser tratado de acordo com o contrato Nameable.

```
Nameable N = D;
```

This code works fine. So, we can safely say that a dog is a nameable entity. This is a simple but effective proof that both inheritance and interfaces constitute an is-a relationship. The interface relationship is more like a “behaves-like-a” when used properly. You might have data interfaces that are “is-a,” but more often you are going to have the former.

Nameable Interface

An interface specifies certain behavior but not the implementation. By implementing the `Nameable` interface, you are saying that you will provide nameable behavior by implementing methods called `getName()` and `setName()`. How you implement these methods is up to you. All you have to do is to provide the methods.

Making a Contract

The simple rule for defining a contract is to provide an unimplemented method, via either an abstract class or an interface. Thus, when a subclass is designed with the intent of complying with the contract, it must provide the implementation for the unimplemented methods in the parent class or interface.

As stated earlier, one of the advantages of a contract is to standardize coding conventions. Let’s explore this concept in greater detail by providing an example of what happens when coding standards are not used. In this case, there are three classes: `Planet`, `Car`, and `Dog`. Each class implements code to name the entity. However, because they are all implemented separately, each class has different syntax to retrieve the name. Consider the following code for the `Planet` class:

Click here to view code image

```
public class Planet {
    String planetName;
    public void getPlanetName() {return planetName;};
}
```

Likewise, the `Car` class might have code like this:

Click here to view code image

```
public class Car {

    String carName;

    public String getCarName() { return carName; };

}
```

And the `Dog` class might have code like this:

```
public class Dog {

    String dogName;

    public String getDogName() { return dogName; };

}
```

The obvious issue here is that anyone using these classes would have to look at the documentation (what a horrible thought!) to figure out how to retrieve the name in each of these cases. Even though looking at the documentation is not the worst fate in the world, it would be nice if all the classes used in a project (or company) would use the same naming convention—it would make life a bit easier. This is where the `Nameable` interface comes in.

O código:

```
Dog D = new Dog();
Nameable N = D;
```

funciona. Assim, podemos dizer que `Dog` é uma entidade `Nameable`. Tanto herança quanto interfaces permitem substituição de tipos; no caso da interface, uma descrição mais intuitiva costuma ser “comporta-se como”. Interfaces de dados podem se aproximar de um verdadeiro “é-um”, mas interfaces comportamentais são muito comuns.

INTERFACE NAMEABLE

Uma interface especifica comportamento, não a implementação. Ao implementar `Nameable`, a classe se compromete a fornecer os métodos `getName()` e `setName()`. Como esses métodos funcionarão internamente é decisão da classe.

CRIANDO UM CONTRATO

Uma regra simples para formar um contrato é declarar um método sem implementação em uma classe abstrata ou interface. Uma subclasse ou classe implementadora que pretenda cumprir o contrato deverá fornecer a implementação necessária.

Um benefício é padronizar convenções. Imagine três classes - `Planet`, `Car` e `Dog` - que precisam possuir nomes. Se cada equipe implementar isso de maneira independente, podemos acabar com `getPlanetName()`, `getCarName()` e `getDogName()`. Quem utiliza essas classes precisa consultar a documentação para descobrir a sintaxe específica de cada uma.

Consultar documentação não é um problema por si só, mas seria mais simples se todas as classes de um projeto ou organização seguissem uma convenção comum. É exatamente esse o papel que `Nameable` pode desempenhar.

The idea would be to make a contract for any type of class that needs to use a name. As users of various classes move from one class to the other, they would not have to figure out the current syntax for naming an object. The `Planet` class, the `Car` class, and the `Dog` class would all have the same naming syntax.

To implement this lofty goal, we can create an interface (we can use the `Nameable` interface that we used previously). The convention is that all classes must implement `Nameable`. In this way, the users have to remember only a single interface for all classes when it comes to naming conventions:

[Click here to view code image](#)

```
public interface Nameable {

    public String getName();
    public void setName(String aName);

}

The new classes, Planet, Car, and Dog, should look like this:
public class Planet implements Nameable {

    String planetName;

    public String getName() {return planetName;}
    public void setName(String myName) { planetName = myName; }

}

public class Car implements Nameable {

    String carName;

    public String getName() {return carName;}
    public void setName(String myName) { carName = myName;}

}

public class Dog implements Nameable {

    String dogName;

    public String getName() {return dogName;}
    public void setName(String myName) { dogName = myName;}

}
```

In this way, we have a standard interface, and we’ve used a contract to ensure that it is the case. In fact, one of the major benefits of using a modern IDE is that, when implementing an interface, the IDE will automatically stub out the required methods. This feature saves lots of time and effort when using interfaces.

There is one little issue that you might have thought about. The idea of a contract is great as long as everyone plays by the rules, but what if some shady individual doesn’t want to play by the rules (the rogue programmer)? The bottom line is that there is nothing to stop people from breaking the standard contract; however, in some cases, doing so will get them in deep trouble.

On one level, a project manager can insist that everyone use the contract, just like team members must use the same variable naming conventions and configuration management system. If a team member fails to abide by the rules, he could be reprimanded, or even fired.

A proposta é estabelecer um contrato para qualquer classe que possua um nome. Assim, ao passar de uma classe para outra, o programador não precisa reaprender a sintaxe. `Planet`, `Car` e `Dog` usam a mesma convenção.

A interface `Nameable` declara:

```
public interface Nameable {
    public String getName();
    public void setName(String aName);
}
```

`Planet`, `Car` e `Dog` passam então a declarar “implements `Nameable`” e cada uma implementa `getName()` e `setName()` usando seu próprio campo interno. O contrato é comum; a implementação continua encapsulada em cada classe.

Isso produz uma interface padronizada e garante, via compilador, que as classes que afirmam implementar `Nameable` forneçam os métodos necessários. IDEs modernas ainda ajudam criando automaticamente os stubs dos métodos exigidos, economizando trabalho repetitivo.

Existe, entretanto, uma questão organizacional: o contrato só funciona se as regras forem seguidas. Um desenvolvedor pode simplesmente decidir não implementar a interface em uma classe que deveria adotá-la. Em alguns projetos isso será detectado por revisão, testes, arquitetura ou processos de equipe; em outros, pode resultar em problemas sérios.

Em um nível administrativo, o gerente pode exigir o uso do contrato da mesma forma que exige convenções de nomes ou um sistema comum de controle de configuração.

Enforcing rules is one way to ensure that contracts are followed, but there are instances in which breaking a contract will result in unusable code. Consider the Java interface `Runnable`. Old-style Java applets implement the `Runnable` interface because it requires that any class implementing `Runnable` must implement a `run()` method. This is important because the browser that calls the applet will call the `run()` method within `Runnable`. If the `run()` method does not exist, things will break.

System Plug-in Points

Basically, contracts are “plug-in points” into your code. Anyplace where you want to make parts of a system abstract, you can use a contract. Instead of coupling to objects of specific classes, you can connect to any object that implements the contract. You need to be aware of where contracts are useful; however, you can overuse them. You want to identify common features such as the `Nameable` interface, as discussed in this chapter. However, be aware that there is a trade-off when using contracts. They might make code reuse more of a reality, but they make things somewhat more complex.

AN E-BUSINESS EXAMPLE

It’s sometimes hard to convince a decision maker, who may have no development background, of the monetary savings of code reuse. However, when reusing code, it is pretty easy to understand the advantage to the bottom line. In this section, we’ll walk through a simple but practical example of how to create a workable framework using inheritance, abstract classes, interfaces, and composition.

An E-Business Problem

Perhaps the best way to understand the power of reuse is to present an example of how you would reuse code. In this example, we’ll use inheritance (via interfaces and abstract classes) and composition. Our goal is to create a framework that will make code reuse a reality, reduce coding time, and reduce maintenance—all the typical software development wish-list items.

Let’s start our own Internet business. Let’s assume that we have a client, a small pizza shop called Papa’s Pizza. Despite the fact that it is a small, family-owned business, Papa realizes that a Web presence can help the business in many ways. Papa wants his customers to access his website, find out what Papa’s Pizza is all about, and order pizzas right from the comfort of their browsers.

At the site we develop, customers will be able to access the website, select the products they want to order, and select a delivery option and time for delivery. They can eat their food at the restaurant, pick up the order, or have the order delivered. For example, a customer decides at 3:00 that he wants to order a pizza dinner (with salads, breadsticks, and drinks), to be delivered to his home at 6:00. Let’s say the customer is at work (on a break, of course). He gets on the Web and selects the pizzas, including size, toppings, and crust; the salads, including dressings; breadsticks; and drinks. He chooses the delivery option and requests that the food be delivered to his home at 6:00. Then he pays for the order by credit card, gets a confirmation number, and exits. Within a few minutes he gets an email confirmation as well. We will set up accounts so that when people bring up the site, they will get a greeting reminding them of who they are, what their favorite pizza is, and what new pizzas have been created this week.

When the software system is finally delivered, it is deemed a total success. For the next several weeks, Papa’s customers happily order pizzas and other food and drinks over the Internet. During this rollout period, Papa’s brother-in-law, who owns a donut shop called Dad’s Donuts,

Regras organizacionais são uma forma de garantir contratos, mas há casos em que quebrá-los torna o código efetivamente inutilizável.

Considere a interface Java `Runnable`. Em modelos antigos de applets e em outros contextos, componentes que dependem de `Runnable` esperam um método `run()`. Se a classe não fornecer esse ponto de entrada conforme o contrato, o mecanismo que tenta chamá-lo não funcionará.

PONTOS DE PLUG-IN DO SISTEMA

Contratos funcionam como “pontos de plug-in” no código. Onde desejamos tornar uma parte do sistema abstrata, podemos depender do contrato em vez de uma classe concreta. Assim, qualquer objeto que implemente o contrato pode ser conectado naquele ponto.

É preciso identificar onde isso é útil sem exagerar. Interfaces comuns como `Nameable` aumentam a possibilidade de reuso e desacoplamento, mas também adicionam camadas e complexidade. Há sempre uma troca.

UM EXEMPLO DE E-BUSINESS

Às vezes é difícil demonstrar a um tomador de decisão sem formação técnica o valor financeiro do reuso. Um exemplo concreto facilita enxergar a economia.

Nesta seção, o livro constrói um framework simples usando herança, classes abstratas, interfaces e composição, com o objetivo de reduzir tempo de codificação e manutenção.

O cenário começa com uma pequena pizzeria familiar, Papa’s Pizza, que deseja criar presença na web. Clientes poderão conhecer os produtos, montar pedidos, escolher consumo no local, retirada ou entrega, definir horário e pagar com cartão. O sistema pode manter contas, preferências e enviar confirmações.

O projeto é entregue com sucesso e, após algumas semanas, surge uma nova oportunidade: o cunhado de Papa possui uma loja de donuts chamada Dad’s Donuts...

pays Papa a visit. Papa shows Dad the system, and Dad falls in love with it. The next day, Dad calls our company and asks us to develop a Web-based system for his donut shop. This is great, and exactly what we had hoped for. Now, how can we leverage the code that we used for the pizza shop in the system for the donut shop?

How many more small businesses, besides Papa’s Pizza and Dad’s Donuts, could take advantage of our framework to get on the Web? If we can develop a good, solid framework, we will be able to efficiently deliver Web-based systems at lower costs than we were able to do before. There will also be an added advantage that the code will have been tested and implemented previously, so debugging and maintenance should be greatly reduced.

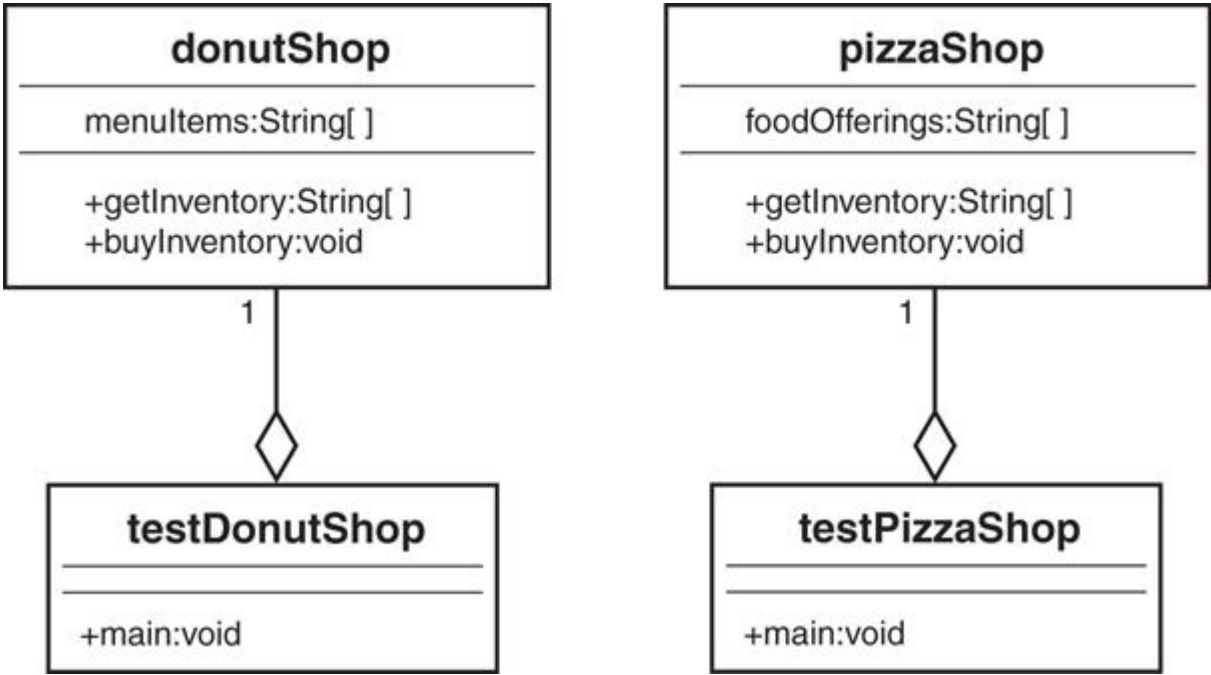
The Non-Reuse Approach

For many reasons, the concept of code reuse has not been as successful as some software developers would like. First, many times reuse is not even considered when developing a system. Second, even when reuse is entered into the equation, the issues of schedule constraints, limited resources, and budgetary concerns often short-circuit the best intentions.

In many instances, code ends up highly coupled to the specific application for which it was written. This means that the code within the application is highly dependent on other code within the same application.

A lot of code reuse is the result of using cut, copy, and paste operations. While one application is open in a text editor, you copy code and then paste it into another application. Sometimes certain functions or routines can be used without any change. As is unfortunately often the case, even though most of the code may remain identical, a small bit of code must change to work in a specific application.

For example, consider two separate applications, as represented by the UML diagram in [Figure 8.6](#).



8.6 Applications on divergent paths.

Figure

...faz uma visita a Papa, vê o sistema e se encanta com a solução. No dia seguinte, liga para nossa empresa e pede um sistema web semelhante para sua loja de donuts. Essa é exatamente a oportunidade esperada: como aproveitar o código criado para a pizzeria no novo sistema?

Quantos outros pequenos negócios poderiam utilizar o mesmo framework? Se construirmos uma base sólida, poderemos entregar sistemas web mais rapidamente e a menor custo. Além disso, grande parte do código já terá sido implementada e testada, reduzindo depuração e manutenção.

A ABORDAGEM SEM REUSO

O reuso de código nem sempre alcança o sucesso esperado. Muitas vezes ele nem é considerado desde o início; em outras, prazos, recursos limitados e orçamento fazem com que boas intenções sejam abandonadas.

O código acaba fortemente acoplado à aplicação específica. Reuso então vira “copiar e colar”: trechos são copiados de um sistema para outro. Às vezes funcionam sem mudança, mas frequentemente pequenas adaptações específicas começam a separar as versões.

A Figura 8.6 representa duas aplicações independentes, testDonutShop e testPizzaShop, que seguem caminhos distintos.

Figura 8.6 - Aplicações em caminhos divergentes.

In this example, the applications `testDonutShop` and `testPizzaShop` are totally independent code modules. The code is kept separate, and there is no interaction between the modules. However, these applications might use some common code. In fact, some code might have been copied verbatim from one application to another. At some point, someone involved with the project might decide to create a library of these shared pieces of code to use in these and other applications. In many well-run and disciplined projects, this approach works well. Coding standards, configuration management, change management, and so on are all very well run. However, in many instances, this discipline breaks down.

Anyone who is familiar with the software development process knows that when bugs crop up and time is of the essence, there is the temptation to put some fixes or additions into a system that are specific to the application currently in distress. This might fix the problem for the distressed application but could have unintended, possibly harmful, implications for other applications. Thus, in situations like these, the initially shared code can diverge, and separate code bases must be maintained.

For example, one day Papa’s website crashes. He calls us in a panic, and one of our developers is able to track down the problem. The developer fixes the problem, knowing that the fix works but is not quite sure why. The developer also does not know what other areas of the system the fix might inadvertently affect. So the developer makes a copy of the code, strictly for use in the Papa’s Pizza system. This is affectionately named Version 2.01papa. Because the developer does not yet totally understand the problem and because Dad’s system is working fine, the code is not migrated to the donut shop’s system.

Tracking Down a Bug

The fact that the bug turned up in the pizza system does not mean that it will also turn up in the donut system. Even though the bug caused a crash in the pizza shop, the donut shop might never encounter it. It may be that the fix to the pizza shop's code is more dangerous to the donut shop than the original bug.

The next week Dad calls in a panic, with a totally unrelated problem. A developer fixes it, again not knowing how the fix will affect the rest of the system, makes a separate copy of the code, and calls it Version 2.03dad. This scenario gets played out for all the sites we now have in operation. There are now a dozen or more copies of the code, with various versions for the various sites. This becomes a mess. We have multiple code paths and have crossed the point of no return. We can never merge them again. (Perhaps we could, but from a business perspective, this would be costly.)

Our goal is to avoid the mess of the previous example. Although many systems must deal with legacy issues, fortunately for us, the pizza and donut applications are brand-new systems. Thus, we can use a bit of foresight and design this system in a reusable manner. In this way, we will not run into the maintenance nightmare just described. What we want to do is factor out as much commonality as possible. In our design, we will focus on all the common business functions that exist in a Web-based application. Instead of having multiple application classes like `testPizzaShop` and `testDonutShop`, we can create a design that has a class called `Shop` that all the applications will use.

Notice that `testPizzaShop` and `testDonutShop` have similar interfaces, `getInventory()` and `buyInventory()`. We will factor out this commonality and require that all applications that conform to our `Shop` framework implement `getInventory()` and `buyInventory()` methods. This requirement to conform to a standard is sometimes called a contract. By explicitly setting forth a contract of services, you isolate the code from a single implementation. In Java, you can implement a contract by using an interface or an abstract class. Let’s explore how this is accomplished.

No exemplo, `testDonutShop` e `testPizzaShop` são módulos totalmente independentes. Podem conter trechos idênticos copiados de um para o outro e, eventualmente, uma equipe pode criar uma biblioteca com partes compartilhadas. Em projetos disciplinados, com bons padrões, controle de configuração e gestão de mudanças, isso pode funcionar. Mas a disciplina muitas vezes se deteriora sob pressão.

Quando surge um bug urgente, existe a tentação de fazer uma correção específica para a aplicação em crise. A correção pode resolver aquele caso, mas ter efeitos imprevistos em outros sistemas. O código originalmente comum começa a divergir e várias bases precisam ser mantidas.

Imagine que o site da pizzeria caia. Um desenvolvedor corrige o problema sem compreender totalmente suas consequências e cria uma versão exclusiva, “2.01papa”. Como o sistema da loja de donuts está funcionando, a mudança não é propagada. Depois, Dad enfrenta outro problema e surge “2.03dad”. Repetindo isso em vários clientes, logo há uma dúzia de versões incompatíveis. O custo para reunificá-las torna-se proibitivo.

O objetivo é evitar esse pesadelo. Como os sistemas ainda são novos, podemos projetá-los para reuso desde o início. Em vez de aplicações separadas, criamos uma abstração `Shop` para concentrar as funções comuns.

`testPizzaShop` e `testDonutShop` possuem operações semelhantes, `getInventory()` e `buyInventory()`. Podemos estabelecer que toda implementação do framework `Shop` deverá fornecer esses métodos. Essa exigência é o contrato. Em Java, o contrato pode ser implementado com interface ou classe abstrata.

An E-Business Solution

Now let’s show how to use a contract to factor out some of the commonality of these systems. In this case, we will create an abstract class to factor out some of the implementation, and an interface (our familiar `Nameable`) to factor out some behavior.

Our goal is to provide customized versions of our Web application with the following features:

- An interface, called `Nameable`, which is part of the contract.
- An abstract class, called `Shop`, which is also part of the contract.
- A class called `CustList`, which we use in composition.
- A new implementation of `Shop` for each customer we service.

The UML Object Model

The newly created `Shop` class is where the functionality is factored out. Notice in [Figure 8.7](#) that the methods `getInventory()` and `buyInventory()` have been moved up the hierarchy tree from `DonutShop` and `PizzaShop` to the abstract class `Shop`. Now, whenever we want to provide a new, customized version of `Shop`, we plug in a new implementation of `Shop` (such as a grocery shop). `Shop` is the contract that the implementations must abide by.

[Click here to view code image](#)

```
public abstract class Shop {  
  
    CustList customerList;  
  
    public void CalculateSaleTax() {  
  
        System.out.println("Calculate Sales Tax");  
  
    }  
  
    public abstract String[] getInventory();  
  
    public abstract void buyInventory(String item);  
  
}
```

UMA SOLUÇÃO DE E-BUSINESS

Agora o exemplo usa um contrato para extrair a parte comum dos sistemas. Uma classe abstrata concentrará implementação compartilhada e a interface `Nameable` concentrará comportamento padronizado.

O objetivo é produzir versões customizadas da aplicação web com:
- interface `Nameable`, parte do contrato;
- classe abstrata `Shop`, também parte do contrato;
- classe `CustList` usada por composição;
- uma implementação de `Shop` para cada cliente.

O MODELO DE OBJETOS UML

Na Figura 8.7, `Shop` concentra a funcionalidade comum. `getInventory()` e `buyInventory()`, antes repetidos em `DonutShop` e `PizzaShop`, sobem para a classe abstrata `Shop`. Para um novo cliente, como um mercado, basta criar outra implementação que cumpra o contrato.

O código de `Shop` mantém uma `CustList`, possui um método concreto `CalculateSaleTax()` e declara `getInventory()` e `buyInventory()` como abstratos. Assim, parte do comportamento é reutilizada e parte é obrigatoriamente especializada pelas subclasses.

Figura 8.7 - Diagrama UML do modelo Shop.

Para mostrar a composição, Shop contém uma lista de clientes. A classe CustList oferece operações como findCust() e addCust().

Para mostrar o uso de interface, Nameable declara:

```
public interface Nameable {
    public abstract String getName();
    public abstract void setName(String name);
}
```

Podemos ter muitas implementações diferentes enquanto o restante da aplicação permanece igual. Em um exemplo pequeno, a economia de código parece modesta; em aplicações reais, pode ser significativa.

A implementação DonutShop estende Shop e implementa Nameable. Ela mantém companyName, uma lista menuItems e fornece versões concretas de getInventory(), buyInventory(), getName() e setName(). O código completo aparece nesta e na próxima página.

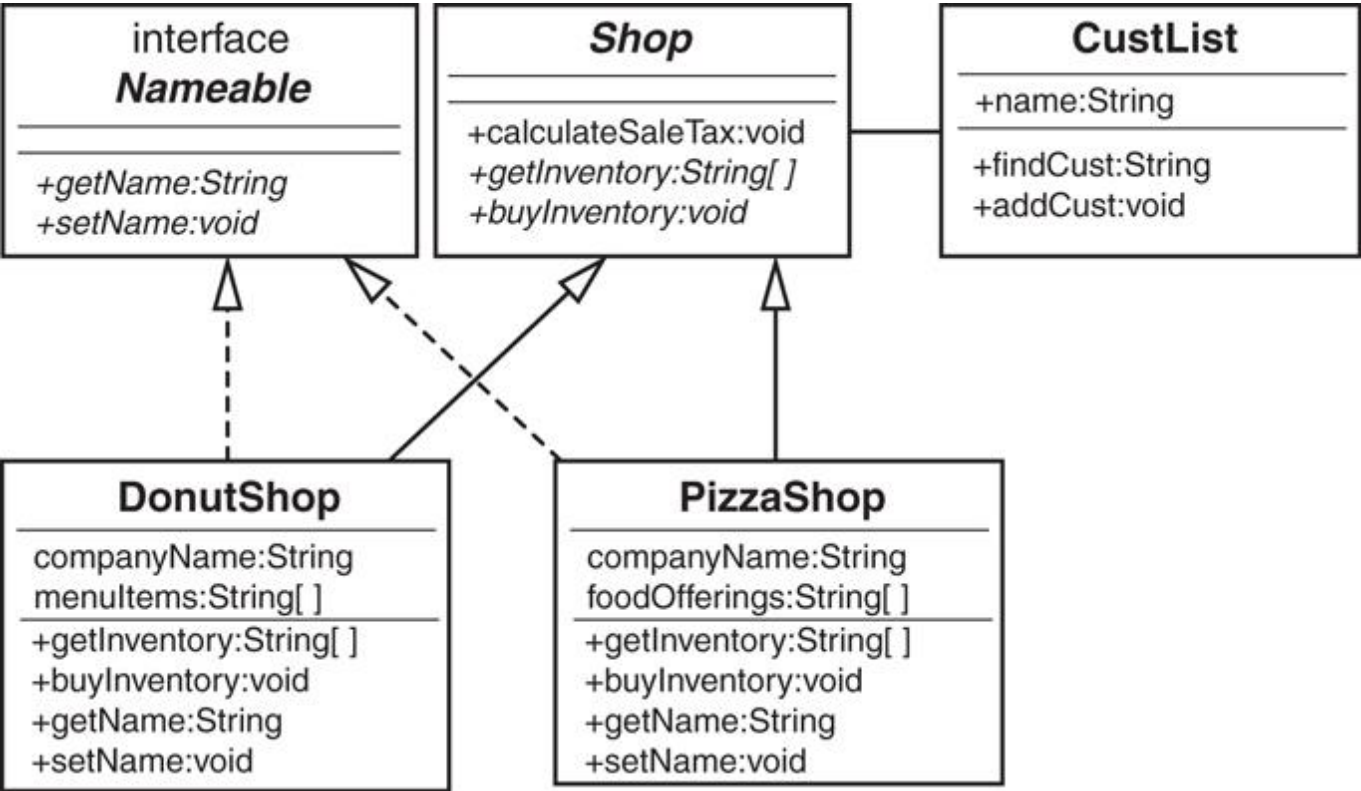


Figure 8.7 A UML diagram of the Shop model.

To show how composition fits into this picture, the `Shop` class has a customer list. Thus, the class `CustList` is contained within `Shop`:

[Click here to view code image](#)

```
public class CustList {

    String name;

    public String findCust() {return name;}
    public void addCust(String Name){}

}
```

To illustrate the use of an interface in this example, an interface called `Nameable` is defined:

[Click here to view code image](#)

```
public interface Nameable {

    public abstract String getName();
    public abstract void setName(String name);

}
```

We could potentially have a large number of different implementations, but all the rest of the code (the application) is the same. In this small example, the code savings might not look like a lot. But in a large, real-world application, the code savings is significant. Let's take a look at the donut shop implementation:

[Click here to view code image](#)

```
public class DonutShop extends Shop implements Nameable {

    String companyName;
```

A classe DonutShop define seu inventário com itens como donuts, muffins, doces, café e chá. `getInventory()` retorna essa lista; `buyInventory()` processa uma compra; `getName()/setName()` atendem ao contrato `Nameable`.

A implementação `PizzaShop` é muito semelhante, mas possui seu próprio conjunto de produtos, como pizza, spaghetti, salada, antipasto e calzone. Ela também implementa os mesmos métodos exigidos por `Shop` e `Nameable`.

O ponto principal não é o conteúdo trivial dos métodos, mas a arquitetura: cada negócio fornece apenas suas particularidades enquanto a aplicação trabalha contra contratos comuns. O código Java da página deve ser preservado conforme o original à esquerda; a tradução explica seu papel no modelo.

```
String[] menuItems = {
    "Donuts",
    "Muffins",
    "Danish",
    "Coffee",
    "Tea"
};

public String[] getInventory() {

    return menuItems;

}

public void buyInventory(String item) {

    System.out.println("\nYou have just purchased " + item);

}

public String getName(){

    return companyName;

}

public void setName(String name){

    companyName = name;

}
}
```

The pizza shop implementation looks very similar:

[Click here to view code image](#)

```
public class PizzaShop extends Shop implements Nameable {

    String companyName;

    String[] foodOfferings = {
        "Pizza",
        "Spaghetti",
        "Garden Salad",
        "Antipasto",
        "Calzone"
    }

    public String[] getInventory() {

        return foodOfferings;

    }

    public void buyInventory(String item) {

        System.out.println("\nYou have just purchased " + item);

    }

    public String getName(){

        return companyName;

    }

    public void setName(String name){
```

A implementação de `PizzaShop` termina com `setName()`, completando os métodos exigidos.

Diferentemente da abordagem inicial, agora temos uma classe principal abstrata, `Shop`, e implementações customizadas como `PizzaShop` e `DonutShop`. A aplicação não precisa estar acoplada a detalhes específicos dessas classes; ela depende do contrato `Shop` e de `Nameable`.

Ainda resta um problema: como escolher dinamicamente qual implementação utilizar? Se escrevemos diretamente:

```
DonutShop myShop = new DonutShop();
```

ou

```
PizzaShop myShop = new PizzaShop();
```

a aplicação continua escolhendo um tipo concreto em tempo de compilação.

Uma alternativa em Java é criar objetos dinamicamente a partir do nome da classe. O exemplo recebe `className` por argumento e usa reflexão:

```
shop = (Shop) Class.forName(className).newInstance();
```

Assim, a mesma aplicação pode decidir em execução qual implementação de `Shop` instanciar. O exemplo `TestShop` começa nesta página: recebe o nome da classe, tenta instanciá-la, obtém o inventário e trabalha apenas com a referência `Shop`.

```
        companyName = name;
    }

}
```

Unlike the initial case, where a large number of customized applications exist, we now have only a single primary class (`Shop`) and various customized classes (`PizzaShop`, `DonutShop`). There is no coupling between the application and any of the customized classes. The only thing the application is coupled to is the contract (`Shop`). The contract specifies that any implementation of `Shop` must provide an implementation for two methods, `getInventory()` and `buyInventory()`. It also must provide an implementation for `getName()` and `setName()` that relates to the interface `Nameable` that is implemented.

Although this solution solves the problem of highly coupled implementations, we still have the problem of deciding which implementation to use. With the current strategy, we would still have to have separate applications. In essence, you must provide one application for each `Shop` implementation. Even though we are using the `Shop` contract, we still have the same situation as before we used the contract:

```
DonutShop myShop= new DonutShop();
```

```
PizzaShop myShop = new PizzaShop ();
```

How do we get around this problem? We can create objects dynamically. In Java, we can use code like this:

Click here to view code image

```
String className = args[0];
```

```
Shop  myShop;
```

```
myShop = (Shop)Class.forName(className).newInstance();
```

In this case, you set `className` by passing a parameter to the code. (There are other ways to set `className`, such as by using a system property.)

Let's look at `Shop` using this approach. (Note that there is no exception handling and nothing else besides object instantiation.)

Click here to view code image

```
class TestShop {
```

```
    public static void main (String args[]) {
```

```
        Shop shop = null;
```

```
        String className = args[0];
```

```
        System.out.println("Instantiate the class:" + className + "\n");
```

```
        try {
```

```
            // new pizzaShop();
            shop = (Shop)Class.forName(className).newInstance();
```

```
        } catch (Exception e) {
```

```
            e.printStackTrace();
        }
```

```
        String[] inventory = shop.getInventory();
```

```
        // list the inventory
```

```
        for (int i=0; i<inventory.length; i++) {
            System.out.println("Argument" + i + " = " + inventory[i]);
        }

        // buy an item

        shop.buyInventory(inventory[1]);

    }
}
```

In this way, we can use the same application code for both `PizzaShop` and `DonutShop`. If we add a `GroceryShop` application, we only have to provide the implementation and the appropriate string to the main application. No application code needs to change.

CONCLUSION

When designing classes and object models, it is vitally important to understand how the objects are related to each other. This chapter discusses the primary topics of building objects: inheritance, interfaces, and composition. In this chapter, you have learned how to build reusable code by designing with contracts.

In [Chapter 9](#), “[Building Objects and Object-Oriented Design](#),” we complete our OO journey and explore how objects that might be totally unrelated can interact with each other.

REFERENCES

Booch, Grady and Robert A. Maksimchuk and Michael W. Engel and Bobbi J. Young and Jim Conallen and Kelli A. Houston. 2007. *Object-Oriented Analysis and Design with Applications*, Third Edition. Boston, MA: Addison-Wesley.

Coad, Peter, and Mark Mayfield. 1997. *Java Design*. Upper Saddle River, NJ: Prentice Hall.

Meyers, Scott. 2005. *Effective C++*, Third Edition. Boston, MA: Addison-Wesley Professional.

`TestShop` percorre o inventário retornado e chama `buyInventory()` sem precisar saber se o objeto concreto é `PizzaShop` ou `DonutShop`.

Com essa estratégia, o mesmo código de aplicação atende às duas lojas. Se criarmos `GroceryShop`, basta fornecer uma nova implementação compatível e informar seu nome; o código principal não precisa mudar.

CONCLUSÃO

Ao projetar classes e modelos de objetos, é essencial compreender como os objetos se relacionam. Este capítulo reuniu herança, interfaces e composição e mostrou como contratos ajudam a construir código reutilizável.

No Capítulo 9, o foco passa para a interação entre objetos que podem não ter qualquer relação de herança entre si.

REFERÊNCIAS

Booch et al. Object-Oriented Analysis and Design with Applications, Third Edition. Addison-Wesley, 2007.

Coad, Peter; Mayfield, Mark. Java Design. Prentice Hall, 1997.

Meyers, Scott. Effective C++, Third Edition. Addison-Wesley Professional, 2005.

[Os títulos bibliográficos foram preservados no idioma original.]

9. Building Objects and Object-Oriented Design

The previous two chapters cover the topics of inheritance and composition. In [Chapter 7](#), “[Mastering Inheritance and Composition](#),” we learned that inheritance and composition represent the primary ways to build objects. In [Chapter 8](#), “[Frameworks and Reuse: Designing with Interfaces and Abstract Classes](#),” we learned that there are varying degrees of inheritance and how inheritance, interfaces, abstract classes, and composition all fit together.

This chapter covers the issue of how objects are related to each other in an overall design. You might say that this topic was already introduced, and you would be correct. Both inheritance and composition represent ways in which objects interact. However, inheritance and composition have one significant difference in the way objects are built. When inheritance is used, the end result is, at least conceptually, a single class that incorporates all the behaviors and attributes of the inheritance hierarchy. When composition is used, one or more classes are used to build another class.

Although it is true that inheritance is a relationship between two classes, what is really happening is that a parent is created that incorporates the attributes and methods of a child class. Let’s revisit the example of the `Person` and `Employee` classes (see [Figure 9.1](#)). Although there are indeed two separately designed classes here, the relationship is not simply interaction—it is inheritance. Basically, an employee is a person. An `Employee` object does not send a message to a `Person` object. An `Employee` object does need the services of a `Person` object. This is because an `Employee` object is a `Person` object.

9. CONSTRUINDO OBJETOS E PROJETO ORIENTADO A OBJETOS

Os dois capítulos anteriores trataram de herança e composição. O Capítulo 7 apresentou esses mecanismos como formas fundamentais de construir objetos; o Capítulo 8 mostrou diferentes graus de herança e como interfaces, classes abstratas e composição se relacionam.

Este capítulo examina como objetos se relacionam no projeto como um todo. Herança e composição já são formas de relacionamento, mas existe uma diferença importante.

Com herança, o resultado conceitual é uma classe que incorpora atributos e comportamentos da hierarquia. Com composição, uma classe é construída utilizando uma ou mais outras classes.

Considere novamente `Person` e `Employee`, na Figura 9.1. Embora existam duas definições de classe, a relação não é simplesmente uma troca de mensagens entre dois objetos independentes. `Employee` é `Person`. Um objeto `Employee` não precisa enviar uma mensagem a um objeto `Person` para obter seus serviços; ele já é, por herança, uma especialização de `Person`.

Figura 9.1 - Uma relação de herança.

Composição é diferente: representa interações entre objetos distintos. Enquanto o capítulo anterior concentrou-se nas variações de herança, este capítulo explorará as variações de composição e a forma como objetos independentes interagem.

RELAÇÕES DE COMPOSIÇÃO

Já vimos que composição representa uma relação parte-todo. Herança é normalmente descrita como “é-um”; composição como “tem-um”. Intuitivamente, um carro tem um volante, conforme a Figura 9.2.

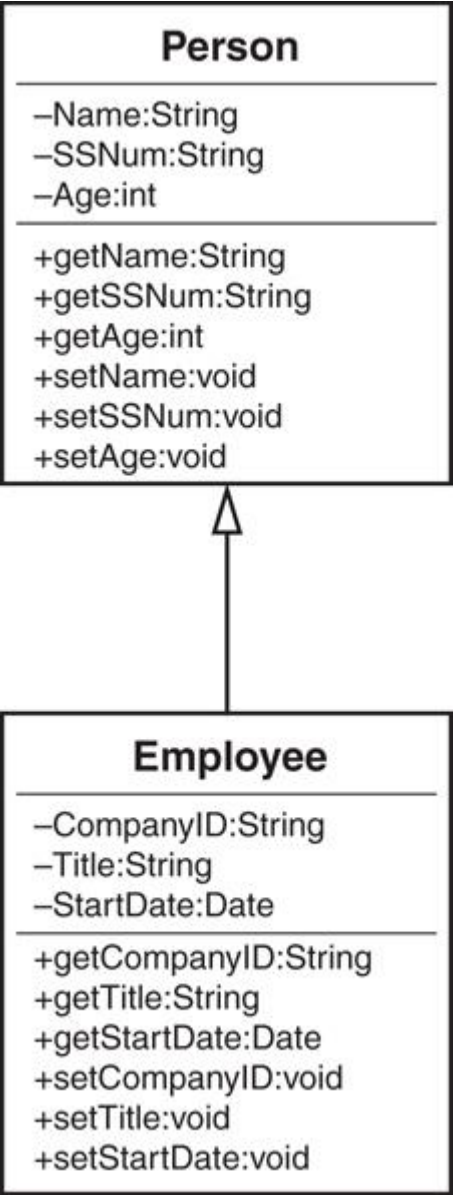


Figure 9.1 An inheritance relationship.

However, composition is a different situation. Composition represents interactions between distinct objects. So, whereas [Chapter 8](#) primarily covers the different flavors of inheritance, this chapter delves into the various flavors of composition and how objects interact with each other.

COMPOSITION RELATIONSHIPS

We have already seen that composition represents a part of a whole. Although the inheritance relationship is stated in terms of is-a, composition is stated in terms of has-a. We know intuitively that a car “has-a” steering wheel (see [Figure 9.2](#)).

Figura 9.2 - Uma relação de composição.

É-UM E TEM-UM

O autor mantém a expressão “has-a” de maneira padronizada para formular a regra, mesmo quando a gramática inglesa exigiria “has an”. Em português, usaremos simplesmente “é-um” e “tem-um”.

A composição ajuda a construir sistemas combinando partes menos complexas. Pessoas naturalmente raciocinam assim. Em vez de manter mentalmente volante, pneus, motor e dezenas de componentes, usamos a abstração “carro”. Isso simplifica comunicação e compreensão.

Composição também favorece peças intercambiáveis. Se componentes obedecem ao mesmo contrato, a identidade específica de uma peça pode ser irrelevante. Em software, essa intercambiabilidade se traduz em reuso.

Gilbert e McCarty exploram associação e composição em maior profundidade em *Object-Oriented Design in Java*. Aqui serão abordados os pontos fundamentais.

CONSTRUINDO EM FASES

Outra grande vantagem da composição é permitir que sistemas e subsistemas sejam desenvolvidos, testados e mantidos de forma relativamente independente.

Sistemas de software são complexos. Uma regra central para produzir software de qualidade é manter as coisas tão simples quanto possível. Grandes sistemas precisam ser quebrados em partes menores e administráveis.

Herbert Simon, em “The Architecture of Complexity” (1962), observou que sistemas complexos estáveis geralmente assumem forma hierárquica: cada sistema é construído a partir de subsistemas mais simples, que por sua vez são construídos a partir de componentes ainda mais simples. Esse princípio também está por trás da decomposição funcional do desenvolvimento procedural.

A Car has a Steering Wheel

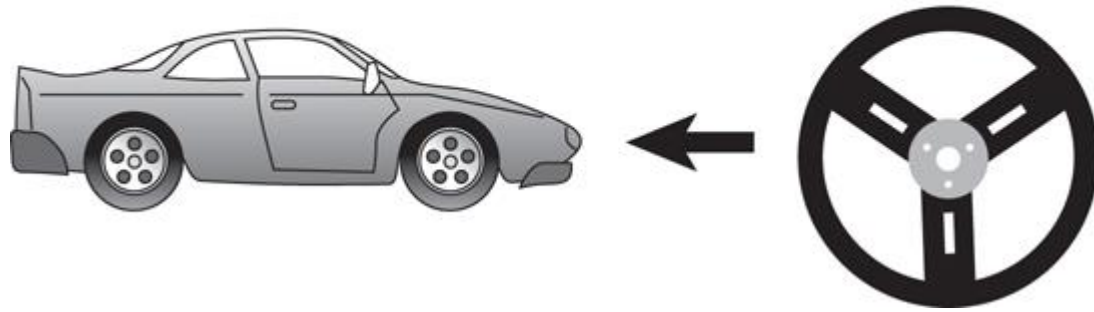


Figure 9.2 A

composition relationship.

Is-a and Has-a

Please forgive my grammar: For consistency, I will stick with “has a engine,” even though “has an engine” is grammatically correct. I do this because I want to simply state the rules as “is-a” and “has-a.”

The reason to use composition is that it builds systems by combining less complex parts. This is a common way for people to approach problems. Studies show that even the best of us can keep, at most, seven chunks of data in our short-term memory at one time. Thus, we like to use abstract concepts. Instead of saying that we have a large unit with a steering wheel, four tires, an engine, and so on, we say that we have a car. This makes it easier for us to communicate and keep things clear in our heads.

Composition also helps in other ways, such as making parts interchangeable. If all steering wheels are the same, it does not matter which specific steering wheel is installed in a specific car. In software development, interchangeable parts mean reuse.

In [Chapters 7](#) and [8](#) of their book *Object-Oriented Design in Java*, Stephen Gilbert and Bill McCarty present many examples of associations and composition in much more detail. I highly recommend referencing this material for a more in-depth look into these subjects. Here we address some of the more fundamental points of these concepts and explore some variations of their examples.

BUILDING IN PHASES

Another major advantage in using composition is that systems and subsystems can be built independently, and perhaps more importantly, tested and maintained independently.

There is no question that software systems are quite complex. To build quality software, you must follow one overriding rule to be successful: Keep things as simple as possible. For large software systems to work properly and be easily maintained, they must be broken into smaller, more manageable parts. How do you accomplish this? In a 1962 article titled “The Architecture of Complexity,” Nobel Prize winner Herbert Simon noted the following thoughts regarding stable systems:

- **“Stable complex systems usually take the form of a hierarchy, where each system is built from simpler subsystems, and each subsystem is built from simpler subsystems still.**—You might already be familiar with this principle because it forms the basis for functional decomposition, the method behind procedural software development.

In object-oriented design, you apply the same principles to composition—building complex objects from simpler pieces.

- **“Stable, complex systems are nearly decomposable.”**—This means you can identify the parts that make up the system and can tell the difference between interactions between the parts and inside the parts. Stable systems have fewer links between their parts than they have inside their parts. Thus, a modular stereo system, with simple links between the speakers, turntable, and amplifier, is inherently more stable than an integrated system, which isn’t easily decomposable.
- **“Stable complex systems are almost always composed of only a few different kinds of subsystems, arranged in different combinations.”**—Those subsystems, in turn, are generally composed of only a few different kinds of parts.
- **“Stable systems that work have almost always evolved from simple systems that worked.”**—Rather than build a new system from scratch—reinventing the wheel—the new system builds on the proven designs that went before it.

In our stereo example (see [Figure 9.3](#)), suppose the stereo system was totally integrated and was not built from components (that is, the stereo system was one big black-box system). In this case, what would happen if the CD player broke and became unusable? You would have to take in the entire system for repair. Not only would this be more complicated and expensive, but you would not have the use of any of the other components.

No projeto orientado a objetos, o mesmo princípio é aplicado à composição: objetos complexos são construídos a partir de partes mais simples.

Simon também observa:

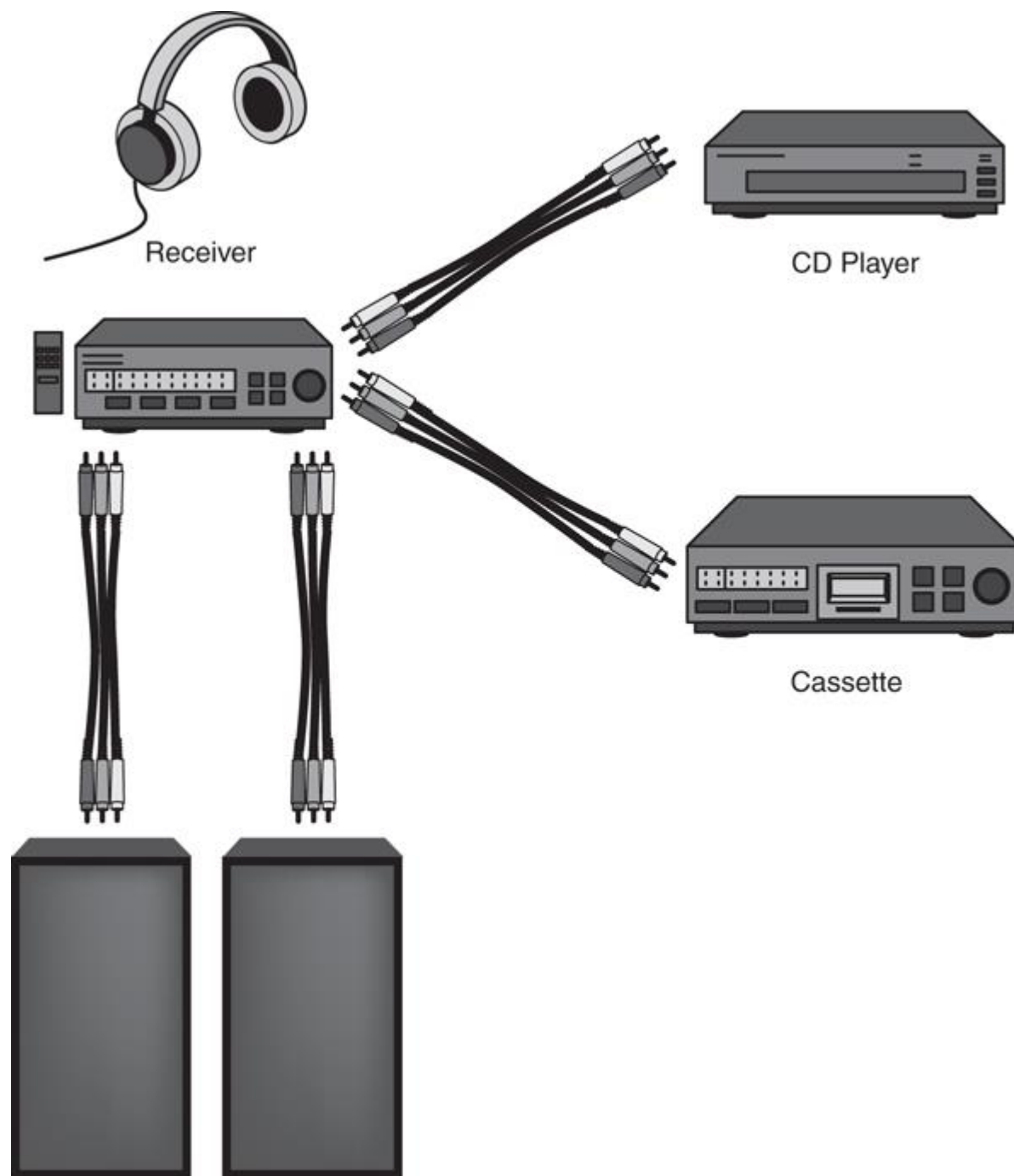
- Sistemas complexos estáveis são “quase decomponíveis”. É possível identificar suas partes e distinguir interações internas de interações entre módulos. Sistemas estáveis costumam possuir mais vínculos dentro dos módulos do que entre módulos.
- Sistemas complexos estáveis geralmente são formados por poucos tipos de subsistemas combinados de maneiras diferentes; esses subsistemas também são construídos com poucos tipos de partes.
- Sistemas estáveis que funcionam quase sempre evoluíram a partir de sistemas simples que já funcionavam. Em vez de reinventar tudo, novos sistemas se apoiam em projetos comprovados.

Retomando o exemplo do aparelho de som, imagine um sistema totalmente integrado, como uma única caixa preta. Se o tocador de CD quebrar, todo o sistema precisa ser levado ao conserto. O reparo se torna mais complicado e caro e, durante esse período, nenhum outro componente pode ser utilizado.

Figura 9.3 - Construindo, testando e verificando um sistema completo passo a passo.

Esse princípio é especialmente importante em ambientes como Java e .NET, nos quais componentes podem ser carregados dinamicamente. Um projeto desacoplado facilita manutenção e distribuição. Se apenas uma classe Java precisar ser corrigida, idealmente podemos redistribuir somente o componente alterado, em vez de todo o sistema.

Se o aparelho de som for composto por módulos independentes, um tocador de CD defeituoso pode ser desconectado e enviado para reparo sem retirar o restante do sistema de operação. O processo é mais simples, barato e rápido. Além disso, podemos substituir temporariamente o componente por outro compatível.



Figure

9.3 Building, testing, and verifying a complete system one step at a time.

This concept becomes very important to languages such as Java and those included in the .NET framework. Because objects are dynamically loaded, decoupling the design is quite important. For example, if you distribute a Java application and one of the class files needs to be re-created (for bug fixes or maintenance), you would be required to redistribute only that particular class file. If all code was in a single file, the entire application would need to be redistributed.

Suppose the system is broken into components rather than a single unit. In this case, if the CD player broke, you could disconnect the CD player and take it in for repair. (Note that all the components are connected by patch cords.) This would be less complicated and less expensive, and it would take less time than having to deal with a single, integrated unit. As an added benefit, you could still use the rest of the system. You could even buy another CD player because

it is a component. The repairperson could then plug your broken CD player into his repair systems to test and fix it. All in all, the component approach works quite well. Composition is one of the primary strategies that you, as a software designer, have in your arsenal to fight software complexity.

One major advantage of using components is that you can use components that were built by other developers within the organization, or even third-party vendors. However, using a software component from another source requires a certain amount of trust. Third-party components must come from a reliable source, and you must feel comfortable that the software is properly tested, not to mention that it must perform the advertised functions properly. There are still many who would rather build their own than trust components built by others.

TYPES OF COMPOSITION

Generally, there are two types of composition: association and aggregation. In both cases, these relationships represent collaborations between the objects. The stereo example we just used to explain one of the primary advantages of composition represents an association.

Is Composition a Form of Association?

Composition is another area in OO technologies where there is a question of which came first, the chicken or the egg. Some texts say that composition is a form of association, and some say that an association is a form of composition. In any event, in this book, we consider inheri-tance and composition the two primary ways to build classes. Thus, in this book, association is considered a form of composition.

All forms of composition include a has-a relationship. However, subtle differences exist between associations and aggregations based on how you visualize the parts of the whole. In an aggregation, you normally see only the whole, and in associations, you normally see the parts that make up the whole.

Aggregations

Perhaps the most intuitive form of composition is aggregation. Aggregation means that a complex object is composed of other objects. A TV set is a clean, neat package that you use for entertainment. When you look at your HD TV, you see a single unit. Most of the time, you do not stop to think about the fact that the HD TV contains some microchips, a screen, a tuner, and so on. Sure, you see a switch to turn the set on and off, and you certainly see the picture screen. However, this is not the way people normally think of HD TVs. When you go into an appliance store, the salesperson does not say, “Let me show you this aggregation of microchips, a picture screen, a tuner, and so on.” The salesperson says, “Let me show you this HD TV.”

Similarly, when you go to buy a car, you do not pick and choose all the individual components of the car. You do not decide which spark plugs to buy or which door handles to buy. You go to buy a car. Of course, you do choose some options, but for the most part, you choose the car as a whole, a complex object made up of many other complex and simple objects (see [Figure 9.4](#)).

Por ser um componente independente, o tocador de CD também pode ser conectado ao equipamento do técnico para diagnóstico e reparo. Essa abordagem modular funciona bem e a composição é uma das principais estratégias para controlar a complexidade de software.

Outra vantagem é reutilizar componentes construídos por outras equipes ou fornecedores. Isso exige confiança: componentes de terceiros devem vir de fontes confiáveis, ter sido bem testados e cumprir o comportamento anunciado. Alguns desenvolvedores ainda preferem construir tudo internamente, mas o reuso pode gerar ganhos significativos.

TIPOS DE COMPOSIÇÃO

De modo geral, o livro trata dois tipos de composição: associação e agregação. Em ambos, os objetos colaboram entre si. O exemplo do sistema de som representa uma associação.

COMPOSIÇÃO É UMA FORMA DE ASSOCIAÇÃO?

A terminologia varia. Alguns autores dizem que composição é uma forma de associação; outros dizem o contrário. Neste livro, herança e composição são consideradas as duas formas principais de construir relações entre classes, e associação é tratada como uma forma de composição.

Todas as formas de composição expressam uma relação “tem-um”. A diferença entre associação e agregação está na forma de perceber as partes e o todo.

AGREGAÇÕES

Agregação representa um objeto complexo formado por outros objetos, mas normalmente enxergado como uma unidade. Uma televisão contém chips, tela, sintonizador etc., porém o consumidor pensa no conjunto como “uma TV”.

Da mesma forma, ao comprar um carro, normalmente escolhemos o carro como um todo, não cada vela de ignição ou maçaneta separadamente. O carro é um objeto complexo formado por muitos objetos simples e complexos, conforme a Figura 9.4.

Figura 9.4 - Hierarquia de agregação de um carro.

ASSOCIAÇÕES

Enquanto uma agregação costuma ser percebida como um todo, numa associação as partes permanecem mais claramente independentes.

Considere um computador desktop tradicional, Figura 9.5. O sistema como um todo inclui monitor, teclado, mouse e gabinete. Cada componente é um objeto separado, conectado ao conjunto. O computador utiliza os serviços do mouse, do teclado e do monitor; ele não implementa internamente essas funções, mas depende de objetos externos conectados por portas e cabos.

Essa colaboração entre objetos independentes caracteriza uma associação.

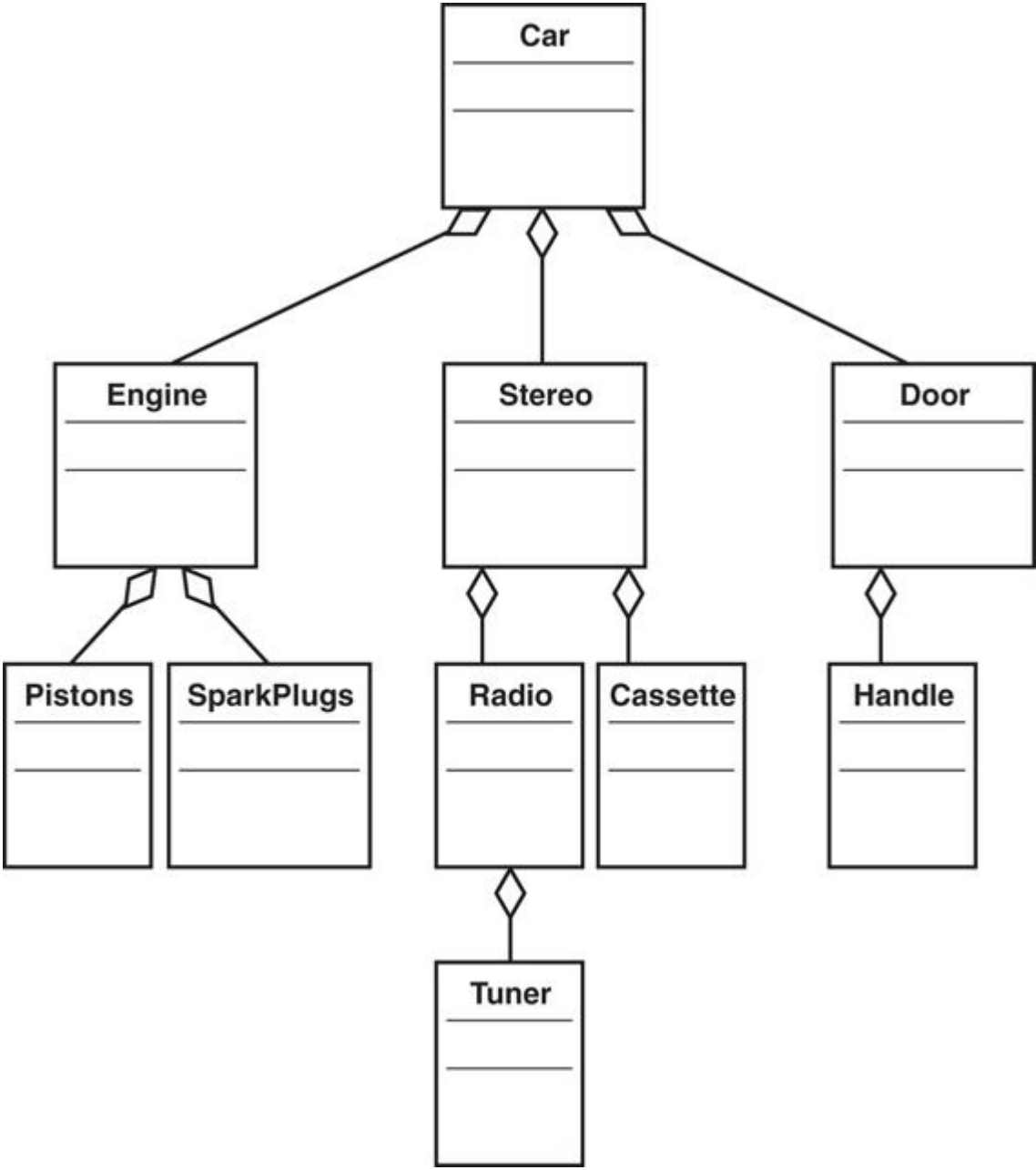


Figure 9.4 An

aggregation hierarchy for a car.

Associations

Whereas aggregations represent relationships where you normally see only the whole, associations present both the whole and the parts. As stated in the stereo example, the various components are presented separately and connect to the whole by use of patch cords (the cords that connect the various components).

Consider a traditional desktop computer system as an example (see [Figure 9.5](#)); the whole is the computer system. The components are the monitor, keyboard, mouse, and main box. Each is a separate object, but together they represent the whole of the computer system. The main computer is using the keyboard, the mouse, and the monitor to delegate some of the work. In other words, the computer box needs the service of a mouse but does not have the capability to provide this service by itself. Thus, the computer box requests the service from a separate mouse via the specific port and cable connecting the mouse to the box.

Figura 9.5 - Associações como serviços separados.

AGREGAÇÃO VERSUS ASSOCIAÇÃO

Uma agregação é um objeto complexo composto por outros objetos. Uma associação aparece quando um objeto solicita que outro objeto lhe forneça um serviço.

USANDO ASSOCIAÇÕES E AGREGAÇÕES JUNTAS

Na prática, a fronteira entre associação e agregação pode ser pouco nítida. Muitas decisões de projeto envolvem exatamente escolher como representar essa relação.

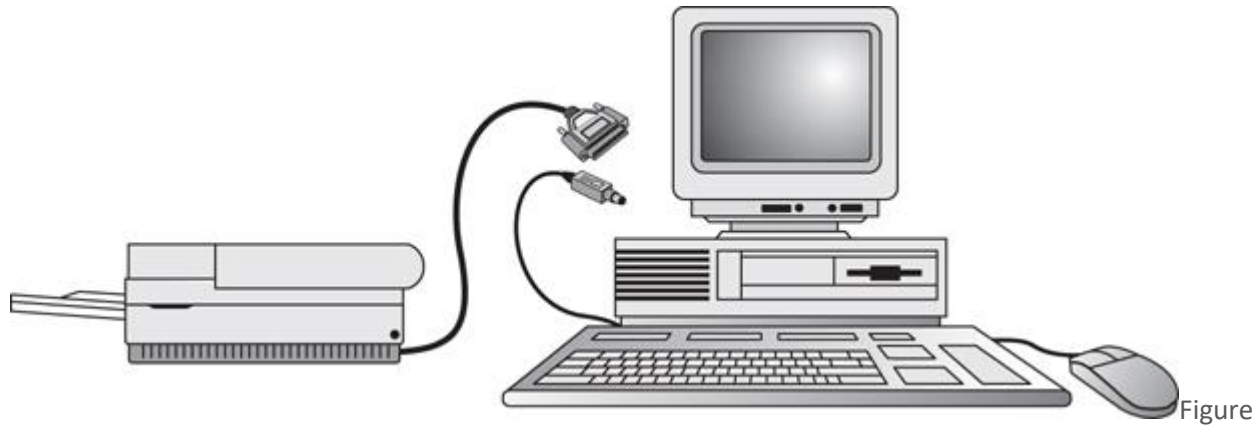
O exemplo do computador mistura ambos os conceitos. Monitor, teclado e mouse estão associados ao gabinete como componentes separados. O próprio gabinete, porém, é uma agregação de placa-mãe, chips, placas de vídeo etc.

Um Employee pode conter Address e também ter uma associação com Spouse. Se tanto o empregado quanto o cônjuge forem funcionários, demitir um deles não destrói o objeto do outro; apenas a associação é desfeita.

O mesmo ocorre no aparelho de som: o receiver se associa a alto-falantes e ao tocador, enquanto cada componente pode internamente ser uma agregação de outras partes. Carros e computadores reais misturam agregações e associações.

NÃO EXISTE UMA ÚNICA RESPOSTA CORRETA

Projeto não é ciência exata. Existem boas regras gerais, mas decisões concretas dependem do problema, dos requisitos e dos compromissos envolvidos.



9.5 Associations as a separate service.

Aggregation Versus Association

An aggregation is a complex object composed of other objects. An association is used when one object wants another object to perform a service for it.

Using Associations and Aggregations Together

One thing you might have noticed in all the examples is that the dividing lines between what is an association and what is an aggregation are often blurred. Suffice it to say that many of your most interesting design decisions will come down to whether to use associations or aggregations.

For example, the desktop computer system example used to describe associations also contains some aggregation. Although the interaction between the computer box, the monitor, the keyboard, and the mouse is association, the computer box itself represents aggregation. You see only the computer box, but it is actually a complex system made up of other objects, including chips, motherboards, video cards, and so on.

Consider that an `Employee` object might be composed of an `Address` object and a `Spouse` object. You might consider the `Address` object as an aggregation (basically a part of the `Employee` object), and the `Spouse` object as an association. To illustrate, suppose both the employee and the spouse are employees. If the employee is fired, the spouse is still in the system but the association is broken.

Similarly, in the stereo example, the receiver has an association with the speakers as well as the CD. Yet, the speakers and the CD are themselves aggregations of other objects, such as power cords.

In the car example, although the engine, spark plugs, and doors represent composition, the stereo also represents an association relationship. In reality, cars and desktop computers are a mix of aggregations *and* associations.

No One Right Answer

As usual, there isn't a single, absolutely correct answer when it comes to making a design decision. Design is not an exact science. Although we can make general rules to live by, these rules are not hard and fast.

AVOIDING DEPENDENCIES

When using composition, it is desirable to avoid making objects highly dependent on one another. One way to make objects very dependent on each other is to mix domains. In the best of all worlds, an object in one domain should not be mixed with an object in another domain, except under certain circumstances. We can return again to the stereo example to explain this concept.

By keeping all the components in separate domains, the stereo system is easier to maintain. For example, if the CD component breaks, you can send the CD player off to be repaired individually. In this case, the CD player and the MP3 player have separate domains. This provides flexibility, such as buying the CD player and the MP3 player from separate manufacturers. So, if you decide you want to swap out the CD player with a brand from another manufacturer, you can.

Sometimes there is a certain convenience in mixing domains. A good example of this, one that I have been using for years, pertains to the existence of the old-style TV/VCR combinations. Granted, it is convenient to have both in the same module. However, if the TV breaks, the VCR is unusable—at least as part of the unit it was purchased in.

You need to determine what is more important in specific situations: whether you want convenience or stability. There is no right answer. It all depends on the application and the environment. In the case of the TV/VCR combination, we decided that the convenience of the integrated unit far outweighed the risk of lower unit stability (see [Figure 9.6](#)). Revisit the stereo system in [Figure 9.3](#) to reinforce what a non-integrated system looks like.

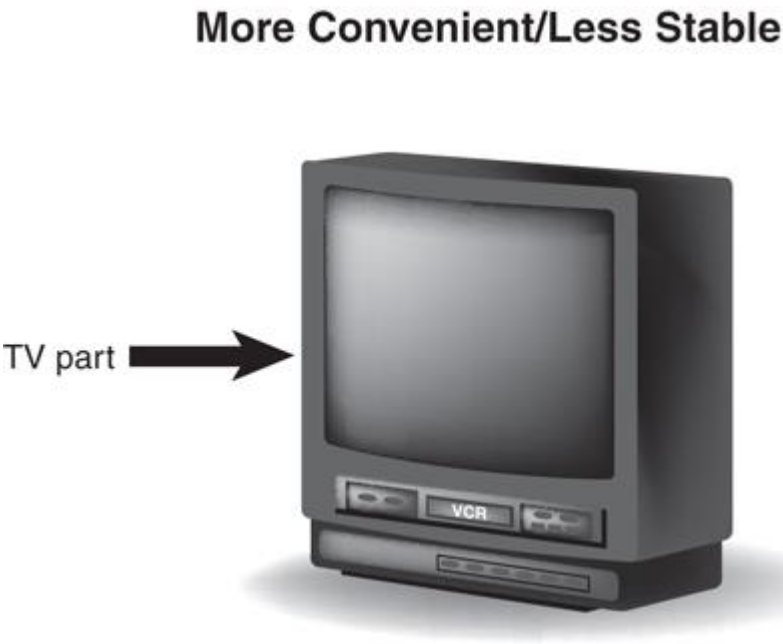


Figure 9.6 Convenience versus stability.

Interfaces solve this and managing dependencies is a major part of this. If interfaces are defined in a shared library and implementations are defined in more concrete classes, you can afford to mix domains by using the behavior contracts.

Mixing Domains

EVITANDO DEPENDÊNCIAS

Ao usar composição, é desejável evitar dependências excessivas entre objetos. Uma forma de gerar acoplamento forte é misturar domínios desnecessariamente.

No exemplo do sistema de som, manter componentes em domínios separados facilita manutenção. Se o tocador de CD quebrar, ele pode ser reparado isoladamente. CD e MP3 podem vir de fabricantes diferentes e ser substituídos independentemente.

Às vezes, misturar domínios oferece conveniência. Um exemplo histórico é o aparelho TV/VCR integrado: é compacto e prático, mas se a televisão quebra, o videocassete também fica indisponível como parte da unidade.

É necessário decidir o que pesa mais em cada situação: conveniência ou estabilidade. Não há resposta universal. A Figura 9.6 ilustra essa troca; a Figura 9.3 representa o caso modular.

Interfaces ajudam a administrar esse problema. Se os contratos forem definidos em uma biblioteca compartilhada e as implementações ficarem em classes concretas separadas, componentes de diferentes domínios podem colaborar por meio de contratos comportamentais, reduzindo dependências diretas.

MISTURANDO DOMÍNIOS

A conveniência de misturar domínios é uma decisão de projeto. Se o benefício de uma unidade integrada compensar o risco e o tempo de indisponibilidade, essa pode ser a escolha adequada.

The convenience of mixing domains is a design decision. If the power of having a TV/VCR combination outweighs the risk and potential downtime of the individual components, the mixing of domains may well be the preferred design choice.

CARDINALITY

In their book *Object-Oriented Design in Java*, Gilbert and McCarty describe cardinality as the number of objects that participate in an association and whether the participation is optional or mandatory. To determine cardinality, Gilbert and McCarty ask the following questions:

- Which objects collaborate with which other objects?
- How many objects participate in each collaboration?
- Is the collaboration optional or mandatory?

For example, let’s consider the following example. We are creating an `Employee` class that inherits from `Person` and has relationships with the following classes:

- `Division`
- `JobDescription`
- `Spouse`
- `Child`

What do these classes do? Are they optional? How many does an `Employee` need?

- `Division`
 - This object contains the information relating to the division that the employee works for.
 - Each employee must work for a division, so the relationship is mandatory.
 - The employee works for one, and only one, division.
- `JobDescription`
 - This object contains a job description, most likely containing information such as salary grade and salary range.
 - Each employee must have a job description, so the relationship is mandatory.
 - The employee can hold various jobs during the tenure at a company. Thus, an employee can have many job descriptions. These descriptions can be kept as a history if an employee changes jobs, or it is possible that an employee might hold two different jobs at one time. For example, a supervisor might take on an employee’s responsibilities if the employee quits and a replacement has not yet been hired.
- `Spouse`
 - In this simplistic example, the `Spouse` class contains only the anniversary date.
 - An employee can be married or not married. Thus, a spouse is optional.
 - An employee can have only one spouse.
- `Child`
 - In this simple example, the `Child` class contains only the string `FavoriteToy`.
 - An employee can have children or not have children.

CARDINALIDADE

Gilbert e McCarty definem cardinalidade como a quantidade de objetos que participam de uma associação e se essa participação é opcional ou obrigatória. Para determiná-la, perguntamos:

- Quais objetos colaboram entre si?
- Quantos objetos participam de cada colaboração?
- A colaboração é opcional ou obrigatória?

Considere `Employee`, que herda de `Person` e se relaciona com `Division`, `JobDescription`, `Spouse` e `Child`.

DIVISION

Contém informações sobre a divisão em que o funcionário trabalha. Todo funcionário deve pertencer a uma divisão, portanto a relação é obrigatória e existe exatamente uma `Division` por `Employee`.

JOBDESCRIPTION

Representa dados como nível e faixa salarial. Todo funcionário deve possuir uma descrição de cargo. Ao longo do tempo, porém, pode existir um histórico de vários cargos, e um funcionário pode até acumular responsabilidades. Assim, a cardinalidade pode ser de um a muitos.

SPOUSE

No exemplo simplificado, contém apenas a data de aniversário de casamento. Um funcionário pode não ser casado, portanto a relação é opcional; quando existe, há no máximo um cônjuge.

CHILD

No exemplo, contém apenas `FavoriteToy`. Um funcionário pode não ter filhos ou pode ter vários. O sistema precisa decidir, se necessário, um limite prático.

- An employee can have no children or an infinite number of children (wow!). You could make a design decision as to the upper limit of the number of children that the system can handle.

To sum up, [Table 9.1](#) represents the cardinality of the associations of the classes we just considered.

Table 9.1 Cardinality of Class Associations

Optional/Association	Cardinality	Mandatory
Employee/Division	1	Mandatory
Employee/JobDescription	1 . . n	Mandatory
Employee/Spouse	0 . . 1	Optional
Employee/Child	0 . . n	Optional

Cardinality Notation

The notation of 0 . . 1 means that an employee can have either zero or one spouse. The notation of 0 . . n means that an employee can have any number of children from zero to an unlimited number. The *n* basically represents infinity.

[Figure 9.7](#) shows the class diagram for this system. Note that in this class diagram, the cardinality is indicated along the association lines. Refer to [Table 9.1](#) to see whether the association is mandatory.

Em resumo, a Tabela 9.1 representa as cardinalidades:

Employee/Division: 1, obrigatória.
Employee/JobDescription: 1..n, obrigatória.
Employee/Spouse: 0..1, opcional.
Employee/Child: 0..n, opcional.

NOTAÇÃO DE CARDINALIDADE

0..1 significa que pode existir zero ou um objeto associado. 0..n significa que podem existir de zero a muitos objetos; n representa uma quantidade não limitada conceitualmente.

A Figura 9.7 mostra essas cardinalidades diretamente nas linhas de associação do diagrama UML. A tabela também registra se cada relação é obrigatória ou opcional.

Figura 9.7 - Cardinalidade em um diagrama UML.

ASSOCIAÇÕES COM MÚLTIPLOS OBJETOS

Como representar em código uma associação que pode conter vários objetos, como zero ou muitos filhos? O exemplo Employee começa nesta página.

A classe herda de Person e mantém atributos próprios como CompanyID, Title e StartDate. Para as relações, mantém referências a Spouse, Division e coleções/arrays de Child e JobDescription.

A listagem Java completa continua na próxima página. O ponto importante é que relações um-para-muitos precisam de alguma estrutura capaz de armazenar múltiplas referências.

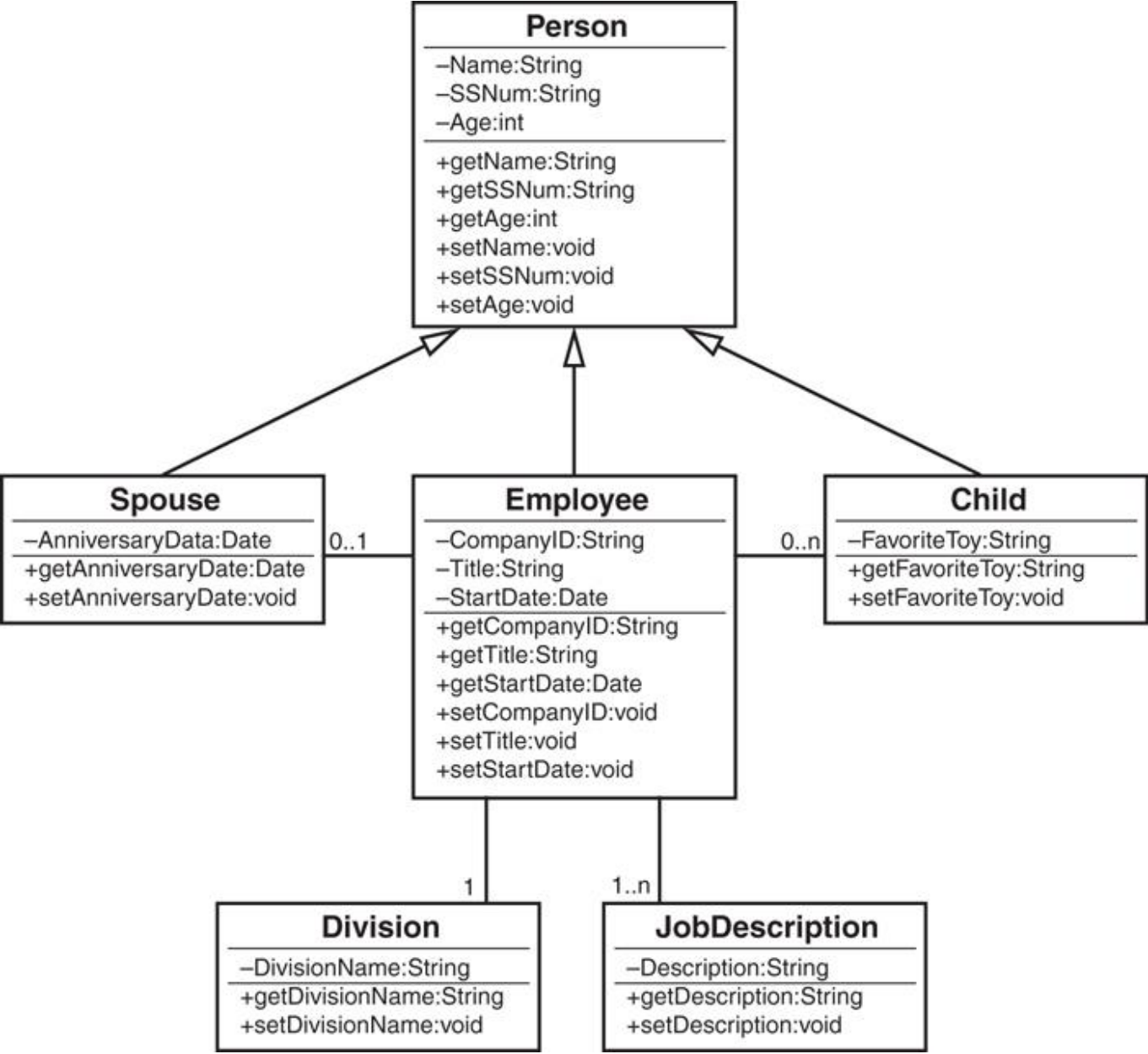


Figure 9.7 Cardinality in a UML diagram.

Multiple Object Associations

How do we represent an association that might contain multiple objects (such as 0 to many children) in code? Here is the code for the `Employee` class:

[Click here to view code image](#)

```
import java.util.Date;

public class Employee extends Person{

    private String CompanyID;
    private String Title;
    private Date StartDate;

    private Spouse spouse;
    private Child[] child;
    private Division division;
    private JobDescription[] jobDescriptions;
```

```
public String getCompanyID() {return CompanyID;}
public String getTitle() {return Title;}
public Date getStartDate() {return StartDate;}

public void setCompanyID(String CompanyID) {}
public void setTitle(String Title) {}
public void setStartDate(int StartDate) {}
}
```

Note that the classes that have a one-to-many relationship are represented by arrays in the code:

```
private Child[] child;
private JobDescription[] jobDescriptions;
```

Optional Associations

One of the most important issues when dealing with associations is to make sure that your application is designed to check for optional associations. This means that your code must check to see whether the association is `null`.

Suppose in the previous example that your code assumes that every employee has a spouse. However, if one employee is not married, the code will have a problem (see [Figure 9.8](#)). If your code does indeed expect a spouse to exist, it may well fail and leave the system in an unstable state. The bottom line is that the code must check for a `null` condition, and must handle this as a valid condition.

Object Mary

```
public String getSpouse(Employee e) {
    return Spouse;
}
```

OOPS!! Mary has no spouse



Must check all optional associations for null!!! Figure 9.8 Checking all optional associations.

For example, if no spouse exists, the code must not attempt to invoke a spouse behavior. This could lead to an application failure. Thus, the code must be able to process an `EmployeeObject` that has no spouse.

No código, as relações com multiplicidade são representadas por arrays:

```
private Child[] child;
private JobDescription[] jobDescriptions;
```

ASSOCIAÇÕES OPCIONAIS

Ao trabalhar com associações, uma questão crítica é tratar corretamente relações opcionais. O código precisa verificar se a referência existe ou se é `null`.

Suponha que o programa assuma que todo `Employee` possui `Spouse`. Se um funcionário não for casado, essa suposição falha. A Figura 9.8 reforça que toda associação opcional deve ser verificada.

Se `spouse` não existir, o programa não pode invocar métodos por meio dessa referência. Em vez disso, deve tratar “sem cônjuge” como um estado perfeitamente válido. Caso contrário, uma operação sobre uma referência nula pode provocar falha e deixar a aplicação instável.

Figura 9.8 - Verificando todas as associações opcionais.

TYING IT ALL TOGETHER: AN EXAMPLE

Let’s work on a simple example that will tie the concepts of inheritance, interfaces, composition, associations, and aggregations together into a single, short system diagram.

Consider the example used in [Chapter 8](#), with one addition: We will add an `Owner` class that will take the dog out for walks. Recall that the `Dog` class inherits directly from the `Mammal` class. The solid arrow represents this relationship between the `Dog` class and the `Mammal` class in [Figure 9.9](#). The `Nameable` class is an interface that `Dog` implements, which is represented by the dashed arrow from the `Dog` class to the `Nameable` interface.

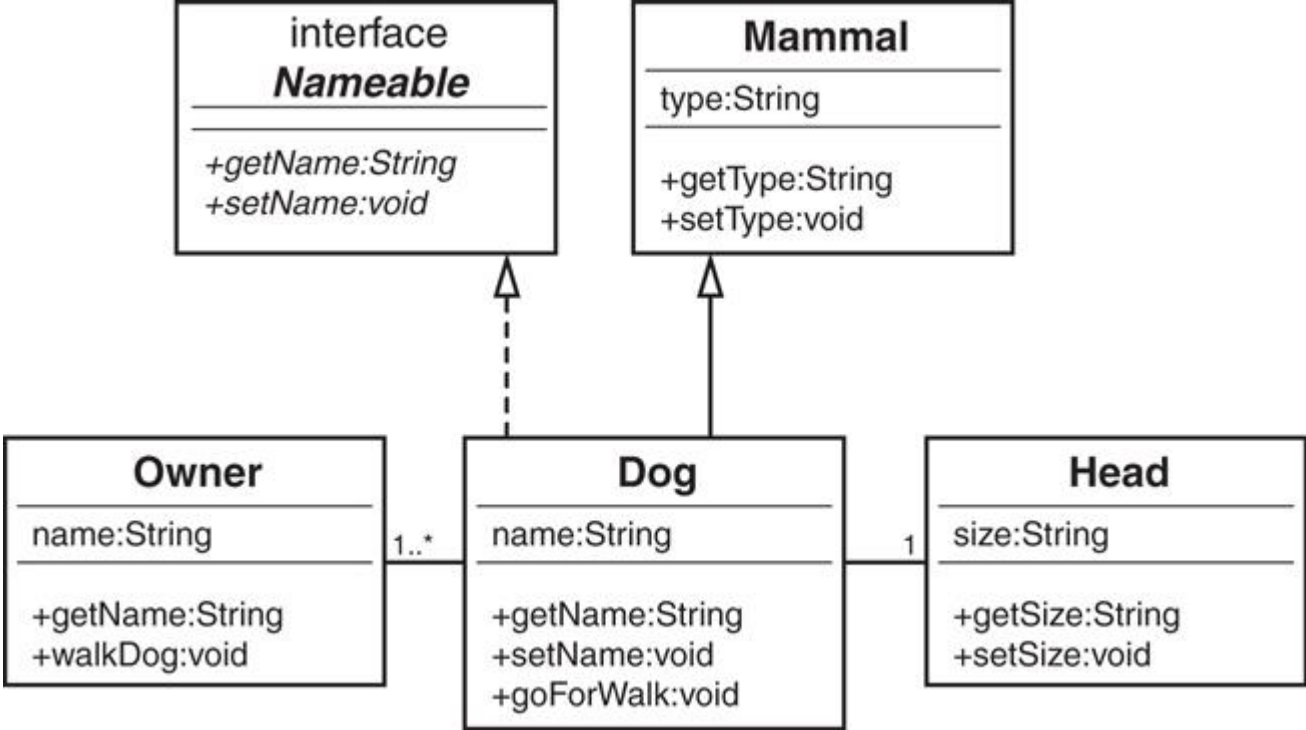


Figure 9.9 A UML diagram for the `Dog` example.

In this chapter, we are mostly concerned with associations and aggregations. The relationship between the `Dog` class and the `Head` class is considered aggregation because the head is actually part of the dog. The cardinality on the line connecting the two class diagrams specifies that a dog can have only a single head.

The relationship between the `Dog` class and the `Owner` class is association. The owner is clearly not part of the dog, or vice versa, so we can safely eliminate aggregation. However, the dog does require a service from the owner—the act of taking him on a walk. The cardinality on the line connecting the `Dog` and `Owner` classes specifies that a dog can have one or more owners (for example, a wife and husband can both be considered owners, with shared responsibility for walking the dog).

These relationships—inheritance, interfaces, composition, associations, and aggregations—represent the bulk of the design work you will encounter when designing OO systems.

Where Is the Head?

You might decide that it makes sense to attach the `Head` class to the `Mammal` class instead of the `Dog` class, because all mammals supposedly have a head. For this model, I was using the `Dog` class as the focal point of the example, so that is why I attached the `Head` to the `Dog` itself.

JUNTANDO TUDO: UM EXEMPLO

Vamos reunir herança, interfaces, composição, associação e agregação em um único diagrama simples.

Retomamos o exemplo do Capítulo 8 e acrescentamos `Owner`, responsável por levar o cão para passear. `Dog` herda diretamente de `Mammal`; no diagrama, essa relação é mostrada pela seta sólida. `Dog` também implementa a interface `Nameable`, indicada pela seta tracejada.

Figura 9.9 - Diagrama UML do exemplo `Dog`.

O foco deste capítulo são associação e agregação. A relação `Dog-Head` é uma agregação porque a cabeça é parte do cão; a cardinalidade indica que há uma única `Head`.

A relação `Dog-Owner` é uma associação. `Owner` não faz parte de `Dog` e `Dog` não faz parte de `Owner`, mas o cão utiliza um serviço do dono: ser levado para passear. A cardinalidade indica que um cão pode ter um ou mais donos, por exemplo duas pessoas compartilhando essa responsabilidade.

Essas relações - herança, interfaces, composição, associações e agregações - representam grande parte das decisões encontradas no projeto de sistemas OO.

ONDE DEVERIA FICAR HEAD?

Também seria razoável associar `Head` a `Mammal`, pois todos os mamíferos possuem cabeça. No modelo mostrado, `Dog` foi escolhido como foco didático, por isso `Head` foi ligada diretamente a `Dog`.

CONCLUSION

In this chapter, we have explored some of the finer points of composition and its two primary types: aggregation and association. Whereas inheritance represents a new kind of already existing object, composition represents the interactions between various objects.

The past three chapters have covered the basics of inheritance and composition. Using these concepts and your skills in the software development process, you are on your way to designing solid classes and object models. Chapter 10, “Design Patterns,” explores how to use UML class diagrams to assist in the modeling of object models.

REFERENCES

Booch, Grady and Robert A. Maksimchuk and Michael W. Engel and Bobbi J. Young and Jim Conallen and Kelli A. Houston. 2007. *Object-Oriented Analysis and Design with Applications*, Third Edition. Boston, MA: Addison-Wesley.

Coad, Peter, and Mark Mayfield. 1997. *Java Design*. Upper Saddle River, NJ: Prentice Hall.

Gilbert, Stephen, and Bill McCarty. 1998. *Object-Oriented Design in Java*. Berkeley, CA: The Waite Group Press.

Meyers, Scott. 2005. *Effective C++*, Third Edition. Boston, MA: Addison-Wesley Professional.

CONCLUSÃO

Este capítulo explorou detalhes de composição e de seus dois tipos principais no livro: agregação e associação. Enquanto herança representa uma nova espécie/especialização de algo já existente, composição representa interações entre objetos distintos.

Os três capítulos anteriores forneceram a base de herança e composição. Com esses conceitos e com habilidades gerais de desenvolvimento de software, é possível começar a projetar classes e modelos de objetos sólidos.

O Capítulo 10, “Design Patterns”, mostra como padrões e diagramas UML podem apoiar o projeto e a modelagem de objetos.

REFERÊNCIAS

Booch et al. Object-Oriented Analysis and Design with Applications, Third Edition. Addison-Wesley, 2007.

Coad, Peter; Mayfield, Mark. Java Design. Prentice Hall, 1997.

Gilbert, Stephen; McCarty, Bill. Object-Oriented Design in Java. The Waite Group Press, 1998.

Meyers, Scott. Effective C++, Third Edition. Addison-Wesley Professional, 2005.

[Os títulos bibliográficos foram preservados no idioma original.]

10. Design Patterns

One of the interesting things about software development is that when you create a software system, you are actually modeling a real-world system. For example, in the Information Technology industry, it is safe to say that IT *is* the business—or at least IT *implements* the business. To write the business software systems, the developers must thoroughly understand the business models. As a result, the developers often have the most intimate knowledge of a company’s business processes.

We have seen this concept throughout this book as it relates to our educational discussions. For example, when we discussed using inheritance to abstract out the behaviors and attributes of mammals, the model was based on the true real-life model, not a contrived model that we created for our own purposes.

Thus, when we create a mammal class, we can use it to build countless other classes, such as dogs and cats and so on, because all mammals share certain behaviors and attributes. This works when we study dogs, cats, squirrels, and other mammals because we can see patterns. These patterns allow us to inspect an animal and make the determination that it is indeed a mammal, or perhaps a reptile, which would have other patterns of behaviors and attributes.

Throughout history, humans have used patterns in many aspects of life, including engineering. These patterns go hand-in-hand with the holy grail of software development: software reuse. In this chapter, we consider design patterns, a relatively new area of software development (the seminal book on design patterns was published in 1995).

Design patterns are perhaps one of the most influential developments that have come out of the object-oriented movement in the past several years. Patterns lend themselves perfectly to the concept of reusable software development. Because object-oriented development is all about reuse, patterns and object-oriented development go hand-in-hand.

The basic concept of design patterns revolves around the principle of best practices. By *best practices*, we mean that when good and efficient solutions are created, these solutions are documented in a way that others can benefit from previous successes—as well as learn from the failures.

One of the most important books on object-oriented software development is *Design Patterns: Elements of Reusable Object-Oriented Software* by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. This book was an important milestone for the software industry and has become so entrenched in the computer science lexicon that the book’s authors have become known as the Gang of Four. In writings on object-oriented topics, you will often see the Gang of Four referred to as the GoF.

The intent of this chapter is to explain what design patterns are. (Explaining each design pattern is far beyond the scope of this book and would take more than one volume.) To accomplish this, we explore each of the three categories of design patterns (creational, structural, and behavioral) as defined by the Gang of Four and provide a concrete example of one pattern in each category.

WHY DESIGN PATTERNS?

The concept of design patterns did not necessarily start with the need for reusable software. In fact, the seminal work on design patterns is about constructing buildings and cities. As Christopher Alexander noted in *A Pattern Language: Towns, Buildings, Construction*, “Each

10. PADRÕES DE PROJETO

Uma característica interessante do desenvolvimento de software é que, ao construir um sistema, estamos modelando algum sistema do mundo real. Em tecnologia da informação, frequentemente o software implementa diretamente processos do negócio; por isso, desenvolvedores precisam compreender profundamente o domínio que automatizam.

Esse princípio apareceu ao longo do livro. Quando abstraímos atributos e comportamentos de mamíferos, por exemplo, o modelo foi baseado em padrões observados no mundo real.

Reconhecemos cães, gatos e outros mamíferos porque há padrões comuns. Da mesma forma, a humanidade utiliza padrões em engenharia e em muitos outros campos. No software, eles se relacionam ao objetivo recorrente de reutilização.

Padrões de projeto tornaram-se um dos desenvolvimentos mais influentes do movimento OO. Eles combinam naturalmente com a ideia de reuso: soluções que funcionam bem são documentadas para que outras pessoas possam aplicar princípios semelhantes e aprender tanto com sucessos quanto com falhas.

Uma obra decisiva é Design Patterns: Elements of Reusable Object-Oriented Software, de Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides, autores conhecidos como Gang of Four (GoF).

O objetivo deste capítulo não é explicar os 23 padrões individualmente, mas apresentar o conceito. Serão estudadas as três categorias definidas pelo GoF - criacionais, estruturais e comportamentais - com um exemplo concreto de cada.

POR QUE PADRÕES DE PROJETO?

A ideia de padrões não nasceu especificamente do software. Christopher Alexander, em A Pattern Language: Towns, Buildings, Construction, descreve um padrão como uma solução essencial para um problema recorrente, capaz de ser reutilizada inúmeras vezes sem ser aplicada exatamente da mesma forma.

pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use the solution a million times over, without ever doing it the same way twice.”

The Four Elements of a Pattern

The GoF describe a pattern as having four essential elements:

- The *pattern name* is a handle we can use to describe a design problem, its solutions, and consequences in a word or two. Naming a pattern immediately increases our design vocabulary. It lets us design at a higher level of abstraction. Having a vocabulary for patterns lets us talk about them with our colleagues, in our documentation, and even to ourselves. It makes it easier to think about designs and to communicate them and their trade-off to others. Finding good names has been one of the hardest parts of developing our catalog.?
- The *problem* describes when to apply the pattern. It explains the problem and its content. It might describe specific design problems, such as how to represent algorithms as objects. It might describe class or object structures that are symptomatic of an inflexible design. Sometimes the problem will include a list of conditions that must be met before it makes sense to apply the pattern.
- The *solution* describes the elements that make up the design, their relationships, responsibilities, and collaborations. The solution doesn't describe a particular concrete design or implementation, because a pattern is like a template that can be applied in many situations. Instead, the pattern provides an abstract description of a design problem, and how a general arrangement of elements (classes and objects in our case) solves it.
- The *consequences* are the results and trade-offs of applying the pattern. Although consequences are often unvoiced, when we describe design decisions, they are critical for evaluating design alternatives and for understanding the costs and benefits of the applying pattern. The consequences for software often concern space and time trade-offs. They might address language and implementation issues as well. Because reuse is often a factor in object-oriented design, the consequences of a pattern include its impact on a system's flexibility, extensibility, or portability. Listing the consequences explicitly helps you understand and evaluate them.

SMALLTALK’S MODEL/VIEW/CONTROLLER

For historical perspective, we need to consider the Model/View/Controller (MVC) introduced in Smalltalk (and used in other object-oriented languages). MVC is often used to illustrate the origins of design patterns. The Model/View/Controller paradigm was used to create user interfaces in Smalltalk. Smalltalk was perhaps the first *popular* object-oriented language.

Smalltalk

Smalltalk is the result of several great ideas that emerged from Xerox PARC. These ideas included the mouse and using a windowing environment, among others. Smalltalk is a wonderful language that provided the foundation for all the object-oriented languages that followed. One of the complaints about C++ is that it's not really object-oriented, whereas Smalltalk is. Although C++ had a larger following in the early days of OO, Smalltalk has always had a very dedicated core group of supporters. Java is a mostly OO language that embraced the C++ developer base.

Design Patterns defines the MVC components in the following manner:

The Model is the application object, the View is the screen presentation, and the Controller defines the way the user interface reacts to user input.

OS QUATRO ELEMENTOS DE UM PADRÃO

O GoF descreve quatro elementos essenciais:

NOME DO PADRÃO - Um identificador curto para o problema, a solução e suas consequências. Dar nomes aos padrões amplia o vocabulário de projeto, permite raciocinar em nível mais alto de abstração e facilita comunicação entre desenvolvedores e documentação.

PROBLEMA - Explica quando aplicar o padrão e quais condições caracterizam a situação. Pode descrever uma dificuldade específica, uma estrutura inflexível ou pré-condições para que o padrão faça sentido.

SOLUÇÃO - Descreve os elementos do projeto, seus relacionamentos, responsabilidades e colaborações. Não é uma implementação concreta; é um modelo abstrato capaz de ser aplicado em diferentes contextos.

CONSEQUÊNCIAS - São os resultados e compromissos do uso do padrão: custos, benefícios, impacto em tempo, espaço, implementação, flexibilidade, extensibilidade, portabilidade e reuso. Torná-los explícitos ajuda a avaliar alternativas.

MODEL/VIEW/CONTROLLER DO SMALLTALK

Historicamente, o Model/View/Controller (MVC) do Smalltalk é frequentemente usado para ilustrar as origens de padrões de projeto em interfaces gráficas.

Smalltalk foi uma das primeiras linguagens OO populares e incorporou ideias surgidas no Xerox PARC. Teve grande influência sobre linguagens orientadas a objetos posteriores.

- No MVC:
- Model é o objeto/modelo da aplicação;
 - View é a apresentação na tela;
 - Controller define como a interface reage às entradas do usuário.

Em paradigmas anteriores, Model, View e Controller muitas vezes eram misturados em uma única entidade. MVC os separa por interfaces distintas. Assim, se apenas a interface visual precisar mudar, idealmente altera-se View sem modificar a lógica do domínio.

Figura 10.1 - Paradigma Model/View/Controller.

Grande parte do projeto OO envolve separar interface de implementação. Também devemos evitar misturar interfaces que representam responsabilidades não relacionadas. MVC foi uma das primeiras formulações populares dessa separação para o problema recorrente de ligar interface gráfica, lógica de negócio e dados.

Quando GUI, regras de negócio e dados são desacoplados, o sistema tende a ficar mais flexível. A interface pode estar no cliente, a lógica em um servidor de aplicação e os dados em um servidor próprio. Alterar a aparência da GUI não precisa modificar o cálculo de negócio; alterar regras não precisa afetar a apresentação; trocar o banco pode ser possível sem reescrever as outras camadas - desde que os contratos entre elas permaneçam estáveis.

EXEMPLO MVC

Uma lista de telefones pode servir de exemplo: a listbox exibida é a View, a coleção de números é o Model e o Controller contém a lógica que conecta os dois.

LIMITAÇÕES DO MVC

MVC é um bom projeto, mas adiciona complexidade e exige atenção ao planejamento inicial. Esse é um dilema geral do projeto OO: existe uma fronteira entre estrutura útil e arquitetura excessivamente pesada. É preciso decidir quanta complexidade vale a pena para o problema real.

The problem with previous paradigms is that the Model, View, and Controller used to be lumped together in a single entity. For example, a single object would have included all three of the components. With the MVC paradigm, these three components have separate and distinct interfaces. So if you want to change the user interface of an application, you only have to change the View. **Figure 10.1** illustrates what the MVC design looks like.

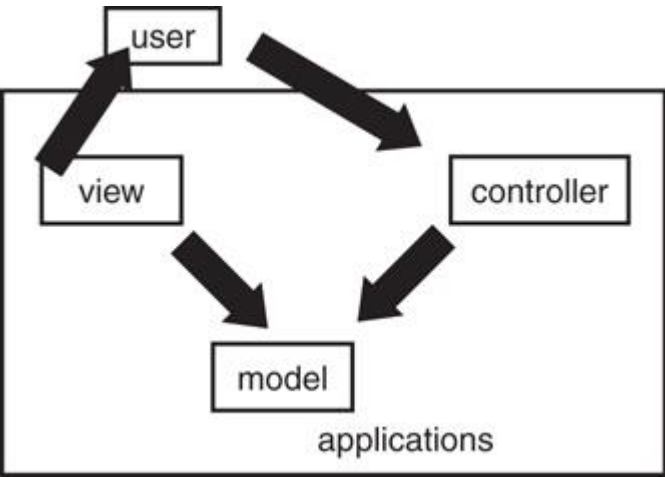


Figure 10.1 Model/View/Controller paradigm.

Remember that much of what we have been learning about object-oriented development has to do with interfaces versus implementation. As much as possible, we want to separate the interface from the implementation. We also want to separate interface from interface as much as possible. For example, we do not want to combine multiple interfaces that do not have anything to do with one another (or the solution to the problem at hand). The MVC was one of the early pioneers in this separation of interfaces. The MVC explicitly defines the interfaces between specific components pertaining to a very common and basic programming problem—the creation of user interfaces and their connection to the business logic and data behind them.

If you follow the MVC concept and separate the user interface, business logic, and data, your system will be much more flexible and robust. For example, assume that the user interface is on a client machine, the business logic is on an application server, and the data is located on a data server. Developing your application in this way would allow you to change the way the GUI looks without having an impact on the business logic or the data. Likewise, if your business logic changes and you calculate a specific field differently, you can change the business logic without having to change the GUI. And finally, if you want to swap databases and store your data differently, you can change the way the data is stored on the data server without affecting either the GUI or the business logic. This assumes, of course, that the interfaces between the three do not change.

MVC Example

One example is that of a listbox used in a user interface. Consider a GUI that includes a list of phone numbers. The listbox is the view, the phone list is the model, and the controller is the logic that binds the listbox to the phone list.

MVC Drawbacks

Although the MVC is a great design, it can be somewhat complex in that a lot of attention must be paid to the upfront design. This is a problem with object-oriented design in general—there is a fine line between a good design and a cumbersome design. The question remains: How much complexity should you build into the system with regard to a complete design?

TYPES OF DESIGN PATTERNS

Design Patterns features 23 patterns grouped into the three categories that follow. Most of the examples are written in C++, with some written in Smalltalk. The time of the book’s publication is indicative of the use of C++ and Smalltalk. The publication date of 1995 was right at the cusp of the Internet revolution and the corresponding popularity of the Java programming language. After the benefit of design patterns became apparent, many other books rushed in to fill the newly created market.

In any event, the actual language used is irrelevant. *Design Patterns* is inherently a design book, and the patterns can be implemented in any number of languages. The authors of the book divided the patterns into three categories:

- *Creational patterns* create objects for you, rather than having you instantiate objects directly. This gives your program more flexibility in deciding which objects need to be created for a given case.
- *Structural patterns* help you compose groups of objects into larger structures, such as complex user interfaces or accounting data.
- *Behavioral patterns* help you define the communication between objects in your system and how the flow is controlled in a complex program.

The following sections discuss one example from each of these categories to provide a flavor of design patterns. For a comprehensive list and description of individual design patterns, refer to the books listed at the end of this chapter.

Creational Patterns

The creational patterns consist of the following categories:

- Abstract factory
- Builder
- Factory method
- Prototype
- Singleton

As stated earlier, the scope of this chapter is to describe what a design pattern is—not to describe each and every pattern in the GoF book. Thus, we will cover a single pattern in each category. With this in mind, let’s consider an example of a creational pattern and look at the factory pattern.

The Factory Method Design Pattern

Creating, or instantiating, objects may well be one of the most fundamental concepts in object oriented programming. It goes without saying that you can’t use an object unless that object exists. When writing code, the most obvious way to instantiate an object is to use the `new` keyword.

To illustrate, let’s revisit the `Shape` example used throughout this book. Here we have the familiar parent class `Shape`, which is abstract, and the child class `Circle`, which is the concrete implementation. We instantiate a `Circle` class in the usual way by employing the `new` keyword:

```
abstract class Shape {  
  
}
```

TIPOS DE PADRÕES DE PROJETO

Design Patterns apresenta 23 padrões agrupados em três categorias. Embora os exemplos originais usem principalmente C++ e Smalltalk, a linguagem é secundária: padrões descrevem projeto e podem ser implementados em diversas linguagens.

As três categorias são:

PADRÕES CRIACIONAIS - Criam objetos de forma indireta, em vez de exigir instanciação concreta em todos os pontos do programa. Isso aumenta a flexibilidade para decidir quais objetos devem existir.

PADRÕES ESTRUTURAIS - Ajudam a compor grupos de objetos em estruturas maiores, como interfaces complexas ou modelos de dados.

PADRÕES COMPORTAMENTAIS - Organizam a comunicação entre objetos e o fluxo de controle do sistema.

O capítulo apresenta um exemplo de cada categoria.

PADRÕES CRIACIONAIS

O grupo criacional inclui Abstract Factory, Builder, Factory Method, Prototype e Singleton.

Como exemplo, o livro escolhe Factory Method.

O PADRÃO FACTORY METHOD

Criar/instanciar objetos é uma operação fundamental em POO. A forma direta é usar `new`. Retomando `Shape` e `Circle`, normalmente poderíamos escrever `Circle circle = new Circle();`.

Isso funciona, mas espalhar a lógica de criação por muitos pontos do sistema pode aumentar o acoplamento. O padrão Factory centraliza essa responsabilidade.

```
class Circle extends Shape {  
  
}
```

```
Circle circle = new Circle();
```

Although this code certainly works, there may be many other places in your code where you need to instantiate a `Circle`, or any other `Shape` for that matter. In many cases, you will have specific object creation parameters that need to be handled each time you create a `Shape`. As a result, any time you change the way objects are created, the code must be changed in every location where a `Shape` object is instantiated. The code is highly coupled because a change in one location necessitates code changes in potentially many other locations. Another problem with this approach is that it exposes the object creation logic to the programmers using the classes.

To remedy these situations, we can implement a factory method. In short, the factory method is responsible for encapsulating all instantiation so that it is uniform across the implementation. You use the factory to instantiate, and the factory is responsible for instantiating properly.

Factory Method Pattern

The fundamental intent of the factory method pattern is to create objects without having to specify the exact class—in effect, using interfaces to create new types of objects. To illustrate how to implement a factory pattern, let’s create a factory for the `Shape` class example. The class diagram in [Figure 10.2](#) helps visualize how the various classes in the example interact.

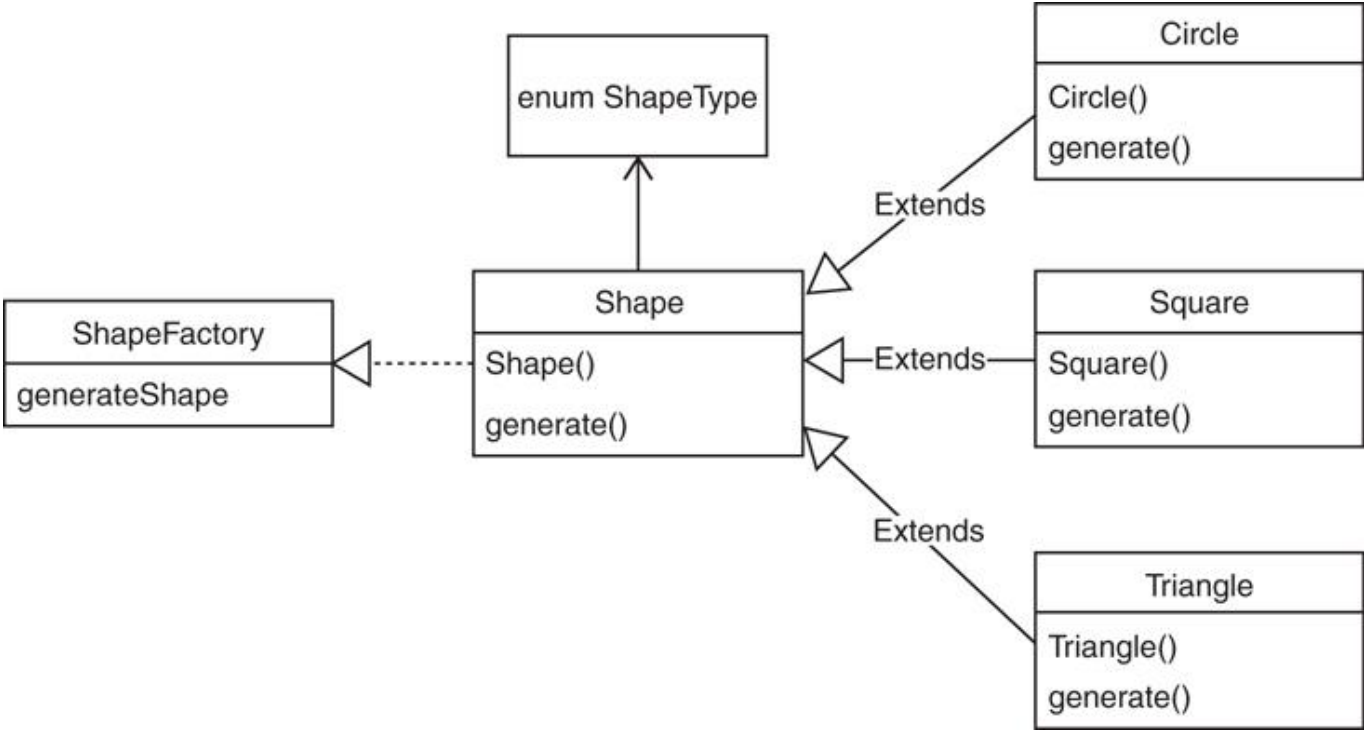


Figure 10.2 Creating a factory for the `Shape` class.

In some ways, you can think of a factory as a wrapper. Consider the fact that there may be some significant logic involved in instantiating an object and you don’t want the programmer (user) to be concerned with this logic. It is almost like the concept of an accessor method (getters and setters) when the retrieval of a value is inside some logic (like when a password is required). Using a factory method is useful when you don’t know ahead of time which specific class you might need. For example, you may know that a shape is required, but you don’t know the

`Circle` pode ser instanciado diretamente com `new Circle()`, mas se muitos pontos do sistema criarem `Shapes` e a lógica de construção mudar, todos esses locais precisarão ser modificados. Além disso, detalhes de criação ficam expostos aos usuários da API.

Uma `Factory Method` encapsula a instanciação e oferece um ponto uniforme para criação. O cliente solicita um objeto à fábrica; a fábrica conhece os detalhes necessários.

PADRÃO FACTORY METHOD

A intenção fundamental é criar objetos sem obrigar o cliente a especificar diretamente a classe concreta em cada ponto, utilizando abstrações para escolher o tipo apropriado.

A Figura 10.2 mostra uma fábrica para a hierarquia `Shape`.

Figura 10.2 - Criando uma fábrica para `Shape`.

A fábrica pode ser vista como um wrapper para a lógica de criação. Isso é útil quando a construção exige regras que não devem preocupar o consumidor da classe.

Também é útil quando sabemos que precisamos de um `Shape`, mas ainda não sabemos qual forma concreta. Todos os tipos criados devem pertencer a uma hierarquia ou contrato comum. Se a instância concreta já fosse conhecida e fixa, poderíamos simplesmente fornecê-la por construtor ou setter.

specific shape (at least not yet). With this in mind, all possible classes must be in the same hierarchy; that is, all the classes in this example must be a subclass of `Shape`. In fact, a factory is used precisely because you don't know what you need, allowing you to add some of the classes later. If you knew what you needed, you could simply "inject" the instance via a constructor or a setter method.

Basically, this is the definition of polymorphism.

We create an `enum` to contain the types of shapes. In this case, we will define `CIRCLE`, `SQUARE`, and `TRIANGLE`.

```
enum ShapeType {  
    CIRCLE, SQUARE, TRIANGLE  
}
```

We define the `Shape` class as abstract with just a constructor and an abstract method called `generate()`.

```
abstract class Shape {  
  
    private ShapeType sType = null;  
  
    public Shape(ShapeType sType) {  
        this.sType = sType;  
    }  
  
    // Generate the shape  
    protected abstract void generate();  
  
}
```

The child classes, `CIRCLE`, `SQUARE`, and `TRIANGLE`, extend the `Shape` class, identify themselves, and provide the concrete implementation of the `generate()` method.

[Click here to view code image](#)

```
class Circle extends Shape {  
  
    Circle() {  
        super(ShapeType.CIRCLE);  
        generate();  
    }  
  
    @Override  
    protected void generate() {  
        System.out.println("Generating a Circle");  
    }  
}
```

```
class Square extends Shape {  
  
    Square() {  
        super(ShapeType.SQUARE);  
        generate();  
    }  
  
    @Override  
    protected void generate() {  
        System.out.println("Generating a Square");  
    }  
}
```

```
class Triangle extends Shape {  
  
    Triangle() {  
        super(ShapeType.TRIANGLE);  
    }  
}
```

Esse comportamento é uma aplicação de polimorfismo.

O exemplo cria um `enum ShapeType` com `CIRCLE`, `SQUARE` e `TRIANGLE`. `Shape` é abstrata, guarda o tipo e declara um método abstrato `generate()`.

As classes `Circle`, `Square` e `Triangle` estendem `Shape`. Cada construtor informa seu `ShapeType` à superclasse e chama `generate()`; cada classe fornece sua própria implementação de `generate()`, imprimindo uma mensagem correspondente.

O código Java completo aparece no lado original. A estrutura é importante: todas as classes concretas compartilham o mesmo tipo abstrato, permitindo que a fábrica retorne `Shape` independentemente da especialização criada.


```
        generate();
    }

    @Override
    protected void generate() {
        System.out.println("Generating a Triangle");
    }
}
```

The `ShapeFactory` class, as the name implies, is the actual factory. Focus on the `generate()` method. While a `Factory` provides many advantages, note that the `generate()` method is the only location within the application that actually instantiates a `Shape`.

[Click here to view code image](#)

```
class ShapeFactory {
    public static Shape generateShape(ShapeType sType) {
        Shape shape = null;
        switch (sType) {

            case CIRCLE:
                shape = new Circle();
                break;

            case SQUARE:
                shape = new Square();
                break;

            case TRIANGLE:
                shape = new Triangle();
                break;

            default:
                // throw an exception
                break;
        }
        return shape;
    }
}
```

The traditional approach to instantiating these individual objects is to have the programmer directly instantiate the objects using the `new` keyword as follows:

[Click here to view code image](#)

```
public class TestFactoryPattern {
    public static void main(String[] args) {

        Circle circle = new Circle();
        Square square = new Square();
        Triangle triangle = new Triangle();

    }
}
```

However, properly using the factory requires that the programmer use the `ShapeFactory` class to obtain any `Shape` object:

[Click here to view code image](#)

```
public class TestFactoryPattern {
    public static void main(String[] args) {

        ShapeFactory.generateShape(ShapeType.CIRCLE);
        ShapeFactory.generateShape(ShapeType.SQUARE);
        ShapeFactory.generateShape(ShapeType.TRIANGLE);
    }
}
```

A classe `ShapeFactory` é a fábrica propriamente dita. Seu método `generateShape(ShapeType sType)` é o único local da aplicação que instancia `Shapes` concretos.

O método escolhe o tipo por switch:

- CIRCLE -> new Circle();
- SQUARE -> new Square();
- TRIANGLE -> new Triangle().

Depois retorna a referência como `Shape`.

Sem a fábrica, o cliente escreveria diretamente `Circle circle = new Circle()`, `Square square = new Square()` etc. Com a fábrica, solicita:

```
ShapeFactory.generateShape(ShapeType.CIRCLE);
ShapeFactory.generateShape(ShapeType.SQUARE);
ShapeFactory.generateShape(ShapeType.TRIANGLE);
```

Assim, detalhes de construção ficam concentrados em um único componente. Alterações na forma de instanciar objetos tendem a afetar menos pontos do sistema.

```
}  
}
```

Structural Patterns

Structural patterns are used to create larger structures from groups of objects. The following seven design patterns are members of the structural category:

- Adapter
- Bridge
- Composite
- Decorator
- Facade
- Flyweight
- Proxy

As an example from the structural category, let’s take a look at the adapter pattern. The adapter pattern is also one of the most important design patterns. This pattern is a good example of how the implementation and interface are separated.

The Adapter Design Pattern

The adapter pattern is a way for you to create a different interface for a class that already exists. The adapter pattern basically provides a class wrapper. In other words, you create a new class that incorporates (wraps) the functionality of an existing class with a new and—ideally—better interface. A simple example of a wrapper is the Java class `Integer`. The `Integer` class wraps a single `Integer` value inside it. You might wonder why you would bother to do this. Remember that in an object-oriented system, everything is an object. In Java, primitives, such as ints, floats, and so on, are not objects. When you need to perform functions on these primitives, such as conversions, you need to treat them as objects. Thus, you create a wrapper object and “wrap” the primitive inside it. Thus, you can take a primitive like the following:

```
int myInt = 10;
```

and wrap it in an `Integer` object:

```
Integer myIntWrapper = new Integer (myInt);
```

Now you can do a conversion, so you can treat it as a string:

```
String myString = myIntWrapper.toString();
```

This wrapper enables you to treat the original integer as an object, thus providing all the advantages of an object.

As for the adapter pattern itself, consider the example of a mail tool interface. Let’s assume you have purchased some code that provides all the functionality you need to implement a mail client. This tool provides everything you want in a mail client, except you would like to change the interface slightly. In fact, all you want to do is change the API to retrieve your mail.

The following class provides a very simple example of a mail client for this example:

[Click here to view code image](#)

```
package MailTool;
```

PADRÕES ESTRUTURAIS

Os padrões estruturais criam estruturas maiores a partir de grupos de objetos. O GoF lista: Adapter, Bridge, Composite, Decorator, Facade, Flyweight e Proxy.

Como exemplo, o capítulo usa Adapter, um padrão importante para separar interface de implementação.

O PADRÃO ADAPTER

Adapter cria uma interface diferente para uma classe já existente. Em essência, construímos um wrapper que incorpora a funcionalidade original e apresenta uma interface nova, idealmente mais adequada ao contexto.

Um exemplo simples de wrapper em Java é Integer envolvendo um valor int. Tipos primitivos não são objetos, e um wrapper permite tratá-los por meio de uma interface de objeto e utilizar operações adicionais. O exemplo histórico da página mostra um int sendo envolvido em Integer e convertido para String.

No padrão Adapter propriamente dito, imagine comprar uma biblioteca que oferece toda a funcionalidade necessária para um cliente de e-mail, mas cuja API para recuperar mensagens não é a interface que desejamos expor. Podemos criar um adaptador que traduza nossa interface preferida para a interface da biblioteca existente.

A página inicia a classe de exemplo do pacote MailTool, cujo código será adaptado nas páginas seguintes.

```
public class MailTool {
    public MailTool () {
    }
    public int retrieveMail() {

        System.out.println ("You've Got Mail");

        return 0;
    }
}
```

When you invoke the `retrieveMail()` method, your mail is presented with the very original greeting “You’ve Got Mail.” Now let’s suppose you want to change the interface in all your company’s clients from `retrieveMail()` to `getMail()`. You can create an interface to enforce this:

```
package MailTool;
interface MailInterface {
    int getMail();
}
```

You can now create your own mail tool that wraps the original tool and provide your own interface:

[Click here to view code image](#)

```
package MailTool;
class MyMailTool implements MailInterface {
    private MailTool yourMailTool;
    public MyMailTool () {
        yourMailTool= new MailTool();
        setYourMailTool(yourMailTool);
    }
    public int getMail() {
        return getYourMailTool().retrieveMail();
    }
    public MailTool getYourMailTool() {
        return yourMailTool ;
    }
    public void setYourMailTool(MailTool newYourMailTool) {
        yourMailTool = newYourMailTool;
    }
}
```

Inside this class, you create an instance of the original mail tool that you want to retrofit. This class implements `MailInterface`, which will force you to implement a `getMail()` method. Inside this method, you literally invoke the `retrieveMail()` method of the original mail tool.

To use your new class, you instantiate your new mail tool and invoke the `getMail()` method.

[Click here to view code image](#)

```
package MailTool;
public class Adapter
{
    public static void main(String[] args)
    {
        MyMailTool myMailTool = new MyMailTool();

        myMailTool.getMail();
    }
}
```

When you invoke the `getMail()` method, you are using this new interface to invoke the `retrieveMail()` method from the original tool. This is a very simple example; however, by

A classe `MailTool` original fornece `retrieveMail()`, que exibe a mensagem “You’ve Got Mail”. Suponha que a empresa queira padronizar a interface e utilizar `getMail()` em todos os clientes.

Podemos criar `MailInterface` com `getMail()` e depois um adaptador `MyMailTool` que implementa essa interface e contém internamente uma instância do `MailTool` original.

`MyMailTool.getMail()` simplesmente delega para `yourMailTool.retrieveMail()`. Assim, o código existente não precisa ser modificado; uma nova camada traduz a interface desejada para a interface antiga.

Para usar a solução, o cliente cria `MyMailTool` e chama `getMail()`. Internamente, o adaptador chama `retrieveMail()` no componente original.

Esse é um exemplo simples do Adapter: o wrapper preserva a funcionalidade existente, mas oferece uma nova API e pode acrescentar funcionalidades próprias.

creating this wrapper, you can enhance the interface and add your own functionality to the original class.

The concept of an adapter is quite simple, but you can create new and powerful interfaces using this pattern.

Behavioral Patterns

The behavioral patterns consist of the following categories:

- Chain of response
- Command
- Interpreter
- Iterator
- Mediator
- Memento
- Observer
- State
- Strategy
- Template method
- Visitor

As an example from the behavioral category, let’s take a look at the iterator pattern. This is one of the most commonly used patterns and is implemented by several programming languages.

The Iterator Design Pattern

Iterators provide a standard mechanism for traversing a collection, such as a vector. Functionality must be provided so that each item of the collection can be accessed one at a time. The iterator pattern provides information hiding, keeping the internal structure of the collection secure. The iterator pattern also stipulates that more than one iterator can be created without interfering with each other. Java provides its own implementation of an iterator. The following code creates a vector and then inserts a number of strings into it:

Click here to view code image

```
package Iterator;

import java.util.*;

public class Iterator {
    public static void main(String args[]) {

        // Instantiate an ArrayList.
        ArrayList<String> names = new ArrayList();

        // Add values to the ArrayList
        names.add(new String("Joe"));
        names.add(new String("Mary"));
        names.add(new String("Bob"));
        names.add(new String("Sue"));

        //Now Iterate through the names
        System.out.println("Names:");
```

Ao criar um wrapper, podemos enriquecer a interface sem alterar a classe original. O conceito de Adapter é simples, mas permite construir interfaces novas e poderosas sobre código existente.

PADRÕES COMPORTAMENTAIS

A categoria comportamental inclui Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method e Visitor.

Como exemplo, o livro apresenta Iterator, um dos padrões mais usados e já incorporado em várias linguagens.

O PADRÃO ITERATOR

Iterators fornecem um mecanismo padronizado para percorrer uma coleção, acessando seus elementos um de cada vez. O padrão também promove ocultação de informação: o consumidor não precisa conhecer a estrutura interna da coleção. Além disso, vários iteradores podem existir simultaneamente sem interferir uns nos outros.

Java oferece implementações próprias para esse conceito. O exemplo cria um ArrayList<String> chamado names e adiciona Joe, Mary, Bob e Sue. Em seguida, a aplicação chama um método iterate() para percorrer a coleção.

```
        iterate(names);
    }

    private static void iterate(ArrayList<String> arl) {
        for(String listItem : arl) {
            System.out.println(listItem.toString());
        }
    }
}
```

Then we create an enumeration so that we can iterate through it. The method `iterate()` is provided to perform the iteration functionality. In this method, we use the Java enumeration method `hasMoreElements()`, which traverses the vector and lists all the names.

ANTIPATTERNS

Although a design pattern evolves from experiences in a positive manner, *antipatterns* can be thought of as collections of experiences that have gone awry. It is well documented that most software projects are ultimately deemed unsuccessful. In fact, as indicated in the article “Creating Chaos” by Johnny Johnson, fully one-third of all projects are cancelled outright. It would seem obvious that many of these failures are caused by poor design decisions.

The term *antipattern* derives from the fact that design patterns are created to proactively solve a specific type of problem. An antipattern, on the other hand, is a reaction to a problem and is gleaned from bad experiences. In short, whereas design patterns are based on solid design practices, antipatterns can be thought of as practices to avoid.

In the November 1995 *C++ Report*, Andrew Koenig described two facets of antipatterns:

- Those that describe a bad solution to a problem, which result in a bad situation.
- Those that describe how to get out of a bad situation and how to proceed from there to a good solution.

Many people believe that antipatterns are more useful than design patterns. This is because antipatterns are designed to solve problems that have already occurred. This boils down to the concept of root-cause analysis. A study can be conducted with data that might indicate why the original design, perhaps an actual design pattern, did not succeed. It might be said that antipatterns emerge from the failure of previous solutions. Thus, antipatterns have the benefit of hindsight.

For example, in his article “Reuse Patterns and Antipatterns,” Scott Ambler identifies a pattern called a *robust artifact* and defines it as follows:

An item that is well-documented, built to meet general needs instead of project-specific needs, thoroughly tested, and has several examples to show how to work with it. Items with these qualities are much more likely to be reused than items without them. A Robust Artifact is an item that is easy to understand and work with.

However, there are certainly many situations when a solution is declared reusable and then no one ever reuses it. Thus, to illustrate an antipattern, he writes:

Someone other than the original developer must review a Reuseless Artifact to determine whether or not anyone might be interested in it. If so, the artifact must be reworked to become a Robust Artifact.

O método `iterate()` percorre `ArrayList` com a sintaxe de iteração da linguagem e imprime cada item. O objetivo é demonstrar que a forma de percorrer a coleção fica padronizada e separada de detalhes internos.

ANTIPADRÕES

Se padrões de projeto registram experiências bem-sucedidas, antipadrões registram experiências que deram errado. Muitos projetos de software fracassam ou são cancelados, e decisões ruins de projeto contribuem para isso.

Um padrão procura antecipar e resolver um problema recorrente. Um antipadrão identifica uma solução recorrente que produz uma situação ruim e, em muitos casos, descreve como sair dela e alcançar uma alternativa melhor.

Andrew Koenig destacou duas facetas:
- soluções ruins que levam a situações ruins;
- orientações para sair de uma situação ruim e evoluir para uma boa solução.

Antipadrões podem ser especialmente úteis porque analisam problemas já ocorridos, permitindo estudar causas e consequências com retrospectiva.

Scott Ambler usa o contraste entre Robust Artifact e Reuseless Artifact. Um artefato robusto é bem documentado, atende necessidades gerais, é extensivamente testado e possui exemplos de uso. Um artefato considerado “reutilizável” mas que ninguém consegue ou deseja reutilizar precisa ser revisto e transformado em algo realmente compreensível e útil.

Thus, antipatterns lead to the revision of existing designs, and the continuous refactoring of those designs until a workable solution is found.

Some Good Examples of Antipatterns

- Singleton
- Service locator
- Magic strings/magic numbers
- Interface bloat
- Coding by exception
- Error hiding/swallowing

CONCLUSION

In this chapter, we explored the concept of design patterns. Patterns are part of everyday life, and this is just the way you should be thinking about object-oriented designs. As with many things pertaining to information technology, the roots for solutions are founded in real-life situations.

Although this chapter covered design patterns only briefly, you should explore this topic in greater detail by picking up one of the books referenced at the end of this chapter.

REFERENCES

Alexander, Christopher, et al. 1977. *A Pattern Language: Towns, Buildings, Construction*. Cambridge, UK: Oxford University Press.

Ambler, Scott. “Reuse Patterns and Antipatterns.” *2000 Software Development Magazine*.

Gamma, Erich, et al. 1995. *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston, MA: Addison-Wesley.

Grand, Mark. 2002. *Patterns in Java: A Catalog of Reusable Design Patterns Illustrated with UML*, Second Edition, volume 1. Hoboken, NJ: Wiley.

Jaworski, Jamie. 1999. *Java 2 Platform Unleashed*. Indianapolis, IN: Sams Publishing.

Johnson, Johnny. “Creating Chaos.” *American Programmer*, July 1995.

Larman, Craig. 2004. *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*, Third Edition. Hoboken, NJ: Wiley.

Assim, antipadrões podem conduzir à revisão e refatoração contínua de projetos até que uma solução adequada seja alcançada.

EXEMPLOS DE ANTIPADRÕES

A página lista, entre outros: Singleton, Service Locator, magic strings/magic numbers, interface bloat, coding by exception e ocultação/engolimento de erros. A classificação de algumas técnicas como antipadrão depende do contexto e das consequências de uso.

CONCLUSÃO

Este capítulo apresentou o conceito de padrões de projeto. Padrões fazem parte da vida cotidiana e podem orientar nossa forma de pensar sobre sistemas OO. Muitas soluções de software possuem raízes em problemas e estruturas observados fora da computação.

O assunto foi apresentado apenas de maneira introdutória; as referências recomendadas permitem aprofundamento.

REFERÊNCIAS

Alexander et al. A Pattern Language: Towns, Buildings, Construction. 1977.

Ambler, Scott. “Reuse Patterns and Antipatterns.” 2000.

Gamma et al. Design Patterns: Elements of Reusable Object-Oriented Software. 1995.

Grand, Mark. Patterns in Java. 2002.

Jaworski, Jamie. Java 2 Platform Unleashed. 1999.

Johnson, Johnny. “Creating Chaos.” 1995.

Larman, Craig. Applying UML and Patterns. 2004.

[Os títulos bibliográficos foram preservados no idioma original.]

11. Avoiding Dependencies and Highly Coupled Classes

As presented in [Chapter 1](#), “[Introduction to Object-Oriented Concepts](#),” the traditional criteria of classical object-oriented programming are encapsulation, inheritance, and polymorphism. Theoretically, to consider a programming language as an object-oriented language, it must follow these three principles. In addition, as also covered in [Chapter 1](#), I like to include composition.

Thus, when I teach object-oriented programming, my list of fundamental concepts looks like this:

- Encapsulation
- Inheritance
- Polymorphism
- Composition

Tip

Perhaps I should add interfaces to this list, but I have always considered interfaces to be a specific type of inheritance.

Adding composition to this list is even more important in today’s development environment because of the debate over how to use inheritance appropriately. Concerns about using inheritance are not a recent phenomenon. In the past several years, this debate has heated up. Many developers I talk to advocate for using composition rather than inheritance (often called composition over inheritance). In fact, some avoid using inheritance at all, or at least limit the use of inheritance to a single hierarchical level.

The reason for focusing on how to use inheritance revolves around the issue of coupling. The arguments for using inheritance are, most certainly, reusability, extensibility, and polymorphism; however, inheritance can cause problems by creating dependencies between classes—in effect, coupling the classes. These dependencies create potential problems for maintenance and testing. [Chapter 7](#), “[Mastering Inheritance and Composition](#),” discussed how inheritance might actually weaken encapsulation, which seems counterintuitive because they are both fundamental concepts. Nevertheless, this is actually part of the fun, and requires that we really think about how we should use inheritance.

Caution

Be aware that I am not advocating avoiding inheritance. The discussion here is actually about avoiding dependencies and highly coupled classes. When to use inheritance is an important part of this discussion.

This debate leads to the following question: if not inheritance, then what? The short answer is to use composition. This should not be surprising because throughout the book I contend that there are really only two ways to reuse classes: using inheritance and using composition. You can either create a child from a parent class via inheritance or contain one class within another class using composition.

11. EVITANDO DEPENDÊNCIAS E CLASSES FORTEMENTE ACOPLADAS

Como apresentado no Capítulo 1, os critérios clássicos de POO são encapsulamento, herança e polimorfismo. O autor também inclui composição. Assim, sua lista fundamental é:

- Encapsulamento
- Herança
- Polimorfismo
- Composição

Interfaces poderiam ser acrescentadas, embora o autor as considere uma forma específica de herança/contrato.

A composição ganha ainda mais importância no debate moderno sobre uso apropriado de herança. Muitos desenvolvedores preferem “composição sobre herança” e alguns limitam a profundidade das hierarquias.

A principal preocupação é acoplamento. Herança oferece reuso, extensibilidade e polimorfismo, mas também cria dependências entre classes. Essas dependências podem dificultar manutenção e testes. No Capítulo 7 vimos inclusive como herança pode enfraquecer encapsulamento.

Isso não significa que o livro defenda evitar herança. O objetivo é evitar dependências desnecessárias e classes excessivamente acopladas, escolhendo a ferramenta correta.

Se não usamos herança, uma alternativa é composição. Em termos gerais, podemos reutilizar classes herdando de uma classe pai ou contendo/dependendo de outros objetos.

If, as some people advocate, inheritance is to be avoided, why do we spend time learning it? The answer is simple: A lot of code utilizes inheritance. As most developers soon come to understand, the vast majority of the code encountered appears in maintenance mode. Thus, it is imperative to understand how to fix, enhance, and maintain code written using inheritance. You may even write some new code using inheritance. In short, a programmer needs to cover all the possible bases and learn the entire developers’ toolkit. However, this also means that we have to keep adding tools to that kit as well as rethink how we use them.

Again, please understand that I am not making any value judgments here. I am not claiming that inheritance is problematic and to avoid it. What I am saying is that it is important to fully grasp how inheritance is used, carefully study alternative ways of design, and then decide for yourself. Thus, the intent of the examples in this chapter is not necessarily to describe the optimal way to design your classes; they are educational exercises meant to get you thinking about the issues associated with deciding between inheritance and composition. Remember that it is important for all technologies to evolve, keep the good, and refine the not-so good.

Moreover, composition poses its own coupling issues. In [Chapter 7](#) I discussed the various types of composition: associations and aggregations. Aggregations are objects that are embedded in other objects (created with the `new` keyword) while associations are objects that are passed into other objects via a parameter list. Because aggregations are embedded in objects, they are highly coupled, which we want to avoid.

Therefore, while inheritance may have obtained a reputation as encouraging highly coupled classes, composition (using aggregations) also can create highly coupled classes. Let’s revisit the stereo component example used in [Chapter 9](#), “[Building Objects and Object-Oriented Design](#),” to bring all of these concepts together in a specific example.

Creating a stereo with aggregations can be likened to creating a boombox, which is a product that has all the components embedded inside a single unit. In many situations, this can be very convenient. It can be picked up, moved easily, and requires no special assembly. However, this design can also lead to many problems. If one component, say the MP3 player, breaks, you must take in the entire unit for repair. Even worse, many problems may arise to render the entire boombox unusable, such as an electrical issue.

Creating a stereo with associations can mitigate many of the problems encountered with aggregations. Think of a component stereo system as a bunch of associations connected by patch cords (or wireless). In this design, there is a central object called a receiver connected to several other objects such as speakers, CD players, even turntables and cassette players. In fact, think of this as a vendor-neutral solution because we can simply obtain a component off the shelf, which is a major advantage.

In this situation, if the CD player breaks, you simply disconnect it, providing the opportunity to either fix the CD player (while still enjoying the use of the other components) or swapping it out with a new CD player that works. This is the advantage of using associations and keeping the coupling between classes to a minimum.

Tip

As pointed out in [Chapter 9](#), although highly coupled classes are generally frowned upon, there might be times when you are willing to accept the risk of a highly coupled design. The boombox is one such example. Despite the fact that it has a highly coupled design, it is sometimes the preferred choice.

Now that we have reviewed the coupling issues of both inheritance and composition, let’s explore examples of some highly coupled designs using both inheritance and composition. As I

Se alguns desenvolvedores procuram evitar herança, por que estudá-la? Porque enorme quantidade de código existente a utiliza. A maior parte do trabalho profissional envolve manutenção, correção e evolução de sistemas já escritos. Um desenvolvedor precisa conhecer todo o conjunto de ferramentas, inclusive técnicas que escolheria usar menos em código novo.

O objetivo aqui não é declarar herança “ruim”, mas entender profundamente seus efeitos, estudar alternativas e escolher conscientemente.

Composição também pode gerar acoplamento. O livro distingue agregações, em que objetos são criados e embutidos internamente, de associações, em que dependências podem ser fornecidas externamente por parâmetros. Quando um objeto instancia diretamente seus componentes com `new`, o vínculo com implementações concretas pode ser forte.

Retomando o aparelho de som: uma boombox integra todos os componentes em uma única unidade. É conveniente e portátil, mas se o MP3 player ou a alimentação quebrar, todo o aparelho pode ficar indisponível.

Um sistema de componentes separados usa associações: receiver, alto-falantes, CD player etc. conectam-se por cabos ou interfaces. Se um componente falha, podemos removê-lo, repará-lo ou substituí-lo sem perder o restante do sistema. Também podemos misturar fornecedores.

Acoplamento forte nem sempre é proibido. Às vezes, como na boombox, a conveniência justifica o risco. O ponto é compreender o compromisso.

A seguir, o capítulo evolui exemplos de herança e composição até chegar a dependency injection como técnica para reduzir acoplamento.

often do in the classroom, we will iterate through these examples until we use a technique called dependency injection to mitigate the coupling issues.

COMPOSITION VERSUS INHERITANCE AND DEPENDENCY INJECTION

To begin, we can focus on how to take an inheritance model (gleaned from examples often used in this book) and redesign it, not with inheritance but with composition. The second example shows how we can redesign with composition—albeit using aggregation, which is not necessarily an optimal solution. The third example shows how to avoid aggregations and design with associations instead—the concept of *dependency injection*.

1) Inheritance

Whether or not you buy into the argument of composition over inheritance, let’s begin by presenting a straightforward example of inheritance and explore how it might otherwise be implemented using composition, revisiting the mammal example used throughout the book. In this case, we introduce a bat—a mammal that can fly, as seen in [Figure 11.1](#).

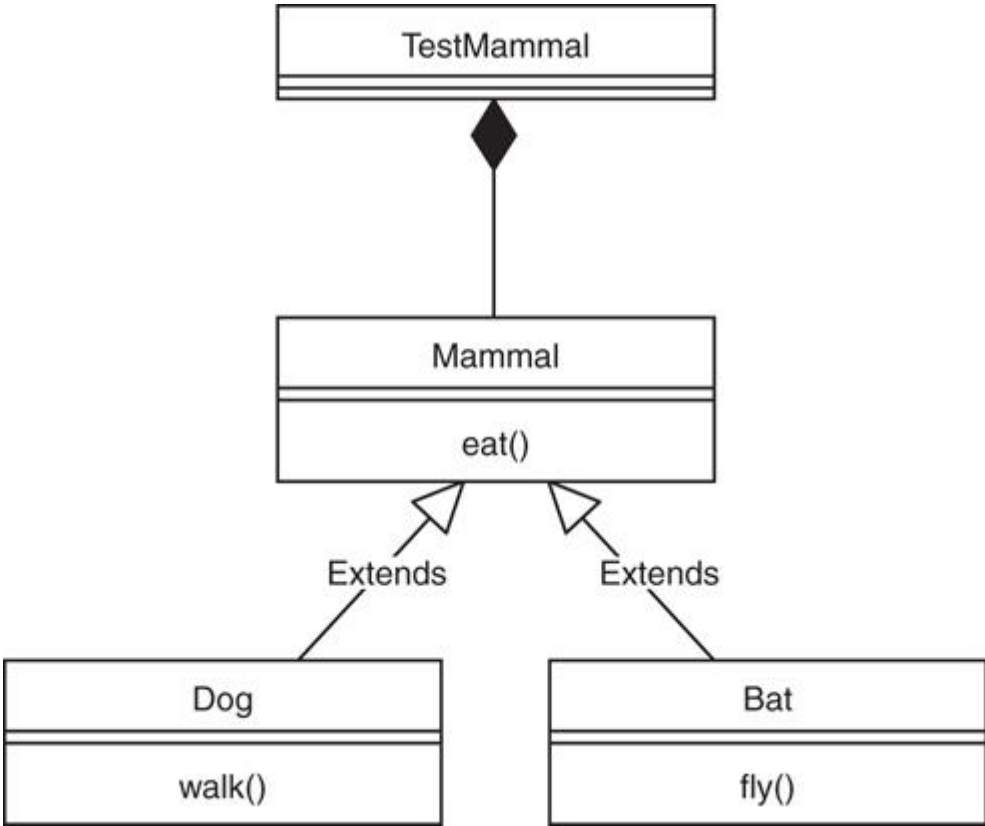


Figure 11.1 Using inheritance

to create mammals.

In this example specifically, inheritance appears to be the obvious choice. Creating a `Dog` class that inherits from `Mammal` is a slam dunk— isn’t it? Look at the following code, which utilizes inheritance in this manner:

[Click here to view code image](#)

```
class Mammal {
    public void eat () {System.out.println("I am Eating");}
}
class Bat extends Mammal {
    public void fly () {System.out.println("I am Flying");}
```

COMPOSIÇÃO VERSUS HERANÇA E INJEÇÃO DE DEPENDÊNCIA

O capítulo começa com um modelo baseado em herança, depois o redesenha com composição por agregação e finalmente substitui agregações por associações usando dependency injection.

1) HERANÇA

Retomemos o exemplo de mamíferos e acrescentemos um morcego, mamífero capaz de voar. A Figura 11.1 mostra Mammal como superclasse de Dog e Bat.

Figura 11.1 - Usando herança para criar mamíferos.

À primeira vista, a escolha parece natural: Dog é Mammal e Bat é Mammal. O código define eat() em Mammal, walk() em Dog e fly() em Bat. O programa TestMammal cria Fido e um morcego e executa os comportamentos herdados e específicos.

```
}
class Dog extends Mammal {
    public void walk () {System.out.println("I am Walking");}
}
public class TestMammal {

    public static void main(String args[]) {

        System.out.println("Composition over Inheritance");

        System.out.println("\nDog");
        Dog fido = new Dog();
        fido.eat();
        fido.walk();
        System.out.println("\nBat");
        Bat brown = new Bat();
        brown.eat();
        brown.fly();
    }
}
```

In this design, a `Mammal` has a single behavior, `eat()`, assuming that all mammals must eat. However, we start to see the problem with inheritance immediately when we add two `Mammal` subclasses, `Bat` and `Dog`. While a dog can walk, not all mammals walk. In addition, while a bat can indeed fly, not all mammals fly. So the question is, where do these methods go? Just like in our earlier penguin example, because not all birds fly, deciding where to place methods in an inheritance hierarchy can be tricky. Separating the `Mammal` class into `FlyingMammals` and `WalkingMammals` is not a very elegant solution because this is only the tip of the proverbial iceberg. Some mammals can swim, some mammals even lay eggs. Moreover, there are likely countless other behaviors that individual mammal species possess, and it might be impractical to create a separate class for all of these behaviors. Thus, rather than approaching this design as an *is-a* relationship, perhaps we should explore it using a *has-a* relationship.

2) Composition

In this strategy, rather than embedding the behaviors in the classes themselves, we create individual classes for each behavior. Therefore, rather than placing behaviors in an inheritance hierarchy, we can create classes for each behavior and then build individual mammals by including just the behaviors that they require (via aggregation).

Thus, we create a class called `Walkable` and a class called `Flyable`, as seen in [Figure 11.2](#).

No modelo, `Mammal` contém `eat()`, assumindo que todos os mamíferos comem. Mas o problema aparece rapidamente quando adicionamos comportamentos específicos. Cães caminham, mas isso não caracteriza todos os mamíferos. Morcegos voam, mas a maioria dos mamíferos não.

Criar hierarquias separadas como `FlyingMammals` e `WalkingMammals` não escala bem. Alguns mamíferos nadam, alguns põem ovos e inúmeras espécies possuem combinações particulares. Tentar representar cada comportamento por uma camada de herança pode gerar uma árvore complexa.

Talvez, em vez de modelar tudo como “é-um”, possamos usar relações “tem-um”.

2) COMPOSIÇÃO

Nessa estratégia, comportamentos são transformados em classes independentes. Em vez de inseri-los na hierarquia, criamos `Walkable`, `Flyable` etc. e construímos cada mamífero agregando apenas os comportamentos de que necessita.

A Figura 11.2 mostra essa composição.

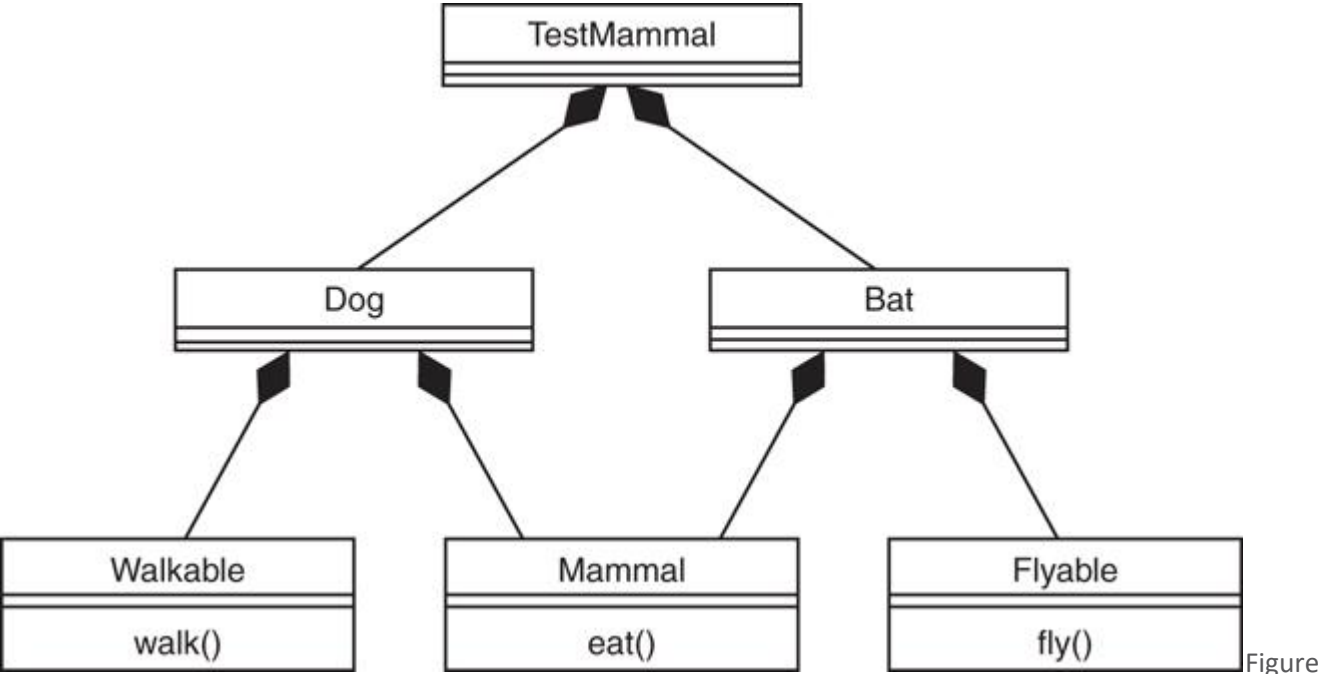
Figura 11.2 - Usando composição para criar mamíferos.

O código mantém `Mammal` com `eat()`, mas `Dog` e `Bat` agora obtêm seus comportamentos por agregação. Existem classes `Walkable` e `Flyable`. `Dog` contém objetos `Mammal` e `Walkable`; `Bat` contém `Mammal` e `Flyable`.

CUIDADO

Esse é apenas um passo intermediário. Embora substitua herança por composição, ainda usa agregação com `new` dentro das classes, portanto continua havendo acoplamento significativo entre implementações concretas.

No programa de teste, Fido delega alimentação ao objeto `dog` e caminhada ao objeto `walker`. O morcego delega alimentação ao objeto `bat` e voo ao objeto `flyer`. A próxima página continua a execução e analisa as limitações desse desenho.



11.2 Using composition to create mammals.

For example, look at the following code. We still have the `Mammal` class with its `eat()` method, and we still have the `Dog` and `Bat` classes. The major design difference here is that the `Dog` and `Bat` classes obtain their behaviors via aggregation using composition.

Caution

Be aware that the term aggregation is used in the preceding paragraph. This example illustrates how composition can be used in lieu of inheritance; however, in this example, we are using aggregation, which still contains significant coupling. Thus, consider this an intermediate, educational step moving toward the next example using interfaces.

[Click here to view code image](#)

```
class Mammal {
    public void eat () {System.out.println("I am Eating");}
}
class Walkable {
    public void walk () {System.out.println("I am Walking");}
}
class Flyable {
    public void fly () {System.out.println("I am Flying");}
}
class Dog {
    Mammal dog = new Mammal();
    Walkable walker = new Walkable();
}
class Bat {
    Mammal bat = new Mammal();
    Flyable flyer = new Flyable();
}
public class TestMammal {

    public static void main(String args[]) {

        System.out.println("Composition over Inheritance");
        System.out.println("\nDog");
        Dog fido = new Dog();
        fido.dog.eat();
```

O exemplo de composição executa os comportamentos por meio dos objetos agregados.

Isso não significa que herança deva ser eliminada. Se todo Mammal realmente compartilha eat(), pode ser razoável manter esse comportamento na superclasse e fazer Dog/Bat herdarem de Mammal. É uma decisão de projeto.

O centro da discussão é acoplamento. Herança pode quebrar parte do encapsulamento porque uma mudança em Mammal exige reavaliação/recompilação das subclasses. Com composição, adicionar Whale pode ser feito criando Swimmable e Whale sem alterar as classes existentes. Swimmable ainda pode ser reutilizada por Dolphin.

Uma regra prática é reservar herança para situações genuinamente polimórficas. Circle e Rectangle herdando de Shape podem ser um bom exemplo. Comportamentos como caminhar e voar podem ser melhores como capacidades separadas, evitando métodos sem sentido como fly() em Dog.

Entretanto, o exemplo de composição ainda possui uma falha séria: os objetos estão fortemente acoplados porque Whale cria diretamente seus colaboradores com new, por exemplo new Mammal() e new Swimmable(). O próximo passo é remover essa dependência concreta.

```
fido.walker.walk();

System.out.println("\nBat");;
Bat brown = new Bat();
brown.bat.eat();
brown.flyer.fly();

}

}
```

Note

The intent of this example is to illustrate how to use composition in lieu of inheritance; that does not mean that you cannot use inheritance at all in your designs. If you determine that absolutely all mammals eat, then, for example, perhaps you would decide to place the eat() method in the Mammal class and have Dog and Bat inherit from Mammal. As always, this is a design decision.

Perhaps the heart of this discussion lies in the concept we covered earlier, that inheritance breaks encapsulation. This is easy to understand because a change in the Mammal class would require a recompilation (and perhaps even a redeployment) of all the Mammal subclasses. This means that the classes are highly coupled, and this is counter to our stated goal of uncoupling classes as much as possible. In our composition example, if we wanted to add a Whale class, none of the previously written classes would need a rewrite. You would add a class called Swimmable and a class called Whale. Then the Swimmable class could be reused for, say, a Dolphin class.

[Click here to view code image](#)

```
class Swimmable {
    public void fly () {System.out.println("I am Swimming");;}
}
class Whale {
    Mammal whale = new Mammal();
    Walkable swimmer = new Swimmable ();
}
```

The main application can add this functionality with no changes to the classes that previously existed.

```
System.out.println("\nWhale");
Whale shamu = new Whale();
shamu.whale.eat();
shamu.swimmer.swim();
```

One rule of thumb is to use inheritance in only truly polymorphic situations. Thus, Circlesand Rectangles inheriting from Shape may well be a legitimate use of inheritance. On the other hand, behaviors such as walking and flying might not be good candidates for inheritance because overriding them could be problematic. For example, if you overrode the fly() method in Dog, the only obvious option would be a no-op (do nothing). Again, as we have seen with the earlier Penguin example, you don’t want a Dog to run over a cliff, execute the available fly() method and then, to Fido’s great chagrin, find that the fly() method doesn’t do anything.

While this example does indeed implement this solution using composition, there is a serious flaw to the design. The objects are highly coupled, since the use of the new keyword is obvious.

```
class Whale {
    Mammal whale = new Mammal();
    Walkable swimmer = new Swimmable ();
}
```

To complete our exercise of decoupling the classes, we introduce the concept of *dependency injection*. In short, rather than creating objects inside other objects, we will inject the objects from the outside via parameter lists. The discussion focuses solely on the concept of injecting dependencies.

Dependency Injection

The example in the previous section uses composition (with aggregation) to provide the Dog with a behavior called Walkable. The Dog class literally created a new Walkable object within the Dog class itself, as the following code fragment illustrates:

```
class Dog {
    Walkable walker = new Walkable();
}
```

Although this does in fact work, the classes remain highly coupled. To completely decouple the classes in the previous example, let’s implement the concept of dependency injection mentioned previously. Dependency injection and inversion of control are often covered together. One definition of inversion of control (IOC) is to make it someone else’s responsibility to make an instance of the dependency and pass it to you. This is exactly what we will implement in this example.

Because not all mammals walk, fly, or swim, to begin the decoupling process, we create interfaces to represent the behaviors for our various mammals. For this example, I will focus on the walking behavior by creating an interface called IWalkable as seen in Figure 11.3.

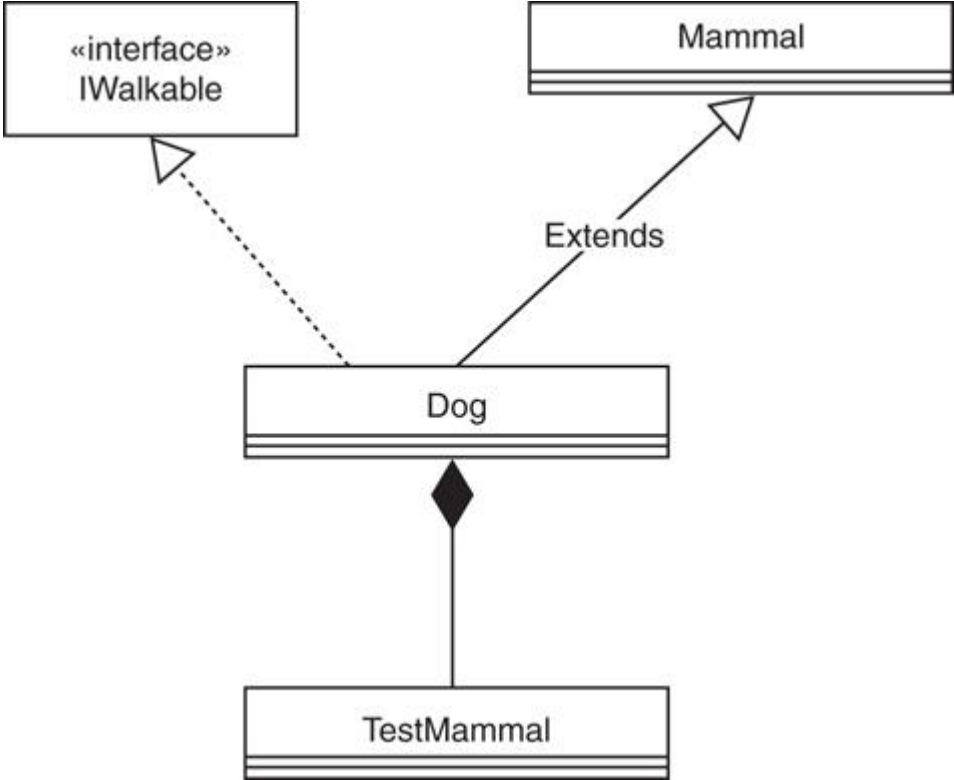


Figure 11.3 Using interfaces to

create mammals.

The code for the IWalkable interface is as follows:

```
interface IWalkable {
    public void walk();
}
```

The only method in this interface is walk(), which is left to the concrete class to provide the implementation.

Para completar o desacoplamento, introduzimos injeção de dependência. Em vez de criar colaboradores dentro do objeto, eles são fornecidos de fora por parâmetros.

INJEÇÃO DE DEPENDÊNCIA

No exemplo anterior, Dog cria diretamente new Walkable(). Isso funciona, mas mantém as classes acopladas.

Com dependency injection e inversão de controle (IoC), outra parte do sistema assume a responsabilidade de criar a dependência e fornecê-la ao objeto que precisa dela.

Como nem todos os mamíferos caminham, voam ou nadam, começamos definindo contratos para os comportamentos. A Figura 11.3 mostra IWalkable.

Figura 11.3 - Usando interfaces para criar comportamentos de mamíferos.

A interface é simples:

```
interface IWalkable {
    public void walk();
}
```

Ela declara o comportamento; a implementação fica a cargo de uma classe concreta.

[Click here to view code image](#)

```
class Dog extends Mammal implements IWalkable{
    Walkable walker;
    public void setWalker (Walkable w) {
        this.walker=w;
    }
    public void walk () {System.out.println("I am Walking");};
}
```

Note that the `Dog` class extends the `Mammal` class and implements the `IWalkable` interface. Also note that the `Dog` class provides a reference and a constructor that provides the mechanism to inject the dependency.

```
Walkable walker;
public void setWalker (Walkable w) {
    this.walker=w;
}
```

In a nutshell, this is what dependency injection is. The `Walkable` behavior is not created inside the `Dog` class using the `new` keyword; it is injected into the `Dog` class via the parameter list.

Here is the complete example:

[Click here to view code image](#)

```
class Mammal {
    public void eat () {System.out.println("I am Eating");};
}
interface IWalkable {
    public void walk();
}
class Dog extends Mammal implements IWalkable{
    Walkable walker;
    public void setWalker (Walkable w) {
        this.walker=w;
    }
    public void walk () {System.out.println("I am Walking");};
}
public class TestMammal {
    public static void main(String args[]) {
        System.out.println("Composition over Inheritance");
        System.out.println("\nDog");
        Walkable walker = new Walkable();
        Dog fido = new Dog();
        fido.setWalker(walker);
        fido.eat();
        fido.walker.walk();
    }
}
```

While this example uses injection by constructor, it is not the only way to handle dependency injection.

Injection by Constructor

One way to *inject* the `Walkable` behavior is to create a constructor within the `Dog` class that, when invoked, will accept an argument from the main application as follows:

```
class Dog {
    Walkable walker;
    public Dog (Walkable w) {
        this.walker=w;
    }
}
```

In this approach, the application instantiates a `Walkable` object and *inserts* it into the `Dog` via the constructor.

`Dog` estende `Mammal` e implementa `IWalkable`. Em vez de criar `Walkable` internamente, mantém apenas uma referência `walker` e recebe o objeto por um método `setWalker(Walkable w)`.

Esse é o núcleo da injeção de dependência: `Walkable` não é criado dentro de `Dog` com `new`; é fornecido externamente.

O exemplo completo cria `Mammal`, `IWalkable` e `Dog`. Na aplicação principal:

```
Walkable walker = new Walkable();
Dog fido = new Dog();
fido.setWalker(walker);
```

Depois `Fido` usa `eat()` e o comportamento de caminhada fornecido.

INJEÇÃO PELO CONSTRUTOR

Uma forma comum é receber a dependência no construtor:

```
class Dog {
    Walkable walker;
    public Dog(Walkable w) {
        this.walker = w;
    }
}
```

A aplicação instancia `Walkable` e cria `Dog` passando esse objeto. A construção da dependência fica fora de `Dog`.

```
Walkable walker = new Walkable();
```

```
Dog fido = new Dog(walker);
```

Injection by Setter

Although a constructor will *initialize* attributes when an object is instantiated, there is often a need to reset values during the lifetime of an object. This is where accessor methods come into play—in the form of setters. The `Walkable` behavior can be *inserted* into the `Dog` class by using a setter, here called `setWalker()`:

```
class Dog {
    Walkable walker;
    public void setWalker (Walkable w) {
        this.walker=w;
    }
}
```

As with the constructor technique, the application instantiates a `Walkable` object and inserts it into the `Dog` via the setter:

```
Walkable walker = new Walkable();
Dog fido = new Dog();
fido.setWalker(walker);
```

CONCLUSION

Dependency injection decouples your class’s construction from the construction of its dependencies. It is like buying something off the shelf (*from a vendor*) rather than building it on your own each time.

This plays to the heart of the discussion of Inheritance and composition. It is very important to note that this is simply a discussion. The purpose of this chapter is not necessarily to describe the “optimal” way to design your classes but to get you thinking about the issues associated with deciding between Inheritance and composition. In the next chapter, we explore The SOLID principles of object-oriented design, concepts highly regarded and accepted by the software development community.

REFERENCES

Martin, Robert, et al. *Agile Software Development, Principles, Patterns, and Practices*. 2002. Boston: Pearson Education, Inc.

Martin, Robert, et al. *Clean Code*. 2009. Boston: Pearson Education, Inc.

A aplicação pode escrever:

```
Walkable walker = new Walkable();
Dog fido = new Dog(walker);
```

INJEÇÃO POR SETTER

Outra opção é fornecer a dependência depois da construção. Isso é útil quando a referência pode ser alterada durante a vida do objeto.

Dog mantém walker e expõe setWalker(Walkable w). A aplicação cria os dois objetos separadamente e chama fido.setWalker(walker).

CONCLUSÃO

Dependency injection desacopla a construção de uma classe da construção de suas dependências. É semelhante a comprar um componente pronto de um fornecedor, em vez de fabricar internamente cada peça sempre que ela for necessária.

Essa discussão se conecta diretamente à escolha entre herança e composição. O objetivo do capítulo não é impor uma única arquitetura “ótima”, mas estimular análise sobre dependências e acoplamento.

No próximo capítulo são apresentados os princípios SOLID, amplamente utilizados para orientar projetos mais flexíveis e sustentáveis.

REFERÊNCIAS

Martin, Robert et al. Agile Software Development, Principles, Patterns, and Practices. 2002.
Martin, Robert et al. Clean Code. 2009.

12. The SOLID Principles of Object-Oriented Design

One of the most common statements that many developers make regarding object-oriented programming is that a primary advantage of OOP is that it models the real world. I admit that I use these words a lot when I discuss classical object-oriented concepts. According to Robert Martin (in at least one lecture that I viewed on YouTube), the idea that OO is closer to the way we think is simply marketing. Instead, he states that OO is about managing dependencies by inverting key dependencies to prevent rigid code, fragile code, and non-reusable code.

For example, in classical object-oriented programming courses, the practice often models the code directly to real-life situations. For example, if a dog *is-a* mammal, then this relationship is an obvious choice for inheritance. The strict *has-a* and *is-a* litmus test has been part of the OO mindset for years.

However, as we have seen throughout this book, trying to force an inheritance relationship can cause design problems (remember the barkless dog?). Is trying to separate barkless dogs from barking dogs, or flying birds from flightless birds, a smart inheritance design choice? Was this all put in place by object-oriented marketers? OK; forget the hype. As we saw in the previous chapter, perhaps focusing on a strict *has-a* and *is-a* decision is not necessarily the best approach. Perhaps we should focus more on decoupling the classes.

In the lecture I mentioned previously, Robert Martin, often referred to as Uncle Bob, defines these three terms to describe non-reusable code:

- **Rigidity**—When a change to one part of a program can break another part
- **Fragility**—When things break in unrelated places
- **Immobility**—When code cannot be reused outside its original context

SOLID was introduced to address these problems and strive to attain these goals. It defines five design principles that Robert Martin introduced to “make software designs more understandable, flexible, and maintainable.” According to Robert Martin, though they apply to any object-oriented design, the SOLID principles can also form a core philosophy for methodologies such as agile development or adaptive software development. The SOLID acronym was introduced by Michael Feathers.

The five SOLID principles are

- **SRP**—Single Responsibility Principle
- **OCP**—Open/Close Principle
- **LSP**—Liskov Substitution Principle
- **IPS**—Interface Segregation Principle
- **DIP**—Dependency Inversion Principle

This chapter focuses on covering these five principles and relates them to the classical object-oriented principles that have been in place for decades. My goal in covering SOLID is to explain the concepts in very simple examples. There is a lot of content online, including several very good YouTube videos. Many of these videos target developers, not necessarily students new to programming.

12. OS PRINCÍPIOS SOLID DO PROJETO ORIENTADO A OBJETOS

É comum afirmar que POO modela o mundo real e se aproxima da maneira como pensamos. Robert Martin (Uncle Bob) propõe outra ênfase: OO é, sobretudo, uma forma de administrar dependências, invertendo dependências importantes para evitar código rígido, frágil e não reutilizável.

Cursos clássicos frequentemente transformam relações do mundo real diretamente em herança: se Dog é Mammal, então “é-um” parece justificar a relação. Porém, exemplos anteriores mostraram que forçar hierarquias pode criar problemas - cães que não latem, aves que não voam etc.

Talvez, em vez de focar apenas no teste “é-um/tem-um”, devamos perguntar como reduzir acoplamento.

Martin usa três termos para código difícil de reutilizar:

- Rigidez: uma mudança em uma parte pode exigir mudanças em outras.
- Fragilidade: alterações quebram pontos aparentemente não relacionados.
- Imobilidade: código não consegue ser reutilizado fora do contexto original.

SOLID procura combater esses problemas e tornar projetos mais compreensíveis, flexíveis e manuteníveis. O acrônimo foi popularizado por Michael Feathers.

Os cinco princípios são:

- SRP - Single Responsibility Principle.
- OCP - Open/Closed Principle.
- LSP - Liskov Substitution Principle.
- ISP - Interface Segregation Principle.
- DIP - Dependency Inversion Principle.

Este capítulo apresenta cada princípio por exemplos simples e os relaciona aos conceitos OO clássicos.

As I have attempted to do with all the examples in this book, my intent is not to get overly complicated but to distill the examples to the lowest common denominator for educational purposes.

THE SOLID PRINCIPLES OF OBJECT-ORIENTED DESIGN

In [Chapter 11](#), “[Avoiding Dependencies and Highly Coupled Classes](#),” we discussed some of the fundamental concepts leading up to our discussion of the five SOLID principles. In this chapter, we dive right in and cover each of the SOLID principles in more detail. All SOLID definitions are from the Uncle Bob site: <http://butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod>.

1) SRP: Single Responsibility Principle

The Single Responsibility Principle states that a class should have only a single reason to change. Each class and module in a program should focus on a single task. Thus, don’t put methods that change for different reasons in the same class. If the description of the class includes the word “and,” you might be breaking the SRP. In other words, every module or class should have responsibility over a single part of the functionality provided by the software, and that responsibility should be entirely encapsulated in the class.

Creating a shape hierarchy is one of the classic illustrations of inheritance. It is used often as a teaching example, and I use it a lot throughout this chapter (as well as the book). In this example, a `Circle` class inherits from an abstract `Shape` class. The `Shape` class provides an abstract method called `calcArea()` as the contract for the subclass. Any class that inherits from `Shape` must provide its own implementation of `calcArea()`:

```
abstract class Shape{
    protected String name;
    protected double area;
    public abstract double calcArea();
}
```

In this example, we have a `Circle` class that inherits from `Shape` and, as required, provides its implementation of `calcArea()`:

```
class Circle extends Shape{
    private double radius;

    public Circle(double r) {
        radius = r;
    }
    public double calcArea() {
        area = 3.14*(radius*radius);
        return (area);
    };
}
```

Caution

In this example, we are only going to include a `Circle` class to focus on the Single Responsibility Principle and keep the example as simple as possible.

A third class called `CalculateAreas` sums the areas of different shapes contained in a `Shape` array. The `Shape` array is of unlimited size and can contain different shapes, such as squares and triangles.

[Click here to view code image](#)

```
class CalculateAreas {
    Shape[] shapes;
    double sumTotal=0;
```

O objetivo é manter os exemplos didáticos e reduzidos ao essencial.

OS PRINCÍPIOS SOLID

O capítulo anterior preparou a discussão sobre dependências e acoplamento. Agora cada princípio é abordado diretamente.

1) SRP: SINGLE RESPONSIBILITY PRINCIPLE

O Princípio da Responsabilidade Única afirma que uma classe deve ter apenas uma razão para mudar. Cada classe ou módulo deve concentrar-se em uma responsabilidade. Se a descrição da classe exige vários “e”, isso pode indicar violação do SRP.

O exemplo usa a hierarquia Shape. Circle herda de Shape e implementa calcArea(). Em seguida, uma classe CalculateAreas recebe uma coleção/array de Shapes e soma as áreas chamando calcArea() polimorficamente.

A página apresenta Shape abstrata e Circle concreta com radius e a fórmula da área. A próxima página adiciona a classe que soma resultados e evidencia o problema de responsabilidade misturada.

CalculateAreas soma as áreas, mas também possui um método output() para exibir o resultado. Assim, cálculo e apresentação estão acoplados na mesma classe.

O programa TestShape cria Circle, coloca-o em um array de Shape, instancia CalculateAreas, executa sumAreas() e depois output(). Funciona, mas viola SRP.

O problema aparece quando queremos alterar apenas a forma de saída. Por exemplo, trocar texto simples por HTML exige modificar, recompilar e redistribuir CalculateAreas, mesmo que o algoritmo de soma não tenha mudado.

Segundo SRP, a mudança de uma responsabilidade não deveria obrigar a tocar em outra. “Uma classe deve ter uma, e somente uma, razão para mudar.”

A solução proposta é separar cálculo e saída em classes distintas: CalculateAreas fica responsável pela soma; OutputAreas passa a cuidar da apresentação.

```
public CalculateAreas(Shape[] sh){
    this.shapes = sh;
}
public double sumAreas() {
    sumTotal=0;
    for (inti=0; i<shapes.length; i++) {
        sumTotal = sumTotal + shapes[i].calcArea();
    }
    return sumTotal;
}
public void output() {
    System.out.println("Total of all areas = " + sumTotal);
}
}
```

Note that the CalculateAreas class also handles the output for the application, which is problematic. The area calculation behavior and the output behavior are coupled—contained in the same class.

We can verify that this code works with the following test application called TestShape:

Click here to view code image

```
public class TestShape {
    public static void main(String args[]) {

        System.out.println("Hello World!");

        Circle circle = new Circle(1);

        Shape[] shapeArray = new Shape[1];
        shapeArray[0] = circle;

        CalculateAreas ca = new CalculateAreas(shapeArray);

        ca.sumAreas();
        ca.output();
    }
}
```

Now with the test application in place, we can focus on the issue of the Single Responsibility Principle. Again, the issue is with the CalculateAreas class and that this class contains behaviors for *summing* the various areas as well as the *output*. The fundamental point (and problem) here is this: If you want to change the functionality of the output() method, it requires a change to the CalculateAreas class regardless of whether the method for summing the areas changes. For example, if at some point we want to present the output to the console in HTML rather than in simple text, we must recompile and redeploy the code that sums the area because the responsibilities are coupled.

According to the Single Responsibility Principle, the goal is that a change to one method would not affect the other method, thus preventing unnecessary recompilations. “A class should have one, and only one, reason to change—a single responsibility to change.”

To address this, we can put the two methods in separate classes, one for the original console output and one for the newly included HTML output:

Click here to view code image

```
class CalculateAreas {
    Shape[] shapes;
    double sumTotal=0;

    public CalculateAreas(Shape[] sh){
        this.shapes = sh;
    }
}
```

```

    }

    public double sumAreas() {
        sumTotal=0;

        for (inti=0; i<shapes.length; i++) {

            sumTotal = sumTotal + shapes[i].calcArea();

        }

        return sumTotal;
    }
}
class OutputAreas {
    double areas=0;
    public OutputAreas(double a){
        this.areas = a;
    }
    public void console() {
        System.out.println("Total of all areas = " + areas);
    }

    public void HTML() {
        System.out.println("<HTML>");
        System.out.println("Total of all areas = " + areas);
        System.out.println("</HTML>");
    }
}

```

Now, using the newly written class, we can add functionality for HTML output without impacting the code for the area summing:

[Click here to view code image](#)

```

public class TestShape {
    public static void main(String args[]) {

        System.out.println("Hello World!");

        Circle circle = new Circle(1);

        Shape[] shapeArray = new Shape[1];
        shapeArray[0] = circle;

        CalculateAreas ca = new CalculateAreas(shapeArray);

        CalculateAreas sum = new CalculateAreas(shapeArray);
        OutputAreasoAreas = new OutputAreas(sum.sumAreas());

        oAreas.console();    // output to console
        oAreas.HTML();       // output to HTML

    }
}

```

The main point here is that you can now send the output to various destinations depending on requirements. If you want to add another output possibility, such as JSON, you can add it to the `OutputAreas` class without having to change the `CalculateAreas` class. As a result, you can redistribute the `CalculateAreas` class independently without having to do anything to the other classes.

`CalculateAreas` agora contém apenas o estado e o algoritmo necessários para somar áreas. `OutputAreas` recebe o resultado e oferece formas de apresentação, como `console()` e `HTML()`.

A aplicação principal cria `CalculateAreas`, calcula o total, cria `OutputAreas` com o valor e escolhe uma ou mais saídas.

O principal benefício é separar razões para mudança. Se amanhã surgir saída em JSON, a lógica de cálculo não precisa ser alterada. `CalculateAreas` pode evoluir e ser distribuída de forma independente da camada de apresentação.

Esse exemplo ilustra SRP de maneira simples: uma classe para calcular, outra para apresentar.

2) OCP: Open/Close Principle

The Open/Close Principle states that you should be able to extend a class's behavior, without modifying it.

Let's revisit the shape example yet again. In the following code, we have a class called `ShapeCalculator` that accepts a `Rectangle` object, calculates the area of that object, and then returns that value. It is a simple application but it works only for rectangles.

[Click here to view code image](#)

```
class Rectangle{
    protected double length;
    protected double width;

    public Rectangle(double l, double w) {
        length = l;
        width = w;
    };
}
class CalculateAreas {

    private double area;

    public double calcArea(Rectangle r) {

        area = r.length * r.width;

        return area;

    }
}
public class OpenClosed {
    public static void main(String args[]) {

        System.out.println("Hello World");

        Rectangle r = new Rectangle(1,2);

        CalculateAreas ca = new CalculateAreas ();

        System.out.println("Area = "+ ca.calcArea(r));

    }
}
```

The fact that this application works only for rectangles brings us to a constraint that illustrates the Open/Closed Principle: If we want to add a `Circle` to the `CalculateArea` class (change what it does), we must change the module itself. Obviously, this is at odds with the Open/Closed Principle, which stipulates that we should not have to change the module to change what it does. To comply with the Open/Closed Principle, we can revisit our tried and true shape example, where an abstract class called `Shape` is created and then all shapes must inherit from the `Shape` class, which has an abstract method called `getArea()`.

At this point, we can add as many different classes as we want without having to change the `Shape` class itself (for example, a `Circle`). We can now say that the `Shape` class is *closed*.

The following code implements this solution for a rectangle and a circle, and allows for the creation of unlimited shapes:

[Click here to view code image](#)

```
abstract class Shape {
```

2) OCP: OPEN/CLOSED PRINCIPLE

O Princípio Aberto/Fechado afirma que deve ser possível estender o comportamento de uma unidade sem modificar seu código já estabelecido.

O exemplo inicial possui `Rectangle` e uma `CalculateAreas` cuja função `calcArea(Rectangle r)` conhece especificamente retângulos. A aplicação funciona, mas apenas para esse tipo.

Se quisermos acrescentar `Circle`, teremos de modificar `CalculateAreas`. Isso conflita com OCP: a unidade deveria estar aberta à extensão, mas fechada à modificação.

A solução retoma `Shape` abstrata com `getArea()`. Cada forma concreta implementa seu próprio cálculo. `CalculateAreas` passa a depender de `Shape`, não de `Rectangle`.

Assim, novas formas podem ser adicionadas criando subclasses/implementações compatíveis, sem alterar a lógica central que apenas chama `s.getArea()`.


```
        public abstract double getArea();
    }
    class Rectangle extends Shape
    {
        protected double length;
        protected double width;

        public Rectangle(double l, double w) {
            length = l;
            width = w;
        };
        public double getArea() {
            return length*width;
        }
    }

    class Circle extends Shape
    {
        protected double radius;

        public Circle(double r) {
            radius = r;
        };
        public double getArea() {
            return radius*radius*3.14;
        }
    }
    class CalculateAreas {
        private double area;

        public double calcArea(Shape s) {
            area = s.getArea();
            return area;
        }
    }
    public class OpenClosed {
        public static void main(String args[]) {

            System.out.println("Hello World");

            CalculateAreas ca = new CalculateAreas();

            Rectangle r = new Rectangle(1,2);

            System.out.println("Area = " + ca.calcArea(r));

            Circle c = new Circle(3);

            System.out.println("Area = " + ca.calcArea(c));

        }
    }
```

Note that in this implementation, the `CalculateAreas()` method does not have to change when you add a new `Shape`.

You can scale your code without having to worry about legacy code. At its core, the Open/Closed Principle states that you should extend your code via subclasses and the original class does not need to be changed. However, the word *extension* is problematic in several discussions relating to SOLID. As we will cover in detail, if we are to favor composition over inheritance, how does this affect the Open/Closed Principle?

O código implementa `Shape` abstrata, `Rectangle` e `Circle` concretas. Cada uma fornece `getArea()` apropriado.

`CalculateAreas` recebe um `Shape`:

```
public double calcArea(Shape s) {
    return s.getArea();
}
```

A aplicação pode passar `Rectangle`, `Circle` ou novas formas futuras. Quando um novo `Shape` é acrescentado, `CalculateAreas` não precisa ser modificado.

Essa é a ideia de estar “aberto para extensão e fechado para modificação”. Podemos escalar o sistema adicionando tipos compatíveis sem alterar código legado estável.

O livro observa, porém, que a palavra “extensão” merece cuidado. Se preferimos composição em muitos cenários, OCP não deve ser interpretado como uma ordem para criar hierarquias profundas. O objetivo é permitir novos comportamentos por abstrações estáveis.

When following one of the SOLID principles, code may also comply with one of the other SOLID principles. For example, when designing to follow the Open/Closed Principle, the code may also comply with the Single Responsibility Principle.

3) LSP: Liskov Substitution Principle

The Liskov Substitution Principle states that the design must provide the ability to replace any instance of a parent class with an instance of one of its child classes. If a parent class can do something, a child class must also be able to do it.

Let’s examine some code that might look reasonable but violates the Liskov Substitution Principle. In the following code, we have the typical abstract class called `Shape`. `Rectangle` then inherits from `Shape` and overrides its abstract method `calcArea()`. `Square`, in turn, inherits from `Rectangle`.

[Click here to view code image](#)

```
abstract class Shape{
    protected double area;

    public abstract double calcArea();
}
class Rectangle extends Shape{
    private double length;
    private double width;

    public Rectangle(double l, double w){
        length = l;
        width = w;
    }
    public double calcArea() {
        area = length*width;
        return (area);
    };
}

class Square extends Rectangle{
    public Square(double s){
        super(s, s);
    }
}

public class LiskovSubstitution {
    public static void main(String args[]) {

        System.out.println("Hello World");

        Rectangle r = new Rectangle(1,2);

        System.out.println("Area = " + r.calcArea());

        Square s = new Square(2);

        System.out.println("Area = " + s.calcArea());

    }
}
```

So far so good: a rectangle *is-a* shape so everything looks fine. Because a square *is-a* rectangle we are still fine—or are we?

Os princípios SOLID frequentemente se reforçam. Um projeto que segue OCP pode, por exemplo, também melhorar SRP.

3) LSP: LISKOV SUBSTITUTION PRINCIPLE

O Princípio da Substituição de Liskov afirma que uma instância de uma classe pai deve poder ser substituída por uma instância de uma classe filha sem quebrar as expectativas do programa. Se o tipo pai promete determinado comportamento, o subtipo precisa preservar esse contrato.

O exemplo cria `Shape` abstrata, `Rectangle` concreta e `Square` herdando de `Rectangle`. À primeira vista, parece natural: um retângulo é `Shape` e um quadrado é um tipo de retângulo.

`Rectangle` recebe `length` e `width` e calcula `area = length * width`. `Square` chama o construtor de `Rectangle` usando o mesmo valor para os dois lados.

O programa cria um `Rectangle` e um `Square` e calcula áreas. Até aqui tudo parece funcionar. Mas o capítulo pergunta: embora geometricamente “um quadrado seja um retângulo”, essa hierarquia preservará todas as expectativas comportamentais de `Rectangle`? A próxima seção explora o problema.

Now we enter into a somewhat philosophical discussion: Is a square really a rectangle? Many people would say yes. However, while the square may well be a specialized type of a rectangle, it does have different properties than a rectangle. A rectangle is a parallelogram (opposite sides are congruent), as is a square. Yet, a square is also a rhombus (all sides are congruent), whereas a rectangle is not. Therefore, there are some differences.

The geometry is not really the issue when it comes to OO design. The issue is how we build rectangles and squares. Here is the constructor for the `Rectangle` class:

```
public Rectangle(double l, double w){
    length = l;
    width = w;
}
```

The constructor obviously requires two parameters. However, the `Square` constructor requires just one, even though its parent class, `Rectangle`, is expecting two.

```
class Square extends Rectangle{
    public Square(double s){
        super(s, s);
    }
}
```

In actuality, the functionality to compute area is subtly different for the two classes. In fact, the `Square` is kind of faking the `Rectangle` out by passing it the same parameter twice. This may seem like an acceptable workaround, but it really is something that may confuse someone maintaining the code and could very well cause unintended maintenance headaches down the road. This is an inconsistency at minimum and, perhaps, a questionable design decision. When you see a constructor calling another constructor, it might be a good idea to pause and reconsider the design—it might not be a proper child class.

How do you address this specific dilemma? Simply put, a square is not a *substitute* for a rectangle and should not be a child class. Thus, they should be separate classes.

[Click here to view code image](#)

```
abstract class Shape {
    protected double area;

    public abstract double calcArea();
}
```

```
class Rectangle extends Shape {

    private double length;
    private double width;

    public Rectangle(double l, double w) {
        length = l;
        width = w;
    }

    public double calcArea() {
        area = length*width;
        return (area);
    };
}
```

```
class Square extends Shape {
    private double side;

    public Square(double s){
        side = s;
    }
    public double calcArea() {
        area = side*side;
    }
}
```

Entramos agora em uma discussão conceitual: um quadrado é realmente um retângulo? Geometricamente, muitos diriam que sim, mas `Square` possui propriedades mais restritas. Ambos são paralelogramos; porém o quadrado também exige todos os lados congruentes.

Para o projeto OO, a questão não é apenas geometria, mas o contrato dos objetos. `Rectangle` é construído com dois parâmetros, comprimento e largura. `Square` necessita de apenas um lado e, para reutilizar o construtor do pai, passa o mesmo valor duas vezes.

Isso funciona tecnicamente, mas revela uma inconsistência comportamental e pode confundir manutenção. Quando uma subclasse precisa “enganar” a abstração do pai para encaixar-se nela, vale reconsiderar a hierarquia.

Nesse exemplo, a solução proposta é não fazer `Square` herdar de `Rectangle`. Ambas herdam diretamente de `Shape` e implementam `calcArea()` de acordo com suas próprias regras.

O código da página mostra `Shape` abstrata, `Rectangle` com `length/width` e `Square` com `side`. Assim, cada tipo respeita o contrato comum sem impor uma relação de substituição inadequada.

```
        return (area);
    };
}
public class LiskovSubstitution {
    public static void main(String args[]) {

        System.out.println("Hello World");

        Rectangle r = new Rectangle(1,2);

        System.out.println("Area = " + r.calcArea());

        Square s = new Square(2);

        System.out.println("Area = " + s.calcArea());

    }
}
```

4) IPS: Interface Segregation Principle

The Interface Segregation Principle states that it is better to have many small interfaces than a few larger ones.

In this example, we are creating a *single* interface that includes *multiple* behaviors for a Mammal, eat() and makeNoise():

[Click here to view code image](#)

```
interface IMammal {
    public void eat();
    public void makeNoise();
}
class Dog implements IMammal {
    public void eat() {
        System.out.println("Dog is eating");
    }
    public void makeNoise() {
        System.out.println("Dog is making noise");
    }
}
public class MyClass {
    public static void main(String args[]) {

        System.out.println("Hello World");

        Dog fido = new Dog();
        fido.eat();
        fido.makeNoise();

    }
}
```

Rather than creating a single interface for Mammal, we can create separate interfaces for all the behaviors:

[Click here to view code image](#)

```
interface IEat {
    public void eat();
}
interface IMakeNoise {
    public void makeNoise();
}
class Dog implements IEat, IMakeNoise {
    public void eat() {
```

O programa de teste cria Rectangle e Square separadamente e calcula suas áreas por suas próprias implementações de calcArea().

4) ISP: INTERFACE SEGREGATION PRINCIPLE

O Princípio da Segregação de Interfaces afirma que é melhor ter várias interfaces pequenas e focadas do que poucas interfaces grandes e abrangentes.

O primeiro exemplo define IMammal com dois comportamentos, eat() e makeNoise(). Dog implementa ambos. Embora funcione, a interface agrupa responsabilidades que poderiam variar independentemente.

Uma alternativa é separar os comportamentos:

```
interface IEat { void eat(); }
interface IMakeNoise { void makeNoise(); }
```

Dog implementa IEat e IMakeNoise. A classe recebe apenas os contratos de que realmente necessita. Esse desenho evita obrigar outros tipos a implementar métodos irrelevantes.

```
        System.out.println("Dog is eating");
    }
    public void makeNoise() {
        System.out.println("Dog is making noise");
    }
}
public class MyClass {
    public static void main(String args[]) {

        System.out.println("Hello World");

        Dog fido = new Dog();
        fido.eat();
        fido.makeNoise();
    }
}
```

In reality, we are decoupling the behaviors from the `Mammal` class. Thus, rather than creating a single `Mammal` entity via inheritance (actually interfaces) we are moving to a composition-based design, similar to the strategy taken in the previous chapter.

In short, by using this approach, we can build `Mammals` with composition rather than being forced to utilize behaviors contained in a single `Mammal` class. For example, suppose someone discovers a `Mammal` that doesn’t eat but instead absorbs nutrients through its skin. If we were inheriting from a single `Mammal` class that contains the `eat()` behavior, the new mammal would not need this behavior. However, if we separate all the behaviors into separate, single interfaces, we can build each mammal in exactly the way it presents itself.

5) DIP: Dependency Inversion Principle

The Dependency Inversion Principle states that code should depend on abstractions. It often seems like the terms *dependency inversion* and *dependency injection* are used interchangeably; however, here are some key terms to understand as we discuss this principle:

- **Dependency inversion**—The principle of inverting the dependencies
- **Dependency injection**—The act of inverting the dependencies
- **Constructor injection**—Performing dependency injection via the constructor
- **Parameter injection**—Performing dependency injection via the parameter of a method, like a setter

The goal of dependency inversion is to couple to something abstract rather than concrete.

Although at some point you obviously have to create something concrete, we strive to create a concrete object (by using the `new` keyword) as far up the chain as possible, such as in the `main()` method. Perhaps a better way of thinking of this is to revisit the discussion presented in [Chapter 8](#),

“Frameworks and Reuse: Designing with Interfaces and Abstract Classes,” where we discuss loading classes at runtime, and in [Chapter 9](#), “[Building Objects and Object-Oriented Design](#),” where we talk about decoupling and creating small classes with limited responsibilities.

In the same vein, one of the goals of the Dependency Inversion Principle is to choose objects at runtime, not at compile time. (You can change the behavior of your program at runtime.) You can even write new classes without having to recompile old ones (in fact, you can write new classes and inject them).

Ao separar `IEat` e `IMakeNoise`, desacoplamos comportamentos da ideia única de `Mammal`. Isso aproxima o desenho de composição: cada tipo pode ser construído a partir das capacidades que realmente possui.

Se algum mamífero hipotético obtivesse nutrientes sem “comer” no sentido modelado, ele não deveria ser obrigado a implementar `eat()`. Interfaces pequenas permitem selecionar apenas os contratos adequados.

5) DIP: DEPENDENCY INVERSION PRINCIPLE

O Princípio da Inversão de Dependência afirma que o código deve depender de abstrações, não de detalhes concretos.

Termos importantes:

- dependency inversion: o princípio de inverter dependências;
- dependency injection: o ato/técnica de fornecer dependências externamente;
- constructor injection: injeção pelo construtor;
- parameter/setter injection: injeção por parâmetro de método.

O objetivo é acoplar módulos a algo abstrato. Objetos concretos obviamente precisam ser criados em algum ponto, mas procuramos empurrar `new` para as camadas mais externas, como `main()`, deixando o núcleo dependente de contratos.

Outro objetivo é permitir escolhas em tempo de execução. Com abstrações e injeção, novas implementações podem ser conectadas sem recompilar classes antigas que dependem apenas do contrato.

Much of the foundation for this discussion was put forth in [Chapter 11](#), “[Avoiding Dependencies and Highly Coupled Classes](#).” Let’s build on that as we consider the Dependency Inversion Principle.

Step 1: Initial Example

For the first step in this example, we revisit yet again one of the classical object-oriented design examples used throughout this book, that of a `Mammal` class, along with a `Dog` and a `Cat` class that inherit from `Mammal`. The `Mammal` class is abstract and contains a single method called `makeNoise()`.

```
abstract class Mammal
{
    public abstract String makeNoise();
}
```

The subclasses, such as `Cat`, use inheritance to take advantage of `Mammal`’s behavior, `makeNoise()`:

```
class Cat extends Mammal
{
    public String makeNoise()
    {
        return "Meow";
    }
}
```

The main application then instantiates a `Cat` object and invokes the `makeNoise()` method:

```
Mammal cat = new Cat();;
```

```
System.out.println("Cat says " + cat.makeNoise());
```

The complete application for the first step is presented in the following code:

[Click here to view code image](#)

```
public class TestMammal {
    public static void main(String args[]) {

        System.out.println("Hello World\n");

        Mammal cat = new Cat();;
        Mammal dog = new Dog();

        System.out.println("Cat says " + cat.makeNoise());
        System.out.println("Dog says " + dog.makeNoise());

    }
}
abstract class Mammal
{
    public abstract String makeNoise();
}
class Cat extends Mammal
{
    public String makeNoise()
    {
        return "Meow";
    }
}
class Dog extends Mammal
{
    public String makeNoise()
    {
        return "Bark";
    }
}
```

O Capítulo 11 preparou a base para essa discussão. Agora o livro constrói o exemplo DIP em etapas.

ETAPA 1: EXEMPLO INICIAL

Retomamos `Mammal`, `Dog` e `Cat`. `Mammal` é abstrata e declara `makeNoise()`. `Cat` e `Dog` herdam e implementam o método retornando, respectivamente, “Meow” e “Bark”.

A aplicação principal instancia objetos concretos e os trata como `Mammal`:

```
Mammal cat = new Cat();
Mammal dog = new Dog();
```

Em seguida chama `makeNoise()` polimorficamente.

O código é válido, mas comportamento e tipo do animal continuam ligados por herança. A próxima etapa separa a capacidade de produzir som da classe `Mammal`.

Step 2: Separating Out Behavior

The preceding code has a potentially serious flaw: It couples the mammals and the behavior (`makingNoise`). There may be a significant advantage to separating the mammal behaviors from the mammals themselves. To accomplish this, we create a class called `MakingNoise` that can be used by all mammals as well as non-mammals.

In this model, a `Cat`, `Dog`, or `Bird` can then extend the `MakeNoise` class and create their own noise-making behavior specific to their needs, such as the following code fragment for a `Cat`:

```
abstract class MakingNoise
{
    public abstract String makeNoise();
}
class CatNoise extends MakingNoise
{
    public String makeNoise()
    {
        return "Meow";
    }
}
```

With the `MakingNoise` behavior separated from the `Cat` class, we can use the `CatNoise` class in place of the hard coded behavior in the `Cat` class itself, as the following code fragment illustrates:

```
abstract class Mammal
{
    public abstract String makeNoise();
}
class Cat extends Mammal
{
    CatNoise behavior = new CatNoise();
    public String makeNoise()
    {
        return behavior.makeNoise();
    }
}
```

The following is the complete application for the second step:

[Click here to view code image](#)

```
public class TestMammal {
    public static void main(String args[]) {

        System.out.println("Hello World\n");

        Mammal cat = new Cat();
        Mammal dog = new Dog();

        System.out.println("Cat says " + cat.makeNoise());
        System.out.println("Dog says " + dog.makeNoise());

    }
}

abstract class MakingNoise
{
    public abstract String makeNoise();
}
class CatNoise extends MakingNoise
{
    public String makeNoise()
    {
        return "Meow";
    }
}
```

ETAPA 2: SEPARANDO O COMPORTAMENTO

O código anterior acopla mamíferos ao comportamento `makeNoise()`. Pode ser vantajoso extrair a produção de som para uma abstração reutilizável também por outros tipos.

Cria-se `MakingNoise` e classes como `CatNoise`. `CatNoise` implementa o comportamento específico “Meow”. `Cat` então contém um objeto `CatNoise` e delega `makeNoise()` para `behavior.makeNoise()`.

Essa separação remove parte da lógica específica de dentro de `Cat` e permite pensar no comportamento como um componente independente.

A página apresenta o início da aplicação completa, incluindo `MakingNoise` abstrata e `CatNoise`. A próxima página mostra `DogNoise` e analisa o acoplamento ainda restante.

```
}
class DogNoise extends MakingNoise
{
    public String makeNoise()
    {
        return "Bark";
    }
}
abstract class Mammal
{
    public abstract String makeNoise();
}

class Cat extends Mammal
{
    CatNoise behavior = new CatNoise();
    public String makeNoise()
    {
        return behavior.makeNoise();
    }
}
class Dog extends Mammal
{
    DogNoise behavior = new DogNoise();
    public String makeNoise()
    {
        return behavior.makeNoise();
    }
}
```

The problem is that although we have decoupled a major part of the code, we still haven’t reached our goal of dependency inversion because the `Cat` is still instantiating the `Cat` noise-making behavior.

```
CatNoise behavior = new CatNoise();
```

The `Cat` is coupled to the low-level module `CatNoise`. In other words, the `Cat` should not be coupled to `CatNoise` but to the abstraction for making noise. In fact, the `Cat` class should not instantiate its noise-making behavior but instead receive the behavior via injection.

Step 3: Dependency Injection

In this final step, we totally abandon the inheritance aspects of our design and examine how to utilize dependency injection via composition. You do not need inheritance hierarchies, which is one of the major reasons why the concept of composition over inheritance is gaining momentum. You compose a subtype rather than creating a subtype from a hierarchical model.

To illustrate, in the original implementation, the `Cat` and the `Dog` basically contain the same exact code; they simply return a different noise. As a result, a significant percentage of the code is redundant. Thus, if you had many different mammals, there would be a lot of noise-making code. Perhaps a better design is to take the code to make noise out of the `mammal`.

The major leap here would be to abandon the specific mammals (`Cat` and `Dog`) and simply use the `Mammal` class as shown here:

```
class Mammal
{
    MakingNoise speaker;

    public Mammal(MakingNoisesb)
    {
        this.speaker = sb;
    }
    public String makeNoise()
```

A aplicação completa possui `CatNoise` e `DogNoise`, além de `Cat` e `Dog`. Cada mamífero contém diretamente uma instância concreta de seu comportamento:

```
CatNoise behavior = new CatNoise();
DogNoise behavior = new DogNoise();
```

Apesar da melhoria, ainda não atingimos DIP. `Cat` continua acoplada ao módulo concreto `CatNoise`. O ideal é que `Cat` dependa da abstração `MakingNoise` e receba a implementação de fora.

ETAPA 3: DEPENDENCY INJECTION

Na etapa final, o exemplo abandona a hierarquia específica `Cat/Dog` e compõe `Mammal` com um comportamento injetado.

`Mammal` mantém uma referência `MakingNoise speaker`, recebida no construtor. `makeNoise()` apenas delega para `speaker.makeNoise()`.

Em vez de “criar uma subclasse `Cat`”, podemos compor um mamífero com comportamento de gato fornecendo `CatNoise`. Essa abordagem ilustra como composição + injeção podem substituir parte de uma hierarquia quando a variação relevante é comportamental.

Agora criamos objetos assim:

```
Mammal cat = new Mammal(new CatNoise());
Mammal dog = new Mammal(new DogNoise());
```

A classe Mammal não conhece CatNoise nem DogNoise concretamente; conhece apenas a interface MakingNoise. As implementações CatNoise e DogNoise cumprem esse contrato e retornam seus sons específicos.

O programa principal escolhe e injeta as dependências. Assim, o comportamento pode ser trocado sem alterar Mammal.

Uma decisão importante em dependency injection é onde instanciar os objetos concretos. Mesmo num sistema orientado a abstrações, algo precisa executar new. O objetivo é concentrar essa criação nas bordas/níveis mais altos da aplicação e manter as classes de domínio dependentes de abstrações sempre que isso trazer benefício.

```
    {
        return this.speaker.makeNoise();
    }
}
```

Now we can instantiate a Cat noise-making behavior and provide it to the Animal class, to make a mammal that behaves like a Cat. In fact, you can always assemble a Cat by injecting behaviors rather than using the traditional techniques of class building.

```
Mammal cat = new Mammal(new CatNoise());
```

The following is the complete application for the final step:

[Click here to view code image](#)

```
public class TestMammal {
    public static void main(String args[]) {

        System.out.println("Hello World\n");

        Mammal cat = new Mammal(new CatNoise());
        Mammal dog = new Mammal(new DogNoise());

        System.out.println("Cat says " + cat.makeNoise());
        System.out.println("Dog says " + dog.makeNoise());

    }
}
class Mammal
{
MakingNoise speaker;

    public Mammal(MakingNoisesb)
    {
        this.speaker = sb;
    }
    public String makeNoise()
    {
        return this.speaker.makeNoise();
    }
}
interface MakingNoise
{
    public String makeNoise();
}
class CatNoise implements MakingNoise
{
    public String makeNoise()
    {
        return "Meow";
    }
}
class DogNoise implements MakingNoise
{
    public String makeNoise()
    {
        return "Bark";
    }
}
```

When discussing dependency injection, when to actually instantiate an object is now a key consideration. Even though the goal is to compose objects via injection, you obviously must instantiate objects at some point. As a result, the design decisions revolve around when to do this instantiation.

As stated earlier in this chapter, the goal of dependency inversion is to couple to something abstract rather than concrete, even though you obviously must create something concrete at some point. Thus, one simple goal is to create a concrete object (by using `new`) as far up the chain as possible, such as in the `main()` method. Always evaluate things when you see a `new` keyword.

CONCLUSION

This concludes the discussion of SOLID. The SOLID principles are one of the most influential sets of object-oriented guidelines used today. What is interesting about studying these principles is how they relate to the fundamental object-oriented encapsulation, inheritance, polymorphism, and composition, specifically in the debate of composition over inheritance.

For me, the most interesting point to take away from the SOLID discussion is that nothing is cut and dried. It is obvious from the discussion on composition over inheritance that even the age-old fundamental OO concepts are open for reinterpretation. As we have seen, a bit of time, along with the corresponding evolution in various thought processes, is good for innovation.

REFERENCES

Martin, Robert, et al. *Agile Software Development, Principles, Patterns, and Practices*. 2002. Boston: Pearson Education, Inc.

Martin, Robert, et al. *Clean Code*. 2009. Boston: Pearson Education, Inc.

Como afirmado anteriormente, DIP busca acoplamento com abstrações, mesmo sabendo que implementações concretas precisam existir. Uma regra simples é criar objetos concretos o mais “alto” possível na cadeia - por exemplo, em `main()` ou em um componente dedicado à composição. Sempre que encontrar `new` dentro de uma classe de domínio, vale avaliar se aquela dependência deveria ser injetada.

CONCLUSÃO

Com isso termina a discussão sobre SOLID. Esses princípios estão entre as diretrizes de projeto OO mais influentes atualmente. Um aspecto especialmente interessante é a relação deles com os conceitos clássicos - encapsulamento, herança, polimorfismo e composição - e com o debate “composição sobre herança”.

A principal conclusão é que decisões de projeto raramente são absolutas. Até conceitos fundamentais podem ser reinterpretados à luz de novas experiências. A evolução das práticas e a revisão crítica de ideias estabelecidas são parte importante da inovação em engenharia de software.

REFERÊNCIAS

Martin, Robert et al. *Agile Software Development, Principles, Patterns, and Practices*. 2002.
Martin, Robert et al. *Clean Code*. 2009.

[Os títulos bibliográficos foram preservados no idioma original.]



Photo by izusek/gettyimages

Register Your Product at informit.com/register

Access additional benefits and **save 35%** on your next purchase

- Automatically receive a coupon for 35% off your next purchase, valid for 30 days. Look for your code in your InformIT cart or the Manage Codes section of your account page.
- Download available product updates.
- Access bonus material if available.*
- Check the box to hear from us and receive exclusive offers on new editions and related products.

**Registration benefits vary by product. Benefits will be listed on your account page under Registered Products.*

InformIT.com—The Trusted Technology Learning Source

InformIT is the online home of information technology brands at Pearson, the world's foremost education company. At InformIT.com, you can:

- Shop our books, eBooks, software, and video training
- Take advantage of our special offers and promotions (informit.com/promotions)
- Sign up for special offers and content newsletter (informit.com/newsletters)
- Access thousands of free chapters and video lessons

Connect with InformIT—Visit informit.com/community



informIT
The Trusted Technology Learning Source

Addison-Wesley • Adobe Press • Cisco Press • Microsoft Press • Pearson IT Certification • Prentice Hall • Que • Sams • Peachpit Press

